BayesFilter

Bayesian Estimation for Structural State-Space Models

HMC-Safe Filtering, Analytic Likelihood Derivatives,
and Industrial-Scale Inference

Chak Wong

May 28, 2026

Contents

Part I

Scope and Interfaces

Chapter 1

Introduction

BayesFilter is a monograph and software-design project for Bayesian estimation of structural state-space models. The word structural is important. The latent state is not merely an anonymous vector passed to a numerical routine: its coordinates may be stochastic shocks, deterministic lags, accounting identities, endogenous completions, auxiliary variables, or measurement-only summaries. A filtering backend that ignores those roles can evaluate the wrong likelihood, produce the wrong gradient, or make a sampler explore states the model never generates.

The project was extracted from DSGE, macro-finance, and analytic Kalman derivative work, but its purpose is broader and cleaner than any one source project. BayesFilter owns the reusable contracts for state-space likelihoods, derivatives, diagnostics, and compiled Bayesian sampling. The economics of NK, EZ, SGU, CIP, AFNS, and future NAWM-scale applications remain downstream case studies that stress those contracts.

The organizing premise is that filtering is not a small implementation detail inside Bayesian state-space inference. For Hamiltonian Monte Carlo, the filter is part of the posterior target: it supplies the log likelihood, the gradient path, the invalid-parameter behavior, and much of the geometry seen by the sampler. A filtering backend that is adequate for point likelihood evaluation may still be unsuitable for HMC if it has hidden non-finite branches, unstable spectral derivatives, dynamic shapes, structural state violations, or opaque compilation behavior.

This monograph separates four layers:

(i) the structural state-space model contract;

(ii) the likelihood and derivative backend;

(iii) the HMC target and diagnostic policy;

(iv) application case studies that stress the generic contracts.

The separation lets the linear Gaussian and analytic derivative spine be audited before nonlinear sigma-point filters, particle filters, transport maps, or DSGE-specific experiments are used as evidence.

1.1 Central Design Thesis

BayesFilter follows one design thesis:

> model projects own structural maps, and BayesFilter owns filters.

For an autoregression, the structural map may be a companion-form lag shift. For a macro-finance model, it may be an affine no-arbitrage state recursion with masked yields. For a DSGE model, it may be a perturbation solution that separates exogenous shocks from endogenous predetermined variables. These maps belong to the model project. The common filtering machinery, derivative validation, diagnostics, and HMC target discipline belong to BayesFilter.

This separation prevents each application project from rediscovering the same numerical failure modes. It also prevents a successful linear Gaussian implementation from being misused as a nonlinear structural filter simply because both expose a state vector and covariance matrix.

1.2 Provenance and Claim Discipline

The source documents for this project are read-only inputs. BayesFilter consolidates them into one notation system and one production-readiness policy, but it does not treat old prose as automatically authoritative. Every substantive claim should be traceable to one of four sources: a local source chapter or reset memo listed in `docs/source_map.yml`, a reviewed literature artifact, a code/test reference, or an explicitly recorded project decision such as the TensorFlow Probability NUTS benchmark.

The most important negative rule is equally simple: no backend is described as HMC-safe merely because it returns a finite likelihood on a smoke test. HMC safety requires a target contract, a derivative policy, finite failure handling, static-shape or compilation discipline where compilation is claimed, and diagnostics that can reveal when the sampler is exploring a numerically fragile region.

1.3 Reader Map

The book is written from the stable core outward. Part I fixes notation and interfaces, including structural state partitions. Part II develops the exact linear Gaussian likelihood because it is the regression oracle for more complicated filters. Part III records analytic derivative and custom-gradient policy before nonlinear filters are promoted into HMC experiments. Part IV discusses nonlinear filters, deterministic completions, and spectral risks with those derivative requirements already in place. Part V states the HMC, JIT, and diagnostic contracts. The case studies appear last so they can be judged against the generic BayesFilter checklist rather than driving the definitions.

Chapter 2

State-Space Model Contracts

BayesFilter treats a state-space model as a contract between a structural model and a filtering backend. The structural model may be a DSGE perturbation solver, an affine term-structure model, a macro-finance system, or a synthetic validation problem. The filter should not need to know that origin; it should receive typed maps with stable shapes and declared derivative policies.

Definition 2.1 (BayesFilter state-space contract)**.** *For parameter vector $\theta \in \Theta \subseteq \mathbb{R}^{d_\theta}$, a BayesFilter state-space contract supplies latent states $x_t \in \mathbb{R}^{n_x}$, observations $y_t \in \mathbb{R}^{n_y}$, transition law*

$$p_\theta(x_{t+1} \mid x_t),$$

observation law

$$p_\theta(y_t \mid x_t),$$

initial law $p_\theta(x_0)$, and an observation mask or selection rule when the available components of y_t vary over time. A linear Gaussian specialization supplies matrices and vectors

$$c_\theta, \quad F_\theta, \quad Q_\theta, \quad a_\theta, \quad H_\theta, \quad R_\theta,$$

with dimensions fixed by the model instance.

Definition 2.2 (Structural state partition)**.** *A structural state partition is model metadata that decomposes the latent state coordinates into declared roles. The minimal BayesFilter partition records*

$$x_t = \mathrm{pack}(s_t, d_t, a_t),$$

where s_t is the stochastic block receiving current innovations directly or through a declared stochastic transition, d_t is a deterministic-completion block, and a_t is an optional auxiliary block used for lags, bookkeeping, or backend-specific summaries. The partition also records the innovation dimension and the pack/unpack order.

The partition is part of the model contract. It should not be inferred only from a covariance matrix. A zero row of a shock-impact matrix is a useful diagnostic, but it is not a complete timing declaration. Conversely, a small positive nugget inserted for numerical regularization does not turn a deterministic identity into a structural source of randomness.

Assumption 2.1 (Shape and value contract)**.** *For a fixed model instance, tensor ranks and leading dimensions are static throughout a compiled likelihood evaluation. Parameter-dependent covariance objects are symmetric positive semidefinite or positive definite as*

required by the backend, and invalid parameter values are reported through an explicit finite-failure policy rather than accidental linear-algebra exceptions.

The contract separates model maps from filter maps. A model map builds state-space objects from θ. A filter map consumes those objects and the data. A derivative provider supplies first or second derivatives of the state-space objects, or declares that a backend relies on autodiff through primitive operations. This distinction is essential for large models: the derivative of a structural solver and the derivative of a Kalman or sigma-point recursion are different mathematical layers.

Remark 2.1 (Structural deterministic dynamics are part of the contract)**.** *For DSGE models, the state-space contract must not hide the distinction between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states and current shocks. A collapsed linear transition may be sufficient for an exact Kalman filter, but nonlinear sigma-point and particle filters need the structural transition map. This is a release-gating issue; see Chapter 19.*

2.1 Degenerate Transitions and Lag Stacks

Structural deterministic coordinates are not special to DSGE models. An autoregression of order two already has the same issue. One companion-form representation is

$$m_t = \phi_1 m_{t-1} + \phi_2 \ell_{t-1} + \sigma \varepsilon_t, \tag{2.1}$$
$$\ell_t = m_{t-1}, \tag{2.2}$$
$$x_t = (m_t, \ell_t). \tag{2.3}$$

The lag coordinate ℓ_t is deterministic conditional on the previous state. In an exact linear Kalman backend, the corresponding transition covariance is singular:

$$Q = \begin{bmatrix} \sigma^2 & 0 \\ 0 & 0 \end{bmatrix}.$$

That singularity is structural information, not an implementation failure. For nonlinear sigma-point or particle filters, quadrature or simulation should be applied to the innovation or stochastic block and then lifted through the lag-shift map.

The same contract covers accumulated sums, stock-flow identities, mixed-frequency aggregation states, and deterministic no-arbitrage recursions. BayesFilter should therefore test degenerate transitions directly rather than only through full-rank synthetic examples.

2.2 Structural Nonlinear Transition Contract

For a nonlinear structural model, a backend should know whether it integrates over innovations, stochastic states, or the full latent state. A convenient contract is

$$s_t = T_s(s_{t-1}, \varepsilon_t; \theta), \tag{2.4}$$
$$d_t = T_d(d_{t-1}, s_{t-1}, s_t; \theta), \tag{2.5}$$
$$x_t = \mathrm{pack}(s_t, d_t, a_t), \tag{2.6}$$
$$y_t \sim p_\theta(y_t \mid x_t). \tag{2.7}$$

When a mixed structural model is evaluated by a full-state Gaussian approximation, the result must be labeled as an approximation. The default nonlinear filter for a mixed structural model should preserve deterministic identities pointwise for propagated sigma points or particles.

Assumption 2.2 (Approximation labeling)**.** *If a backend integrates over a larger stochastic space than the declared model transition, or injects artificial noise into deterministic coordinates, the run metadata must record an approximation label. The label is part of the likelihood provenance and must be visible to HMC diagnostics and case-study reports.*

2.3 Masks and Mixed Frequency

A sparse panel should be represented by a deterministic mask or selection operator, not by changing the model definition at each time. At time t, the filter may work with the observed subvector $y_{t,\mathcal{O}_t}$ and the corresponding selected observation map. The public contract must state whether an all-missing time step performs only prediction, whether ragged panels are batched by pattern, and whether the implementation keeps static shapes for JIT compilation.

2.4 Backend Classes

BayesFilter distinguishes the following backend roles:

- exact linear Gaussian likelihood backends;
- square-root or solve-form variants of exact linear Gaussian backends;
- approximate nonlinear Gaussian filters such as EKF, UKF, CKF, and SVD-factored sigma-point filters;
- particle-filter or simulation-based likelihood estimators;
- diagnostic or surrogate backends that do not define the final HMC target without an explicit correction policy.

The same model can be evaluated by more than one backend, but the monograph must state which backend defines the posterior target and which backends are only diagnostic approximations.

Chapter 3

Posterior Targets and HMC Requirements

HMC samples an unconstrained or transformed parameter vector by integrating Hamiltonian dynamics for a log target. In BayesFilter, that log target usually contains a filtering likelihood. The target contract must therefore be stated before discussing sampler tuning.

Definition 3.1 (Filtering posterior target)**.** *Let $u \in \mathbb{R}^{d_u}$ denote the coordinates used by the sampler and let $\theta = T(u)$ be the model parameter transform. A filtering posterior target has the form*

$$\log \pi(u \mid y_{1:T}) = \log p(T(u)) + \ell(T(u); y_{1:T}) + \log \left|\det DT(u)\right|,$$

where ℓ is the log likelihood returned by the chosen filtering backend.

For HMC, the value alone is not enough. The backend and transform jointly define the gradient seen by the sampler. If a custom gradient is used, it must be the gradient of the same scalar target value, not the gradient of a nearby surrogate unless the sampler is explicitly corrected for that surrogate.

3.1 Finite Failure Policy

Invalid parameters are ordinary events during HMC warmup and exploration. A BayesFilter target must decide how to handle them. The preferred policy is to return a deterministic low log density and finite fallback gradient for known invalid regions, while separately recording diagnostics for the reason. A linear-algebra failure that aborts the process is not an HMC-compatible failure mode.

The policy must be explicit for:

- parameter transforms and prior support;
- stationarity or determinacy restrictions;
- covariance positive-definiteness checks;
- failed factorizations or solve residuals;
- non-finite likelihood or gradient values.

3.2 Compilation and Differentiability

When a backend is advertised as JIT or XLA compatible, the claim refers to the complete value-and-gradient path needed by the sampler. Partial tracing of an outer function is not enough if the filter, derivative provider, or failure branch executes outside the compiled region.

The derivative policy should be one of:

- analytic score or Hessian recursion;
- custom gradient wrapping a certified analytic derivative;
- autodiff through smooth primitive operations;
- stopped-gradient approximation used only with an explicit correction or diagnostic label;
- not HMC-safe.

Chapter 4

BayesFilter API Design

The monograph is not an API reference, but its mathematical contracts should be precise enough that an implementation can be audited against them. The API shape below is therefore a documentation contract rather than a final Python interface.

4.1 Core Objects

A BayesFilter implementation should expose five separable objects:

- a parameter transform from sampler coordinates u to model parameters θ;
- a structural state partition describing stochastic, deterministic, and auxiliary coordinates;
- a model provider that builds state-space objects from θ;
- a filter backend that returns log likelihood and diagnostics;
- a derivative provider or autodiff policy that returns the score and, when needed, curvature information.

For a linear Gaussian model, the model provider returns

$$(c, F, Q, a, H, R, m_0, P_0)$$

and optionally their first and second derivatives with respect to θ. For a nonlinear model, it returns transition and observation maps together with the sigma-point, linearization, or simulation policy required by the selected backend.

4.2 Structural Model Protocol

The exact software interface may change, but implementations should be auditable against the following documentation contract:

`partition` :	state names, role indices, innovation dimension,
`initial_law` :	(m_0, P_0) or a sampling rule,
`innovation_law` :	Σ_ε or an innovation sampler,
`unpack_state`, `pack_state` :	$x \leftrightarrow (s, d, a)$,
`propagate_stochastic` :	$(s_{t-1}, \varepsilon_t, \theta) \mapsto s_t$,
`complete_deterministic` :	$(d_{t-1}, s_{t-1}, s_t, \theta) \mapsto d_t$,
`observe` :	$x_t \mapsto p_\theta(y_t \mid x_t)$.

Here s_t denotes the declared stochastic block, d_t the deterministic completion block, and a_t any auxiliary or backend-specific block. A model with no deterministic block can set d_t empty; a linear Gaussian companion form can still expose the partition even if the exact Kalman backend consumes only the collapsed matrices.

DSGE, MacroFinance, and other application projects should implement this protocol through adapters. BayesFilter should not require those projects to copy filter internals, and those projects should not define private near-duplicates of BayesFilter filters.

4.3 Filter Run Metadata

Every backend should return metadata that records what target was actually evaluated. At minimum, diagnostics should include:

- backend name and version;
- structural partition summary;
- integration space: innovation, stochastic-state, or full-state;
- deterministic-completion policy;
- approximation label, if any;
- derivative policy;
- compiled/eager execution status when relevant;
- finite-failure or warning codes.

This metadata prevents a collapsed approximation, a diagnostic surrogate, or a partially compiled run from being misreported as a structural HMC target.

For mixed structural models, the metadata must also be strong enough to audit deterministic completion. A sigma-point, cubature, or particle backend should therefore expose:

- whether deterministic identities were checked pointwise;
- the maximum deterministic identity residual when such a check is available;

- whether a full-state integration path was used for a mixed model;
- the approximation label attached to any enlarged stochastic space;
- whether artificial jitter was only a numerical linear-algebra device or part of the model transition being evaluated.

These fields are not cosmetic. They are the evidence that the backend followed the `structural_filter_step` contract from Chapter 19. If they are missing, the safest interpretation is that the result has not yet earned a structural filtering claim.

4.4 Structural Filter Implementation Requirements

A BayesFilter structural filter implementation should satisfy the following requirements before a client adapter uses it as an HMC likelihood:

(i) **Fail-closed partition validation:** mixed models must declare stochastic, deterministic-completion, and auxiliary coordinates before a non-full-state backend runs.

(ii) **Visible integration space:** the result must state whether the backend integrated over innovations, stochastic states, or the full state.

(iii) **Deterministic identity diagnostics:** test fixtures with known identities, such as AR(p) lag shifts or $k_t - \phi k_{t-1} - \gamma m_t^2 = 0$, must be checked pointwise for propagated points or particles.

(iv) **Approximation-label propagation:** if a mixed model is evaluated by a full-state Gaussian approximation, the label must survive into the likelihood result, HMC target wrapper, case-study tables, and reset memo.

(v) **Reference-oracle tests:** affine Gaussian cases must recover the exact Kalman value, and low-dimensional nonlinear cases should be compared with dense quadrature, high-particle simulation, or another explicit reference.

(vi) **Derivative and compilation gates:** value-side structural correctness is not a gradient guarantee. SVD/eigen derivatives, static shapes, eager/compiled parity, and HMC diagnostics need separate gates.

DSGE and MacroFinance adapters should supply model timing and economics. BayesFilter should supply this generic validation and filtering machinery. That boundary keeps the common structural-filtering rule in one place while still allowing client projects to own their own solution methods and priors.

4.5 Return Contract

The minimum likelihood return is

$$(\ell,\ d),$$

where ℓ is a scalar log likelihood and d is a diagnostics record. A gradient-capable backend returns

$$(\ell,\ \nabla_\theta \ell,\ d),$$

and a curvature backend returns

$$(\ell,\ \nabla_\theta \ell,\ \nabla^2_{\theta\theta} \ell,\ d).$$

Diagnostics should include enough information to decide whether the result is a valid target evaluation, a finite invalid-region return, or a numerical warning that should influence sampler adaptation or backend selection.

4.6 Backend Promotion

BayesFilter should promote a backend through stages:

(i) value-only likelihood evaluation;

(ii) gradient validation on small problems;

(iii) compiled value-and-gradient execution;

(iv) stress tests under masks, boundaries, and near-singular covariances;

(v) HMC diagnostics on controlled targets;

(vi) application case studies.

No backend should skip directly from value-only likelihood tests to industrial HMC use.

4.7 Client Adapter Boundary

The adapter boundary is intentionally strict. A client project owns:

- structural equations and timing;
- parameter transformations and priors;
- model-specific observation equations;
- perturbation, no-arbitrage, or equilibrium solution objects;
- application-level HMC targets and case-study data.

BayesFilter owns:

- generic filter recursions;
- structural partition validation;
- square-root and SVD numerical backends;
- derivative validation ladders;
- finite-failure and diagnostic conventions;
- backend promotion gates.

The boundary is a maintenance device. It lets MacroFinance reuse analytic Kalman derivatives, DSGE reuse structural nonlinear filtering, and future large models reuse the same validation ladder without forking the numerical core.

Part II

Linear Gaussian Filtering

Chapter 5

Prediction-Error Decomposition

The linear Gaussian model is the first BayesFilter reference target because its likelihood is exact and its derivative recursions can be audited against code and automatic-differentiation tests. This chapter records the value-only likelihood spine; score and Hessian recursions are deferred to Chapters 9–10.

5.1 Linear Gaussian Specialization

Under the contract in Definition 2.1, the linear Gaussian specialization is

$$x_{t+1} = c(\theta) + F(\theta)x_t + u_{t+1}, \qquad u_{t+1} \sim \mathcal{N}(0, Q(\theta)), \tag{5.1}$$

$$y_t = a(\theta) + H(\theta)x_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R(\theta)). \tag{5.2}$$

For the exact likelihood backend, $Q(\theta) \succeq 0$ and $R(\theta) \succ 0$ are required at every accepted target evaluation. The initial moments are denoted $m_{0|0}$ and $P_{0|0}$.

The semidefinite allowance for Q is intentional. Companion-form lag stacks, mixed-frequency accumulators, and structural deterministic coordinates often produce process covariances with zero rows or deficient rank. This is not a reason to abandon the exact linear Gaussian likelihood. It is a requirement that the backend distinguish structural degeneracy from accidental numerical failure.

Assumption 5.1 (Innovation regularity)**.** *For every time point contributing an observation, the selected innovation covariance S_t defined below is positive definite on the observed coordinates. If the model or data mask makes S_t singular, the likelihood backend must return a named finite-failure diagnostic or use a separately documented diffuse/singular-observation treatment.*

5.2 Kalman Recursion

Let

$$m_{t|s} = \mathbb{E}[x_t \mid y_{1:s}], \qquad P_{t|s} = \mathrm{Var}(x_t \mid y_{1:s}).$$

The prediction step is

$$m_{t|t-1} = c + F m_{t-1|t-1}, \tag{5.3}$$

$$P_{t|t-1} = F P_{t-1|t-1} F^\top + Q. \tag{5.4}$$

The innovation and innovation covariance are

$$v_t = y_t - a - Hm_{t|t-1}, \tag{5.5}$$
$$S_t = HP_{t|t-1}H^\top + R. \tag{5.6}$$

When $S_t \succ 0$, the Kalman gain and filtered moments are

$$K_t = P_{t|t-1}H^\top S_t^{-1}, \tag{5.7}$$
$$m_{t|t} = m_{t|t-1} + K_t v_t, \tag{5.8}$$
$$P_{t|t} = (I - K_t H)P_{t|t-1}. \tag{5.9}$$

The covariance update in (5.9) is the compact textbook form. Production implementations may use the Joseph form

$$P_{t|t} = (I - K_t H)P_{t|t-1}(I - K_t H)^\top + K_t R K_t^\top$$

or may update an algebraically equivalent factor rather than materializing a full covariance matrix. These are numerical representations of the same exact linear Gaussian likelihood, not different statistical models.

5.3 Log Likelihood

The prediction-error decomposition gives

$$\ell(\theta) = -\frac{1}{2}\sum_{t=1}^{T}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_{y,t}\log(2\pi)\right], \tag{5.10}$$

where $n_{y,t}$ is the number of observed components at time t. In a dense panel, $n_{y,t} = n_y$. In a masked panel, the same formula applies after selecting the observed components and the corresponding rows and columns of the observation objects.

If no components are observed at a time point, the measurement update and likelihood contribution are skipped and the filter carries the prediction forward. This convention treats missingness as part of the observation contract rather than as a change in the latent transition law.

Equation (5.10) is the exact likelihood for (5.1)–(5.2). It is also the reference value against which approximate nonlinear filters and alternative linear algebra backends should be tested whenever a controlled linear Gaussian case is available.

5.4 Initialization

Initial moments are part of the model contract. If $m_{0|0}$ and $P_{0|0}$ are fixed by design, derivative initial conditions in later chapters are zero. If $P_{0|0}$ is the stationary covariance, then its derivatives are governed by the derivative Lyapunov equations for the transition system. BayesFilter must record which initialization policy defines the target, because changing it changes the likelihood.

5.5 Reference Role

The prediction-error likelihood is the value oracle for the rest of the monograph. Later chapters may evaluate it through covariance, solve-form, Cholesky, QR, or SVD linear-algebra primitives, but they must agree on (5.10) within documented tolerances. Nonlinear filters should also recover this value when their model is specialized to a linear Gaussian case.

Derivative chapters differentiate this same scalar likelihood. A custom gradient, analytic score recursion, or Hessian recursion is valid for HMC only if it corresponds to the value defined here, plus the prior and parameter transform terms in Chapter 3.

Chapter 6

Stable Linear Filtering

The covariance-form equations in Chapter 5 are algebraically correct, but they are not always the best production primitive. Large panels, nearly collinear observations, low measurement noise, and repeated covariance updates can turn small roundoff differences into large differences in precision matrices and Hessian contractions.

6.1 Backend Ladder

BayesFilter uses a backend ladder rather than a single Kalman implementation:

(i) covariance-form recursion for clarity and small reference problems;

(ii) solve-form likelihood evaluation using factored innovation covariances;

(iii) Cholesky square-root propagation where positive pivots are reliable;

(iv) QR square-root propagation when reconstruction and solve diagnostics are needed;

(v) SVD or spectral fallbacks only with explicit diagnostic labels.

The ladder is a production policy, not a mathematical change to the likelihood. All exact linear Gaussian backends must agree with (5.10) within documented tolerances.

6.2 Covariance Form as Reference Algebra

The covariance-form recursion is the clearest way to state the likelihood, but it is not automatically the safest way to compute it. In finite precision, the compact covariance update can lose symmetry or positive semidefiniteness after many time steps. Implementations should therefore symmetrize only as a documented numerical projection, not as a way to hide model invalidity.

For small reference tests, covariance form is useful because it exposes the same moments that appear in the mathematical recursion. For production likelihoods, BayesFilter should prefer solve and factor forms that avoid explicit inverses and provide residual diagnostics.

6.3 Solve Form

Let $S_t = L_t L_t^\top$ and define $z_t = L_t^{-1} v_t$. Then

$$\log\det S_t = 2\sum_k \log L_{t,kk}, \qquad v_t^\top S_t^{-1} v_t = z_t^\top z_t.$$

The likelihood can therefore be evaluated with triangular solves and log diagonal pivots rather than by explicitly forming S_t^{-1}. This is the preferred reference form for implementation because the same solves appear in the analytic score and Hessian recursions.

The solve-form contract is:

- factor the selected innovation covariance;
- solve for the whitened innovation;
- accumulate log determinant and quadratic form from the factor and solve result;
- record factor pivots and solve residuals.

The backend should not form an explicit inverse except in small diagnostic tests where the numerical error is itself being measured.

6.4 Square-Root and Spectral Backends

Cholesky square-root filters are efficient when positive pivots are reliable. QR square-root filters are useful when the implementation needs stable stack updates, reconstruction checks, and a clear path to derivative-factor validation. SVD or eigenspectral factorizations are useful for rank-deficient or nearly rank-deficient linear Gaussian problems, but they should be reported as spectral backend choices with their own diagnostics.

This distinction matters because the word SVD appears in two different parts of the monograph. An SVD-factored exact linear Gaussian backend is still evaluating (5.10). A nonlinear SVD sigma-point filter is an approximate nonlinear Gaussian filter whose value and gradient policy require separate tests. Success of the former does not certify the latter.

6.5 Backend Agreement Tests

Every exact linear Gaussian backend should be tested against at least one reference backend on controlled models. A minimal agreement test records:

- log-likelihood difference;
- maximum filtered-mean difference;
- maximum filtered-covariance or reconstructed-factor difference;
- minimum pivot or singular value;
- solve residual norm;
- whether the process covariance is full rank or structurally singular.

For singular Q examples, the test should assert that the likelihood is finite and that the structural degeneracy is reported as model structure or backend telemetry, not as an unexplained numerical exception.

6.6 Diagnostics

Stable filtering backends should return diagnostics, not only a scalar likelihood. The MacroFinance large-scale backend ladder tests track:

- minimum factor pivots;
- reconstruction errors for prediction, innovation, and filtered factors;
- solve residual norms;
- finite log likelihood, gradient, and Hessian values;
- backend deltas against the solve-form reference.

Those tests classify SVD fallback triggers as no trigger, candidate trigger, or confirmed trigger. BayesFilter should preserve that caution: a spectral fallback is a diagnostic event, not evidence that the HMC gradient path is automatically safe.

The diagnostic record should travel with the likelihood into HMC reports. A finite log likelihood with repeated tiny pivots or frequent fallback triggers is not the same engineering result as a finite log likelihood with comfortable factor margins. Both may be mathematically valid evaluations, but they imply different sampler risk.

Chapter 7

Missing Data and Mixed Frequency

Real macro-finance panels are rarely dense. Missing observations, mixed-frequency releases, publication lags, and ragged starts must be handled as part of the filtering contract rather than as ad hoc data cleaning outside the likelihood.

7.1 Selection Form

Let M_t be a deterministic selection matrix that extracts the observed components of y_t. The dense observation equation

$$y_t = a + Hx_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R)$$

becomes

$$M_t y_t = M_t a + M_t H x_t + M_t \varepsilon_t, \qquad M_t \varepsilon_t \sim \mathcal{N}(0, M_t R M_t^\top).$$

The prediction-error likelihood in (5.10) then uses the selected innovation and selected innovation covariance. If no components are observed at time t, the filter performs prediction without a measurement update.

Equivalently, write $\mathcal{O}_t$ for the observed row set. The per-period objects are

$$v_{t,\mathcal{O}_t} = y_{t,\mathcal{O}_t} - a_{\mathcal{O}_t} - H_{\mathcal{O}_t} m_{t|t-1}, \qquad S_{t,\mathcal{O}_t} = H_{\mathcal{O}_t} P_{t|t-1} H_{\mathcal{O}_t}^\top + R_{\mathcal{O}_t,\mathcal{O}_t}.$$

The likelihood contribution uses the dimension $|\mathcal{O}_t|$. If $\mathcal{O}_t$ is empty, the contribution is zero and the filtered moments equal the predicted moments. This convention is important for ragged panels because it makes missingness explicit in the likelihood rather than hidden in preprocessed data.

7.2 Mixed-Frequency State Augmentation

Mixed-frequency models often require deterministic auxiliary states. Examples include cumulators for quarterly growth, publication-lag states, and running averages used to align daily, monthly, and quarterly releases. These coordinates are part of the structural state contract from Definition 2.2. Their process covariance may be singular because the auxiliary rows are deterministic functions of lagged states and current high-frequency states.

BayesFilter should therefore distinguish two operations:

(i) augmenting the state so the observation equation is linear or conditionally linear at the target sampling frequency;

(ii) selecting the observed rows at each time point.

Both operations preserve the prediction-error likelihood when the augmented transition and observation maps are the declared model. A small ridge added only for numerical conditioning must be recorded as numerical regularization, not as a new source of structural shock variation.

7.3 Implementation Policy

The public backend must state whether it supports sparse panels. A dense-only backend should fail before likelihood evaluation if the observations contain NaNs, infinities, or masks that imply unsupported missing-data behavior. The MacroFinance missing-data policy tests explicitly check that unsupported sparse observation masks raise a named pending-support error rather than leaking NaNs into the Kalman recursion.

For compiled likelihoods, there are two viable policies:

- group time steps by mask pattern and keep each compiled path static;
- keep a fixed dense shape and use selection or masking operations whose shape is known at trace time.

Either policy is acceptable if the likelihood value and derivative contract are unchanged and the diagnostics identify which observations contributed to each time step.

The diagnostic record should include at least:

- number of observed rows at each time or by mask pattern;
- count of all-missing prediction-only steps;
- mask-pattern identifiers used for compiled grouping;
- whether missingness was handled by row selection, dense masking, or an unsupported fallback;
- any regularization applied to augmented deterministic rows.

7.4 Derivative Deferral

The value contract above is independent of the derivative implementation. Later derivative chapters must state whether mask patterns are differentiated as fixed data, whether all-missing steps contribute zero derivative to the measurement likelihood, and whether mixed-frequency augmentation parameters enter the transition, the observation map, or both. This chapter fixes the likelihood object; it does not yet certify any derivative backend.

Chapter 8

Large-Scale Linear Gaussian State-Space Models

Large linear Gaussian models are the bridge between textbook Kalman filtering and industrial Bayesian state-space inference. They are still exact, but their dimension exposes the numerical and compilation issues that small examples can hide.

8.1 Scale Targets

A BayesFilter scale target should record:

- state dimension n_x;
- observation dimension n_y;
- time length T;
- parameter dimension d_θ;
- dense or sparse observation policy;
- backend ladder and fallback rules;
- memory layout and compilation policy.

The MacroFinance large-scale LGSSM tests include scenarios such as `baseline_10x3x5`, low-measurement-noise collinearity, ill-conditioned process noise, and near-rank-loss candidates. BayesFilter uses these as infrastructure evidence, not as a claim that every NAWM-sized model is already solved.

8.2 Backend Registry

Large-scale runs should be described by a backend registry rather than by a single method name. A registry entry should identify:

- the value backend, such as covariance form, solve form, Cholesky square root, QR square root, or spectral fallback;

- the derivative backend, such as tape autodiff, analytic score, analytic Hessian, custom gradient, or finite-difference diagnostic;
- the initialization policy;
- the missing-data policy;
- static-shape and compilation assumptions;
- finite-failure diagnostics and fallback labels.

This registry is part of the scientific result. A reported HMC run without backend metadata is not reproducible enough to diagnose whether a divergence, small effective sample size, or slow compilation came from the posterior geometry, the filtering algebra, the derivative path, or the execution mode.

8.3 Validation Ladder

The validation ladder should move from exact values to sampler behavior:

(i) value parity against the prediction-error decomposition in Chapter 5;

(ii) filtered-moment and backend-delta parity across covariance, solve, and square-root implementations on controlled small cases;

(iii) stress cases with low measurement noise, nearly collinear observations, singular or ill-conditioned process covariance, and long panels;

(iv) derivative parity on small cases before the derivative backend is used by HMC;

(v) compiled-path checks for static shape, finite diagnostics, and stable recompilation behavior;

(vi) short HMC smoke runs that exercise the target without being used as convergence evidence;

(vii) medium and strict HMC recovery only after the value and derivative gates have passed.

The same ladder should be reused for MacroFinance adapters, DSGE adapters, and future BayesFilter-native examples. Passing it for one model family is useful evidence about the backend, but it does not automatically transfer to another state partition, observation mask policy, or parameterization.

8.4 Stress Scenarios

The stress suite should include both numerical and modeling edge cases:

- structurally singular Q from AR(p) companion states, mixed-frequency accumulators, or deterministic structural coordinates;

- nearly singular innovation covariance from collinear measurements or very small measurement noise;
- long panels where repeated updates test covariance symmetry and factor reconstruction;
- ragged panels and all-missing time steps;
- parameter draws near admissibility boundaries;
- backend fallback triggers and spectral-gap telemetry.

Each scenario should define the expected diagnostic outcome before it is used as an HMC target. Some cases should pass with comfortable numerical margins; others should return named finite failures. Both outcomes are useful if they are intentional and reproducible.

8.5 Readiness Criteria

A large-scale LGSSM backend is ready for HMC experiments only after:

(i) the value backend matches a solve-form or covariance-form oracle;

(ii) gradient and Hessian backends match finite differences or autodiff on small controlled cases;

(iii) square-root or QR backends match the solve-form backend within stated tolerances on dimension-scaled cases;

(iv) diagnostics remain finite under stress scenarios;

(v) shape and compilation behavior are documented.

The current evidence supports this ladder as a validation design. It does not remove the need to rerun the ladder for each new model family, parameterization, or missing-data policy.

8.6 Derivative Consolidation Hypotheses

The next analytic-derivative pass should test the following hypotheses against the MacroFinance derivation and implementation:

(i) the Chapter 5 value contract is identical to the likelihood differentiated in the MacroFinance analytic Kalman note;

(ii) analytic score recursions match autodiff or finite differences on small dense and singular-Q cases;

(iii) Hessian recursions match finite-difference-on-gradient diagnostics on small cases before they are used for mass-matrix or curvature proposals;

(iv) solve-form derivatives avoid explicit innovation-covariance inverses in production code;

(v) derivative diagnostics identify the first time step and matrix block responsible for nonfinite values or large parity errors.

These are hypotheses, not completed certification. They should be promoted to claims only after source derivations, code, and numerical tests agree.

Part III

Analytic Derivatives

Chapter 9

Kalman Score Recursions

The score recursion differentiates the prediction-error decomposition from Chapter 5. In BayesFilter this is the first derivative contract for exact linear Gaussian HMC targets.

For parameter component θ_i, write

$$\dot{z}^{(i)} = \frac{\partial z}{\partial \theta_i}.$$

All objects in the filtering recursion may depend on θ:

$$c_t, \quad F_t, \quad Q_t, \quad a_t, \quad H_t, \quad R_t.$$

Time subscripts are kept in this chapter because missing-data masks and mixed-frequency observation systems can make the effective observation equation time varying even when the underlying structural model is time homogeneous.

9.1 Prediction Derivatives

The differentiated prediction step is

$$\dot{m}^{(i)}_{t|t-1} = \dot{c}^{(i)}_t + \dot{F}^{(i)}_t m_{t-1|t-1} + F_t \dot{m}^{(i)}_{t-1|t-1}, \tag{9.1}$$

$$\dot{P}^{(i)}_{t|t-1} = \dot{F}^{(i)}_t P_{t-1|t-1} F_t^\top + F_t \dot{P}^{(i)}_{t-1|t-1} F_t^\top + F_t P_{t-1|t-1} \dot{F}^{(i)\top}_t + \dot{Q}^{(i)}_t. \tag{9.2}$$

These are source labels `eq:dm_pred_note` and `eq:dP_pred_note` in the MacroFinance analytic derivative note. They also define the initial-condition contract: if the implementation fixes $(m_{0|0}, P_{0|0})$ as data, the initial derivatives are zero; if it uses a stationary initial covariance, the derivatives must come from the corresponding derivative Lyapunov equations and be reported as part of the structural provider.

9.2 Innovation Derivatives

The source note derives the innovation derivatives

$$\dot{v}^{(i)}_t = -\dot{a}^{(i)}_t - \dot{H}^{(i)}_t m_{t|t-1} - H_t \dot{m}^{(i)}_{t|t-1}, \tag{9.3}$$

$$\dot{S}^{(i)}_t = \dot{H}^{(i)}_t P_{t|t-1} H_t^\top + H_t \dot{P}^{(i)}_{t|t-1} H_t^\top + H_t P_{t|t-1} \dot{H}^{(i)\top}_t + \dot{R}^{(i)}_t. \tag{9.4}$$

These are source labels `eq:dv_note` and `eq:dS_note`. Using $\partial_i S^{-1} = -S^{-1}\dot{S}^{(i)}S^{-1}$, the per-time score contribution is

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} S_t^{-1} v_t - v_t^\top S_t^{-1}\dot{S}_t^{(i)} S_t^{-1} v_t\right]. \tag{9.5}$$

This is consolidated from source label `eq:score_note` in the MacroFinance analytic Kalman derivative note.

9.3 Solve-Form Score

For production code, BayesFilter prefers the solve form. Let $S_t w_t = v_t$. Then the same score can be written as

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right]. \tag{9.6}$$

This is source label `eq:solve_score_proved`. The implementation reference is the MacroFinance solve differentiated Kalman backend. The trace term may be computed by explicit materialization in small tests, but production code should prefer factor solves or trace estimators appropriate to the dimension and backend.

9.4 Gain and Update Derivatives

The score contribution at time t can be computed as soon as (9.3)–(9.4) are available. The recursion must still propagate derivatives to the next time step. For the covariance-form Kalman update,

$$K_t = P_{t|t-1} H_t^\top S_t^{-1},$$

the gain derivative is

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1}^{(i)} H_t^\top S_t^{-1} + P_{t|t-1}\dot{H}_t^{(i)\top} S_t^{-1} - P_{t|t-1} H_t^\top S_t^{-1}\dot{S}_t^{(i)} S_t^{-1}. \tag{9.7}$$

Equivalently, if the implementation treats the gain as the solved matrix $S_t K_t^\top = H_t P_{t|t-1}$, then

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t}\left(\dot{H}_t^{(i)} P_{t|t-1} + H_t \dot{P}_{t|t-1}^{(i)} - \dot{S}_t^{(i)} K_t^\top\right). \tag{9.8}$$

The filtered mean and covariance derivatives are

$$\dot{m}_{t|t}^{(i)} = \dot{m}_{t|t-1}^{(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \tag{9.9}$$

$$\dot{P}_{t|t}^{(i)} = -\dot{K}_t^{(i)} H_t P_{t|t-1} - K_t \dot{H}_t^{(i)} P_{t|t-1} + (I - K_t H_t)\dot{P}_{t|t-1}^{(i)}. \tag{9.10}$$

Equation (9.10) differentiates the simplified covariance update $P_{t|t} = (I - K_t H_t) P_{t|t-1}$. A Joseph-form or square-root backend may use a different algebraic update for numerical reasons, but it must produce derivatives of the same filtered covariance it uses in the next prediction step.

9.5 Masks and Time-Varying Observation Blocks

For missing data, let M_t select the observed rows at time t. The effective observation objects are

$$y_t^o = M_t y_t, \qquad a_t^o = M_t a_t, \qquad H_t^o = M_t H_t, \qquad R_t^o = M_t R_t M_t^\top.$$

The score recursion above applies after replacing (y_t, a_t, H_t, R_t) by $(y_t^o, a_t^o, H_t^o, R_t^o)$. The mask is data, not a differentiated parameter, so $\dot{M}_t = 0$. If no rows are observed, the likelihood and score contribution for that time are zero and the update is the identity:

$$m_{t|t} = m_{t|t-1}, \qquad P_{t|t} = P_{t|t-1},$$

with the same equalities for first derivatives. This convention ensures that the derivative path corresponds to the same masked likelihood value used by the filter.

9.6 Evidence Status

The current MathDevMCP AST audit identifies Cholesky/solve, prediction, update, quadratic-form, and derivative-recursion structure in that file, but it does not by itself certify the algebra. The stronger evidence is the MacroFinance test suite comparing solve, QR square-root, autodiff, and finite difference results on controlled LGSSMs.

Chapter 10

Kalman Hessian and Observed Information

The Hessian layer is necessary for observed-information diagnostics, local identification, and mass-matrix construction, but it is also the easiest place to overstate the state of the mathematics. BayesFilter therefore records the Hessian as a source-backed contract and validation target, not as a finished proof for every backend.

For second derivatives, write

$$\ddot{z}^{(ij)} = \frac{\partial^2 z}{\partial\theta_i \partial\theta_j}.$$

The MacroFinance note expands the second-order prediction, innovation, and gain recursions by product rule. In solve form, the source introduces

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t}\left(\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t\right), \tag{10.1}$$

where $\mathcal{S}_{S_t}(b)$ denotes solving $S_t x = b$. It then decomposes the per-time Hessian contribution into log-determinant, innovation-linear, and quadratic terms:

$$\frac{\partial^2 \ell_t}{\partial\theta_i \partial\theta_j} = -\frac{1}{2}\left[T_{ij}^{\log\det} + T_{ij}^{v} - T_{ij}^{S}\right]. \tag{10.2}$$

This statement is consolidated from source label `eq:solve_hessian_proved`.

10.1 Second-Order Prediction

The second-order prediction step is the product-rule derivative of (9.1)–(9.2). For the predicted mean,

$$\ddot{m}_{t|t-1}^{(ij)} = \ddot{c}_t^{(ij)} + \ddot{F}_t^{(ij)} m_{t-1|t-1} + \dot{F}_t^{(i)} \dot{m}_{t-1|t-1}^{(j)} + \dot{F}_t^{(j)} \dot{m}_{t-1|t-1}^{(i)} + F_t \ddot{m}_{t-1|t-1}^{(ij)}. \tag{10.3}$$

For the predicted covariance,

$$\begin{aligned}
\ddot{P}_{t|t-1}^{(ij)} = {} & \ddot{F}_t^{(ij)} P_{t-1|t-1} F_t^\top + \dot{F}_t^{(i)} \dot{P}_{t-1|t-1}^{(j)} F_t^\top + \dot{F}_t^{(i)} P_{t-1|t-1} \dot{F}_t^{(j)\top} \\
& + \dot{F}_t^{(j)} \dot{P}_{t-1|t-1}^{(i)} F_t^\top + F_t \ddot{P}_{t-1|t-1}^{(ij)} F_t^\top + F_t \dot{P}_{t-1|t-1}^{(i)} \dot{F}_t^{(j)\top} \\
& + \dot{F}_t^{(j)} P_{t-1|t-1} \dot{F}_t^{(i)\top} + F_t \dot{P}_{t-1|t-1}^{(j)} \dot{F}_t^{(i)\top} + F_t P_{t-1|t-1} \ddot{F}_t^{(ij)\top} + \ddot{Q}_t^{(ij)}.
\end{aligned} \tag{10.4}$$

These equations are the second-order source recursions in the MacroFinance analytic derivative note.

10.2 Second-Order Innovation Objects

The innovation derivatives are

$$\ddot{v}_t^{(ij)} = -\ddot{a}_t^{(ij)} - \ddot{H}_t^{(ij)} m_{t|t-1} - \dot{H}_t^{(i)} \dot{m}_{t|t-1}^{(j)} - \dot{H}_t^{(j)} \dot{m}_{t|t-1}^{(i)} - H_t \ddot{m}_{t|t-1}^{(ij)}, \tag{10.5}$$

$$\begin{aligned} \ddot{S}_t^{(ij)} = {} & \ddot{H}_t^{(ij)} P_{t|t-1} H_t^\top + \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} H_t^\top + \dot{H}_t^{(i)} P_{t|t-1} \dot{H}_t^{(j)\top} \\ & + \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} H_t^\top + \dot{H}_t^{(j)} P_{t|t-1} \dot{H}_t^{(i)\top} + H_t \ddot{P}_{t|t-1}^{(ij)} H_t^\top \\ & + H_t \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} + H_t \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} + H_t P_{t|t-1} \ddot{H}_t^{(ij)\top} + \ddot{R}_t^{(ij)}. \end{aligned} \tag{10.6}$$

Together with the first-order innovation objects from Chapter 9, these are the only model-specific inputs needed by the solve-form likelihood Hessian.

10.3 Solve-Form Hessian

The compact statement above is useful only if the three terms are kept explicit. Let $S_t w_t = v_t$ and define $\dot{w}_t^{(j)}$ by (10.1). The second derivative of the solved vector is

$$\ddot{w}_t^{(ij)} = \mathcal{S}_{S_t}\left(\ddot{v}_t^{(ij)} - \ddot{S}_t^{(ij)} w_t - \dot{S}_t^{(i)} \dot{w}_t^{(j)} - \dot{S}_t^{(j)} \dot{w}_t^{(i)}\right). \tag{10.7}$$

The log-determinant term is

$$T_{ij}^{\log\det} = \operatorname{tr}\left(S_t^{-1} \ddot{S}_t^{(ij)} - S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)}\right). \tag{10.8}$$

The innovation-linear term is

$$T_{ij}^{v} = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top} \dot{w}_t^{(j)}. \tag{10.9}$$

The quadratic term is

$$T_{ij}^{S} = 2\dot{w}_t^{(j)\top} \dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t. \tag{10.10}$$

Substituting (10.8)–(10.10) into (10.2) gives the observed Hessian contribution. The formula requires only the innovation objects

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}.$$

It is therefore backend-neutral. A linear Kalman filter, a square-root Kalman filter, or a sigma-point filter may all use the same likelihood Hessian layer once their own recursions supply these six objects.

10.4 Second-Order Gain and Update

The covariance-form backend also propagates the Hessian of the filtered state objects. Define

$$\dot{S}_t^{-1(i)} = -S_t^{-1} \dot{S}_t^{(i)} S_t^{-1}, \tag{10.11}$$

$$\ddot{S}_t^{-1(ij)} = S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} + S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} - S_t^{-1} \ddot{S}_t^{(ij)} S_t^{-1}. \tag{10.12}$$

Then

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1}^{(i)} H_t^\top S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} H_t^\top \dot{S}_t^{-1(i)}, \tag{10.13}$$

$$\begin{aligned} \ddot{K}_t^{(ij)} = {} & \ddot{P}_{t|t-1}^{(ij)} H_t^\top S_t^{-1} + \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} S_t^{-1} + \dot{P}_{t|t-1}^{(i)} H_t^\top \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} \ddot{H}_t^{(ij)\top} S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} H_t^\top \dot{S}_t^{-1(i)} + P_{t|t-1} \dot{H}_t^{(j)\top} \dot{S}_t^{-1(i)} + P_{t|t-1} H_t^\top \ddot{S}_t^{-1(ij)}. \end{aligned} \tag{10.14}$$

The filtered mean Hessian is

$$\ddot{m}_{t|t}^{(ij)} = \ddot{m}_{t|t-1}^{(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \tag{10.15}$$

For the simplified covariance update,

$$\begin{aligned} \ddot{P}_{t|t}^{(ij)} = {} & -\ddot{K}_t^{(ij)} H_t P_{t|t-1} - \dot{K}_t^{(i)} \dot{H}_t^{(j)} P_{t|t-1} - \dot{K}_t^{(i)} H_t \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} \dot{H}_t^{(i)} P_{t|t-1} - K_t \ddot{H}_t^{(ij)} P_{t|t-1} - K_t \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} H_t \dot{P}_{t|t-1}^{(i)} - K_t \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} + (I - K_t H_t) \ddot{P}_{t|t-1}^{(ij)}. \end{aligned} \tag{10.16}$$

The gain and covariance formulas are useful as a covariance-form audit oracle. Solve-form or square-root implementations may avoid explicit (10.11)–(10.12) internally, but they must agree with these recursions on controlled well-conditioned systems.

10.5 Information Objects and Sign Convention

This chapter computes Hessians of the log likelihood contribution $\ell_t(\theta)$. The observed information contribution is

$$\mathcal{J}_t(\theta) = -\nabla_\theta^2 \ell_t(\theta),$$

and posterior precision at a mode additionally includes the negative Hessian of the log prior. BayesFilter should not call a matrix a mass matrix, local covariance, or identification object unless the sign convention and prior contribution have been stated explicitly.

10.6 Hessian-Ready Backend Contract

A backend is Hessian-ready when it can return, for every time step and active parameter pair,

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}$$

for the same likelihood value it evaluates. To propagate the scan, it must also return either the covariance-form update derivatives (10.15)–(10.16) or an equivalent factor/backend-specific derivative update with reconstruction diagnostics. This is the bridge used later by the SVD sigma-point chapter: the nonlinear backend supplies approximate innovation derivatives, and the solve-form Hessian layer remains the same.

10.7 Evidence Status

The source derivation is detailed, but BayesFilter should keep three evidence levels distinct:

- formula provenance from the MacroFinance analytic derivative note;
- structural code evidence from MathDevMCP AST audits;
- numerical parity evidence from tests against autodiff and finite differences.

The current MathDevMCP audit of the MacroFinance solve differentiated Kalman backend found the Kalman operation graph and solve structure but reported a mismatch for a strict `logdet/trace` operation query and missing explicit shape and covariance guards. This should be treated as a production-readiness prompt, not as a proof failure. The generic LGSSM autodiff-validation tests are the stronger mathematical check for the current implementation because they compare likelihood, gradient, and Hessian against solve backends, TensorFlow autodiff, and finite differences on controlled small-dimensional models.

Chapter 11

Structural Derivatives

Kalman derivative recursions assume that the derivatives of the state-space objects are already available. Structural derivatives are the upstream layer: they map model parameters into

$$c_\theta,\ F_\theta,\ Q_\theta,\ a_\theta,\ H_\theta,\ R_\theta$$

and their first and second derivatives.

11.1 Provider Map

Let $\psi \in \mathbb{R}^p$ denote the parameter vector used by the numerical provider. In HMC this is usually an unconstrained or transformed parameter vector, not necessarily the raw economic vector. The structural provider declares the map

$$g(\psi) = \mathrm{vec}(c(\psi), F(\psi), Q(\psi), a(\psi), H(\psi), R(\psi), m_0(\psi), P_0(\psi))\,. \tag{11.1}$$

The MacroFinance source map uses the same contract for (b, F, Q, a, H, R) under source label `eq:structural_map_g`; BayesFilter adds (m_0, P_0) because the filtering scan cannot be differentiated without an initial-condition policy.

For every active coordinate ψ_i, the first-order provider contract is

$$\dot{G}^{(i)} = \left(\dot{c}^{(i)}, \dot{F}^{(i)}, \dot{Q}^{(i)}, \dot{a}^{(i)}, \dot{H}^{(i)}, \dot{R}^{(i)}, \dot{m}_0^{(i)}, \dot{P}_0^{(i)}\right). \tag{11.2}$$

For every active pair (ψ_i, ψ_j), the second-order contract is

$$\ddot{G}^{(ij)} = \left(\ddot{c}^{(ij)}, \ddot{F}^{(ij)}, \ddot{Q}^{(ij)}, \ddot{a}^{(ij)}, \ddot{H}^{(ij)}, \ddot{R}^{(ij)}, \ddot{m}_0^{(ij)}, \ddot{P}_0^{(ij)}\right). \tag{11.3}$$

Equations (11.2)–(11.3) are the objects consumed by Chapters 9 and 10. A backend may store them as dense tensors, sparse blocks, named arrays, or lazy linear operators, but the contract is mathematical: each block must mean a derivative of the same state-space law used to evaluate the likelihood.

For BayesFilter, a derivative provider must declare:

- parameter order and units;
- shape of every first and second derivative tensor;
- whether derivatives are analytic, autodiff, finite-difference, or unavailable;
- which parameters affect which reduced-form objects;
- whether cross-derivatives are implemented or intentionally zero.

11.2 Parameter Transforms

Let $\phi = T(\psi)$ be the raw economic parameter vector and let $M(\phi)$ be any reduced-form block among $c, F, Q, a, H, R, m_0, P_0$. Write

$$M_\alpha = \frac{\partial M}{\partial \phi_\alpha}, \qquad M_{\alpha\beta} = \frac{\partial^2 M}{\partial \phi_\alpha \partial \phi_\beta},$$

and

$$T_{\alpha,i} = \frac{\partial \phi_\alpha}{\partial \psi_i}, \qquad T_{\alpha,ij} = \frac{\partial^2 \phi_\alpha}{\partial \psi_i \partial \psi_j}.$$

Then

$$\dot{M}^{(i)} = \sum_\alpha M_\alpha T_{\alpha,i}, \tag{11.4}$$

$$\ddot{M}^{(ij)} = \sum_{\alpha,\beta} M_{\alpha\beta} T_{\alpha,i} T_{\beta,j} + \sum_\alpha M_\alpha T_{\alpha,ij}. \tag{11.5}$$

These are the BayesFilter versions of the MacroFinance transformed-parameter chain-rule equations. They matter because HMC often runs on $\psi = \log \sigma$, $\psi = \log \kappa$, Cholesky coordinates, Fisher transforms, or stationarity transforms. A correct derivative with respect to a raw standard deviation is wrong for HMC if it is passed off as the derivative with respect to the log standard deviation.

11.3 Initial Conditions

The initial moments in (11.1) are part of the model law. There are three common cases:

(i) fixed diffuse or fixed proper initialization: $\dot{m}_0 = \dot{P}_0 = \ddot{m}_0 = \ddot{P}_0 = 0$ because the initial moments are data of the filtering problem;

(ii) stationary initialization: m_0 and P_0 solve model equations and must be differentiated together with those equations;

(iii) estimated or hierarchical initialization: m_0 and P_0 have their own parameter blocks and cross derivatives.

For a stationary linear system with $m_0 = c + Fm_0$, the first derivative satisfies

$$(I - F)\dot{m}_0^{(i)} = \dot{c}^{(i)} + \dot{F}^{(i)} m_0. \tag{11.6}$$

For $P_0 = FP_0F^\top + Q$, the first derivative satisfies the Lyapunov derivative equation

$$\dot{P}_0^{(i)} = F\dot{P}_0^{(i)}F^\top + \dot{F}^{(i)}P_0F^\top + FP_0\dot{F}^{(i)\top} + \dot{Q}^{(i)}. \tag{11.7}$$

Second derivatives follow by differentiating (11.6) and (11.7). BayesFilter should record whether these equations were solved analytically, by a differentiable Lyapunov solver, by autodiff through the stationary solver, or intentionally not used.

11.4 Sparsity, Masks, and Zero Blocks

A derivative tensor entry equal to zero is different from a derivative entry that is unavailable. The provider must distinguish:

$$\ddot{M}^{(ij)} = 0 \quad \text{declared structural zero,} \qquad \ddot{M}^{(ij)} \text{ not implemented.}$$

Structural zeros are useful for large models because they allow the score and Hessian scan to skip blocks without changing the mathematical target. Missing entries are not a mathematical shortcut; they mean the backend cannot certify the requested derivative object.

Observation masks are data-side selections. If J_t selects observed coordinates at time t, the provider may expose either the full (a_t, H_t, R_t) together with J_t or the already-masked blocks

$$a_t^o = J_t a_t, \qquad H_t^o = J_t H_t, \qquad R_t^o = J_t R_t J_t^\top.$$

The same selection applies to derivatives. Since J_t is data, not a parameter, no derivative of the mask is taken. This convention is the structural upstream counterpart of the masking rule in Section 9.5.

11.5 Metadata and Shape Contract

The provider must make the following metadata inspectable before likelihood evaluation:

- names and order of ψ_i;
- transform type and raw unit for each parameter;
- state and observation coordinate names;
- dimensions of each reduced-form block;
- tensor layout conventions, including whether Hessian tensors store all pairs or only the symmetric upper triangle;
- symmetry and positive-definiteness policies for Q, R, and P_0;
- deterministic or algebraic state-coordinate roles when the provider is used by a structural nonlinear filter.

These declarations are not decorative. Without them, a finite likelihood can be computed from a different parameter ordering, unit system, or deterministic state convention than the derivative tensors assume.

The MacroFinance large-scale derivative-provider tests check derivative shapes, finite-difference parity by object type, reduced-form rank diagnostics, and parameter-unit rescaling. BayesFilter should preserve these as public contract tests before a structural model is allowed to feed an HMC target.

11.6 Provider Validation

The minimum validation ladder for a structural provider is:

(i) finite value check: all blocks in (11.1) are finite, conformable, symmetric where required, and admissible before filtering;

(ii) shape check: (11.2)–(11.3) have the declared dimensions for every active parameter and pair;

(iii) transform check: (11.4)–(11.5) agree with finite differences in transformed coordinates;

(iv) block finite-difference check: each reduced-form object is checked before it is inserted into a long filtering scan;

(v) end-to-end check: the resulting likelihood score and Hessian agree with finite differences or autodiff on small systems where that comparison is reliable;

(vi) diagnostic check: structural zeros, unavailable derivatives, regularization, rejected parameter points, and mask policies are reported.

Only after these checks pass should the structural provider feed an HMC target or an observed-information diagnostic. This is the upstream analogue of the factor parity gates in Chapter 12.

Chapter 12

QR and Cholesky Factor Derivatives

Square-root filters propagate factors rather than covariance matrices. Their derivative contract is therefore not only a Kalman contract; it is also a factor calculus contract. The likelihood score and Hessian in Chapters 9–10 are expressed in terms of v_t, S_t, and their derivatives. A factor backend is acceptable only if the factors it propagates reproduce those covariance objects and the same derivative objects.

BayesFilter uses three levels of factor evidence:

(i) Cholesky or QR identities stated in the source derivation;

(ii) reconstruction diagnostics proving that differentiated factors reconstruct the differentiated covariance objects;

(iii) backend parity against solve-form likelihood, score, and Hessian.

The QR square-root tests in MacroFinance require prediction, innovation, and filtered factor derivative reconstruction errors below tight tolerances on small-dimensional parameter grids, and they compare QR likelihood, gradient, and Hessian against the solve-form backend. Those tests are currently the right standard for BayesFilter factor-derivative chapters: a factor formula is not production-ready until both reconstruction and end-to-end likelihood parity pass.

12.1 Reconstruction Contract

Let $A(\theta)$ be any covariance-like symmetric positive semidefinite object used by the filter, and let an implemented factor satisfy

$$A_\star(\theta) = C(\theta)C(\theta)^\top.$$

The subscript $\star$ is deliberate. For a Cholesky backend, $A_\star = A$ on a positive-definite branch. For a spectral backend, $A_\star$ may be a rank-truncated, floored, or symmetrized version of A. The first derivative reconstructed by the factor is

$$\dot{A}_\star^{(i)} = \dot{C}^{(i)}C^\top + C\dot{C}^{(i)\top}. \tag{12.1}$$

The second derivative reconstructed by the factor is

$$\ddot{A}_\star^{(ij)} = \ddot{C}^{(ij)}C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \tag{12.2}$$

Equations (12.1)–(12.2) are the universal audit equations for every factor backend. The derivative may be obtained by analytic formulas, custom gradients, or autodiff, but the resulting C, $\dot{C}$, and $\ddot{C}$ must reconstruct the implemented covariance branch that produced the likelihood value.

12.2 Cholesky Factor Derivatives

Suppose $A = LL^\top$, where A is positive definite and L is lower triangular with positive diagonal. Define the lower-triangular half-diagonal operator

$$\Phi(X)_{ab} = \begin{cases} X_{ab}, & a > b, \\ X_{aa}/2, & a = b, \\ 0, & a < b. \end{cases}$$

For a first derivative, form

$$B^{(i)} = L^{-1}\dot{A}^{(i)}L^{-\top}, \qquad G^{(i)} = \Phi\left(B^{(i)}\right).$$

Then the Cholesky factor derivative is

$$\dot{L}^{(i)} = LG^{(i)}. \tag{12.3}$$

This is the BayesFilter restatement of source label `eq:first_cholesky_derivative`.

For a second derivative, remove the known first-order products before applying the same triangular split:

$$C^{(ij)} = L^{-1}\left(\ddot{A}^{(ij)} - \dot{L}^{(i)}\dot{L}^{(j)\top} - \dot{L}^{(j)}\dot{L}^{(i)\top}\right)L^{-\top}, \qquad H^{(ij)} = \Phi\left(C^{(ij)}\right).$$

Then

$$\ddot{L}^{(ij)} = LH^{(ij)}. \tag{12.4}$$

This restates source label `eq:second_cholesky_derivative`. The formulas are local smooth-branch formulas: they require a positive diagonal and become numerically fragile near a Cholesky pivot boundary. A backend that adds jitter before Cholesky must report whether its derivatives target the original A or the implemented $A + \delta I$.

12.3 QR Factor Derivatives

QR square-root filters often factor a stack rather than a covariance matrix. Let $M(\theta) \in \mathbb{R}^{m\times n}$ have full column rank, $m \geq n$, and use the thin convention

$$M = QR, \qquad Q^\top Q = I_n, \qquad R \text{ upper triangular}, \qquad R_{aa} > 0.$$

The positive diagonal fixes the local sign convention. Define

$$A^{(i)} = Q^\top \dot{M}^{(i)} R^{-1}.$$

Let $\Omega^{(i)}$ be the skew-symmetric matrix identified from

$$\Omega^{(i)}_{ab} = \begin{cases} A^{(i)}_{ab}, & a > b, \\ 0, & a = b, \\ -A^{(i)}_{ba}, & a < b, \end{cases} \qquad T^{(i)} = A^{(i)} - \Omega^{(i)}.$$

Then

$$\dot{R}^{(i)} = T^{(i)}R, \tag{12.5}$$
$$\dot{Q}^{(i)} = Q\Omega^{(i)} + (I - QQ^\top)\dot{M}^{(i)}R^{-1}. \tag{12.6}$$

Equation (12.5) restates source label `eq:first_qr_r_derivative`.

For second derivatives, define the effective second-order stack derivative

$$E^{(ij)} = \ddot{M}^{(ij)} - \dot{Q}^{(i)}\dot{R}^{(j)} - \dot{Q}^{(j)}\dot{R}^{(i)}$$

and

$$B^{(ij)} = Q^\top E^{(ij)} R^{-1}, \qquad C_Q^{(ij)} = -\dot{Q}^{(i)\top}\dot{Q}^{(j)} - \dot{Q}^{(j)\top}\dot{Q}^{(i)}.$$

The matrix $\Gamma^{(ij)} = Q^\top \ddot{Q}^{(ij)}$ is determined by

$$\Gamma_{ab}^{(ij)} = \begin{cases} B_{ab}^{(ij)}, & a > b, \\ C_{Q,aa}^{(ij)}/2, & a = b, \\ C_{Q,ab}^{(ij)} - B_{ba}^{(ij)}, & a < b, \end{cases} \qquad U^{(ij)} = B^{(ij)} - \Gamma^{(ij)}.$$

Then

$$\ddot{R}^{(ij)} = U^{(ij)}R, \tag{12.7}$$
$$\ddot{Q}^{(ij)} = Q\Gamma^{(ij)} + (I - QQ^\top)E^{(ij)}R^{-1}. \tag{12.8}$$

Equation (12.7) restates source label `eq:second_qr_r_derivative`. The QR formulas are local to a fixed rank and sign branch. Pivoting, column-rank loss, or diagonal sign corrections are branch events and must be reported as derivative-policy events, not hidden inside a smooth Hessian claim.

12.4 Square-Root Kalman Stacks

The generic formulas above enter the filter through stack factorizations. If $C_{t-1|t-1}C_{t-1|t-1}^\top = P_{t-1|t-1}$ and $G_tG_t^\top = Q_t$, the prediction covariance can be declared by the stack

$$M_t^P = \begin{bmatrix} F_tC_{t-1|t-1} & G_t \end{bmatrix}, \qquad P_{t|t-1} = M_t^P M_t^{P\top}.$$

Equivalently, a thin QR factorization of $M_t^{P\top} = Q_t^P R_t^P$ gives the lower factor $C_{t|t-1} = R_t^{P\top}$. The first and second derivatives of M_t^P are obtained by product rule from the derivatives of F_t, $C_{t-1|t-1}$, and G_t, and (12.5)–(12.7) then give the derivatives of $C_{t|t-1}$.

Likewise, if $E_tE_t^\top = R_t$,

$$M_t^S = \begin{bmatrix} H_tC_{t|t-1} & E_t \end{bmatrix}, \qquad S_t = M_t^S M_t^{S\top},$$

and QR of $M_t^{S\top}$ gives the innovation factor used by the solve-form likelihood. The filtered covariance can be audited by the Joseph stack

$$M_t^U = \begin{bmatrix} (I - K_tH_t)C_{t|t-1} & K_tE_t \end{bmatrix}, \qquad P_{t|t} = M_t^U M_t^{U\top}.$$

Derivatives of K_t are those in (9.7) and (10.14). Therefore a square-root implementation has two equivalent audit paths: reconstruct (12.1)–(12.2) from differentiated factors, and compare the resulting score/Hessian objects to the covariance-form recursions in Chapters 9–10.

12.5 Spectral Factors

SVD sigma-point filters use a spectral factor, not a triangular factor, to generate point displacements. If

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2},$$

then the factor actually reconstructs

$$P_+ = CC^\top = U\Lambda_+ U^\top,$$

where Λ_+ includes the rank, floor, or smoothing policy used by the implementation. Therefore spectral factor validation must distinguish two questions:

(i) Do C and its derivatives reconstruct P_+ and its derivatives?

(ii) If $P_+ \neq P$, is the difference a declared model approximation, a finite-arithmetic regularization, or an invalid fallback?

This distinction is essential for HMC. A hard eigenvalue floor can produce a finite likelihood while changing the derivative target at the floor boundary. If the floor is hard, derivatives are branch derivatives of P_+, not derivatives of the pre-regularized covariance P. If the floor is smooth, the floor derivative is part of the model actually differentiated. Either way, the factor reconstruction equations in (12.1)–(12.2) must be checked against P_+.

Chapter 18 therefore treats spectral derivatives as a separate contract with gap telemetry, active-floor reporting, and branch diagnostics. Its simple-spectrum SVD/eigen formulas define one smooth-branch policy; they do not remove the need for finite-difference checks across spectral gap, rank, and floor regimes.

12.6 Backend Parity Gates

A factor backend is ready to support analytic gradients and Hessians only after the following checks pass on controlled systems:

(i) value reconstruction: $\|A_\star - CC^\top\|$ is within tolerance for prediction, innovation, and filtered covariance objects;

(ii) first- and second-derivative reconstruction: (12.1)–(12.2) match the declared $\dot{A}_\star$ and $\ddot{A}_\star$;

(iii) solve parity: the factor solve gives the same w_t, score, and Hessian contribution as the covariance solve on well-conditioned systems;

(iv) branch reporting: jitter, rank truncation, pivoting, sign flips, active floors, and fallback paths are emitted as diagnostics;

(v) backend parity: Cholesky-refactorized, QR square-root, and SVD/spectral backends agree where their mathematical branches represent the same covariance law.

These gates prevent a common mistake: a numerically stable factor can make the likelihood finite while silently changing the derivative target. The factor chapter therefore supplies the algebraic contracts that the validation chapter turns into executable tests.

Chapter 13

Custom-Gradient Wrappers for HMC

A custom gradient is safe for HMC only if it returns the derivative of the same scalar log target used in the Metropolis correction. A fast gradient of a nearby surrogate changes the dynamics and can bias the chain unless the sampler is explicitly corrected.

13.1 Same-Scalar Contract

Let the production value path return

$$\mathcal{L}(\psi) = \sum_{t=1}^{T} \ell_t(\psi) + \log p(\psi), \tag{13.1}$$

where the likelihood terms use the same structural provider, masks, factor branch, regularization policy, and parameter transforms declared in Chapters 11–12. The custom-gradient path is valid only if it returns

$$g_i(\psi) = \frac{\partial \mathcal{L}(\psi)}{\partial \psi_i}. \tag{13.2}$$

If the value path evaluates a floored spectral covariance P_+, the gradient must be the derivative of that implemented value path. If the value path uses a jittered innovation covariance $S_t + \delta I$, the gradient must include the same jitter policy or clearly report that the parameter point is outside the certified smooth branch. The wrapper must not evaluate one scalar and return the gradient of another.

BayesFilter's custom-gradient wrapper policy is:

(i) compute the primal log likelihood with the production backend;

(ii) return an analytic score that has been validated against the same primal value on controlled cases;

(iii) expose diagnostics for finite failures and backend fallbacks;

(iv) keep tensor shapes static inside compiled sampler paths;

(v) use Hessians for mass matrices or diagnostics, not inside every HMC leapfrog step unless explicitly justified.

This policy reflects the MacroFinance motivation: nested tape Hessians through full Kalman scans are useful as small-model validation oracles, but they are not the intended large-model production path.

13.2 Gradient Callback Contract

A custom-gradient wrapper should expose a contract equivalent to the following mathematical tuple:

$$\text{value\_and\_score}(\psi) = (\mathcal{L}(\psi), \nabla_\psi \mathcal{L}(\psi), d(\psi)) ,$$

where $d(\psi)$ is a diagnostic record. The diagnostic record must include:

- the structural provider and parameter transform identifiers;
- the active observation-mask policy;
- the covariance/factor backend used by the value path;
- any jitter, nugget, PSD projection, rank truncation, active spectral floor, pivoting, sign correction, or fallback branch;
- the minimum relevant Cholesky pivot, QR diagonal, or spectral gap;
- finite-value and finite-gradient status.

The gradient callback should return a failure status rather than silently replacing the derivative with zeros, prior gradients, or a different backend. Emergency fallbacks can be useful for optimization diagnostics, but HMC must know whether it is using the certified target derivative.

13.3 Hessian and Observed-Information Path

The HMC hot path needs values and gradients. Hessians are usually used outside the leapfrog loop for mass-matrix initialization, local identification, and curvature diagnostics. If a wrapper exposes a Hessian, it should target

$$H_{ij}(\psi) = \frac{\partial^2 \mathcal{L}(\psi)}{\partial \psi_i \, \partial \psi_j}$$

for the same scalar in (13.1). The observed information is $-H(\psi)$, possibly plus a sign-adjusted convention for priors if the likelihood-only and posterior targets are both reported. A Hessian path that omits prior curvature, transformed-parameter curvature, or jitter/floor branch terms must label itself as a partial diagnostic, not as posterior curvature.

The recommended production pattern is:

(i) value-plus-score path for MAP and HMC;

(ii) analytic or custom Hessian path for mode diagnostics and mass-matrix construction;

(iii) small-model autodiff Hessian path as an oracle, not as the large-model default.

13.4 Static-Shape and Compilation Contract

Compiled HMC paths require stable shapes. Parameter dimension, state dimension, observation dimension, maximum active observation size, and the layout of diagnostic records should be static across leapfrog calls. Missing data should be represented by fixed-shape masks or predeclared selection patterns rather than by dynamically changing tensor ranks. Fixed sigma-point rules must keep the number and ordering of points stable. Factor backends must report branch events without changing the differentiable return signature.

This static-shape rule is an implementation constraint, but it has mathematical content: changing the set of coordinates, points, or factors during a compiled scan can turn a smooth derivative contract into a piecewise branch derivative without making the branch visible to the sampler.

13.5 HMC Safety Labels

BayesFilter should distinguish the following labels:

- **value-correct**: the scalar likelihood or posterior value matches a reference value path;
- **gradient-correct**: the returned score matches finite differences or autodiff of that same scalar on certified cases;
- **curvature-correct**: the Hessian or observed information matches the same scalar and sign convention;
- **sampler-usable**: short HMC runs have finite log probabilities, finite gradients, and acceptable diagnostic telemetry;
- **production-gated**: compiled/eager parity, branch diagnostics, mask tests, stress tests, and representative workload tests all pass.

A finite HMC run is not by itself a gradient proof. Conversely, a correct gradient on small smooth systems does not certify spectral floors, rank changes, or DSGE deterministic-completion branches unless those regimes have been validated explicitly.

Chapter 14

Derivative Validation

Derivative validation is a ladder. Passing one rung is useful evidence, but it does not certify the whole industrial target.

14.1 Validation Ladder

BayesFilter derivative providers should pass:

(i) shape and finite-value checks;

(ii) finite-difference checks on scalar or tiny LGSSMs;

(iii) autodiff parity on small models where tape Hessians are feasible;

(iv) backend parity among covariance, solve, Cholesky, and QR variants;

(v) factor reconstruction and solve-residual diagnostics;

(vi) compiled/eager parity for TensorFlow backends;

(vii) stress tests for low noise, collinearity, near rank loss, and masks.

MacroFinance currently supplies examples of these tests, including QR square-root parity against autodiff and finite differences, solve backend agreement with the existing differentiated backend, TensorFlow eager/compiled parity, and large-scale backend ladder diagnostics. BayesFilter should import the testing discipline before importing any claim of production readiness.

14.2 What Is Being Compared

Every derivative test must name the scalar or tensor being compared. For a posterior target,

$$\mathcal{L}(\psi) = \sum_{t=1}^{T} \ell_t(\psi) + \log p(\psi),$$

the score test compares

$$\hat{g}_i(\psi) \quad \text{to} \quad \frac{\mathcal{L}(\psi + he_i) - \mathcal{L}(\psi - he_i)}{2h}$$

or to an autodiff score of the same scalar. The Hessian test compares

$$\hat{H}_{ij}(\psi) \quad \text{to} \quad \frac{\hat{g}_i(\psi + he_j) - \hat{g}_i(\psi - he_j)}{2h}$$

or to a tape Hessian of the same scalar. If the reported analytic derivative is likelihood-only but the finite difference includes the prior, the test is invalid. If the value path contains jitter, floors, masks, or transformed parameters, the comparison value must contain the same choices.

14.3 Recommended Rungs

The derivative validation ladder should be run in this order:

(i) **Finite values and shapes.** Check that value, score, Hessian, state moments, factors, provider tensors, and diagnostics are finite and have declared dimensions.

(ii) **Symmetry and admissibility.** Check that covariance-like objects are symmetric, positive semidefinite/definite under their declared policy, and that Hessians are symmetric within tolerance.

(iii) **Block finite differences.** Validate structural derivatives of $c, F, Q, a, H, R, m_0, P_0$ before running a long filter scan.

(iv) **Likelihood finite differences.** Compare score and Hessian of the complete scalar target on tiny systems.

(v) **Autodiff parity.** Use taped gradients and Hessians as small-system oracles where nested tapes are feasible.

(vi) **Backend parity.** Compare covariance, solve, Cholesky, QR, and SVD/spectral backends on systems where they represent the same law.

(vii) **Factor reconstruction.** Check (12.1)–(12.2) for prediction, innovation, and filtered covariance factors.

(viii) **Mask tests.** Test dense all-observed masks, sparse deterministic masks, and all-missing periods using the convention in Section 9.5.

(ix) **Spectral and branch stress tests.** Vary spectral gaps, Cholesky pivots, QR diagonals, rank thresholds, floors, jitter, and fallback branches while checking diagnostics.

(x) **Eager/compiled parity.** Compare values and derivatives under eager execution, compiled execution, and XLA/JIT paths when those paths are intended for production.

(xi) **Sampler smoke tests.** Run short HMC/NUTS checks only after deterministic value-and-gradient tests pass, and interpret them as sampler usability evidence, not proof of the derivative formulas.

14.4 Finite-Difference Tolerances

Finite differences are not exact tests; they are numerical experiments. The step size should be selected relative to the transformed parameter scale, for example

$$h_i = \eta(1 + |\psi_i|),$$

with several η values checked when possible. A central difference should show a stable error plateau before the result is accepted. Parameters near constraints, branch boundaries, or spectral floors should be tested separately from smooth interior points, because the expected derivative may be a branch derivative rather than a smooth derivative of the unregularized model.

14.5 Backend-Specific Gates

Each backend has a distinct failure mode:

- covariance form can lose symmetry or positive semidefiniteness under finite arithmetic;
- solve form can hide ill conditioning unless solve residuals and log-determinant policies are reported;
- Cholesky form depends on positive pivots and any jitter policy;
- QR form depends on full column rank, diagonal sign convention, and pivot policy;
- SVD/spectral form depends on gap, rank, floor, sign/order, and fallback policy;
- sigma-point form additionally depends on fixed point rules, weights, nonlinear map derivatives, and moment reconstruction.

Backend parity is meaningful only when the compared backends target the same law. A covariance backend using P and an SVD backend using a floored P_+ are not required to have identical derivatives unless $P_+ = P$ on that test case.

14.6 Validation Artifact

Every promoted derivative backend should leave an artifact containing:

- model name, parameter vector, transform policy, and reference point;
- backend name and value branch;
- tests run, tolerances, and maximum absolute/relative errors;
- finite-value, finite-gradient, and finite-Hessian status;
- branch diagnostics: jitter, PSD projection, spectral floors, pivots, rank truncation, sign/order corrections, and fallbacks;
- interpretation: which claim is supported and which regimes remain untested.

This artifact is the bridge between a mathematical derivation and an HMC claim. Without it, phrases such as “analytic Hessian”, “SVD sigma-point gradient”, or “HMC-ready backend” are too ambiguous to audit.

14.7 Audit Record

Each derivative chapter should record the source labels, code files, and tests that support it. If a tool audit is inconclusive, the monograph should say so. An inconclusive symbolic audit can still be valuable: it identifies which guards, shapes, or noncommutative matrix steps need manual proof before a formula is promoted to peer-review-ready status.

For the analytic-derivative chapter set, the current audit record is:

- MacroFinance analytic derivative labels provide the source formulas for score, Hessian, structural provider, Cholesky, and QR contracts;
- MathDevMCP is useful for label lookup and local-context provenance, but it does not replace manual proof of noncommutative matrix calculus steps;
- ResearchAssistant is local/offline and has no current summary hits for the exact SVD/eigen derivative-validation cluster used here;
- production readiness remains a test claim, not a prose claim.

Part IV

Nonlinear Filtering

Chapter 15

Extended Kalman Filtering

The extended Kalman filter is the most direct nonlinear extension of the linear Gaussian spine in Chapter 5. It replaces nonlinear transition and observation maps by local linear approximations and then applies the same prediction-error likelihood form to the resulting Gaussian approximation. In BayesFilter this is an approximate likelihood backend, not an exact replacement for the Kalman filter except in the affine-Gaussian special case.

15.1 Local Linearization

Consider the nonlinear contract

$$x_{t+1} = f_\theta(x_t) + u_{t+1}, \qquad u_{t+1} \sim \mathcal{N}(0, Q_\theta), \tag{15.1}$$

$$y_t = h_\theta(x_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta). \tag{15.2}$$

Let

$$F_t = \left.\frac{\partial f_\theta(x)}{\partial x^\top}\right|_{x=m_{t-1|t-1}}, \qquad H_t = \left.\frac{\partial h_\theta(x)}{\partial x^\top}\right|_{x=m_{t|t-1}}.$$

The EKF prediction and update are

$$m_{t|t-1} = f_\theta(m_{t-1|t-1}), \tag{15.3}$$

$$P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q_\theta, \tag{15.4}$$

$$v_t = y_t - h_\theta(m_{t|t-1}), \tag{15.5}$$

$$S_t = H_t P_{t|t-1} H_t^\top + R_\theta, \tag{15.6}$$

$$K_t = P_{t|t-1} H_t^\top S_t^{-1}, \tag{15.7}$$

$$m_{t|t} = m_{t|t-1} + K_t v_t. \tag{15.8}$$

The covariance may be updated in simplified form or in a Joseph-style linearized form. The backend must state which form defines the numerical recursion, because this choice affects finite-arithmetic behavior even when the first-order algebra is equivalent.

15.2 Likelihood Status

The EKF log likelihood uses the same innovation expression as (5.10), but the innovations and innovation covariances come from local linearization. Therefore the EKF target is

an approximate target unless f_θ and h_θ are affine on the region reached by the filter. An HMC chain using the EKF samples from the posterior induced by that approximate likelihood and prior, not from the exact nonlinear state-space posterior.

For BayesFilter documentation and tests, the EKF should be used in three roles:

(i) a baseline nonlinear Gaussian approximation;

(ii) a regression oracle in cases where the nonlinear maps collapse to affine maps and the exact Kalman likelihood is available;

(iii) a diagnostic comparison for sigma-point and particle backends.

15.3 HMC Contract

An EKF backend is HMC-eligible only after it declares:

- how the Jacobians F_t and H_t are computed;
- whether gradients with respect to θ are tape-based, analytic, or custom;
- what happens when S_t is singular, non-finite, or numerically indefinite;
- whether all loop lengths, masks, and state dimensions remain static under compilation;
- which parity tests compare the EKF to exact linear Gaussian cases.

The older nonlinear-filtering chapters in the source projects motivate this contract, but BayesFilter tightens it for HMC: differentiability and finite failure behavior are part of the target definition, not implementation afterthoughts.

Chapter 16

Sigma-Point Filters

Sigma-point filters replace local linearization with deterministic quadrature. They propagate a finite set of points through nonlinear maps and reconstruct approximate Gaussian moments from weighted averages. This makes them more accurate than the EKF for some nonlinear transformations, while preserving the Gaussian filtering closure used by the prediction-error likelihood. The price is that the likelihood remains approximate and the derivative path becomes more complicated.

16.1 Moment-Matching Interface

At each time step, a sigma-point backend starts from a Gaussian approximation

$$x_t \mid y_{1:t-1} \approx \mathcal{N}(m_{t|t-1}, P_{t|t-1}).$$

A deterministic rule constructs points $\chi_{t|t-1}^{(j)}$ and weights $w_j^{(m)}, w_j^{(c)}$. Passing the points through the observation map gives

$$z_t^{(j)} = h_\theta(\chi_{t|t-1}^{(j)}).$$

The backend then forms approximate predicted observations, innovation covariances, and cross covariances:

$$\bar{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{16.1}$$

$$P_{zz,t} = \sum_j w_j^{(c)} (z_t^{(j)} - \bar{z}_t)(z_t^{(j)} - \bar{z}_t)^\top + R_\theta, \tag{16.2}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (\chi_{t|t-1}^{(j)} - m_{t|t-1})(z_t^{(j)} - \bar{z}_t)^\top. \tag{16.3}$$

The gain is $K_t = P_{xz,t} P_{zz,t}^{-1}$ and the innovation is $v_t = y_t - \bar{z}_t$. The same Gaussian innovation likelihood formula is then used, with $P_{zz,t}$ replacing S_t.

16.2 Conjugate Unscented Transform Rules

The ordinary unscented transform is not the only deterministic quadrature rule that can be used in the interface above. The conjugate unscented transform (CUT) of Adurthi

et al. [2012a,b] constructs higher-order Gaussian point sets by combining symmetric conjugate point families. The experimental file

`/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py`

implements a deprecated TensorFlow square-root CUT4 demonstration for a coordinated-turn tracking model. It is not wired into the current BayesFilter filter hierarchy, but it is a useful design reference because it separates the sigma-point rule from the square-root covariance backend.

For a standard normal variable $Z \in \mathbb{R}^n$, the CUT4-G rule used there has

$$N_{\mathrm{CUT4}} = 2n + 2^n$$

points: $2n$ axis points and 2^n hypercube-corner points. In the convention used by the demo, for $n \geq 3$,

$$r_a^2 = \frac{n+2}{2}, \qquad r_c^2 = \frac{n+2}{n-2},$$

with weights

$$w_a = \frac{4}{(n+2)^2}, \qquad w_c = \frac{(n-2)^2}{2^n(n+2)^2}.$$

More explicitly, define

$$\mathcal{A} = \{\pm r_a e_i : i = 1, \ldots, n\}, \qquad \mathcal{C} = \{r_c s : s \in \{-1, +1\}^n\}.$$

The CUT4-G quadrature operator is

$$\mathcal{Q}_{\mathrm{CUT4}}[\varphi] = w_a \sum_{\xi \in \mathcal{A}} \varphi(\xi) + w_c \sum_{\xi \in \mathcal{C}} \varphi(\xi). \tag{16.4}$$

Proposition 16.1 (CUT4-G Gaussian moment exactness)**.** *For $n \geq 3$, the point set (16.4) integrates every standard-normal monomial in $Z \in \mathbb{R}^n$ of total degree at most five exactly. Equivalently, it is a degree-five Gaussian rule.*

Proof. Both $\mathcal{A}$ and $\mathcal{C}$ are centrally symmetric and sign-symmetric in every coordinate. Hence every monomial with at least one odd coordinate exponent has zero quadrature value, matching the standard normal. This covers all odd total degrees through five and also the mixed even-odd monomials.

It remains to check total degrees 0, 2, and 4. The total mass is

$$2nw_a + 2^n w_c = 1.$$

For a coordinate i,

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^2] = 2w_a r_a^2 + 2^n w_c r_c^2 = 1 = \mathbb{E}[Z_i^2],$$

and, for $i \neq j$, $\mathcal{Q}_{\mathrm{CUT4}}[z_i z_j] = 0 = \mathbb{E}[Z_i Z_j]$ by sign symmetry. The fourth marginal moment is

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^4] = 2w_a r_a^4 + 2^n w_c r_c^4 = 3 = \mathbb{E}[Z_i^4],$$

while the mixed fourth moment is

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^2 z_j^2] = 2^n w_c r_c^4 = 1 = \mathbb{E}[Z_i^2 Z_j^2], \qquad i \neq j.$$

All remaining fourth-degree monomials contain an odd coordinate exponent and therefore vanish on both sides. These identities exhaust the Gaussian monomials of total degree at most five, so the rule has degree-five exactness. □

Thus the rule is symmetric, has positive weights, and integrates Gaussian polynomials through degree five. By contrast, the usual $2n+1$ unscented rule is commonly described as third-degree accurate for Gaussian moment propagation. The extra corner points are what allow CUT4-G to match cross moments such as $\mathbb{E}[Z_i^2 Z_j^2]$ that axis-only rules do not represent in the same way.

The price is exponential growth in the corner set. CUT4-G uses 42 points at $n = 5$, 1044 points at $n = 10$, and becomes unattractive in large structural states unless the quadrature is applied to a low-dimensional stochastic block. This reinforces the structural lesson of Chapter 19: higher-order quadrature should be spent on the declared stochastic integration coordinates, not on deterministic-completion directions that add no stochastic law.

16.3 Approximation Boundary

BayesFilter should label sigma-point filters as approximate likelihood backends unless a special model structure supplies an exactness argument. The source SR-UKF material contains model-specific claims about quadratic observation equations and pruned DSGE structure. Those claims are useful case-study material, but they should not be promoted to a general BayesFilter theorem without a separate derivation in BayesFilter notation and a regression test against an exact reference.

For HMC, this distinction matters. If the sigma-point likelihood is used in the target, the chain targets the sigma-point posterior. If the sigma-point filter is only a surrogate for an exact nonlinear likelihood, then a correction or delayed-acceptance policy is needed before exact-posterior claims are made.

16.4 Derivative and Compilation Requirements

Sigma-point HMC requires differentiating through:

- the covariance factor used to generate points;
- the nonlinear transition and observation maps;
- the weighted covariance assembly;
- linear solves or factorizations for $P_{zz,t}$;
- any PSD projection, jitter, clipping, or fallback branch.

The derivative provider must return the derivative of the same approximate log likelihood that supplies the Metropolis value. Static shapes are also non-negotiable: the number of sigma points, state dimension, observed-data mask shape, and scan length must not vary inside the compiled transition.

16.5 Evidence Ladder

The minimum evidence ladder for a sigma-point backend is:

(i) exact recovery on affine Gaussian systems against the Kalman backend;

(ii) finite value and finite gradient on controlled nonlinear systems;

(iii) eager/compiled parity where a compiled sampler is claimed;

(iv) comparison to a denser quadrature, long simulation, or trusted reference on low-dimensional nonlinear examples;

(v) HMC diagnostics that remain explicitly separated from convergence proof.

This ladder is the consolidation of the source-project SVD/HMC validation plans, not a completed BayesFilter certification result.

CUT backends should pass the same ladder plus one rule-specific check: for standard-normal polynomial test functions up to the declared degree, the implemented points and weights should reproduce the analytic Gaussian moments before the points are used in a filter. This catches mistakes in point enumeration, weights, dimension restrictions, and transformed-coordinate scaling before nonlinear filtering errors obscure the quadrature issue.

Chapter 17

Square-Root Sigma-Point Filters

Square-root sigma-point filters propagate a factor of the covariance rather than the covariance matrix itself. Their purpose is numerical: avoid avoidable loss of symmetry and positive semi-definiteness, reduce solve instability, and make the filtering recursion easier to monitor. In BayesFilter, square-root bookkeeping is a backend discipline. It does not by itself make an approximate likelihood exact or an autodiff gradient safe for HMC.

17.1 Backend Contract

A square-root backend stores a factor C_t such that $P_t = C_t C_t^\top$ or an equivalent factorization. The factor may be Cholesky, QR-derived, SVD/eigen-derived, or another auditable representation. The backend contract must state:

- the factor orientation and reconstruction identity;
- the predict update and measurement update used for the factor;
- whether covariance matrices are ever reconstructed;
- how rank loss, jitter, and PSD projection are handled;
- which operations define the differentiable target path.

The source SR-UKF material is useful here precisely because it exposed a maintenance risk: a method can be called square-root while still reconstructing full covariances and applying simplified covariance updates internally. The BayesFilter monograph should therefore describe the actual implemented recursion, not only the label attached to the filter.

17.2 HMC-Relevant Risks

For HMC, the relevant question is not only whether the factor recursion keeps P_t positive semi-definite in floating point. The sampler also needs a stable derivative of the scalar log target. Cholesky, QR, and SVD factors have different derivative behavior near rank loss or repeated singular values. A backend that is robust for value evaluation can still be fragile as a raw tape-gradient path.

BayesFilter should therefore classify square-root sigma-point evidence in two columns:

(i) value-recursion evidence, such as finite likelihoods, factor reconstruction residuals, and affine-Gaussian parity;

(ii) derivative evidence, such as finite gradients, AD/custom-gradient parity, spectral-gap telemetry, and HMC leapfrog stability.

Only the second column is directly relevant to HMC production readiness.

17.3 When to Prefer Square-Root Forms

Square-root forms are natural when observation noise is small, latent covariances are nearly singular, or long scans accumulate asymmetry. They are also useful for large LGSSMs where factor diagnostics expose numerical problems earlier than a final log likelihood does. They should not be treated as a universal answer to nonlinear filtering: if the moment closure is poor, the filter is still approximating the wrong target; if the derivative of the factorization is unstable, HMC can still fail.

17.4 CUT With Square-Root Backends

The conjugate unscented transform in Section 16.2 is a point-and-weight rule. It can be combined with any square-root factor backend that maps standardized points $\xi^{(j)}$ into state points

$$\chi^{(j)} = m + C\xi^{(j)}, \qquad CC^\top = P_\star.$$

The experimental `/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py` module uses positive CUT4-G weights together with QR square-root covariance updates. It also keeps rank-deficient process noise as a rectangular factor G with $Q = GG^\top$, avoiding an invalid Cholesky factorization of a singular Q.

An SVD/eigen factor can be used in the same location:

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2}.$$

This may make value evaluation more robust when P is singular or nearly singular. It does not change CUT's quadrature degree; it changes the numerical factor used to place the CUT points. Therefore a stable CUT-SVD filter must report the same factor diagnostics as an SVD sigma-point filter: the covariance actually reconstructed, active floors or rank truncations, spectral gaps, sign or order choices, and fallback branches. In short, CUT can be combined with SVD, but the result is a higher-order quadrature rule on top of an SVD factor policy, not a new theorem that removes spectral derivative risk.

Chapter 18

SVD Sigma-Point Filters

The SVD sigma-point filter is the most important nonlinear-filtering lesson from the recent source-project debugging work. It improved value-side robustness by using spectral covariance factors and explicit finite-arithmetic guards, but it also exposed why raw autodiff through spectral decompositions is not a comfortable industrial HMC default. BayesFilter should preserve both lessons.

18.1 What the Source Audit Established

The math/code audit recorded in the source-project SVD filter audit result from 2026-04-25 established the following bounded facts:

- the linear SVD Kalman path uses a Joseph-style covariance update;
- the nonlinear sigma-point path uses the simplified update

$$P^+ = P^- - KP_{zz}K^\top;$$

- PSD projection occurs after the candidate covariance is formed, so it cannot prevent overflow or non-finite values during the formation of P^+;
- the conditioning guard with threshold `_SVD_CONDITIONING_MAX_ABS = 1e100` is an engineering finite-arithmetic guard, not a mathematically derived posterior threshold;
- prior short HMC diagnostics were numerical-path regressions, not convergence evidence.

These are implementation facts, not general theorems about SVD filtering. BayesFilter should cite them as project evidence and should avoid converting them into unsupported production-readiness claims.

18.2 Spectral Differentiation Risk

The Phase 2B literature gate accepted the Matrix Backprop source for the bounded claim that gradients through SVD or eigendecomposition contain terms with denominators involving singular-value or eigenvalue gaps. When two singular values or eigenvalues

are equal, the decomposition is not uniquely differentiable; when they are merely close, derivative terms can become large and numerically unstable.

This is directly relevant to HMC. A large DSGE or NAWM-sized model can visit parameter regions where covariance factors are ill conditioned, nearly rank deficient, or spectrally clustered. A value-side SVD factor may still return a finite covariance approximation, while the tape gradient through the spectral basis becomes noisy, discontinuous, or non-finite. Larger dimensions make close spectral gaps more likely in practice, so BayesFilter should require gap telemetry and derivative validation before using SVD sigma-point gradients inside production HMC.

18.3 Analytic Score and Hessian Recursion

The SVD sigma-point derivative problem has two layers. The likelihood layer is not new: after the sigma-point update has produced

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)},$$

the score and Hessian are exactly the solve-form expressions in Chapter 10. The new work is the sigma-point layer: differentiate the point cloud, the nonlinear maps, and the weighted moment assemblies that generate these innovation objects.

Thus this chapter plugs into the analytic-derivative stack as follows. Chapter 11 supplies derivatives of structural maps and parameter transforms; Chapter 12 defines what differentiated SVD factors must reconstruct; Chapter 13 requires the SVD score to differentiate the same implemented scalar value; and Chapter 14 supplies the finite-difference, backend-parity, branch, and HMC-gating tests.

Let a Gaussian input at some predict or update substep be

$$X \sim \mathcal{N}(m(\theta), P(\theta)).$$

Let an SVD/eigendecomposition factor be written

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2},$$

where Λ_+ denotes the eigenvalue policy actually used for point generation: active positive eigenvalues, a smooth floor, or a hard floor. A fixed sigma-point rule can be written as

$$\chi^{(r)} = m + C\xi^{(r)}, \qquad r = 0, \ldots, N-1, \tag{18.1}$$

where the standardized offsets $\xi^{(r)}$ and weights $w_r^{(m)}, w_r^{(c)}$ are fixed by the UKF, CKF, or another deterministic quadrature rule. For the UKF, for example, the nonzero offsets are $\pm\gamma e_a$. If tuning parameters such as α or κ are estimated, the same formulas hold with additional $\dot{\xi}^{(r)}$, $\ddot{\xi}^{(r)}$, $\dot{w}_r$, and $\ddot{w}_r$ terms. BayesFilter's default derivative contract treats the rule as fixed.

With fixed quadrature offsets, the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \tag{18.2}$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \tag{18.3}$$

Thus all spectral difficulty enters through $\dot{C}$ and $\ddot{C}$; the remaining recursion is ordinary chain rule and product rule.

18.3.1 Map and Moment Derivatives

Let g_θ denote either the transition map in the predict step or the observation map in the update step, and define

$$\eta^{(r)} = g_\theta(\chi^{(r)}).$$

Using $D_x g_\theta$ for the Jacobian in the state argument, and $D^2_{xx} g_\theta[a, b]$ for the second directional derivative in directions a, b, the propagated point derivatives are

$$\dot\eta^{(r,i)} = D_x g_\theta(\chi^{(r)})\dot\chi^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \tag{18.4}$$

$$\begin{aligned}\ddot\eta^{(r,ij)} = {} & D_x g_\theta(\chi^{(r)})\ddot\chi^{(r,ij)} + D^2_{xx} g_\theta(\chi^{(r)})[\dot\chi^{(r,i)}, \dot\chi^{(r,j)}] \\ & + D^2_{x\theta_i} g_\theta(\chi^{(r)})\dot\chi^{(r,j)} + D^2_{x\theta_j} g_\theta(\chi^{(r)})\dot\chi^{(r,i)} + \partial^2_{ij} g_\theta(\chi^{(r)}).\end{aligned} \tag{18.5}$$

These equations are the exact place where the structural model derivatives enter the SVD sigma-point score and Hessian.

The sigma-point mean and covariance of the transformed cloud are

$$\bar\eta = \sum_{r=0}^{N-1} w_r^{(m)} \eta^{(r)}, \tag{18.6}$$

$$M_\eta = \sum_{r=0}^{N-1} w_r^{(c)} \Delta_\eta^{(r)} \Delta_\eta^{(r)\top} + \Omega_\theta, \qquad \Delta_\eta^{(r)} = \eta^{(r)} - \bar\eta, \tag{18.7}$$

where Ω_θ is the process-noise covariance in a predict step or the measurement-noise covariance in an update step. Their derivatives are

$$\dot{\bar\eta}^{(i)} = \sum_r w_r^{(m)} \dot\eta^{(r,i)}, \tag{18.8}$$

$$\ddot{\bar\eta}^{(ij)} = \sum_r w_r^{(m)} \ddot\eta^{(r,ij)}, \tag{18.9}$$

and, with $\dot\Delta_\eta^{(r,i)} = \dot\eta^{(r,i)} - \dot{\bar\eta}^{(i)}$,

$$\dot M_\eta^{(i)} = \sum_r w_r^{(c)} \left[\dot\Delta_\eta^{(r,i)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \dot\Delta_\eta^{(r,i)\top}\right] + \dot\Omega_\theta^{(i)}, \tag{18.10}$$

$$\ddot M_\eta^{(ij)} = \sum_r w_r^{(c)} \left[\ddot\Delta_\eta^{(r,ij)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \ddot\Delta_\eta^{(r,ij)\top} + \dot\Delta_\eta^{(r,i)} \dot\Delta_\eta^{(r,j)\top} + \dot\Delta_\eta^{(r,j)} \dot\Delta_\eta^{(r,i)\top}\right] + \ddot\Omega_\theta^{(ij)}. \tag{18.11}$$

Equations (18.2)–(18.11) are the analytic derivative recursion for any sigma-point predict moment.

18.3.2 UKF Update Objects

At the observation update, take the input mean and covariance to be $m_t^- = m_{t|t-1}$ and $P_t^- = P_{t|t-1}$, construct points $\chi_t^{(r)}$ by (18.1), and propagate

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}).$$

Equations (18.8)–(18.11) give

$$\bar{z}_t, \quad S_t = \sum_r w_r^{(c)} (z_t^{(r)} - \bar{z}_t)(z_t^{(r)} - \bar{z}_t)^\top + R_\theta,$$

and their first and second derivatives. The innovation is

$$v_t = y_t - \bar{z}_t, \qquad \dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \qquad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}, \tag{18.12}$$

unless the data transformation itself depends on parameters.

The cross-covariance used in the Kalman gain is

$$P_{xz,t} = \sum_r w_r^{(c)} \Delta_x^{(r)} \Delta_z^{(r)\top}, \qquad \Delta_x^{(r)} = \chi_t^{(r)} - m_t^-, \qquad \Delta_z^{(r)} = z_t^{(r)} - \bar{z}_t. \tag{18.13}$$

Its first and second derivatives are

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \tag{18.14}$$

$$\ddot{P}_{xz,t}^{(ij)} = \sum_r w_r^{(c)} \left[\ddot{\Delta}_x^{(r,ij)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \ddot{\Delta}_z^{(r,ij)\top} + \dot{\Delta}_x^{(r,i)} \dot{\Delta}_z^{(r,j)\top} + \dot{\Delta}_x^{(r,j)} \dot{\Delta}_z^{(r,i)\top} \right]. \tag{18.15}$$

The gain is best differentiated as a solved matrix equation. Since $K_t = P_{xz,t} S_t^{-1}$, equivalently

$$S_t K_t^\top = P_{xz,t}^\top.$$

Therefore

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t} \left(\dot{P}_{xz,t}^{(i)\top} - \dot{S}_t^{(i)} K_t^\top \right), \tag{18.16}$$

$$\ddot{K}_t^{(ij)\top} = \mathcal{S}_{S_t} \left(\ddot{P}_{xz,t}^{(ij)\top} - \ddot{S}_t^{(ij)} K_t^\top - \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} - \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} \right). \tag{18.17}$$

Finally, the usual sigma-point update

$$m_t^+ = m_t^- + K_t v_t, \tag{18.18}$$

$$P_t^+ = P_t^- - K_t S_t K_t^\top \tag{18.19}$$

has derivatives

$$\dot{m}_t^{+(i)} = \dot{m}_t^{-(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \tag{18.20}$$

$$\ddot{m}_t^{+(ij)} = \ddot{m}_t^{-(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \tag{18.21}$$

For the covariance, define

$$\dot{P}_t^{+(i)} = \dot{P}_t^{-(i)} - \dot{K}_t^{(i)} S_t K_t^\top - K_t \dot{S}_t^{(i)} K_t^\top - K_t S_t \dot{K}_t^{(i)\top}, \tag{18.22}$$

and

$$\begin{aligned} \ddot{P}_t^{+(ij)} = \ddot{P}_t^{-(ij)} &- \ddot{K}_t^{(ij)} S_t K_t^\top - \dot{K}_t^{(i)} \dot{S}_t^{(j)} K_t^\top - \dot{K}_t^{(i)} S_t \dot{K}_t^{(j)\top} \\ &- \dot{K}_t^{(j)} \dot{S}_t^{(i)} K_t^\top - K_t \ddot{S}_t^{(ij)} K_t^\top - K_t \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} \\ &- \dot{K}_t^{(j)} S_t \dot{K}_t^{(i)\top} - K_t \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} - K_t S_t \ddot{K}_t^{(ij)\top}. \end{aligned} \tag{18.23}$$

The next time step then refactors P_t^+ and differentiates the new SVD factor. This is where the spectral contract below enters the recursion again.

18.3.3 Likelihood Score and Hessian

The SVD sigma-point likelihood contribution is the Gaussian innovation likelihood

$$\ell_t = -\frac{1}{2}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right],$$

where S_t and v_t are the sigma-point objects above. Let $S_t w_t = v_t$. The score is

$$\partial_i \ell_t = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right], \tag{18.24}$$

and the Hessian is obtained by substituting the sigma-point $\dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ into (10.1)–(10.10). Thus the UKF/SVD Hessian is not a separate likelihood theorem. It is the Kalman solve-form Hessian applied to approximate moments generated by (18.1)–(18.23).

18.4 Structural Fixed-Support Score

Chapter 19 explains why a structural sigma-point filter places points on the pre-transition uncertainty variable

$$A_t = (x_{t-1}, \varepsilon_t), \qquad A_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_{a,t}, P_{a,t}), \tag{18.25}$$

and then completes the state pointwise:

$$A_t^{(r)} = \mu_{a,t} + C_{a,t}\xi^{(r)}, \qquad X_t^{(r)} = F_\theta(A_t^{(r)}), \qquad Z_t^{(r)} = h_\theta(X_t^{(r)}). \tag{18.26}$$

Equations (18.25)–(18.26) are the derivative version of Proposition 19.1. The filtered state is still x_t; the sigma-point variable is the pre-transition uncertainty whose push-forward generates the predictive law. This is the object that must be differentiated for deterministic-completion examples such as Model C in Chapter 40.

The subtle case is a fixed structural zero direction. Write

$$P_{a,t} = U_t \Lambda_t U_t^\top, \qquad \Lambda_t = \operatorname{diag}(\lambda_{1t}, \ldots, \lambda_{dt}),$$

and split the spectral indices into

$$\mathcal{I}_t^+ = \{a : \lambda_{at} > \tau\}, \qquad \mathcal{I}_t^0 = \{a : \lambda_{at} \le \tau\}.$$

The structural fixed-support branch assumes that the zero-support block is part of the model law, not a numerical floor:

$$u_b^\top \dot{P}_{a,t}^{(i)} u_c = 0 \qquad \text{whenever } b \in \mathcal{I}_t^0 \text{ or } c \in \mathcal{I}_t^0, \tag{18.27}$$

$$|\lambda_{at} - \lambda_{bt}| > g_{\min} \qquad \text{for every differentiated active pair.} \tag{18.28}$$

Here "differentiated active pair" means any pair that enters the derivative of an active factor column: active-active pairs and active-null pairs. Gaps between two null directions do not matter because their factor columns are identically zero on this branch.

The implemented factor is

$$C_{a,t} = \left[\sqrt{\lambda_{1t}}u_{1t} \quad \cdots \quad \sqrt{\lambda_{dt}}u_{dt}\right], \qquad \sqrt{\lambda_{at}}u_{at} = 0 \quad \text{for } a \in \mathcal{I}_t^0. \tag{18.29}$$

For an active column $a \in \mathcal{I}_t^+$,

$$\dot{u}_{at}^{(i)} = \sum_{b \neq a} u_{bt} \frac{u_{bt}^\top \dot{P}_{a,t}^{(i)} u_{at}}{\lambda_{at} - \lambda_{bt}}, \tag{18.30}$$

$$\dot{c}_{at}^{(i)} = \sqrt{\lambda_{at}}\, \dot{u}_{at}^{(i)} + \frac{u_{at}^\top \dot{P}_{a,t}^{(i)} u_{at}}{2\sqrt{\lambda_{at}}}\, u_{at}. \tag{18.31}$$

For a null column $a \in \mathcal{I}_t^0$,

$$c_{at} = 0, \qquad \dot{c}_{at}^{(i)} = 0. \tag{18.32}$$

Under (18.27), these column rules reconstruct the derivative of the same pre-transition covariance:

$$\dot{P}_{a,t}^{(i)} = \dot{C}_{a,t}^{(i)} C_{a,t}^\top + C_{a,t} \dot{C}_{a,t}^{(i)\top}. \tag{18.33}$$

The reconstruction target is not a nuggeted covariance. A positive floor that turns a structural zero direction into positive variance changes the law and must be reported as a different regularized branch.

With this fixed-support factor, the score recursion is the ordinary sigma-point chain rule. For a scalar parameter component θ_i,

$$\dot{A}_t^{(r,i)} = \dot{\mu}_{a,t}^{(i)} + \dot{C}_{a,t}^{(i)} \xi^{(r)}, \tag{18.34}$$

$$\dot{X}_t^{(r,i)} = D_a F_\theta(A_t^{(r)}) \dot{A}_t^{(r,i)} + \partial_i F_\theta(A_t^{(r)}), \tag{18.35}$$

$$\dot{Z}_t^{(r,i)} = D_x h_\theta(X_t^{(r)}) \dot{X}_t^{(r,i)} + \partial_i h_\theta(X_t^{(r)}). \tag{18.36}$$

The moment derivatives are

$$\dot{\hat{x}}_t^{(i)} = \sum_r w_r^{(m)} \dot{X}_t^{(r,i)}, \qquad \dot{\hat{z}}_t^{(i)} = \sum_r w_r^{(m)} \dot{Z}_t^{(r,i)}, \tag{18.37}$$

$$\dot{S}_t^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_z^{(r,i)} \Delta_z^{(r)\top} + \Delta_z^{(r)} \dot{\Delta}_z^{(r,i)\top} \right] + \dot{R}_t^{(i)}, \tag{18.38}$$

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[\dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \tag{18.39}$$

where

$$\Delta_x^{(r)} = X_t^{(r)} - \hat{x}_t, \qquad \Delta_z^{(r)} = Z_t^{(r)} - \hat{z}_t,$$

and dotted centered quantities subtract the corresponding dotted means. The innovation derivative remains

$$v_t = y_t - \hat{z}_t, \qquad \dot{v}_t^{(i)} = -\dot{\hat{z}}_t^{(i)}.$$

Therefore the per-period score is

$$\partial_i \ell_t = -\frac{1}{2} \left[\operatorname{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2 \dot{v}_t^{(i)\top} S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} v_t \right]. \tag{18.40}$$

This sign convention is the direct derivative of the likelihood in (19.91); the term $\dot{v}_t = -\dot{\hat{z}}_t$ is where the observation mean derivative enters.

The diagnostics for this branch must report at least the fixed null count, the maximum fixed-null derivative residual in (18.27), the active spectral gap in (18.28), the factor reconstruction residual in (18.33), and the deterministic-completion residual of the propagated points. Passing these checks means that the score differentiates the implemented structural sigma-point likelihood. It does not certify a nonlinear Hessian, HMC target, or GPU/XLA scaling claim.

18.5 Spectral Factor Derivative Contract

The preceding section assumes $\dot{C}$ and $\ddot{C}$ are available. This subsection states what those derivatives mean for an SVD/eigen factor. Suppose first that $P(\theta)$ has a smooth simple spectrum and no active hard floor:

$$P = \sum_{a=1}^{n} \lambda_a u_a u_a^\top, \qquad \lambda_a > 0, \qquad \lambda_a \neq \lambda_b \ (a \neq b).$$

For a parameter component θ_i,

$$\dot{\lambda}_a^{(i)} = u_a^\top \dot{P}^{(i)} u_a, \tag{18.41}$$

$$\dot{u}_a^{(i)} = \sum_{b \neq a} u_b \frac{u_b^\top \dot{P}^{(i)} u_a}{\lambda_a - \lambda_b}. \tag{18.42}$$

For the column factor $c_a = \sqrt{\lambda_a}\, u_a$,

$$\dot{c}_a^{(i)} = \sqrt{\lambda_a}\, \dot{u}_a^{(i)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} u_a. \tag{18.43}$$

Second derivatives can be written in the same eigenbasis. Define $A_a^{(i)} = \dot{P}^{(i)} - \dot{\lambda}_a^{(i)} I$ and $A_a^{(ij)} = \ddot{P}^{(ij)} - \ddot{\lambda}_a^{(ij)} I$. Then

$$\ddot{\lambda}_a^{(ij)} = u_a^\top \ddot{P}^{(ij)} u_a + 2 \sum_{b \neq a} \frac{(u_b^\top \dot{P}^{(i)} u_a)(u_b^\top \dot{P}^{(j)} u_a)}{\lambda_a - \lambda_b}. \tag{18.44}$$

The second derivative of the eigenvector is determined by differentiating the eigenequation and the normalization condition:

$$\ddot{u}_a^{(ij)} = -u_a\, \dot{u}_a^{(i)\top} \dot{u}_a^{(j)} + \sum_{b \neq a} u_b \frac{u_b^\top \left(A_a^{(ij)} u_a + A_a^{(i)} \dot{u}_a^{(j)} + A_a^{(j)} \dot{u}_a^{(i)} \right)}{\lambda_a - \lambda_b}. \tag{18.45}$$

Consequently

$$\begin{aligned} \ddot{c}_a^{(ij)} = {} & \sqrt{\lambda_a}\, \ddot{u}_a^{(ij)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(j)} + \frac{\dot{\lambda}_a^{(j)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(i)} \\ & + \left(\frac{\ddot{\lambda}_a^{(ij)}}{2\sqrt{\lambda_a}} - \frac{\dot{\lambda}_a^{(i)} \dot{\lambda}_a^{(j)}}{4\lambda_a^{3/2}} \right) u_a. \end{aligned} \tag{18.46}$$

Equations (18.41)–(18.46) are the branch-local eigenderivative formulas for an eigenfactor covariance square root on a smooth simple-spectrum branch, before any active hard-floor or ordering/sign fallback is crossed.

The formulas also show why the derivative path is fragile. Eigenvector derivatives contain denominators $\lambda_a - \lambda_b$; the Hessian contains the same gaps again through $\dot{u}_a$ and through differentiation of the gap. This is the spectral-gap risk identified by matrix-backpropagation calculus [Ionescu et al., 2015]. If eigenvalues are repeated, individual eigenvectors are not uniquely differentiable. If a hard floor $\lambda_a^+ = \max(\lambda_a, \epsilon)$ is used, the

derivative is branch dependent and undefined at $\lambda_a = \epsilon$. A smooth floor $\lambda_a^+ = \phi_\epsilon(\lambda_a)$ replaces $\dot{\lambda}_a$ and $\ddot{\lambda}_a$ by

$$\dot{\lambda}_a^+ = \phi'_\epsilon(\lambda_a)\dot{\lambda}_a, \qquad \ddot{\lambda}_a^+ = \phi'_\epsilon(\lambda_a)\ddot{\lambda}_a + \phi''_\epsilon(\lambda_a)\dot{\lambda}_a^{(i)}\dot{\lambda}_a^{(j)},$$

but it does not remove the eigenvector gap denominators.

For this reason BayesFilter should validate SVD factor derivatives through reconstruction identities, not merely by checking that autodiff returns finite numbers. The reconstruction target is the covariance represented by the implemented factor,

$$P_+ = CC^\top = U\Lambda_+ U^\top,$$

not necessarily the pre-floor covariance P:

$$\dot{P}_+^{(i)} \stackrel{?}{=} \dot{C}^{(i)}C^\top + C\dot{C}^{(i)\top}, \tag{18.47}$$

$$\ddot{P}_+^{(ij)} \stackrel{?}{=} \ddot{C}^{(ij)}C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \tag{18.48}$$

The residuals in (18.47)–(18.48), the minimum spectral gap, the active-floor set, and any fallback branch are part of the derivative output contract.

18.6 Validation Targets for SVD Score and Hessian

The analytical derivation above leads to concrete tests:

(i) On affine Gaussian systems, the SVD sigma-point score and Hessian must match the exact Kalman score and Hessian from Chapters 9–10.

(ii) On small nonlinear systems, the analytical score and Hessian must match finite differences of the same SVD sigma-point likelihood.

(iii) The factor residuals (18.47)–(18.48) must remain small on well-separated spectra.

(iv) Tests must deliberately include clustered eigenvalues, zero eigenvalues, floor crossings, and sign/order changes, and report when the derivative is a branch derivative rather than the derivative of a smooth mathematical SVD.

(v) HMC diagnostics must record whether gradients came from the structural analytic recursion, a custom spectral derivative, raw autodiff, or a regularized fallback.

These tests turn the phrase "SVD sigma-point Hessian" into an auditable object: a sigma-point moment derivative recursion, a solve-form likelihood Hessian, and a declared spectral-factor derivative policy.

18.7 SVD-CUT Score and Hessian

The conjugate unscented transform can be inserted into the same spectral sigma-point filter. The precise object is an SVD-placed CUT rule: use an SVD/eigen factor to place the CUT offsets, then use the ordinary sigma-point moment and likelihood recursions. This is sometimes called CUT-SVD and sometimes SVD-CUT. The name is less important than the contract

$$P_\star = CC^\top,$$

where $P_\star$ is the covariance actually represented by the implemented factor. If eigenvalues have been floored, truncated, or otherwise regularized, $P_\star$ is the floored or truncated covariance, not necessarily the pre-regularized covariance submitted to the factor routine.

Let the SVD branch return a factor $C(\theta) \in \mathbb{R}^{n_x \times q}$ and let $Z \sim \mathcal{N}(0, I_q)$. A CUT rule in the factor coordinates supplies standardized offsets and weights

$$\{(\xi^{(r)}, w_r) : r = 0, \dots, N_{\mathrm{CUT}} - 1\}, \qquad \xi^{(r)} \in \mathbb{R}^q.$$

For CUT4-G, when $q \geq 3$,

$$N_{\mathrm{CUT4}} = 2q + 2^q.$$

The SVD-CUT points are

$$\chi^{(r)} = m + C\xi^{(r)}, \qquad r = 0, \dots, N_{\mathrm{CUT}} - 1. \tag{18.49}$$

The quadrature approximation to an expectation under the implemented Gaussian law $X_\star = m + CZ \sim \mathcal{N}(m, P_\star)$ is

$$\mathcal{Q}_{\text{SVD-CUT}}[\varphi] = \sum_{r=0}^{N_{\mathrm{CUT}}-1} w_r \varphi(m + C\xi^{(r)}). \tag{18.50}$$

Proposition 18.1 (Polynomial exactness under affine SVD placement)**.** *Suppose the standardized CUT rule is exact for standard-normal polynomials on $\mathbb{R}^q$ through degree d. If $CC^\top = P_\star$, then (18.50) integrates every polynomial $p : \mathbb{R}^{n_x} \to \mathbb{R}$ of total degree at most d exactly under $X_\star \sim \mathcal{N}(m, P_\star)$. The statement remains true when $P_\star$ is singular.*

Proof. For any polynomial p of degree at most d, define

$$\psi(z) = p(m + Cz).$$

The affine composition cannot increase total degree, so ψ is a polynomial on $\mathbb{R}^q$ of degree at most d. By standardized CUT exactness,

$$\sum_r w_r \psi(\xi^{(r)}) = \mathbb{E}[\psi(Z)].$$

Substituting the definition of ψ gives

$$\sum_r w_r p(m + C\xi^{(r)}) = \mathbb{E}[p(m + CZ)] = \mathbb{E}[p(X_\star)].$$

No inverse of C is used. Hence rank deficiency of $P_\star = CC^\top$ does not affect the formal quadrature exactness statement; it only changes the affine support on which the degenerate Gaussian law lives. □

This proposition is the clean mathematical answer to the accuracy question: abstracting from finite arithmetic, SVD placement does not by itself destroy CUT accuracy. It is not a recommendation to hide a structural state split inside a large singular covariance. A full zero-padded factor may reproduce the same law, but it also creates duplicate or nearly duplicate points in null directions and pushes the derivative path through the larger spectral branch.

18.7.1 SVD-CUT Moment Derivatives

The analytic gradient and Hessian follow by differentiating (18.49)–(18.50). Assume first that the CUT offsets and weights are fixed. If CUT tuning parameters are estimated, the same derivation applies after adding the explicit $\dot{\xi}^{(r)}$, $\ddot{\xi}^{(r)}$, $\dot{w}_r$, and $\ddot{w}_r$ terms.

For a parameter component θ_i, the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \tag{18.51}$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \tag{18.52}$$

Let g_θ be the nonlinear map being integrated. In a predict step this is the transition map; in an update step it is the observation map. Write

$$y^{(r)} = g_\theta(\chi^{(r)}), \qquad \bar{y} = \sum_r w_r y^{(r)}, \qquad d^{(r)} = y^{(r)} - \bar{y}.$$

The propagated point derivatives are

$$\dot{y}^{(r,i)} = D_x g_\theta(\chi^{(r)})\dot{\chi}^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \tag{18.53}$$

$$\begin{aligned}\ddot{y}^{(r,ij)} = {} & D_x g_\theta(\chi^{(r)})\ddot{\chi}^{(r,ij)} + D^2_{xx} g_\theta(\chi^{(r)})[\dot{\chi}^{(r,i)}, \dot{\chi}^{(r,j)}] \\ & + D^2_{x\theta_i} g_\theta(\chi^{(r)})\dot{\chi}^{(r,j)} + D^2_{x\theta_j} g_\theta(\chi^{(r)})\dot{\chi}^{(r,i)} + \partial^2_{ij} g_\theta(\chi^{(r)}).\end{aligned} \tag{18.54}$$

Therefore

$$\dot{\bar{y}}^{(i)} = \sum_r w_r \dot{y}^{(r,i)}, \tag{18.55}$$

$$\ddot{\bar{y}}^{(ij)} = \sum_r w_r \ddot{y}^{(r,ij)}. \tag{18.56}$$

Let Ω_θ denote the additive covariance in the moment being formed: Q_θ in a predict step or R_θ in an observation update. The SVD-CUT covariance moment is

$$M = \sum_r w_r d^{(r)} d^{(r)\top} + \Omega_\theta. \tag{18.57}$$

With

$$\dot{d}^{(r,i)} = \dot{y}^{(r,i)} - \dot{\bar{y}}^{(i)}, \qquad \ddot{d}^{(r,ij)} = \ddot{y}^{(r,ij)} - \ddot{\bar{y}}^{(ij)},$$

its derivatives are

$$\dot{M}^{(i)} = \sum_r w_r \left[\dot{d}^{(r,i)} d^{(r)\top} + d^{(r)} \dot{d}^{(r,i)\top}\right] + \dot{\Omega}^{(i)}_\theta, \tag{18.58}$$

$$\ddot{M}^{(ij)} = \sum_r w_r \left[\ddot{d}^{(r,ij)} d^{(r)\top} + d^{(r)} \ddot{d}^{(r,ij)\top} + \dot{d}^{(r,i)} \dot{d}^{(r,j)\top} + \dot{d}^{(r,j)} \dot{d}^{(r,i)\top}\right] + \ddot{\Omega}^{(ij)}_\theta. \tag{18.59}$$

Derivation. Equations (18.51) and (18.52) are the first and second derivatives of the affine point map $\chi^{(r)} = m + C\xi^{(r)}$, using fixed $\xi^{(r)}$. Equations (18.53) and (18.54) are the first and second multivariate chain rules for $g_\theta(\chi^{(r)}(\theta))$. Equations (18.55) and (18.56) follow from linearity of the weighted sum. Finally, differentiating $d^{(r)}d^{(r)\top}$ once and twice gives

$$\partial_i(dd^\top) = \dot{d}_i d^\top + d\dot{d}_i^\top$$

and

$$\partial^2_{ij}(dd^\top) = \ddot{d}_{ij} d^\top + d\ddot{d}_{ij}^\top + \dot{d}_i \dot{d}_j^\top + \dot{d}_j \dot{d}_i^\top,$$

which proves (18.58) and (18.59) after summing over the fixed weights and adding the derivatives of Ω_θ. □

18.7.2 SVD-CUT Likelihood Gradient and Hessian

For an observation update, set $g_\theta = h_\theta$ and identify

$$\bar{z}_t = \bar{y}, \qquad S_t = M, \qquad v_t = y_t^{\text{obs}} - \bar{z}_t.$$

If the recorded observation does not depend on θ, then

$$\dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \qquad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}. \tag{18.60}$$

Let $S_t w_t = v_t$. The per-time SVD-CUT log likelihood is

$$\ell_t = -\frac{1}{2}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right]. \tag{18.61}$$

Its score is

$$\partial_i \ell_t = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right]. \tag{18.62}$$

For the Hessian, define

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t}\left(\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t\right). \tag{18.63}$$

Then

$$T_{ij}^{\log\det} = \operatorname{tr}\left(S_t^{-1}\ddot{S}_t^{(ij)} - S_t^{-1}\dot{S}_t^{(j)} S_t^{-1}\dot{S}_t^{(i)}\right), \tag{18.64}$$

$$T_{ij}^{v} = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top}\dot{w}_t^{(j)}, \tag{18.65}$$

$$T_{ij}^{S} = 2\dot{w}_t^{(j)\top}\dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t, \tag{18.66}$$

and

$$\partial_{ij}^2 \ell_t = -\frac{1}{2}\left[T_{ij}^{\log\det} + T_{ij}^{v} - T_{ij}^{S}\right]. \tag{18.67}$$

Derivation. Equations (18.58) and (18.59), with $\Omega_\theta = R_\theta$, supply $\dot{S}_t$ and $\ddot{S}_t$. Equation (18.60) supplies $\dot{v}_t$ and $\ddot{v}_t$. Differentiating the solve $S_t w_t = v_t$ gives

$$S_t \dot{w}_t^{(j)} + \dot{S}_t^{(j)} w_t = \dot{v}_t^{(j)},$$

hence (18.63). The differential identities $\partial_i \log\det S = \operatorname{tr}(S^{-1}\dot{S}_i)$ and $\partial_i S^{-1} = -S^{-1}\dot{S}_i S^{-1}$ give (18.62). Differentiating the three pieces of (18.61) a second time gives (18.64) and (18.67). This is the same solve-form likelihood Hessian as Chapter 10; the SVD-CUT-specific work is the construction of $v_t, S_t, \dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ from the CUT point cloud. □

All dependence on the SVD backend enters through $C, \dot{C}, \ddot{C}$. Consequently the derivative must reconstruct the same implemented covariance branch:

$$\dot{P}_\star^{(i)} = \dot{C}^{(i)} C^\top + C\dot{C}^{(i)\top},$$

and

$$\ddot{P}_\star^{(ij)} = \ddot{C}^{(ij)} C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}.$$

These are exactly the reconstruction identities (18.47)–(18.48). If a hard eigenvalue floor is active, the equations describe the branch derivative of the floored covariance $P_\star$, not the derivative of the pre-floor covariance. If eigenvalues are clustered, repeated, reordered, or sign-flipped, the smooth-branch formulas in Section 18.5 may fail or become ill-conditioned.

18.7.3 Point Count, GPU, and XLA

Modern GPUs and XLA make large sigma-point clouds much less painful than scalar Python loops, but they do not make the point count disappear. The CUT map evaluations are embarrassingly parallel over points and chains, and the point dimension is static when q is fixed. This fits the XLA discipline in Chapter 36: construct a fixed tensor of offsets, evaluate the transition or observation map in batch, and reduce over the point axis inside the compiled value-and-gradient path.

The remaining costs still scale with

$$N_{\mathrm{CUT4}} = 2q + 2^q.$$

For one observation update with output dimension n_y, the value-side moment assembly contains

$$O(N_{\mathrm{CUT}}\, c_h) \quad \text{map work,} \qquad O(N_{\mathrm{CUT}}\, n_y^2) \quad \text{innovation-covariance reductions,}$$

plus $O(N_{\mathrm{CUT}}\, n_x n_y)$ if the cross covariance and Kalman gain are formed. With p parameters, a direct analytic score stores or recomputes $O(N_{\mathrm{CUT}}\, p\, n_y)$ propagated first derivatives, and a direct Hessian stores or recomputes $O(N_{\mathrm{CUT}}\, p^2\, n_y)$ propagated second derivatives, in addition to the SVD-factor derivative work. XLA can fuse many elementwise operations and run the point axis in parallel, but memory traffic, reductions, spectral factorization, compile time, and the sequential time scan remain real constraints.

Thus the correct engineering conclusion is conditional. A large number of CUT points may be acceptable on modern GPU/XLA hardware when the stochastic factor dimension q is modest, the maps are sufficiently expensive to amortize compile and launch overhead, tensor shapes are static, and chains or particles provide enough batch parallelism. It is not generally acceptable to collapse a large DSGE state into a full-state CUT rule and rely on the GPU to absorb the 2^q corner set. The preferred design remains structural SVD-CUT: apply higher-order CUT quadrature only to the declared stochastic integration block, then complete deterministic coordinates through the structural transition map and use SVD regularization only where the covariance factor actually needs it.

18.8 Status Labels for SVD-HMC

The SVD/HMC gap-closure plan introduced four labels that BayesFilter should reuse:

filter-correct the likelihood and gradients agree with a reference behavior on controlled cases;

sampler-usable short or medium HMC runs produce finite chains and useful diagnostics;

converged strict multi-chain posterior diagnostics pass on the intended target;

production-XLA-gated fixed-solution, full-solve, boundary rejection, gradient, and tiny HMC paths compile and run under the production compilation policy.

These labels prevent a recurring mistake: a clean smoke run is not a convergence result, and a finite likelihood is not a validated HMC gradient.

18.9 BayesFilter Policy

BayesFilter may use SVD sigma-point filters as robust approximate likelihood infrastructure and as a diagnostic backend. To use them as HMC production targets, the project must additionally provide:

(i) affine-Gaussian recovery against the exact Kalman backend;

(ii) nonlinear controlled recovery against dense quadrature or a trusted reference;

(iii) finite value and finite gradient stress tests across invalid and near-boundary parameter proposals;

(iv) eigenvalue or singular-value gap telemetry during HMC;

(v) eager/compiled parity for the complete value-and-gradient path;

(vi) a custom-gradient or analytic-gradient policy if raw spectral autodiff fails the preceding checks.

For industrial-size models, the conservative default is prevention rather than debugging: certify a derivative path before relying on SVD-tape gradients in a long HMC run.

Chapter 19

Structural Deterministic Dynamics in DSGE Filtering

Production warning. Do not implement DSGE nonlinear filtering by silently treating every declared state coordinate as an exchangeable noisy Gaussian coordinate. Many DSGE models contain a structural split between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states, controls, and current shocks. This chapter proves the law-level reason: for mixed structural models, the nonlinear prediction target is the pushforward of the lagged filtering law and the current innovation law through the structural transition. A full-state additive-noise approximation is therefore valid only when it preserves that law or is explicitly labeled and tested as an approximation.

19.1 The Structural Split

The state vector in a DSGE likelihood is not merely a numerical vector. It usually carries economic timing. In the language of the structural state partition from Chapter 2, a common DSGE representation separates a declared stochastic block from a deterministic-completion block.

Definition 19.1 (Structural transition notation)**.** *Throughout this chapter, write the full structural state as $x_t = (m_t, k_t)$, where m_t is the declared stochastic or exogenous block and k_t is the deterministic-completion block. A mixed structural transition is*

$$m_t = T_m(m_{t-1}, \varepsilon_t; \theta), \tag{19.1}$$
$$k_t = T_k(k_{t-1}, m_{t-1}, m_t; \theta), \tag{19.2}$$
$$x_t = (m_t, k_t). \tag{19.3}$$

Equivalently, define the structural map

$$F_\theta(x_{t-1}, \varepsilon_t) = \left(T_m(m_{t-1}, \varepsilon_t; \theta), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t; \theta); \theta)\right). \tag{19.4}$$

Let $\pi^x_{t-1|t-1}$ denote the filtering law of x_{t-1} and let ν^ε_θ denote the law of ε_t. The pre-transition law is

$$\pi^a_t = \pi^x_{t-1|t-1} \otimes \nu^\varepsilon_\theta$$

when the current innovation is conditionally independent of the lagged filtering law. The predictive law of x_t *is*

$$\pi^x_{t|t-1}(B) = \Pr_\theta(x_t \in B \mid y_{1:t-1})$$

for measurable state sets B*. The pushforward of* π^a_t *by* F_θ *is the measure*

$$(F_\theta)_\# \pi^a_t(B) = \pi^a_t\big(F_\theta^{-1}(B)\big)\,.$$

To place sigma points on a random variable means to choose weighted points $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ *whose weighted mean and covariance match the Gaussian approximation to that variable, then propagate* $x_t^{(j)} = F_\theta(a_t^{(j)})$ *point by point.*

Thus, for mixed structural models,

$$\pi^x_{t|t-1} = (F_\theta)_\# \pi^a_t,$$

or, in density notation when the densities exist,

$$p_\theta(x_t \mid y_{1:t-1}) = \iint \delta(x_t - F_\theta(x_{t-1}, \varepsilon_t))\, p(\varepsilon_t)\, p_\theta(x_{t-1} \mid y_{1:t-1})\, d\varepsilon_t\, dx_{t-1}.$$

In a perturbation solution, the same economics may also be represented by a compact law of motion

$$x_t = h_x x_{t-1} + \eta \varepsilon_t$$

at first order, with zero or rank-deficient innovation rows for the deterministic-completion block. The compact representation is useful, but it must not erase the timing contract or the structural metadata described in Chapter 4. In nonlinear filtering, the correct random input is therefore the innovation or declared stochastic block. The endogenous block is then generated by the model-implied deterministic-completion map. Here "deterministic" means no independent innovation in that coordinate once (x_{t-1}, ε_t) is fixed; it does not mean zero predictive variance conditional only on the lagged filtering state.

This is the distinction emphasized in the DSGE filtering literature. It is also an instance of the generic BayesFilter structural-transition contract from Chapter 2: the backend integrates over the declared stochastic block and completes the deterministic block pointwise. Standard Bayesian DSGE expositions write the likelihood in state-space form after the model solution has supplied a transition law for predetermined variables and an observation equation for measured macroeconomic series [Herbst and Schorfheide, 2015, An and Schorfheide, 2007]. Pruned perturbation methods make the same point in a different language: the first-order component is propagated stochastically, while the higher-order correction is propagated by a separate deterministic recursion driven by the lower-order state [Kim et al., 2008, Andreasen et al., 2018]. The filtering backend must honor the state-space law supplied by the structural solution, not merely the dimension of the state vector.

19.1.1 Literature Map for This Chapter

The chapter uses four strands of literature, and each supports a different claim.

(i) **State-space likelihoods and DSGE solutions.** Standard state-space and Bayesian DSGE references provide the filtering likelihood setup and the distinction between a structural model solution and the numerical recursion used to evaluate it [Durbin and Koopman, 2012, Herbst and Schorfheide, 2015, An and Schorfheide, 2007].

(ii) **Pruned nonlinear DSGE state spaces.** Pruning papers show that nonlinear DSGE approximations naturally separate lower-order stochastic propagation from higher-order deterministic correction recursions [Kim et al., 2008, Andreasen et al., 2018]. That literature motivates the structural partition used here, but it does not by itself choose a sigma-point implementation.

(iii) **Unscented and sigma-point filtering.** Julier and Uhlmann justify deterministic sigma points as a moment-matching approximation for nonlinear transformations, while van der Merwe's sigma-point Kalman-filter treatment supplies the dynamic state-space notation and augmented-noise filtering pattern used below [Julier and Uhlmann, 1996, 1997, van der Merwe, 2004].

(iv) **Particle and factor backends.** Particle-filter references support the same law-level distinction in a simulation backend: sample the declared transition and weight by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Square-root and SVD chapters in this monograph address numerical factorization and derivative risk; they do not replace the structural-law requirement.

The contribution of this chapter is to connect these strands: in a mixed DSGE transition, sigma-point, SVD sigma-point, cubature, and particle methods should approximate the pushforward law generated by the structural map $F_\theta(x_{t-1}, \varepsilon_t)$.

19.2 Why Linear Kalman Filtering Can Hide the Problem

For a linear Gaussian state-space model,

$$x_t = c + Fx_{t-1} + u_t, \qquad u_t \sim \mathcal{N}(0, Q), \tag{19.5}$$

$$y_t = a + Hx_t + e_t, \qquad e_t \sim \mathcal{N}(0, R), \tag{19.6}$$

the Kalman filter only needs the conditional mean and covariance of $x_t \mid x_{t-1}$. If a DSGE solution implies a singular or nearly singular Q, the exact linear Gaussian likelihood remains well defined when the collapsed representation preserves the same conditional law induced by the structural transition. Numerically stable implementations may then handle that degenerate conditional Gaussian law through diffuse, square-root, solve-form, or carefully regularized recursions as appropriate [Durbin and Koopman, 2012].

This is why the following collapsed first-order representation can be acceptable in an exact Kalman path:

$$Q = \eta\eta^\top, \qquad P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q.$$

The key point is not that structural determinism disappears. The key point is that the collapsed linear representation still encodes the same one-step conditional Gaussian law. Deterministic-completion coordinates are not wrong merely because they are included in x_t. They obtain randomness through their dependence on lagged stochastic coordinates, and their direct innovation variance may be zero. The exact linear Kalman recursion uses only the correct first two conditional moments.

The danger is that this success can train the implementation culture into a bad habit: it makes the state vector look like a homogeneous Gaussian vector. That habit becomes

wrong when the backend changes from exact linear filtering to nonlinear sigma-point or particle filtering. The relevant design decision is then the integration space recorded in the filter metadata, not merely the numerical dimension of x_t.

A related misunderstanding should be removed explicitly. A deterministic- completion coordinate need not have zero one-step predictive variance given the lagged state. It can inherit randomness through the current stochastic block. In the Chapter 2 AR(2) lag-stack example, the lag coordinate has no new innovation of its own but still remains random under the lagged filtering distribution. Likewise, in DSGE models with (19.1)–(19.2), k_t may satisfy

$$\mathrm{Var}(k_t \mid x_{t-1}) > 0$$

even though k_t has no independent innovation once (x_{t-1}, ε_t) is fixed.

19.3 Why Nonlinear Filters Must Respect the Structural Map

Sigma-point and particle filters approximate a transition law, not just a matrix covariance update. In a generic nonlinear state-space model,

$$x_t = f_\theta(x_{t-1}, \varepsilon_t), \qquad y_t = g_\theta(x_t, e_t),$$

the filter has to decide which variables are integrated over and which are deterministic transforms. In BayesFilter language, this is the choice of integration space recorded in the filter metadata. For DSGE models with (19.1)–(19.2), the natural nonlinear prediction step is:

(i) represent the filtering distribution of the lagged structural state;

(ii) draw, integrate, or choose quadrature points for the exogenous innovation ε_t or current exogenous state m_t;

(iii) compute each corresponding endogenous state k_t using $T_k(k_{t-1}, m_{t-1}, m_t; \theta)$;

(iv) evaluate the observation map on the resulting structurally valid points.

The endogenous coordinates should not receive arbitrary independent "sigma-point noise" merely because they are entries of x_t. If a sigma point moves an accounting identity or predetermined endogenous state outside the model-implied support of the structural transition, the filter is no longer approximating the intended DSGE state-space model. It is approximating a different artificial model relative to the declared structural law. This is exactly the approximation boundary from Chapter 16: quadrature may approximate the target law, but injecting additional stochastic coordinates changes the target unless that change is declared and justified.

Particle-filter literature makes the same distinction operationally. A particle filter propagates particles through the state transition and weights them by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Written in the structural notation, a basic propagation and weighting step is

$$x_t^{(i)} = F_\theta\left(x_{t-1}^{(a_i)}, \varepsilon_t^{(i)}\right), \qquad w_t^{(i)} \propto p_\theta\left(y_t \mid x_t^{(i)}\right).$$

When part of the transition is deterministic, particles should be propagated by sampling the declared stochastic block and then deterministically completing the remaining coordinates. Adding artificial noise to deterministic coordinates can be used only as a declared proposal or regularization device, with the proposal/target correction and approximation status stated explicitly. It is not the default filtering law.

19.4 Reusable BayesFilter Structural Filtering Step

The doctrine above can be stated as a reusable backend contract. At every time step, the filter should treat the structural transition as the object being approximated:

$$x_t = F_\theta(x_{t-1}, \varepsilon_t), \qquad y_t \sim p_\theta(y_t \mid x_t).$$

The filtered state is still the full state x_t. The integration variable is the pre-transition uncertainty that generates the predictive law.

Algorithm: `structural_filter_step`.

(1) Start from the filtered approximation to $p_\theta(x_{t-1} \mid y_{1:t-1})$ and the declared innovation law $p_\theta(\varepsilon_t)$.

(2) Form sigma points, quadrature points, particles, or linearization points in the declared integration space: (x_{t-1}, ε_t), $(s_{t-1}, d_{t-1}, \varepsilon_t)$, or an equivalent stochastic-state parameterization.

(3) For each point, call the structural transition map:

$$s_t^{(j)} = T_s(s_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta), \qquad d_t^{(j)} = T_d(d_{t-1}^{(j)}, s_{t-1}^{(j)}, s_t^{(j)}; \theta).$$

Deterministic-completion coordinates are outputs of this map, not independent new stochastic coordinates.

(4) Pack $x_t^{(j)} = \mathrm{pack}(s_t^{(j)}, d_t^{(j)}, a_t^{(j)})$ and evaluate the observation map or density on these structurally valid points.

(5) Apply the selected update: exact Kalman, Gaussian sigma-point closure, particle weighting/resampling, or another declared backend.

(6) Return the likelihood contribution, filtered approximation, and metadata recording integration space, deterministic-completion policy, approximation label, derivative policy, and finite-failure diagnostics.

This algorithm is intentionally backend-neutral. For a linear Gaussian Kalman backend, the structural transition may collapse to matrices (F, Q) as long as the same conditional Gaussian law is preserved. For UKF, CKF, SVD sigma-point, and particle backends, the pointwise structural transition is the default. A mixed-model full-state Gaussian approximation may still be offered as a diagnostic or legacy path, but it must carry an approximation label and must be tested against structural references.

19.5 Degenerate Transitions Are Structural

Degenerate transition covariance is not an exceptional corner case in DSGE models. It is often the mathematical expression of timing, identities, and endogenous accumulation. Capital, debt, lagged output growth, risk-premium terms, and other predetermined variables may be deterministic conditional on the previous state and current shocks. A full covariance matrix with small positive diagonal padding can keep linear algebra routines alive, but it does not by itself define the intended economic transition. BayesFilter therefore separates three ideas that are often confused in practice:

(i) an exact degenerate transition law that already matches the structural model;

(ii) numerical regularization used to evaluate that law stably; and

(iii) an altered transition law that injects new stochastic degrees of freedom into deterministic-completion coordinates.

Only the first two preserve the original structural model automatically.

The implementation policy is therefore:

- distinguish structural determinism from numerical regularization;
- never let a nugget or PSD projection change the declared stochastic dimension without a written approximation label;
- expose deterministic blocks in diagnostics, rather than burying them under a full-rank covariance;
- test degenerate and rank-deficient transition cases directly.

This point is separate from the SVD spectral-gradient issue in Chapter 18. SVD factors may improve value-side robustness for nearly singular covariances, and an SVD sigma-point filter can be the right numerical backend for a structurally declared sigma-point rule. But SVD is a factorization choice, not a state-law declaration. It repairs the reviewer's degeneracy concern only if the sigma-point backend factors the covariance of the correct integration variable, such as (x_{t-1}, ε_t), and then completes deterministic coordinates pointwise. If the backend first invents a full-rank post-transition covariance for deterministic-completion coordinates, an SVD factor merely represents that altered law more stably.

19.6 Standard UKF, Structural UKF, and the Predictive Law

19.6.1 The Original Unscented-Transform Pattern

Julier and Uhlmann's original unscented-transform construction starts from a random variable $X \in \mathbb{R}^L$ with mean $\bar{x}$ and covariance P_x and approximates the moments of $Y = g(X)$ by propagating deterministic sigma points through g [Julier and Uhlmann, 1996, 1997]. The (α, β, κ) scaled rule used for the formulas below belongs to the later

sigma-point Kalman filter convention [van der Merwe, 2004]. In the notation used here, one convenient scaled UKF rule forms

$$\chi_0 = \bar{x}, \tag{19.7}$$

$$\chi_i = \bar{x} + \left[\sqrt{(L+\lambda)P_x}\right]_i, \qquad i = 1, \ldots, L, \tag{19.8}$$

$$\chi_{i+L} = \bar{x} - \left[\sqrt{(L+\lambda)P_x}\right]_i, \qquad i = 1, \ldots, L, \tag{19.9}$$

where $[\sqrt{(L+\lambda)P_x}]_i$ denotes the ith column or row of the chosen matrix square root. With

$$\lambda = \alpha^2(L+\kappa) - L,$$

the usual scaled weights are

$$w_0^{(m)} = \frac{\lambda}{L+\lambda}, \qquad w_0^{(c)} = \frac{\lambda}{L+\lambda} + 1 - \alpha^2 + \beta, \tag{19.10}$$

$$w_i^{(m)} = w_i^{(c)} = \frac{1}{2(L+\lambda)}, \qquad i = 1, \ldots, 2L. \tag{19.11}$$

The propagated points are $\upsilon_i = g(\chi_i)$, and the transformed moments are approximated by

$$\bar{y} \approx \sum_{i=0}^{2L} w_i^{(m)} \upsilon_i, \tag{19.12}$$

$$P_y \approx \sum_{i=0}^{2L} w_i^{(c)} (\upsilon_i - \bar{y})(\upsilon_i - \bar{y})^\top. \tag{19.13}$$

The early unscented-transform report analyzes this construction by Taylor expansion: the method matches the input mean and covariance and pushes the resulting deterministic ensemble through the nonlinear map, yielding lower-order moment accuracy that improves on first-order linearization under the smoothness conditions stated there [Julier and Uhlmann, 1996]. Van der Merwe's sigma-point Kalman filter treatment uses the same family logic: sigma points are selected to capture the important first and second moments of the prior Gaussian random variable and then weighted after nonlinear propagation [van der Merwe, 2004].

This chapter uses that scaled UKF convention for exposition. Later UKF, CDKF, and square-root variants differ in point placement, weights, and factorization details, but the structural question is invariant: which random variable is the sigma-point rule approximating? If the wrong variable is chosen, the transform may approximate the wrong law accurately.

In the additive-noise filtering setting, the unscented-transform idea is usually applied to an augmented Gaussian variable containing the prior state and current additive-noise variables. The sigma points therefore approximate the variables that generate the next predicted state and predicted observation. For mixed structural DSGE transitions, that same principle is exactly why the sigma-point variable is (x_{t-1}, ε_t) or an equivalent pre-transition parameterization, not an artificially perturbed post-transition full state.

19.6.2 Standard Additive-Noise UKF Algorithm

To avoid confusion, write the additive-noise UKF pattern explicitly. Suppose the model is declared as

$$x_t = f_\theta(x_{t-1}, u_t), \qquad y_t = h_\theta(x_t, e_t),$$

with independent Gaussian state innovation $u_t \sim \mathcal{N}(0, Q_t)$ and measurement innovation $e_t \sim \mathcal{N}(0, R_t)$. Given a previous Gaussian filtering approximation, one additive-noise UKF step is:

U1. Inputs. Start from

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(m_{t-1|t-1}, P_{t-1|t-1}).$$

The implementation receives f_θ, h_θ, Q_t, R_t, the observed value y_t, sigma-point parameters (α, β, κ), and a declared covariance-factorization policy.

U2. Augmented variable. Form

$$b_t = (x_{t-1}, u_t, e_t), \qquad b_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_b, P_b),$$

with

$$\mu_b = (m_{t-1|t-1}, 0, 0), \qquad P_b = \text{blockdiag}(P_{t-1|t-1}, Q_t, R_t).$$

If a variant handles measurement noise by adding R_t after observation propagation, omit e_t from b_t and add R_t in step U6.

U3. Sigma points. Generate $\{b_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_b}$ from (μ_b, P_b) using (19.9)–(19.11). Write $b_t^{(j)} = (x_{t-1}^{(j)}, u_t^{(j)}, e_t^{(j)})$.

U4. State propagation. Propagate

$$x_t^{(j)} = f_\theta(x_{t-1}^{(j)}, u_t^{(j)}).$$

Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \qquad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

U5. Observation propagation. Form

$$z_t^{(j)} = h_\theta(x_t^{(j)}, e_t^{(j)})$$

or $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ with R_t added below, depending on the chosen variant.

U6. Innovation moments. Compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.14}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t^{\text{add}}, \tag{19.15}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top, \tag{19.16}$$

where $R_t^{\text{add}} = 0$ if measurement noise was included in b_t and $R_t^{\text{add}} = R_t$ otherwise.

U7. Gaussian update and likelihood. With $v_t = y_t - \hat{z}_t$,

$$K_t = P_{xz,t} S_t^{-1}, \tag{19.17}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \tag{19.18}$$
$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.19}$$
$$\ell_t = -\frac{1}{2}\left[d_y \log(2\pi) + \log|S_t| + v_t^\top S_t^{-1} v_t\right]. \tag{19.20}$$

U8. Return metadata. Return $(\ell_t, \hat{x}_{t|t}, P_{t|t})$ with metadata recording the augmented variable b_t, the factorization method, sigma-point parameters, whether measurement noise was augmented or added, and any PSD/jitter/finiteness interventions.

This is the textbook pattern many readers have in mind. It is perfectly natural when the declared state transition itself is an additive-noise model on the full state.

19.6.3 Structural UKF Algorithm

For a mixed structural model, the update algebra remains the same, but the sigma-point variable is chosen differently because the predictive law is generated differently. For the structural transition (19.1)–(19.2), a structural UKF step is:

S1. Inputs. Start from the filtering approximation

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}).$$

The implementation receives T_m, T_k, the observation map h_θ, the innovation law $\varepsilon_t \sim \mathcal{N}(0, \Sigma_{\varepsilon,t})$, observation covariance R_t, the observed value y_t, sigma-point parameters, and a factorization policy.

S2. Pre-transition variable. Form

$$a_t = (x_{t-1}, \varepsilon_t), \qquad a_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_a, P_a),$$

with

$$\mu_a = (\mu_{t-1}, 0), \qquad P_a = \mathrm{blockdiag}(P_{t-1}, \Sigma_{\varepsilon,t}).$$

This is the sigma-point variable because $\pi^x_{t|t-1} = (F_\theta)_\# \pi^a_t$.

S3. Sigma points. Generate $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_a}$ from (μ_a, P_a). Write $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$.

S4. Stochastic-block propagation. For each point compute

$$m_t^{(j)} = T_m(m_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta).$$

S5. Deterministic completion. For the same point compute

$$k_t^{(j)} = T_k(k_{t-1}^{(j)}, m_{t-1}^{(j)}, m_t^{(j)}; \theta), \qquad x_t^{(j)} = (m_t^{(j)}, k_t^{(j)}).$$

There is no independent sigma-point coordinate for k_t unless the model declares one.

S6. State moments. Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \qquad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

S7. Observation points and moments. Evaluate $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ or the appropriate noise-augmented observation variant, then compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.21}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \tag{19.22}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top. \tag{19.23}$$

S8. Gaussian update and likelihood. Apply exactly the update from U7:

$$v_t = y_t - \hat{z}_t, \qquad K_t = P_{xz,t} S_t^{-1}, \qquad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t,$$

with $P_{t|t} = P_{xx,t} - K_t S_t K_t^\top$ and the same Gaussian innovation log-likelihood formula.

S9. Return metadata. Return the likelihood contribution and filtered Gaussian approximation with metadata recording `integration_space` equal to (x_{t-1}, ε_t), `deterministic_completion` equal to `pointwise`, structural identity diagnostics, sigma-point parameters, factorization method, and any approximation label.

Structural UKF is not a different Kalman-gain formula. It is the same Gaussian update algebra applied to a predictive sigma-point cloud that respects the structural transition law.

19.6.4 UKF in Sequential-Monte-Carlo Kernel Notation

The same point can be expressed in the probability-kernel notation used in Chopin and Papaspiliopoulos [2020]. A state-space model is specified by an initial law $P_{\theta,0}(dx_0)$, transition kernels $P_{\theta,t}(x_{t-1}, dx_t)$, and observation densities $f_{\theta,t}(y_t \mid x_t)$. In the bootstrap Feynman–Kac formalism, the transition kernel is

$$M_{\theta,t}(x_{t-1}, dx_t) = P_{\theta,t}(x_{t-1}, dx_t),$$

and the potential is

$$G_{\theta,t}(x_t) = f_{\theta,t}(y_t \mid x_t).$$

If η_{t-1} denotes the filtering law of x_{t-1} given $y_{1:t-1}$, the exact one-step recursion is

$$\xi_t(dx_t) = \int \eta_{t-1}(dx_{t-1}) P_{\theta,t}(x_{t-1}, dx_t), \tag{19.24}$$

$$\lambda_t = \xi_t(G_{\theta,t}) = \int G_{\theta,t}(x_t)\, \xi_t(dx_t), \tag{19.25}$$

$$\eta_t(dx_t) = \frac{G_{\theta,t}(x_t)\xi_t(dx_t)}{\xi_t(G_{\theta,t})}. \tag{19.26}$$

Equations (19.24)–(19.26) are the filtering target. A particle filter approximates the measures in these equations by random weighted particles. A UKF instead approximates the same prediction/update recursion by deterministic quadrature and Gaussian projection.

For the mixed structural transition in (19.4), the transition kernel is the pushforward kernel

$$P^{\mathrm{str}}_{\theta,t}(x,B) = \int \mathbf{1}\{F_\theta(x,\varepsilon)\in B\}\,\nu^{\varepsilon}_{\theta,t}(d\varepsilon) = \int \delta_{F_\theta(x,\varepsilon)}(B)\nu^{\varepsilon}_{\theta,t}(d\varepsilon), \tag{19.27}$$

for measurable state sets B. This is a perfectly valid probability kernel even when it is singular with respect to Lebesgue measure. Kernel notation is therefore useful here: it does not force a degenerate structural transition to pretend that it has a full-dimensional transition density. The predictive integral in (19.24) becomes, for any test function ψ,

$$\xi^{\mathrm{str}}_t(\psi) = \iint \psi(F_\theta(x_{t-1},\varepsilon_t))\,\eta_{t-1}(dx_{t-1})\nu^{\varepsilon}_{\theta,t}(d\varepsilon_t). \tag{19.28}$$

Thus the kernel version of the structural statement is:

$$P_{\theta,t} = P^{\mathrm{str}}_{\theta,t}, \qquad M_{\theta,t} = P^{\mathrm{str}}_{\theta,t}, \qquad G_{\theta,t}(x) = f_{\theta,t}(y_t \mid x).$$

Now suppose the incoming filter is approximated by a Gaussian $\widehat{\eta}_{t-1} = \mathcal{N}(\widehat{m}_{t-1}, \widehat{P}_{t-1})$. Let

$$\widehat{\alpha}_t(da) = \widehat{\eta}_{t-1}(dx_{t-1})\nu^{\varepsilon}_{\theta,t}(d\varepsilon_t), \qquad a = (x_{t-1},\varepsilon_t).$$

The structural UKF replaces integrals against $\widehat{\alpha}_t$ by the deterministic sigma-point rule

$$\mathcal{U}^{(r)}_t[\psi] = \sum_j w^{(r)}_j \psi(a^{(j)}_t), \qquad r\in\{m,c\}.$$

With

$$x^{(j)}_t = F_\theta(a^{(j)}_t), \qquad z^{(j)}_t = h_\theta(x^{(j)}_t),$$

the UKF predictive and observation moments are

$$\widehat{m}^-_t = \sum_j w^{(m)}_j x^{(j)}_t, \tag{19.29}$$

$$\widehat{P}^-_t = \sum_j w^{(c)}_j (x^{(j)}_t - \widehat{m}^-_t)(x^{(j)}_t - \widehat{m}^-_t)^\top, \tag{19.30}$$

$$\widehat{z}_t = \sum_j w^{(m)}_j z^{(j)}_t, \tag{19.31}$$

$$\widehat{S}_t = \sum_j w^{(c)}_j (z^{(j)}_t - \widehat{z}_t)(z^{(j)}_t - \widehat{z}_t)^\top + R_t, \tag{19.32}$$

$$\widehat{P}_{xz,t} = \sum_j w^{(c)}_j (x^{(j)}_t - \widehat{m}^-_t)(z^{(j)}_t - \widehat{z}_t)^\top. \tag{19.33}$$

Equivalently, UKF builds the Gaussian joint approximation

$$\widehat{J}_t(dx_t,dy_t) = \mathcal{N}\left(\begin{bmatrix}x_t\\ y_t\end{bmatrix}; \begin{bmatrix}\widehat{m}^-_t\\ \widehat{z}_t\end{bmatrix}, \begin{bmatrix}\widehat{P}^-_t & \widehat{P}_{xz,t}\\ \widehat{P}^\top_{xz,t} & \widehat{S}_t\end{bmatrix}\right) dx_t\,dy_t. \tag{19.34}$$

The UKF likelihood factor and update are then the marginal and conditional Gaussian objects induced by (19.34):

$$\widehat{\lambda}_t^{\mathrm{UKF}} = \varphi_{d_y}(y_t; \widehat{z}_t, \widehat{S}_t), \tag{19.35}$$

$$\begin{aligned}\widehat{\eta}_t^{\mathrm{UKF}}(dx_t) &= \widehat{J}_t(dx_t \mid y_t) \\ &= \mathcal{N}\Big(dx_t;\, \widehat{m}_t^- + \widehat{P}_{xz,t}\widehat{S}_t^{-1}(y_t - \widehat{z}_t),\, \widehat{P}_t^- - \widehat{P}_{xz,t}\widehat{S}_t^{-1}\widehat{P}_{xz,t}^\top\Big).\end{aligned} \tag{19.36}$$

The corresponding contribution to the log likelihood is $\log \widehat{\lambda}_t^{\mathrm{UKF}}$. If measurement noise is augmented rather than added, replace $a = (x_{t-1}, \varepsilon_t)$ by $a = (x_{t-1}, \varepsilon_t, e_t)$, set $z_t^{(j)} = h_\theta(F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)}), e_t^{(j)})$, and omit the extra R_t term in (19.33).

This notation also states the equivalence condition for collapsed SVD constructions. If a full-state degenerate SVD implementation uses another kernel

$$\widetilde{P}_{\theta,t}(x, B) = \int \delta_{\widetilde{F}_\theta(x,\eta)}(B)\widetilde{\nu}_{\theta,t}(d\eta),$$

then it is a law-equivalent implementation of the structural model only when

$$\widetilde{P}_{\theta,t}(x, B) = P_{\theta,t}^{\mathrm{str}}(x, B) \qquad \text{for the relevant } x \text{ and measurable } B, \tag{19.37}$$

or at least when the two kernels induce the same predictive measure from the current filtering approximation. Equality of a covariance matrix, or existence of an SVD factor for a singular covariance, is not by itself equality of probability kernels. This is the kernel-notation version of the structural warning in this chapter.

19.6.5 Why These Algorithms Differ

Object	**Standard additive-noise UKF**	**Structural UKF**
Filtered object	Full state x_t	Full state x_t
Sigma-point variable	Declared augmented additive-noise variable (x_{t-1}, u_t, e_t)	Pre-transition structural variable (x_{t-1}, ε_t)
Direct process noise	Whatever the additive model declares	Only the declared stochastic block
Deterministic coordinates	None unless constraints are declared	Completed by T_k pointwise
Target law	Additive-noise predictive law	Structural pushforward law
Failure if misused	Moment approximation error	Model-law error plus moment approximation error
SVD/square-root role	Covariance factorization	Covariance factorization, not structural repair

The update equations are shared. The exact difference is upstream: which random variable receives sigma points and which map generates the predictive state cloud.

19.6.6 Why the Sigma-Point Variable Uses Pre-Transition Uncertainty

Proposition 19.1 (Predictive pushforward and sigma-point space)**.** *Under the structural transition*

$$m_t = T_m(m_{t-1}, \varepsilon_t), \tag{19.38}$$
$$k_t = T_k(k_{t-1}, m_{t-1}, m_t), \tag{19.39}$$
$$x_t = (m_t, k_t), \tag{19.40}$$

with innovation law $p(\varepsilon_t)$ independent of $x_{t-1} \mid y_{1:t-1}$, the following hold:

(i) the one-step predictive law $p(x_t \mid y_{1:t-1})$ is the pushforward of the joint law of (x_{t-1}, ε_t) under the structural map;

(ii) a UKF that approximates this predictive law should place sigma points on (x_{t-1}, ε_t), or on an equivalent parameterization of the same pre-transition uncertainty;

(iii) a naive full-state additive-noise UKF targets the same predictive law only if its induced predictive law coincides with that pushforward law; if it adds a direct perturbation to k_t, it targets a different law.

Proof. Define the structural map

$$F(x_{t-1}, \varepsilon_t) = \big(T_m(m_{t-1}, \varepsilon_t), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t))\big). \tag{19.41}$$

Because the current innovation is independent of the lagged filtering law, the joint pre-transition uncertainty factorizes as

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t). \tag{19.42}$$

Let φ be any bounded test function on the state space of x_t. Then

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.43}$$

Equation (19.43) is exactly the expectation of φ under the pushforward of the joint law of (x_{t-1}, ε_t) through F, which proves part (i).

Moreover, under the structural transition,

$$x_t = F(x_{t-1}, \varepsilon_t) \qquad \text{almost surely.} \tag{19.44}$$

So once (x_{t-1}, ε_t) is fixed, both m_t and k_t are fixed. There is no independent new perturbation attached to k_t under the intended model. Therefore any sigma-point rule intended to approximate $p(x_t \mid y_{1:t-1})$ should be applied to the pre-transition uncertainty variables that generate that law, namely (x_{t-1}, ε_t) or an exactly equivalent parameterization. This proves part (ii).

Finally, if a full-state additive-noise UKF omits ε_t, then it misses genuine current-shock uncertainty. If it instead adds a direct perturbation to k_t, then it enlarges the stochastic space and defines a different predictive law unless that perturbation is degenerate in exactly the way required to reproduce the same pushforward distribution in (19.43). This proves part (iii). □

The proposition does not say that the filter stops targeting x_t. The filtered state remains x_t. The point is that the sigma-point variable is the pre-transition uncertainty whose pushforward generates the predictive law of x_t. This is analogous to process-noise augmentation in a standard additive- noise UKF, but in the mixed structural setting the distinction is sharper: ε_t is a genuine new source of uncertainty, while k_t is a structural completion of that uncertainty rather than an independently shocked coordinate.

Proposition 19.2 (Local structural UKF Taylor accuracy)**.** *Let $A_t = (x_{t-1}, \varepsilon_t)$ have a Gaussian approximation with mean μ_a and covariance P_a, and let $X_t = F_\theta(A_t)$ where F_θ is the structural map in (19.4). Write $P_a = \eta^2 \Sigma_a$, with Σ_a fixed and η measuring the local scale of the input uncertainty. Suppose the sigma-point deviations $\xi_j = a_t^{(j)} - \mu_a$ satisfy*

$$\sum_j w_j^{(m)} \xi_j = 0, \qquad \sum_j w_j^{(m)} \xi_j \xi_j^\top = P_a, \qquad \sum_j w_j^{(c)} \xi_j \xi_j^\top = P_a,$$

and suppose F_θ is three-times continuously differentiable on a neighborhood containing the relevant input mass and sigma points. Let J be the Jacobian of F_θ at μ_a, and let H_r be the Hessian of the rth component of F_θ at μ_a. Then the structural UKF prediction matches the transformed mean through the quadratic Taylor term:

$$\mathbb{E}[X_{t,r} \mid y_{1:t-1}] = F_{\theta,r}(\mu_a) + \frac{1}{2} \operatorname{tr}(H_r P_a) + O(\eta^3), \tag{19.45}$$

$$\hat{x}^{\mathrm{UKF}}_{t|t-1,r} = F_{\theta,r}(\mu_a) + \frac{1}{2} \operatorname{tr}(H_r P_a) + O(\eta^3). \tag{19.46}$$

It also matches the leading second-order covariance term:

$$\operatorname{Cov}(X_t \mid y_{1:t-1}) = J P_a J^\top + O(\eta^3), \tag{19.47}$$

$$P^{\mathrm{UKF}}_{xx,t} = J P_a J^\top + O(\eta^3). \tag{19.48}$$

Thus, under the local small-uncertainty Taylor expansion used here, the structural UKF matches the transformed mean through the quadratic term and matches the leading covariance term for the Gaussian approximation on the correct input law (x_{t-1}, ε_t). This is a local expansion statement for the selected structural input law, not a blanket exactness theorem for nonlinear structural transitions. It does not apply to a naive post-transition full-state sigma-point cloud unless that cloud induces the same law as $(F_\theta)_\# \pi_t^a$.

Proof. By Proposition 19.1, the structural predictive law of x_t is the law of $F_\theta(A_t)$. The structural UKF therefore applies the unscented-transform rule to the transformation $g = F_\theta$ and the input variable A_t.

For a component r, Taylor expansion around μ_a gives

$$F_{\theta,r}(\mu_a + \delta) = F_{\theta,r}(\mu_a) + J_r \delta + \frac{1}{2} \delta^\top H_r \delta + O(\|\delta\|^3).$$

Taking expectation under the Gaussian approximation for $A_t - \mu_a$ yields the first line of (19.46), because $\mathbb{E}[A_t - \mu_a] = 0$ and $\mathbb{E}[(A_t - \mu_a)(A_t - \mu_a)^\top] = P_a$. Taking the weighted sum over sigma points gives the second line of (19.46) by the same first- and second-moment identities for ξ_j.

For covariance, subtract the corresponding mean expansion. The leading centered term is $J(A_t - \mu_a)$ for the true law and $J\xi_j$ for the sigma points. Their second moments

are both $JP_aJ^\top$ by moment matching. The quadratic Taylor terms have size $O(\eta^2)$, so their covariance contribution with the leading linear term is $O(\eta^3)$ and their self-covariance contribution is $O(\eta^4)$. This proves (19.48). These are the local unscented-transform moment calculations for the selected sigma-point rule [Julier and Uhlmann, 1996, van der Merwe, 2004], applied to the structural input variable. If a naive full-state cloud is constructed from a different input law, the same UT calculation applies only to that different law; it gives no accuracy guarantee for the structural pushforward target. □

19.7 Worked Example A: Nonlinear Structural Transition

Let

$$m_t = \rho m_{t-1} + \sigma\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \tag{19.49}$$

$$k_t = \phi k_{t-1} + \gamma m_t^2, \tag{19.50}$$

$$y_t = m_t + k_t + e_t, \qquad e_t \sim \mathcal{N}(0,R). \tag{19.51}$$

The state is $x_t = (m_t, k_t)$, but only the innovation ε_t is new stochastic input at time t. Conditional on $(m_{t-1}, k_{t-1}, \varepsilon_t)$, the coordinate k_t is deterministic.

Suppose the previous filtering distribution is Gaussian:

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}), \qquad \mu_{t-1} = \begin{bmatrix} \bar{m} \\ \bar{k} \end{bmatrix}.$$

The structural UKF should augment the previous state with the innovation. This step needs to be motivated carefully, because it is exactly where the structural UKF departs from the naive full-state construction. The point is not to pad the state arbitrarily. The point is to place sigma points on the variables that actually generate the one-step uncertainty in the structural transition. In this example, the predictive law is generated by the joint uncertainty in the lagged state (m_{t-1}, k_{t-1}) and the current shock ε_t. Once those three quantities are fixed, both m_t and k_t are fixed by the structural map.

That is why the augmented variable is

$$a_t = (x_{t-1}, \varepsilon_t), \tag{19.52}$$

rather than an artificially padded post-transition full state. Standard additive-noise UKF presentations often augment with process noise for the same reason: sigma points should live on the pre-transition uncertainty variables. For a mixed structural model, however, that principle has a sharper consequence. The shock ε_t is a genuine new source of uncertainty, while k_t is not. If the augmentation omitted ε_t, the propagated sigma points would miss current shock uncertainty. If the augmentation instead added a direct perturbation for k_t, the sigma points would describe a different transition law from the one declared by the model.

So the structural UKF should augment the previous state with the innovation. This places sigma points in the same pre-transition uncertainty variables that define the predictive law, rather than in an artificially padded post-transition full state:

$$a_t = \begin{bmatrix} m_{t-1} \\ k_{t-1} \\ \varepsilon_t \end{bmatrix}, \qquad a_t \mid y_{1:t-1} \approx \mathcal{N}\left(\begin{bmatrix} \bar{m} \\ \bar{k} \\ 0 \end{bmatrix}, \begin{bmatrix} P_{t-1} & 0 \\ 0 & 1 \end{bmatrix} \right). \tag{19.53}$$

Let $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2n_a}$ be the usual unscented points and weights for this $n_a = 3$ dimensional augmented Gaussian [Julier and Uhlmann, 1997]. For every point $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$, the structural transition is evaluated as

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \tag{19.54}$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma \left(m_t^{(j)}\right)^2, \tag{19.55}$$

$$x_t^{(j)} = \begin{bmatrix} m_t^{(j)} \\ k_t^{(j)} \end{bmatrix}. \tag{19.56}$$

This is the crucial step: the filter never creates an independent sigma-point perturbation for k_t. It creates sigma points for the previous state and the current innovation, then computes k_t by the structural map (19.50).

The structural and naive full-state UKF constructions now differ at the level that matters for the update. The structural UKF forms sigma points for (x_{t-1}, ε_t), propagates them through the structural transition, and therefore computes moments of the target law declared by the model. A naive full-state UKF instead behaves as if there were an additive-noise model of the form

$$x_t = \tilde{f}(x_{t-1}) + \tilde{u}_t, \qquad \tilde{u}_t \sim \mathcal{N}(0, \tilde{Q}),$$

with a full-state covariance $\tilde{Q}$ that assigns direct new variance to all or most coordinates of x_t. The three failure claims can now be derived in the notation of this toy model.

Prediction-law failure. Under the structural model,

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = \phi k_{t-1} + \gamma m_t^2.$$

So conditional on (m_{t-1}, k_{t-1}), the one-step predictive law is induced by a single scalar shock ε_t. Eliminating the shock-driven update for m_t gives the structural identity

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0. \tag{19.57}$$

Every propagated sigma point in the structural UKF satisfies this identity, so its sigma-point cloud lies on the support of the structural predictive law. A naive full-state UKF instead propagates points

$$\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$$

from an additive full-state covariance model. In general those points satisfy

$$\tilde{k}_t^{(j)} - \phi k_{t-1}^{(j)} - \gamma \big(\tilde{m}_t^{(j)}\big)^2 \neq 0,$$

so the naive sigma-point cloud approximates moments of a different one-step law. That is the prediction-law failure.

Cross-covariance failure. For the observation equation $z_t = m_t + k_t$, the structural UKF computes

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} \big(x_t^{(j)} - \hat{x}_{t|t-1}\big)\big(z_t^{(j)} - \hat{z}_t\big),$$

with

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}, \qquad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma\big(m_t^{(j)}\big)^2.$$

Hence all variation in the second state coordinate enters through lagged-state variation and the common shock channel already represented in $m_t^{(j)}$. Under the naive full-state UKF,

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} \big(\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}\big)\big(\tilde{z}_t^{(j)} - \tilde{z}_t\big),$$

with

$$\tilde{z}_t^{(j)} = \tilde{m}_t^{(j)} + \tilde{k}_t^{(j)}.$$

Because $\tilde{k}_t^{(j)}$ now contains a direct artificial perturbation channel, the second component of $P_{xz,t}^{\text{naive}}$ contains covariance terms between the observation and that artificial variation. Those terms do not exist in the structural model, where k_t is completed deterministically from the same shock path that drives m_t. That is the cross-covariance failure.

Update-law failure. The familiar UKF update equations,

$$K_t = P_{xz,t} S_t^{-1}, \qquad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \qquad P_{t|t} = P_{t|t-1} - K_t S_t K_t^\top,$$

do not fail algebraically. They fail semantically when $(\hat{x}_{t|t-1}, S_t, P_{xz,t})$ are computed from the wrong predictive law. Once the naive full-state approximation changes either $P_{xz,t}$ or S_t, it changes the gain itself:

$$K_t^{\text{naive}} = P_{xz,t}^{\text{naive}} \left(S_t^{\text{naive}}\right)^{-1} \neq P_{xz,t}^{\text{struct}} \left(S_t^{\text{struct}}\right)^{-1} = K_t^{\text{struct}}$$

in general. The existing numerical comparison later in this example shows this explicitly: adding artificial variance to k_t changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$, which then changes the approximate log-likelihood contribution. The same change propagates into the Kalman gain and posterior update. So the update law is not wrong as linear algebra; it is correctly updating the wrong approximate model.

The predicted state moments are

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \tag{19.58}$$

$$P_{xx,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1}\right)\left(x_t^{(j)} - \hat{x}_{t|t-1}\right)^\top. \tag{19.59}$$

For the observation equation (19.51), define

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}.$$

Then the predicted observation, innovation variance, and cross covariance are

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.60}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)^2 + R, \tag{19.61}$$

$$P_{xz,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1}\right)(z_t^{(j)} - \hat{z}_t). \tag{19.62}$$

The Gaussian measurement update is

$$v_t = y_t - \hat{z}_t, \tag{19.63}$$
$$K_t = P_{xz,t} S_t^{-1}, \tag{19.64}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \tag{19.65}$$
$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.66}$$

and the one-step approximate log likelihood contribution is

$$\ell_t = -\frac{1}{2}\left[\log(2\pi S_t) + \frac{v_t^2}{S_t}\right].$$

19.7.1 Reviewer Edge Case: Does $\phi = 0$ Collapse the Example?

The model does not collapse just because $\phi = 0$. In that case

$$k_t = \gamma m_t^2, \qquad m_t = \rho m_{t-1} + \sigma \varepsilon_t,$$

so k_t is still random whenever $\gamma \neq 0$ and the conditional law of m_t is nondegenerate. What disappears is the inherited lagged-k channel, not the current-shock channel. The support becomes the lower-dimensional manifold

$$\{(m, k) : k = \gamma m^2\},$$

conditional on the parameters and the lagged state distribution. It collapses to a point only in degenerate cases, for example if $\gamma = 0$ or if m_t is itself degenerate. With the numerical calibration below but setting $\phi = 0$, the recomputed structural UKF moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.04247 \end{bmatrix},$$

and the observation moments are

$$\hat{z}_t = 0.11024, \qquad S_t = 0.56807, \qquad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.04247 \end{bmatrix}.$$

The k variance is smaller because lagged k_{t-1} no longer feeds into k_t, but it is not zero: it is induced by the quadratic transformation of the shock-driven m_t.

19.7.2 Numerical Illustration

Use the parameters

$$\rho = 0.8, \quad \sigma = 0.5, \quad \phi = 0.7, \quad \gamma = 0.4, \quad R = 0.25,$$

and previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \qquad P_{t-1} = \mathrm{diag}(0.04, 0.09).$$

Use the scaled unscented convention $\alpha = 1$, $\beta = 2$, and $\kappa = 0$. Since $n_a = 3$, this gives $\lambda = \alpha^2(n_a + \kappa) - n_a = 0$, central mean weight $w_0^{(m)} = 0$, central covariance weight $w_0^{(c)} = 2$, and six off-center mean/covariance weights equal to $1/6$. The augmented covariance is

$$\mathrm{diag}(0.04, 0.09, 1),$$

so $\sqrt{(n_a+\lambda)P_a}$ has diagonal entries 0.34641, 0.51962, and 1.73205. The seven sigma points and their deterministic structural completion are:

j	$m_{t-1}^{(j)}$	$k_{t-1}^{(j)}$	$\varepsilon_t^{(j)}$	$m_t^{(j)}$	$k_t^{(j)}$	$z_t^{(j)}$
0	0	0	0	0	0	0
1	0.34641	0	0	0.27713	0.03072	0.30785
2	0	0.51962	0	0	0.36373	0.36373
3	0	0	1.73205	0.86603	0.30000	1.16603
4	−0.34641	0	0	−0.27713	0.03072	−0.24641
5	0	−0.51962	0	0	−0.36373	−0.36373
6	0	0	−1.73205	−0.86603	0.30000	−0.56603

For example, the third positive innovation point gives $m_t = 0.5(1.73205) = 0.86603$ and $k_t = 0.4(0.86603)^2 = 0.30000$. No row contains an independently sampled "new" shock for k_t.

Using the stated covariance weights, the structural propagation gives

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.08657 \end{bmatrix}.$$

The predicted observation moments are

$$\hat{z}_t = 0.11024, \qquad S_t = 0.61217, \qquad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix}.$$

For an observation $y_t = 0.30$,

$$v_t = 0.18976, \qquad K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix},$$

and therefore

$$\hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix}, \qquad P_{t|t} = \begin{bmatrix} 0.15152 & -0.03897 \\ -0.03897 & 0.07433 \end{bmatrix}.$$

The one-step log likelihood contribution is approximately

$$\ell_t = -0.70297.$$

These numbers are illustrative calculations under the stated sigma-point convention and should be read as a worked example rather than general performance evidence.

Side-by-side prediction/update contrast. For this toy model, the structural UKF treats

$$a_t = (x_{t-1}, \varepsilon_t)$$

as the sigma-point object and computes propagated points by

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \qquad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma\big(m_t^{(j)}\big)^2.$$

It therefore forms

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} \big(x_t^{(j)} - \hat{x}_{t|t-1}\big)\big(z_t^{(j)} - \hat{z}_t\big),$$

where every $x_t^{(j)}$ lies on the support generated by the structural map. A naive full-state UKF instead produces an artificial cloud $\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$ with

$$\tilde{k}_t^{(j)} \neq \phi k_{t-1}^{(j)} + \gamma\big(\tilde{m}_t^{(j)}\big)^2 \text{ in general,}$$

and therefore forms

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} \big(\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}\big)\big(\tilde{z}_t^{(j)} - \tilde{z}_t\big),$$

which mixes genuine shock-driven variation with artificial perturbations of the deterministic-completion coordinate.

Structural identity test. A simple way to see the difference is to check whether propagated points satisfy

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0.$$

The structural UKF satisfies this identity point by point for every propagated sigma point. A naive full-state UKF with direct perturbations to k_t does not. That is the diagnostic place where the structural target and this naive full-state approximation part company.

Now compare this with an off-manifold full-state approximation that adds an independent artificial innovation variance 0.04 directly to k_t after the structural map. Nothing in (19.49)–(19.51) justifies that extra shock. It changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$ and changes the same observation's log likelihood contribution from -0.70297 to -0.73282. The numerical difference is small in this toy calibration, but the conceptual difference is large: the latter filter is approximating a different approximate state-space model relative to the declared structural law.

To connect these numbers back to the three failures above, write the naive full-state approximation as using

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.12657 \end{bmatrix},$$

which differs from the structural covariance only by the artificial extra variance in the k_t coordinate. Under the same observation equation $z_t = m_t + k_t$, this gives

$$\tilde{S}_t = 0.65217, \qquad \tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix},$$

and therefore

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19408 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix},$$

for the same observation $y_t = 0.30$.

These values make the three failures concrete:

(i) **Prediction-law failure:** the structural update uses $P_{xx,t} = \text{diag}(0.27560, 0.08657)$, while the naive full-state approximation uses $\tilde{P}_{xx,t} = \text{diag}(0.27560, 0.12657)$ because it has inserted artificial uncertainty directly into k_t.

(ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix}.$$

The second component is larger only because the naive construction lets the observation load directly on artificial variation in k_t.

(iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix}, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19407 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix}.$$

The update formula has not changed, but the gain and posterior correction do because they are now computed from the wrong predictive moments.

So the numerical illustration does not merely show a slightly different likelihood value. It shows that once the naive approximation changes the predictive covariance and cross covariance, the entire measurement update shifts with it.

The lesson is not that UKF is wrong. The lesson is that the UKF must be applied to the correct structural transition. Sigma points belong in the declared stochastic integration space. Endogenous deterministic coordinates are completed by the model point by point. In BayesFilter terms, this is a design rule for mixed structural models and an approximation-label boundary for any backend that instead enlarges the stochastic space.

19.8 Worked Example B: Degenerate Linear Transition with Nonlinear Measurement

The most practically relevant counterpart to the previous example is not a fully stochastic linear-Gaussian state transition. It is a degenerate linear transition together with a nonlinear measurement equation. That case is closer to many DSGE and affine term-structure applications than a fully nonlinear latent transition. The key point is that the latent transition may still have a structural split even when it is linear. A nonlinear measurement equation does not remove that split.

Consider the linear structural transition

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \tag{19.67}$$

$$k_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t, \tag{19.68}$$

$$x_t = (m_t, k_t), \tag{19.69}$$

and the nonlinear exponential-affine measurement equation

$$y_t = \exp(A^\top x_t + \mu) + e_t, \qquad e_t \sim \mathcal{N}(0, R). \tag{19.70}$$

The latent transition is linear, but it is still structurally degenerate: conditional on (x_{t-1}, ε_t), the endogenous coordinate k_t is completed deterministically from the same shock path that drives m_t. So the structural issue from the previous section does not disappear. What changes is where the nonlinearity appears. The latent transition is linear, while the measurement map is nonlinear.

This means there are now two layers to keep separate:

(i) the latent predictive law is still generated by the pre-transition uncertainty variables (x_{t-1}, ε_t), not by direct full-state perturbations of x_t;

(ii) even after the latent cloud is propagated correctly, the UKF must still approximate nonlinear observation moments and cross covariances for the map $x_t \mapsto \exp(A^\top x_t + \mu)$.

The first issue is structural-modeling correctness. The second is observation- side quadrature quality.

19.8.1 Worked structural derivation for the degenerate linear-transition case

Accepted assumptions. For this section, treat the following as explicit modeling assumptions:

(A1) the innovation is conditionally independent of the lagged filtering law, so that

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t); \tag{19.71}$$

(A2) the latent map, observation map, and composed map defined below are measurable and have the finite moments needed for the stated expectations;

(A3) whenever we refer to the observation-side-only error regime, we assume that the latent predictive law from (19.74) is preserved exactly;

(A4) no invertibility of T_k is required: it is enough that k_t is a well-defined measurable deterministic completion of (k_{t-1}, m_{t-1}, m_t);

(A5) if one wants to replace the innovation variable ε_t by the current exogenous state m_t as the sigma-point variable, then for fixed m_{t-1} the map

$$\varepsilon_t \mapsto T_m(m_{t-1}, \varepsilon_t) \tag{19.72}$$

should be injective, or invertible onto its image with measurable inverse. This extra assumption is only needed for that reparameterization step, not for the pushforward identities proved below.

Latent predictive pushforward. Define the degenerate linear structural map

$$F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t) = \Big(\rho m_{t-1} + \sigma\varepsilon_t,\, \phi k_{t-1} + \psi m_{t-1} + \chi(\rho m_{t-1} + \sigma\varepsilon_t)\Big). \tag{19.73}$$

Then for any bounded test function φ on the latent-state space,

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.74}$$

Hence the latent predictive law is the pushforward of the joint law of (x_{t-1}, ε_t) through F_{lin}.

Derivation. Under the state transition (19.67)–(19.69) and the factorization assumption (19.71), the conditional expectation of any bounded test function $\varphi(x_t)$ is obtained by substituting $x_t = F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t)$ and integrating over the joint law of (x_{t-1}, ε_t). This gives (19.74), which is exactly the pushforward identity.

Deterministic-completion identity. Under (19.67)–(19.68), all admissible latent states satisfy

$$k_t - \phi k_{t-1} - \psi m_{t-1} - \chi m_t = 0. \tag{19.75}$$

Therefore, once (x_{t-1}, ε_t) is fixed, both m_t and k_t are fixed; there is no independent new perturbation attached to k_t in the latent transition.

Derivation. Substitute (19.67) into (19.68) and rearrange terms. The resulting identity is exactly (19.75).

Observation predictive pushforward. Define the nonlinear noise-free observation map

$$h(x_t) = \exp(A^\top x_t + \mu). \tag{19.76}$$

and the composed map

$$G(x_{t-1}, \varepsilon_t) = h(F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t))\,. \tag{19.77}$$

Then for any bounded test function ψ_0 on the noise-free observation space,

$$\mathbb{E}\big[\psi_0\big(h(x_t)\big) \mid y_{1:t-1}\big] = \iint \psi_0(G(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.78}$$

Hence the noise-free observation predictive law is the pushforward of the same pre-transition uncertainty through the composed map.

Derivation. Compose the latent pushforward identity (19.74) with the measurable observation map (19.76). This yields (19.78).

Naive full-state perturbation changes the latent law. If a naive full-state UKF introduces a direct perturbation to the endogenous coordinate k_t, then its propagated latent sigma-point cloud does not, in general, satisfy (19.75) pointwise. Therefore it does not target the latent predictive law characterized by (19.74).

Diagnostic check. By the deterministic-completion identity above, every admissible latent state under the structural model satisfies (19.75). A naive full-state UKF that adds an independent perturbation to k_t creates propagated points whose k_t values are not determined solely by (x_{t-1}, ε_t), so the identity fails in general. Hence the resulting sigma-point cloud cannot represent the same latent predictive law.

Failure propagation and observation-side-only error. Under the preceding worked identities:

(i) if a naive full-state UKF directly perturbs k_t, then prediction-law failure, cross-covariance failure, and update-law failure all still apply;

(ii) if the latent predictive law from (19.74) is preserved exactly and only the transformed observation moments from (19.78) are approximated by quadrature, then the error is not structural-model failure; it is only observation-side quadrature error.

Reason. Part (i) follows because the naive construction changes the latent predictive law, and the nonlinear observation law is the pushforward of that same latent law. Once the latent law changes, the observation moments, state-observation cross covariance, and therefore the UKF update objects also change. Part (ii) follows from Assumption (A3): if the latent law is preserved and only the nonlinear observation moments are approximated, then the remaining error arises only in the observation-side quadrature.

The formal statements above separate three layers that are easy to blur in practice: accepted modeling assumptions, exact pushforward identities, and the interpretation of failure modes. The filtered state is still x_t, but the latent predictive law is generated by the pre-transition uncertainty variables (x_{t-1}, ε_t). The nonlinear measurement equation adds another approximation layer, but it does not remove the need to respect the degenerate latent transition. Assumption (A5) is stronger than the pushforward arguments need: it is included only to justify replacing ε_t by m_t as an equivalent sigma-point variable when that reparameterization is desired.

19.8.2 Worked Degenerate Linear-Transition Example

Use

$$\rho = 0.8, \qquad \sigma = 0.5, \qquad \phi = 0.7, \qquad \psi = 0.3, \qquad \chi = 0.4,$$

with previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \qquad P_{t-1} = \operatorname{diag}(0.04, 0.09),$$

and measurement parameters

$$A = \begin{bmatrix} 1.1 \\ 0.6 \end{bmatrix}, \qquad \mu = 0.10, \qquad R = 0.04.$$

Because the latent transition remains structurally degenerate, the correct sigma-point object is still

$$a_t = (x_{t-1}, \varepsilon_t),$$

not a naively perturbed full post-transition state. Using the same scaled unscented construction as in the previous example, the propagated latent points are obtained from

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \tag{19.79}$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \psi m_{t-1}^{(j)} + \chi m_t^{(j)}. \tag{19.80}$$

For each propagated point, define the nonlinear noise-free observation

$$z_t^{(j)} = \exp\left(1.1 m_t^{(j)} + 0.6 k_t^{(j)} + 0.10\right).$$

The same seven sigma points used earlier now produce the following latent and measurement values:

j	$m_{t-1}^{(j)}$	$k_{t-1}^{(j)}$	$\varepsilon_t^{(j)}$	$m_t^{(j)}$	$k_t^{(j)}$	$z_t^{(j)}$
0	0	0	0	0	0	1.10517
1	0.34641	0	0	0.27713	0.21477	1.70524
2	0	0.51962	0	0	0.36373	1.37470
3	0	0	1.73205	0.86603	0.34641	3.52709
4	−0.34641	0	0	−0.27713	−0.21477	0.71626
5	0	−0.51962	0	0	−0.36373	0.88848
6	0	0	−1.73205	−0.86603	−0.34641	0.34629

The predicted latent moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix}.$$

The nonlinear predicted observation moments are

$$\hat{z}_t = 1.42635, \qquad S_t = 1.32191, \qquad P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix}.$$

So the structural UKF gain is

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}.$$

If the realized observation is $y_t = 1.50$, then

$$v_t = 0.07365, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix}.$$

As in the previous example, these numbers are illustrative calculations under the stated sigma-point convention. They demonstrate the law and moment objects being compared; they are not standalone performance evidence for a production filter.

Now compare this with a naive full-state approximation that adds an artificial latent perturbation variance 0.04 directly to the deterministic-completion coordinate k_t. To keep the comparison reproducible, take the naive rule to form a four-dimensional sigma-point variable

$$\tilde{a}_t = (x_{t-1}, \varepsilon_t, \delta_t), \qquad \delta_t \sim \mathcal{N}(0, 0.04),$$

and to propagate

$$\tilde{k}_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t + \delta_t.$$

That artificial perturbation changes the latent predictive covariance to

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

while leaving the mean unchanged. Under the same nonlinear measurement map, the corresponding naive moment approximation is

$$\tilde{z}_t = 1.44483, \qquad \tilde{S}_t = 1.57838, \qquad \tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

Hence the naive full-state gain becomes

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix},$$

and for the same observation $y_t = 1.50$ the naive posterior mean update is

$$\tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

These numbers make the three failures concrete in this linear-transition / nonlinear-measurement setting:

(i) **Prediction-law failure:** the structural latent law uses

$$P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

because it has inserted artificial uncertainty directly into k_t.

(ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix},$$

whereas the naive approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

The extra direct perturbation of k_t changes how the nonlinear observation loads on the second state coordinate.

(iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

The update formula itself is unchanged, but the predicted moments and cross covariance are not, so the gain and posterior correction are computed for a different approximate model.

19.8.3 What Is Similar to the Structural-Deterministic Case?

The similarity is that the structural latent transition is still generated by (x_{t-1}, ε_t), not by independent perturbations of every coordinate of x_t. If a naive full-state UKF directly perturbs k_t, it again changes the latent predictive law before the observation map is even evaluated. So the structural warning remains active even though the latent transition is linear.

The nonlinear observation adds a second approximation layer. Once the latent cloud is propagated correctly, the UKF still has to approximate

$$\hat{y}_t, \qquad S_t, \qquad P_{xz,t}$$

for the map $x_t \mapsto \exp(A^\top x_t + \mu)$. Poor quadrature here still changes the gain and posterior update, just as in other nonlinear Gaussian filters.

19.8.4 What Is Different from the Earlier Nonlinear-State Example?

In the earlier toy example, the latent state equation itself was nonlinear, so the UKF had to approximate both the latent pushforward through the nonlinear state map and the resulting observation moments. Here the latent transition is linear, but structurally degenerate. That removes one source of approximation error but not the structural state-partition issue. The UKF still needs the correct pre-transition uncertainty variable because k_t remains a deterministic-completion coordinate.

So this example isolates a different practical regime:

(i) the latent transition is linear but not full-state stochastic;

(ii) the nonlinear burden sits in the measurement equation;

(iii) the UKF can still fail in two distinct ways:

- structurally, if it perturbs k_t directly and therefore changes the latent predictive law;
- numerically, if it uses the correct latent construction but poorly approximates the nonlinear observation moments.

This is exactly why the structural deterministic-dynamics chapter should not be read as applying only to nonlinear state transitions. The same latent state-partition discipline remains necessary when the state equation is linear but degenerate and the observation map is nonlinear.

19.9 What Structural Correctness Does and Does Not Guarantee

Structural correctness guarantees that the filter is approximating the intended latent predictive law rather than an altered full-state approximation. It does not, by itself, guarantee that the resulting Gaussian closure is exact, that nonlinear observation moments are approximated with negligible error, or that the derivative path is stable enough

for HMC. In BayesFilter terms, structural correctness is a law-specification requirement; it is not a blanket certificate of moment accuracy, derivative accuracy, or production readiness.

In particular, structural correctness does not solve four other problems that must be evaluated separately:

(i) it does not make the Gaussian sigma-point closure exact for a genuinely nonlinear transformation;

(ii) it does not remove observation-side quadrature error in models such as (19.70);

(iii) it does not resolve numerical covariance-factor issues when square roots, SVD factors, or PSD projections are used;

(iv) it does not guarantee a stable value-gradient path for HMC, especially when spectral decompositions or fallback branches are differentiated.

Thus the right mental model is layered: first make sure the latent law is correct, then measure the quality of the nonlinear approximation, then audit the numerical and derivative path.

19.10 Structural Degeneracy, Numerical Degeneracy, and SVD

The literature and the source-project audit motivate a distinction that should be made explicit:

Issue	What it means	What fixes it
Structural degeneracy	Some latent coordinates are deterministic-completion variables under the model law	Respect the declared stochastic integration space and complete the deterministic block pointwise
Numerical degeneracy	A covariance or innovation matrix is singular or ill-conditioned in computation	Use a stable solve, square-root, or spectral factorization appropriate to the declared law
SVD/square-root representation	A numerical backend chooses a robust factorization of a covariance object	This may improve value-side robustness, but it does not by itself correct a structurally wrong latent law
Derivative/spectral risk	Raw autodiff through spectral or fallback branches may be unstable	Requires derivative audits, stress tests, and possibly custom gradients

This is why an SVD sigma-point filter does not automatically solve the problem discussed in this chapter. It may solve a factorization problem while leaving the latent-law specification problem untouched. Conversely, a structural SVD sigma-point filter is a sensible backend when it factors the covariance of the declared pre-transition variable,

propagates those points through F_θ, and records the usual value and derivative diagnostics from Chapters 17 and 18. The question is therefore not "SVD or structural UKF?" The correct question is:

Which law is being factored and propagated?

Proposition 19.3 (Degenerate covariance and formal sigma-point accuracy)**.** *Let $A_\delta = \mu + \delta Z$ be a possibly degenerate Gaussian random variable in $\mathbb{R}^L$, with $\mathbb{E}[Z] = 0$ and $\mathrm{Cov}(Z) = \Sigma \succeq 0$. Thus $\mathrm{Cov}(A_\delta) = P_\delta = \delta^2\Sigma$ may be singular. Let $\{a_\delta^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ be any sigma-point rule whose deviations $\xi_j = a_\delta^{(j)} - \mu = \delta\zeta_j$ have fixed standardized points ζ_j and satisfy*

$$\sum_j w_j^{(m)}\xi_j = 0, \qquad \sum_j w_j^{(m)}\xi_j\xi_j^\top = P_\delta, \qquad \sum_j w_j^{(c)}\xi_j\xi_j^\top = P_\delta.$$

If $G : \mathbb{R}^L \to \mathbb{R}^d$ is three-times continuously differentiable and its third-order Taylor remainders around μ have expected size $O(\delta^3)$ under the Gaussian law and weighted size $O(\delta^3)$ under the sigma-point cloud, then the sigma-point transformed mean and covariance have the same local order as in the nonsingular case:

$$\mathbb{E}[G_r(A_\delta)] = G_r(\mu) + \frac{1}{2}\,\mathrm{tr}(H_r P_\delta) + O(\delta^3), \tag{19.81}$$

$$\hat{g}_r^{\mathrm{SP}} = G_r(\mu) + \frac{1}{2}\,\mathrm{tr}(H_r P_\delta) + O(\delta^3), \tag{19.82}$$

$$\mathrm{Cov}(G(A_\delta)) = JP_\delta J^\top + O(\delta^3), \tag{19.83}$$

$$P_G^{\mathrm{SP}} = JP_\delta J^\top + O(\delta^3), \tag{19.84}$$

where J is the Jacobian of G at μ and H_r is the Hessian of the rth component. Positive definiteness of P_δ is not used. Consequently, abstracting from finite arithmetic and factor-construction errors, rank deficiency of the state-noise covariance does not by itself lower the formal local sigma-point moment accuracy for the law being propagated.

Proof. The usual Taylor proof only uses the first two moments of the input variable and the matching first two weighted moments of the sigma-point cloud. For a component r,

$$G_r(\mu + \Delta) = G_r(\mu) + J_r\Delta + \frac{1}{2}\Delta^\top H_r\Delta + O(\|\Delta\|^3).$$

Taking expectations with $\Delta = A_\delta - \mu$ gives the first expansion because $\mathbb{E}[\Delta] = 0$ and $\mathbb{E}[\Delta\Delta^\top] = P_\delta$. Taking the weighted sum with $\Delta = \xi_j$ gives the sigma-point mean expansion by the same identities. For covariance, the leading centered term is $J\Delta$, whose second moment is $JP_\delta J^\top$ both for the true Gaussian law and for the sigma-point cloud. The quadratic remainder terms enter at order $O(\delta^3)$ or smaller. No inverse, Cholesky pivot, or full-rank support argument appears in this proof, so the conclusion is unchanged when P_δ is only positive semidefinite. □

Proposition 19.3 verifies the reviewer's claim in its precise value-side form. If a model is written as a full-state transition with a degenerate Gaussian innovation, and if that transition induces exactly the same conditional law as the structural pushforward in Proposition 19.1, then an exact SVD sigma-point rule for that degenerate law has the same formal local moment order as the corresponding nondegenerate rule. Degeneracy

is not, by itself, a mathematical objection to sigma-point filtering. This is a formal value-side statement abstracting from finite arithmetic, factor-construction choices, and derivative-branch effects. Those implementation issues still require the diagnostics below.

That statement is an equivalence result, not a recommendation to hide the structural split inside a large singular covariance matrix. To make the comparison precise, let the structural route use

$$A_t = (x_{t-1}, \varepsilon_t), \qquad x_t = F_\theta(A_t), \qquad \pi_t^x = (F_\theta)_\# \pi_t^a,$$

where $\pi_t^a = \pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon$. Let a collapsed full-state SVD route instead use an augmented variable

$$\widetilde{A}_t = (x_{t-1}, \widetilde{u}_t), \qquad \widetilde{u}_t \sim \mathcal{N}(0, \widetilde{Q}_t), \qquad \widetilde{x}_t = \widetilde{F}_\theta(\widetilde{A}_t),$$

with $\widetilde{Q}_t = \widetilde{C}_t \widetilde{C}_t^\top$ obtained by an SVD or spectral rule. The collapsed route is acceptable only when the following mathematical diagnostics are satisfied.

(i) **Law declaration is pushforward equality.** The two declarations are equivalent only if

$$(\widetilde{F}_\theta)_\# \left(\pi_{t-1|t-1}^x \otimes \mathcal{N}(0, \widetilde{Q}_t) \right) = (F_\theta)_\# \left(\pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon \right),$$

or, equivalently, if $\mathbb{E}[\varphi(\widetilde{x}_t) \mid y_{1:t-1}] = \mathbb{E}[\varphi(x_t) \mid y_{1:t-1}]$ for every bounded test function φ. Matching a covariance matrix is only a necessary moment check, not this equality of laws.

Example. If $m_t = \varepsilon_t$ and $k_t = \varepsilon_t^2$, then the structural conditional law is supported on the parabola $\{(m, k) : k = m^2\}$. A collapsed additive Gaussian transition $\widetilde{x}_t = \mu + \widetilde{u}_t$ with any covariance $\widetilde{Q}_t$ has affine Gaussian support. It cannot equal the parabolic pushforward unless ε_t is degenerate. A singular covariance matrix by itself therefore does not encode the nonlinear timing contract.

(ii) **Sigma-point support is a constraint residual.** Let $c_\theta(a, x) = 0$ collect the deterministic identities implied by the structural transition, for example $c_\theta(A_t, x_t) = x_t - F_\theta(A_t)$. A structural sigma-point cloud satisfies

$$\Delta_{\mathrm{id}}^{\mathrm{str}} = \max_j \| c_\theta(a_t^{(j)}, F_\theta(a_t^{(j)})) \| = 0.$$

The collapsed cloud must satisfy the corresponding check

$$\Delta_{\mathrm{id}}^{\mathrm{col}} = \max_j \| c_\theta(\widetilde{a}_t^{(j)}, \widetilde{F}_\theta(\widetilde{a}_t^{(j)})) \| = 0,$$

after translating its variables into the structural coordinates. If this residual is not defined because the collapsed adapter no longer exposes the relevant shock or timing variables, the adapter cannot demonstrate structural equivalence.

Example. For the linear deterministic identity $x_t = (m_t, k_t) = (\varepsilon_t, \varepsilon_t)$, the support condition is $c(m, k) = k - m = 0$. The exact rank-one covariance $\widetilde{Q} = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ has an SVD factor whose points lie on $k = m$. After a nugget, $\widetilde{Q}_\eta = \widetilde{Q} + \eta^2 I$, sigma points have normal coordinate displacement $\sqrt{L + \lambda}\, \eta$ in the direction $(1, -1)/\sqrt{2}$, so $\Delta_{\mathrm{id}}^{\mathrm{col}} > 0$.

(iii) **Dimension and cost are point-count identities.** For a symmetric $2L+1$ rule, the number of model evaluations is determined by the dimension L of the factored variable. If the previous state has dimension n and the declared shock has dimension q, the structural augmented rule uses

$$N_{\mathrm{str}} = 2(n+q)+1.$$

A full-state additive-noise route that augments by an n-dimensional degenerate noise vector uses

$$N_{\mathrm{col}} = 2(n+n)+1, \qquad N_{\mathrm{col}} - N_{\mathrm{str}} = 2(n-q).$$

This overhead is pure bookkeeping if the extra $n-q$ directions are zero-noise directions.

Example. With $n = 100$ state coordinates and $q = 10$ genuine shocks, the structural augmented rule uses 221 points, while the full-state degenerate-noise augmentation uses 401 points. In the two-coordinate toy covariance $\mathrm{diag}(1,0)$ treated as a two-dimensional factor, the zero column produces two sigma points equal to the mean; they add evaluations and alter the weight bookkeeping without adding stochastic information.

(iv) **Finite arithmetic is normal-support leakage.** Let M_t be a matrix whose columns span the exact linear support of a degenerate Gaussian factor, and let N_t span the normal space $(\mathrm{col}\, M_t)^{\perp}$. Exact support preservation requires

$$N_t^{\top} \widetilde{C}_t = 0 \quad \Longleftrightarrow \quad N_t^{\top} \widetilde{Q}_t N_t = 0.$$

A nugget, PSD projection, rank threshold, or asymmetric reconstruction should therefore be reported through

$$\ell_{\perp,t} = \|N_t^{\top} \widetilde{Q}_t N_t\|_{\mathrm{op}} \quad \text{or} \quad \max_j \|N_t^{\top}(\widetilde{x}_t^{(j)} - \bar{x}_t)\|.$$

Example. In the identity example $k = m$, the normal vector is $N = (1,-1)^{\top}/\sqrt{2}$. For $\widetilde{Q}_\eta = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right) + \eta^2 I$,

$$N^{\top} \widetilde{Q}_\eta N = \eta^2.$$

Hence the nominally deterministic relation has positive artificial variance η^2 in finite arithmetic. This is not a harmless representation of the same law; it is a regularized law unless explicitly removed before propagation.

(v) **Gradient risk is controlled by spectral gaps.** If $\widetilde{C}_t(\theta)$ is produced from an eigendecomposition or SVD, derivatives of the spectral vectors contain gap denominators. For a symmetric eigendecomposition, schematically,

$$du_i = \sum_{j \neq i} u_j \frac{u_j^{\top}(d\widetilde{Q}_t)u_i}{\lambda_i - \lambda_j} + \text{terms along } u_i.$$

Therefore the gradient audit must track

$$g_t^{\min} = \min_{i \neq j} |\lambda_i(\widetilde{Q}_t) - \lambda_j(\widetilde{Q}_t)|, \qquad \|\partial_\theta \widetilde{C}_t\| \lesssim \frac{\|\partial_\theta \widetilde{Q}_t\|}{g_t^{\min}}$$

away from rank and ordering branches. Structural completion does not remove factor-gradient issues, but it avoids adding spectral choices for artificial zero-noise coordinates.

Example. Let $\widetilde{Q}(\theta) = \left(\begin{smallmatrix} 1 & \theta \\ \theta & 1 \end{smallmatrix}\right)$. The eigenvalues are $1+\theta$ and $1-\theta$, with gap $2|\theta|$. At $\theta = 0$ the covariance is perfectly smooth but the ordered eigenspace is not unique; raw autodiff through the spectral factor can be discontinuous or unstable even though the value-side covariance is benign.

(vi) **Diagnostics must separate model law from regularization.** A collapsed route should report both the identity residual ((ii)) and the normal leakage ((iv)). Otherwise a finite likelihood can be produced by a different state-space model. For a scalar observation that loads on a deterministic coordinate, this difference is explicit: if the structural law is

$$k_t = 0, \qquad y_t = k_t + e_t, \qquad e_t \sim \mathcal{N}(0, r),$$

then $y_t \sim \mathcal{N}(0, r)$. If a collapsed covariance injects artificial variance η^2 into k_t, then the implemented likelihood uses $y_t \sim \mathcal{N}(0, r+\eta^2)$, and the one-period log-likelihood change is

$$\Delta\ell_t = -\frac{1}{2}\left[\log\frac{r+\eta^2}{r} + y_t^2\left(\frac{1}{r+\eta^2} - \frac{1}{r}\right)\right].$$

A finite value of this likelihood is therefore not evidence that the structural transition was respected; it may only show that the nugget made the altered model numerically evaluable.

These diagnostics matter for the UKF because the UKF update is only as correct as the sigma-point cloud used to build its predictive moments. For a noise-free observation map h_θ and observation noise covariance R_t, the structural UKF forms points

$$x_t^{(j)} = F_\theta(a_t^{(j)}), \qquad z_t^{(j)} = h_\theta(x_t^{(j)}),$$

and then computes

$$\hat{x}_t = \sum_j w_j^{(m)} x_t^{(j)}, \qquad \hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.85}$$

$$P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(x_t^{(j)} - \hat{x}_t)^\top, \tag{19.86}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \tag{19.87}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(z_t^{(j)} - \hat{z}_t)^\top. \tag{19.88}$$

The Gaussian UKF update then uses

$$v_t = y_t - \hat{z}_t, \qquad K_t = P_{xz,t} S_t^{-1}, \tag{19.89}$$

$$\hat{x}_{t|t} = \hat{x}_t + K_t v_t, \qquad P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.90}$$

with log-likelihood contribution

$$\ell_t = -\frac{1}{2}\left[d_y \log(2\pi) + \log\det S_t + v_t^\top S_t^{-1} v_t\right]. \tag{19.91}$$

Thus a collapsed SVD route is not merely changing an implementation detail. It replaces the point cloud $\{F_\theta(a_t^{(j)})\}$ in (19.88) by $\{\widetilde{F}_\theta(\widetilde{a}_t^{(j)})\}$. The error enters through

$$\Delta\hat{z}_t = \widetilde{\hat{z}}_t - \hat{z}_t, \tag{19.92}$$

$$\Delta S_t = \widetilde{S}_t - S_t, \tag{19.93}$$

$$\Delta P_{xz,t} = \widetilde{P}_{xz,t} - P_{xz,t}, \tag{19.94}$$

$$\Delta K_t = \widetilde{P}_{xz,t}\widetilde{S}_t^{-1} - P_{xz,t}S_t^{-1}. \tag{19.95}$$

Even when the formal sigma-point rule is accurate for its own input law, (19.95) is controlled by whether the collapsed cloud matches the structural cloud on the particular functions h_θ, $x\,h_\theta(x)^\top$, and $h_\theta(x)h_\theta(x)^\top$ used in the UKF update. Equality of predictive laws is the robust sufficient condition; accidental equality of a few low-order moments is not a model-contract proof. This is the precise UKF sense in which the law-level diagnostics above are not optional metadata.

The six diagnostics above correspond to the UKF recursion as follows.

(i) **Pushforward equality** is the condition that the two point clouds approximate the same expectations in (19.88). If ((i)) fails, then $\widetilde{\hat{x}}_t$, $\widetilde{\hat{z}}_t$, $\widetilde{S}_t$, and $\widetilde{P}_{xz,t}$ are moments of a different predictive law.

(ii) **Identity residuals** test whether every propagated point used in (19.88) is an admissible structural state. If $\Delta_{\mathrm{id}}^{\mathrm{col}} > 0$, then the UKF averages include states that the DSGE transition assigns probability zero.

(iii) **Point-count overhead** affects the number of evaluations of F_θ and h_θ required to build the same objects $\hat{x}_t, \hat{z}_t, S_t, P_{xz,t}$. Extra zero-noise directions do not add information; they add factorization and map-evaluation work before the update in (19.90).

(iv) **Normal-support leakage** enters the UKF through S_t and $P_{xz,t}$. Artificial variance in a deterministic-completion direction changes the innovation covariance and the state-observation cross covariance, so it changes both the likelihood (19.91) and the gain $K_t = P_{xz,t}S_t^{-1}$.

(v) **Spectral-gap gradient risk** is the derivative of the same UKF likelihood and update with respect to parameters. If $\partial_\theta \widetilde{C}_t$ is unstable, then the gradients of $\widetilde{S}_t$, $\widetilde{P}_{xz,t}$, $\widetilde{K}_t$, and $\widetilde{\ell}_t$ inherit that instability.

(vi) **Diagnostic separation** tells the reader whether (19.90) and (19.91) were evaluated under the structural law or under a regularized law. Without the identity and leakage diagnostics, a finite UKF likelihood cannot distinguish those two cases.

A one-dimensional observation example shows the mechanism without requiring filtering expertise. Let

$$m_t = \varepsilon_t, \qquad k_t = m_t, \qquad y_t = k_t + e_t, \qquad \varepsilon_t \sim \mathcal{N}(0,q), \quad e_t \sim \mathcal{N}(0,r).$$

The structural predictive state has

$$\mathrm{Var}(m_t) = q, \qquad \mathrm{Var}(k_t) = q, \qquad \mathrm{Cov}(m_t,k_t) = q, \qquad S_t^{\mathrm{str}} = q + r.$$

The Kalman gain components implied by the UKF moment objects are therefore

$$K_t^{\mathrm{str}} = \frac{\mathrm{Cov}((m_t, k_t), y_t)}{\mathrm{Var}(y_t)} = \frac{1}{q+r}\begin{pmatrix} q \\ q \end{pmatrix}.$$

If the collapsed SVD route adds a small independent numerical variance η^2 to the deterministic-completion coordinate k_t, then

$$\widetilde{k}_t = m_t + \eta\xi_t, \qquad \xi_t \sim \mathcal{N}(0,1),$$

and the UKF objects become

$$\widetilde{S}_t = q + \eta^2 + r, \qquad \widetilde{K}_t = \frac{1}{q+\eta^2+r}\begin{pmatrix} q \\ q+\eta^2 \end{pmatrix}.$$

The likelihood and update have changed:

$$\widetilde{\ell}_t - \ell_t = -\frac{1}{2}\left[\log\frac{q+\eta^2+r}{q+r} + y_t^2\left(\frac{1}{q+\eta^2+r} - \frac{1}{q+r}\right)\right], \tag{19.96}$$

$$\widetilde{K}_{m,t} - K_{m,t} = -\frac{q\eta^2}{(q+r)(q+\eta^2+r)}, \tag{19.97}$$

$$\widetilde{K}_{k,t} - K_{k,t} = \frac{\eta^2 r}{(q+r)(q+\eta^2+r)}. \tag{19.98}$$

In this example, the six diagnostics above appear in concrete form:

(i) **Pushforward equality** is the difference between the structural law $k_t = m_t$ and the collapsed law $\widetilde{k}_t = m_t + \eta\xi_t$. If $\eta > 0$, the predictive law is no longer supported on $k = m$, and the UKF moments in (19.10) are not the moments in (19.10).

(ii) **Identity residuals** are measured by $c(m,k) = k - m$. The structural points have $c = 0$ pointwise, while the collapsed points have $c(\widetilde{m}_t, \widetilde{k}_t) = \eta\xi_t$ and therefore positive residual whenever the artificial direction is sampled.

(iii) **Point-count overhead** is hidden in the construction of $\widetilde{k}_t$. The structural rule needs points for the one genuine shock ε_t; the collapsed route adds a second numerical direction ξ_t before computing the same scalar observation y_t.

(iv) **Normal-support leakage** is the extra term η^2 in $\widetilde{S}_t = q + \eta^2 + r$ in (19.10). That term is exactly the artificial variance normal to the structural line $k = m$, and it changes the likelihood and gain in (19.98).

(v) **Spectral-gradient risk** enters if η^2 is produced by an SVD threshold, nugget branch, or spectral factor of a nearly singular covariance. Then derivatives of $\widetilde{S}_t$, $\widetilde{K}_t$, and $\widetilde{\ell}_t$ inherit the derivative behavior of that spectral branch.

(vi) **Diagnostic separation** is the need to report whether the likelihood is based on $S_t^{\mathrm{str}} = q + r$ or on $\widetilde{S}_t = q + \eta^2 + r$. Both are finite UKF likelihoods, but (19.98) shows that they are not the same likelihood unless $\eta = 0$.

So the nugget changes both the likelihood weight assigned to the observation and the way the observation is fed back into m_t and k_t. This is the UKF problem in concrete terms: the filter can remain numerically stable and still perform the prediction, likelihood evaluation, and update for a different state-space model.

The practical conclusion is therefore conservative. SVD is a good numerical backend for singular or nearly singular covariance factors, and it can be used inside a structurally correct sigma-point filter. But using SVD on a collapsed full-state covariance is usually a harder implementation contract: it has more linear algebra to audit, more fragile derivatives for HMC, and more ways for a regularization choice to become an unrecorded model change. The structural adapter is preferred because it states the economic law directly and leaves SVD to do the narrower job of factoring the covariance that actually belongs to that law.

For HMC promotion, one more distinction matters. Matrix backpropagation formulas for spectral decompositions contain denominators involving singular-value or eigenvalue gaps, so repeated or nearly repeated spectral values are derivative-risk regions [Ionescu et al., 2015]. That risk is orthogonal to the structural-law issue. A structurally correct SVD sigma-point value path still needs spectral-gap telemetry, finite-gradient stress tests, and compiled parity before it can be treated as a production HMC target.

19.11 Pruned DSGE and Adapter Implications

For pruned second-order DSGE approximations, the source DSGE project already uses the standard first-order/second-order decomposition:

$$x_t^f = h_x x_{t-1}^f + \eta \varepsilon_t, \tag{19.99}$$

$$x_t^s = h_x x_{t-1}^s + \frac{1}{2} H_{xx}(x_{t-1}^f \otimes x_{t-1}^f) + \frac{1}{2} h_{ss}, \tag{19.100}$$

$$y_t = g_x(x_t^f + x_t^s) + \frac{1}{2} G_{xx}(x_t^f \otimes x_t^f) + \frac{1}{2} g_{ss} + e_t. \tag{19.101}$$

That decomposition is necessary, but it is not sufficient for large DSGE filtering. It separates perturbation order; it does not automatically separate exogenous shock states from endogenous deterministic states inside x_t^f. A robust BayesFilter DSGE adapter should therefore carry both bookkeeping axes,

$$\text{perturbation order: } (x^f, x^s), \qquad \text{structural timing: } (m, k).$$

The adapter-facing implication is simple: the model must expose the exogenous transition, the deterministic endogenous transition, the observation map, and a declaration of which variables define the stochastic integration space. For a sigma-point implementation, that adapter contract should compile to the following concrete objects:

(i) dimensions and names for the filtered state, the declared innovation, and deterministic-completion coordinates;

(ii) a function that maps each point $(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$ to $x_t^{(j)} = F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$;

(iii) a deterministic-identity residual, evaluated on every propagated point;

(iv) an observation map evaluated only after structural completion;

(v) an approximation label if any legacy full-state or regularized covariance path is used instead.

The pruned perturbation decomposition tells the adapter what equations exist; the structural metadata tells the filter which variables receive quadrature.

19.12 Source-Project Lesson

The recent source-project audit found a concrete instance of the structural failure discussed in this chapter. The default DSGE SVD sigma-point adapter uses a collapsed transition of the form

$$Q_w = \eta\eta^\top + \epsilon I, \qquad x^f_{t|t-1} = h_x x^f_{t-1|t-1},$$

and propagates the covariance as

$$P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q_w.$$

It tracks the pruned x^s correction separately, but it does not expose a partition between exogenous and endogenous coordinates inside x^f. For the smallest NK example, where the states are only the exogenous demand and monetary-policy shocks, this is not a serious modeling error. For Rotemberg NK, SGU, EZ, and NAWM-scale DSGE models, however, the state vector contains endogenous/predetermined components, so treating every coordinate as an exchangeable noisy Gaussian coordinate becomes a structural filtering bug rather than a mere numerical tuning issue.

This does not mean the generic SVD sigma-point machinery is unusable. The source audit found a more precise split: a generic sigma-point path that accepts user-supplied transition points or a structural transition map can be reused as an approximate backend, while the default mixed-DSGE adapter must be gated unless it exposes the exogenous/endogenous partition and deterministic completion. In other words, reuse the factorization and diagnostics; do not reuse an adapter that silently changes the stochastic integration space.

19.13 Validation Gates and Final Policy Rule

No future BayesFilter report should claim that a nonlinear DSGE filter is value-valid, gradient-valid, sampler-usable, converged, or production-ready in the sense of Chapter 44 unless the following structural tests are passed or explicitly waived with written justification:

1. **Metadata test:** the model exposes exogenous, endogenous, and deterministic state blocks, and their dimensions match the shock-impact matrix and transition maps.

2. **Constraint-support test:** propagated sigma points or particles satisfy deterministic identities to numerical tolerance.

3. **Linear recovery test:** when the structural transition is linear, the structural nonlinear filter recovers the exact Kalman likelihood up to the declared approximation tolerance.

4. **Degenerate-transition test:** zero-innovation endogenous rows remain deterministic except for explicitly declared numerical regularization.

5. **SVD/factorization test:** if a square-root or SVD sigma-point backend is used, diagnostics state whether the factor belongs to the structural integration variable or to a legacy full-state approximation; the latter must carry an approximation label.

6. **Toy DSGE test:** a controlled model with

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = f(k_{t-1}, m_{t-1}, m_t),$$

compares structural propagation against a dense quadrature or high-particle reference.

7. **Case-study test:** Rotemberg NK, SGU, and any EZ/NAWM target must document which state coordinates are shock-driven, which are deterministic conditional on the shock path, and how the filter honors that timing.

8. **Derivative-promotion test:** any HMC claim through SVD or eigenspectral factors must pass spectral-gap, finite-gradient, and eager/compiled parity checks.

19.13.1 Common Misunderstandings

The most common reader errors in this area are:

(i) a deterministic endogenous state need not have zero one-step predictive variance; it can inherit randomness through the declared stochastic block;

(ii) a singular or rank-deficient transition covariance is not, by itself, a modeling error in the exact linear-Gaussian case;

(iii) numerical regularization is not the same thing as declaring a new stochastic state coordinate;

(iv) a full-state nonlinear approximation is not automatically forbidden, but it must be labeled as an approximation if it enlarges the stochastic space beyond the structural transition contract.

> For nonlinear DSGE filtering, the stochastic integration variables are not automatically the entries of the state vector. They are the variables declared stochastic by the structural transition. Deterministic endogenous states must be computed, not noised.

If an implementation violates this rule, it must be labeled as an artificial approximation and justified against a reference. Otherwise, the filter is solving a different approximate state-space model relative to the declared structural law. Failing one of the validation gates above does not automatically make a backend useless, but it does mean the strongest claim labels from Chapter 44 are not yet justified.

Chapter 20

Particle-Filter Foundations

This chapter develops the particle-filter baseline for the differentiable particle-filter program. The goal is not yet to make the filter differentiable. The goal is to establish, in BayesFilter notation, the nonlinear filtering recursion, the empirical particle approximation, the bootstrap particle-filter likelihood estimator, and the exact point at which the later differentiable program departs from the classical construction.

Particle methods matter because the nonlinear filtering problem is generally not closed under finite-dimensional moments. When the predictive law or filtering law is materially non-Gaussian, a Gaussian closure such as the EKF or UKF may be a useful approximation but no longer defines the exact filtering object. The particle filter instead approximates the filtering law directly as a weighted random measure. This makes it the natural baseline for any later attempt to construct particle-flow, resampling-relaxed, or differentiable particle methods.

20.1 Objects, assumptions, and claim-status vocabulary

The later DPF chapters rely on this chapter as the reference object. The following notation is therefore fixed before introducing any differentiable relaxation.

Object	Meaning in this chapter
$x_t \in \mathbb{R}^{n_x}$	latent state at time t
$y_t \in \mathbb{R}^{n_y}$	observation at time t
$\theta \in \Theta$	fixed model parameter while the filtering recursion is defined
$p_\theta(x_t \mid x_{t-1})$	transition density or Markov kernel density
$p_\theta(y_t \mid x_t)$	observation density, evaluated at the realized observation y_t
$p_\theta(x_t \mid y_{1:t-1})$	predictive law before assimilating y_t
$p_\theta(x_t \mid y_{1:t})$	filtering law after assimilating y_t
$p_\theta(y_t \mid y_{1:t-1})$	one-step likelihood factor
$q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$	sequential proposal density
$\tilde{w}_t^{(i)}, w_t^{(i)}$	unnormalized and normalized particle weights
$a_{t-1}^{(i)}$	ancestor index selected during resampling
$\hat{\pi}_t^N$	weighted empirical approximation to the filtering law
Z_t^N	average incremental bootstrap weight at time t
$\hat{p}_\theta^N(y_{1:T})$	bootstrap particle-filter estimator of the marginal likelihood

The standing assumptions for the formulas below are the standard ones needed to make the displayed densities and expectations well defined: the transition and observation kernels are dominated by reference measures; the proposal has support wherever the trajectory target has support; the relevant integrals are finite; and resampling schemes are conditionally unbiased in the sense that the expected empirical measure after resampling equals the normalized empirical measure before resampling. These assumptions are not cosmetic. They mark the difference between an exact model identity, a finite-particle Monte Carlo estimator statement, and an implementation diagnostic.

The chapter uses the following claim-status vocabulary. The filtering recursion and likelihood factorization are exact model identities. Weighted particles are finite-N Monte Carlo approximations. The bootstrap likelihood estimator is an unbiased estimator of the likelihood, not of the log likelihood and not of the score. ESS and ancestor-count summaries are diagnostics, not proofs of posterior correctness or failure.

20.2 DPF-block notation and claim-status index

This chapter begins the DPF block, so the following index fixes the notation used across the later particle-flow, PF-PF, resampling/OT, learned-OT, HMC, and debugging chapters. A symbol may acquire a method-specific subscript in a later chapter, but its mathematical role should not change silently.

Table 20.1: DPF-block notation index.

Category	Symbols	Fixed role across the DPF block
State and observations	x_t, $x_{0:t}$, y_t, $y_{1:t}$	Latent state, latent trajectory, current observation, and observation history for the state-space model.
Parameters	θ, u	Structural/model parameter and, in HMC chapters, unconstrained sampler coordinate after any parameter transform.
Filtering laws	$p_\theta(x_t \mid y_{1:t-1})$, $p_\theta(x_t \mid y_{1:t})$	Predictive and filtering laws; these are the reference statistical objects.
Particles and ancestors	$x_t^{(i)}$, $\tilde{x}_t^{(j)}$, $a_{t-1}^{(i)}$, a_j	Pre- or post-update particles, equal-weight output particles, and discrete ancestor indices.
Weights and empirical measures	$\tilde{w}_t^{(i)}$, $w_t^{(i)}$, $\hat{\pi}_t^N$, $\hat{\pi}_{t,\mathrm{eq}}^N$	Unnormalized/normalized weights and weighted/equal-weight empirical measures.
Likelihood estimates	Z_t^N, $\hat{p}_\theta^N(y_{1:T})$, $\widehat{L}(y \mid \theta, \omega)$	Average incremental particle weight, bootstrap likelihood estimator, and generic randomized likelihood estimator used in pseudo-marginal discussion.
Proposals	q_θ, $q_{t,0}$, $q_{t,1}$, γ_t	Sequential proposal, pre-flow proposal, post-flow proposal, and unnormalized one-step target density.
Flow maps and Jacobians	$\pi_{t,\lambda}$, $v_{t,\lambda}$, Φ_t, J_t, $D\Phi_t$	Homotopy density, pseudo-time velocity field, flow map, and forward Jacobian of the flow.
Transport couplings	Π, $\Pi_0^\star$, $\Pi_\varepsilon^\star$, $\Pi_{\varepsilon,K}$	Unregularized, entropy-regularized, and finite-Sinkhorn transport couplings.
Sinkhorn scalings	C, K_ε, a, b, ε, K	Cost matrix, Gibbs kernel, row/column scalings, regularization level, and finite iteration count.
Learned maps	T_ψ, Y_τ, $R_{\psi,\tau}$, $\mathcal{D}$	Learned student map, teacher output, student-teacher residual, and training distribution.
HMC scalar and gradient	$\ell_\star(u)$, $U_\star(u)$, $g(u)$, $H_\star(u,p)$	Named log target, potential, gradient used by the integrator, and Hamiltonian. The value and gradient must refer to the same scalar.

Table 20.2: DPF-block claim-status vocabulary.

Status phrase	Use only when	Disallowed shortcut
Exact model identity	The statement follows from the specified state-space model, Bayes' rule, change of variables, or a named exact special case.	Calling a closure, relaxation, solver, or learned map exact because it is written in equations.
Unbiased particle estimator	The expectation is taken over the stated particle randomness and the target object is the likelihood or normalizing constant.	Inferring unbiased log likelihood, score, pathwise gradient, or HMC validity.
Consistent approximation	A convergence regime and target object are stated.	Treating increasing N or decreasing tolerance as a finite-sample proof.
Approximate closure	A Gaussian, local-linear, finite-particle, or numerical closure replaces the reference filtering object.	Delaying the approximation label until a later chapter.
Relaxed target	A smooth resampling or transport object replaces the classical categorical operation and the scalar target is the relaxed object.	Implying the original posterior is preserved without a correction.
Learned surrogate	A trained map approximates a named teacher under a declared training distribution.	Using runtime, smoothness, or training loss as posterior validity evidence.
Engineering hypothesis	The claim is a bounded recommendation or diagnostic rule, with required tests stated.	Presenting implementation preference as mathematical validation.
Unsupported claim	The statement lacks a local derivation, reviewed source support, or documented test evidence.	Leaving the statement in the main argument without weakening or removing it.

20.3 Nonlinear state-space model and filtering recursion

Let $x_t \in \mathbb{R}^{n_x}$ denote the latent state and $y_t \in \mathbb{R}^{n_y}$ the observation at time t. Fix a parameter vector $\theta \in \Theta \subset \mathbb{R}^{d_\theta}$. The nonlinear state-space model is specified by a transition density and an observation density,

$$x_t \mid x_{t-1} \sim p_\theta(x_t \mid x_{t-1}), \tag{20.1}$$

$$y_t \mid x_t \sim p_\theta(y_t \mid x_t). \tag{20.2}$$

Write $y_{1:t} = (y_1, \dots, y_t)$ for the observation history. The predictive law and the filtering law satisfy the standard Bayesian recursion

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}, \tag{20.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}, \tag{20.4}$$

where the one-step predictive density of the observation is

$$p_\theta(y_t \mid y_{1:t-1}) = \int p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{20.5}$$

The prediction equation is the Chapman–Kolmogorov identity applied to the conditional law of x_{t-1} given the observations already seen. For any measurable set B,

$$\mathbb{P}_\theta(x_t \in B \mid y_{1:t-1}) = \int \mathbb{P}_\theta(x_t \in B \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}.$$

When the transition kernel has density $p_\theta(x_t \mid x_{t-1})$, this set identity gives (20.3). The update equation is Bayes' rule with the predictive law as prior and the observation density as likelihood:

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t, y_{1:t-1}) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}.$$

The conditional-independence convention of the state-space model reduces $p_\theta(y_t \mid x_t, y_{1:t-1})$ to $p_\theta(y_t \mid x_t)$, giving (20.4). Finally, repeated use of the chain rule gives the marginal likelihood factorization

$$p_\theta(y_{1:T}) = p_\theta(y_1) \prod_{t=2}^{T} p_\theta(y_t \mid y_{1:t-1}),$$

with the same notation covering $t = 1$ by interpreting $y_{1:0}$ as the empty history. The marginal likelihood then factorizes as

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{1:t-1}). \tag{20.6}$$

Equations (20.3)–(20.6) are exact. The difficulty is computational rather than conceptual: outside special classes such as the linear-Gaussian case, the integrals are rarely available in closed form. Particle filters replace these exact distributions by random empirical approximations whose quality improves with particle count under appropriate assumptions.

20.4 Empirical filtering measures

A particle filter represents the filtering law by a weighted empirical measure. At time t, let $\{(x_t^{(i)}, w_t^{(i)})\}_{i=1}^N$ be particles and normalized weights, with $w_t^{(i)} \geq 0$ and $\sum_{i=1}^N w_t^{(i)} = 1$. The empirical filtering measure is

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^{N} w_t^{(i)} \, \delta_{x_t^{(i)}}(dx), \tag{20.7}$$

where δ_z is the Dirac point mass at z. The approximation of a test function φ under the filtering law is then

$$\int \varphi(x) \, \hat{\pi}_t^N(dx) = \sum_{i=1}^{N} w_t^{(i)} \varphi(x_t^{(i)}). \tag{20.8}$$

Thus the filter approximates the entire posterior law by a finite atomic measure, not only its first two moments.

This is the main structural distinction between particle methods and the Gaussian-approximation filters of the previous chapters. The particle filter is not built by closing the recursion in a finite-dimensional moment family. Instead, it propagates a random discrete approximation to the law itself.

20.5 Sequential importance sampling

To derive the basic particle recursion, introduce a proposal density $q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$ from which particles can be sampled. Assume that the proposal is positive whenever the trajectory target below is positive. Without this support condition the importance ratio is not a valid correction; particles may miss regions that carry target probability. Suppose a full trajectory proposal factors as

$$q_\theta(x_{0:t} \mid y_{1:t}) = q_\theta(x_0 \mid y_1) \prod_{s=1}^{t} q_\theta(x_s \mid x_{0:s-1}, y_{1:s}). \tag{20.9}$$

The target filtering distribution over trajectories is proportional to

$$p_\theta(x_{0:t}, y_{1:t}) = p_\theta(x_0) \prod_{s=1}^{t} p_\theta(x_s \mid x_{s-1}) p_\theta(y_s \mid x_s). \tag{20.10}$$

The unnormalized importance weight is therefore

$$\tilde{w}_t(x_{0:t}) = \frac{p_\theta(x_{0:t}, y_{1:t})}{q_\theta(x_{0:t} \mid y_{1:t})}. \tag{20.11}$$

This ratio is a trajectory-level object. It is not yet a resampling rule and not yet a differentiable likelihood map. It is the Radon–Nikodym correction that converts expectations under the proposal path law into expectations under the unnormalized trajectory target, provided the support and integrability conditions above hold. Using the factorization in (20.9)–(20.10), one obtains the recursive update

$$\tilde{w}_t^{(i)} = \tilde{w}_{t-1}^{(i)} \frac{p_\theta(y_t \mid x_t^{(i)}) p_\theta(x_t^{(i)} \mid x_{t-1}^{(i)})}{q_\theta(x_t^{(i)} \mid x_{0:t-1}^{(i)}, y_{1:t})}. \tag{20.12}$$

Normalizing gives

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{j=1}^{N} \tilde{w}_t^{(j)}}. \tag{20.13}$$

Equation (20.12) is the generic sequential importance sampling (SIS) update. It already shows the two main mathematical stresses of particle filtering:

(i) the proposal must be chosen carefully to avoid explosive weight variability;

(ii) even when the estimator is well defined, the normalized weights tend to concentrate over time, which leads to degeneracy.

20.6 Sequential importance resampling and the bootstrap filter

Sequential importance sampling alone is typically unstable over long horizons, because the weights become increasingly uneven. Sequential importance resampling (SIR) introduces a resampling step to refresh the cloud when the weight distribution becomes too concentrated.

The most classical special case is the bootstrap particle filter, which uses the transition prior itself as the proposal:

$$q_\theta(x_t \mid x_{0:t-1}, y_{1:t}) = p_\theta(x_t \mid x_{t-1}). \tag{20.14}$$

Substituting (20.14) into (20.12) gives the simplified incremental weight

$$\tilde{w}_t^{(i)} \propto w_{t-1}^{(i)} p_\theta(y_t \mid x_t^{(i)}). \tag{20.15}$$

The bootstrap recursion at time t can therefore be written as:

Step 1: Select ancestors: if resampling is performed before propagation, draw $a_{t-1}^{(i)}$ from the current normalized weights; otherwise keep $a_{t-1}^{(i)} = i$.

Step 2: Propagate: sample $x_t^{(i)} \sim p_\theta(x_t \mid x_{t-1}^{(a_{t-1}^{(i)})})$.

Step 3: Weight: compute $\tilde{w}_t^{(i)} = p_\theta(y_t \mid x_t^{(i)})$ and normalize the weights.

Step 4: Resample: after recording the likelihood factor, either resample using the normalized weights and reset weights to $1/N$, or carry the weighted cloud forward until the next resampling decision.

This yields the standard bootstrap/SIR particle filter of Gordon et al. [1993], elaborated in the SMC text of Doucet et al. [2001]. The particle-MCMC literature [Andrieu and Roberts, 2009, Andrieu et al., 2010] later uses the likelihood estimator generated by this construction, but that pseudo-marginal role is a statement about an unbiased value-side estimator inside a Metropolis–Hastings construction. It is not a statement that the realized resampling path is smooth in θ.

20.7 The bootstrap particle-filter likelihood estimator

The bootstrap particle filter supplies a natural estimator of the factors in the marginal-likelihood decomposition (20.6). Define the average incremental weight at time t by

$$Z_t^N = \frac{1}{N} \sum_{i=1}^{N} \tilde{w}_t^{(i)}. \tag{20.16}$$

The corresponding likelihood estimator is

$$\hat{p}_\theta^N(y_{1:T}) = \prod_{t=1}^{T} Z_t^N. \tag{20.17}$$

The estimator-status result BayesFilter needs at this stage is the following.

Proposition 20.1 (Bootstrap particle-filter likelihood-estimator status)**.** *Fix the observations $y_{1:T}$ and the parameter θ. Assume the model densities are dominated and integrable, the bootstrap propagation step samples from $p_\theta(x_t \mid x_{t-1})$, and the Feynman–Kac quantities used below are finite: for every bounded measurable φ used in the induction, the*

operator $Q_t\varphi$ is well defined, the incremental weights have finite conditional expectation, and the displayed conditional expectations are finite. Assume also that every resampling step used by the algorithm is conditionally unbiased for the current normalized empirical measure. Here conditional unbiasedness means that, for every bounded measurable test function ψ in the same finite-expectation class, the expected post-resampling empirical average equals the pre-resampling weighted empirical average, conditional on the current weighted cloud. Biased or deterministic approximation schemes that do not satisfy this identity are not covered by the proposition. Then the estimator (20.17) is an unbiased estimator of the marginal likelihood:

$$\mathbb{E}\big[\hat{p}_\theta^N(y_{1:T})\big] = p_\theta(y_{1:T}).$$

This claim is about the likelihood itself. It does not claim that $\log \hat{p}_\theta^N(y_{1:T})$ is an unbiased estimator of $\log p_\theta(y_{1:T})$, and it does not claim that differentiating a realized particle system gives an unbiased score.

Proof. For compactness, define the bootstrap Feynman–Kac operator

$$(Q_t\varphi)(x_{t-1}) = \int \varphi(x_t) p_\theta(y_t \mid x_t) p_\theta(x_t \mid x_{t-1})\, dx_t$$

for bounded measurable test functions φ. Also define the unnormalized filtering functional

$$\gamma_t(\varphi) = p_\theta(y_{1:t}) \int \varphi(x_t) p_\theta(x_t \mid y_{1:t})\, dx_t.$$

The prediction/update recursion implies $\gamma_t(\varphi) = \gamma_{t-1}(Q_t\varphi)$ and $\gamma_t(1) = p_\theta(y_{1:t})$.

The filtration convention is as follows. At the end of time $t-1$ the algorithm has a weighted empirical measure $\hat{\pi}_{t-1}^N$ and a normalizing-constant estimate $\prod_{s=1}^{t-1} Z_s^N$. Any resampling randomness used to choose the particles propagated at time t is then drawn, and the transition propagation at time t is conditionally independent across particles given that post-resampling cloud. All test functions in the induction are bounded measurable, or otherwise integrable enough that the conditional expectations below are finite.

Let $\mathcal{F}_{t-1}^N$ denote the sigma-field generated by the particle system just before propagation at time t, after any resampling decision at time $t-1$. Let $\bar{\pi}_{t-1}^N$ be the empirical measure of the particles from which the transition propagation is drawn. If resampling is used, the assumption of conditional unbiasedness means

$$\mathbb{E}[\bar{\pi}_{t-1}^N(\psi) \mid \hat{\pi}_{t-1}^N] = \hat{\pi}_{t-1}^N(\psi)$$

for bounded ψ; if resampling is not used, then $\bar{\pi}_{t-1}^N = \hat{\pi}_{t-1}^N$.

After bootstrap propagation and weighting,

$$Z_t^N \hat{\pi}_t^N(\varphi) = \frac{1}{N} \sum_{i=1}^N p_\theta(y_t \mid x_t^{(i)}) \varphi(x_t^{(i)}).$$

Conditional on $\mathcal{F}_{t-1}^N$, the propagated particles are drawn from the transition kernel, so

$$\mathbb{E}\big[Z_t^N \hat{\pi}_t^N(\varphi) \mid \mathcal{F}_{t-1}^N\big] = \bar{\pi}_{t-1}^N(Q_t\varphi).$$

Taking the conditional expectation over any resampling randomness replaces $\bar{\pi}_{t-1}^N$ by $\hat{\pi}_{t-1}^N$ in expectation. Multiplying by the previous normalizing-constant estimate, which is measurable with respect to the previous particle history, gives

$$\mathbb{E}\left[\left(\prod_{s=1}^{t} Z_s^N\right)\hat{\pi}_t^N(\varphi)\right] = \mathbb{E}\left[\left(\prod_{s=1}^{t-1} Z_s^N\right)\hat{\pi}_{t-1}^N(Q_t\varphi)\right].$$

Iterating this identity and using $\gamma_t(\varphi) = \gamma_{t-1}(Q_t\varphi)$ gives

$$\mathbb{E}\left[\left(\prod_{s=1}^{t} Z_s^N\right)\hat{\pi}_t^N(\varphi)\right] = \gamma_t(\varphi).$$

Taking $\varphi \equiv 1$ yields

$$\mathbb{E}\left[\prod_{s=1}^{t} Z_s^N\right] = \gamma_t(1) = p_\theta(y_{1:t}).$$

Setting $t = T$ proves the displayed likelihood-unbiasedness identity. This is the local BayesFilter version of the standard SMC normalizing-constant unbiasedness argument; see Doucet et al. [2001] and Andrieu et al. [2010] for the full particle-system treatment. □

This proposition is mathematically central for the later DPF program. It tells us that the classical bootstrap particle filter supplies an unbiased estimator of the marginal likelihood, even though it does not supply a pathwise differentiable likelihood map. Later differentiable constructions must therefore be compared against this baseline carefully: a differentiable surrogate may be smoother to optimize or differentiate, but it need not preserve this exact estimator status.

20.8 Differentiability boundary

The unbiasedness result above is a value-side Monte Carlo statement. It should not be read as a pathwise differentiability statement. For a fixed random seed, the propagation step may be reparameterizable in some models, but the resampling step is a categorical map from weights to ancestor indices. Small changes in θ can leave the selected ancestor unchanged over an interval and then switch it discontinuously when a cumulative weight boundary is crossed. The realized resampled cloud is therefore not a smooth function of θ.

This distinction creates three separate objects that later chapters must not blur:

(i) the true likelihood $p_\theta(y_{1:T})$;

(ii) an unbiased randomized estimator $\hat{p}_\theta^N(y_{1:T})$ of that likelihood;

(iii) a pathwise differentiable scalar produced by a relaxed, transported, or learned particle computation.

The first object is the statistical target, the second can support pseudo-marginal value-side constructions under their own assumptions, and the third may be useful for optimization or HMC only after its target status is stated. A gradient through the third object is not automatically a gradient of the first or of the expectation of the second.

20.9 Degeneracy and effective sample size

The main numerical weakness of the particle filter is weight degeneracy. As the observation sequence grows, the normalized weights frequently concentrate on a small subset of particles. A standard summary is the effective sample size (ESS),

$$\mathrm{ESS}_t = \frac{1}{\sum_{i=1}^{N} (w_t^{(i)})^2}. \tag{20.18}$$

The extreme cases are informative:

- if all weights are equal, then $\mathrm{ESS}_t = N$;
- if one particle carries almost all the mass, then $\mathrm{ESS}_t \approx 1$.

When ESS collapses, the atomic approximation (20.7) still exists, but its practical variance may become extremely large. Resampling alleviates this by discarding low-weight particles and duplicating high-weight particles, but resampling does not remove the deeper dimensionality problem. The classical literature shows that in many high-dimensional settings the number of particles needed to stabilize the weight distribution can grow very rapidly with the effective dimension of the observation problem [Bengtsson et al., 2008, Snyder et al., 2008].

This is the exact point where the later chapters enter. Particle-flow methods, PF-PF corrections, differentiable resampling, OT relaxations, and learned OT operators are all responses to some combination of:

(i) degeneracy under standard resampling;

(ii) lack of differentiability of the resampling step;

(iii) computational burden of high-quality resampling or transport maps;

(iv) mismatch between a useful learning objective and a valid HMC target.

A mathematically clean DPF monograph must therefore begin here: by making the classical particle-filter object precise before modifying it.

20.10 Baseline audit table

Table 20.3: Classical particle-filter baseline audit.

Object or claim	Status	Assumptions	Implementation diagnostic
Filtering recursion and likelihood factorization	Exact model identity	Dominated transition and observation kernels; fixed θ; finite normalizers.	Compare predictive and filtering factors against a Kalman reference in a linear-Gaussian model.
Trajectory importance ratio	Exact algebraic identity for the specified target/proposal pair	Proposal support covers target support; finite weights.	Check target and proposal log densities separately on a small trajectory.
Weighted empirical measure	Finite-N Monte Carlo approximation	Particle system generated by the declared proposal and weighting rule.	Compare bounded test-function expectations across increasing N.

Object or claim	Status	Assumptions	Implementation diagnostic
Bootstrap likelihood estimator	Unbiased estimator of likelihood, not log likelihood or score	Bootstrap proposal; conditionally unbiased resampling; integrable likelihood factors.	In a controlled model, average repeated estimates and compare to the analytic likelihood.
ESS and ancestor diagnostics	Engineering diagnostic, not posterior-validity proof	Weights normalized and finite; diagnostic interpreted with model dimension and observation informativeness.	Track ESS, log-weight variance, and unique ancestor count by time and parameter region.
Pathwise gradient through resampling	Not supplied by the classical bootstrap filter	Categorical ancestor map is used without relaxation.	Finite-difference a fixed-seed resampled path and identify discontinuities or zero-almost-everywhere regions.

20.11 What this chapter does and does not claim

This chapter establishes the exact nonlinear filtering recursion and the classical bootstrap particle-filter baseline. It does *not* yet claim any of the following:

- a pathwise differentiable likelihood map;
- a particle-flow approximation;
- a proposal-corrected flow filter;
- a differentiable resampling rule;
- an HMC-ready particle backend.

Those belong to the later chapters. The role of the present chapter is more basic and more important: it fixes the reference object against which every later DPF construction must be interpreted.

Chapter 21

Particle-Flow Foundations

This chapter develops the mathematical foundations of particle-flow methods. Where the previous chapter fixed the classical particle-filter baseline, the present chapter studies a different idea: instead of relying on repeated reweighting and resampling to move from the predictive law to the filtering law, one may attempt to transport particles continuously along a pseudo-time path.

This transport point of view is attractive because it addresses a real weakness of classical particle filters: in sharply informative or moderately high-dimensional problems, repeated weight concentration can make the empirical measure numerically poor long before the state-space model itself ceases to be well defined. Particle-flow methods attempt to reduce that instability by constructing a velocity field that pushes the predictive cloud toward the posterior cloud. The mathematical difficulty is that the transport law is not free: one must define the interpolating densities, the continuity equation, the velocity field, and the assumptions under which the resulting flow is exact or only approximate.

This chapter therefore treats particle flow first as a transport problem, not as a final likelihood engine. In particular, the present chapter does *not* yet claim that a raw particle flow defines the final HMC target. It first develops the homotopy and flow mathematics, identifies the Gaussian- closure approximation behind the classical EDH construction, and states the exact special case in which the transport recovers the Kalman update. The subsequent PF-PF chapter then asks how to restore a clearer proposal/target relationship by importance correction.

21.1 Objects, notation, and source roles

The particle-flow literature is easy to misread because the same word "flow" can refer to several distinct objects: a density path, a velocity field, a particle ODE, a numerical map produced by integrating that ODE, or a proposal mechanism later corrected by an importance weight. This chapter uses the following objects.

Symbol or phrase	Meaning in this chapter
$\pi_{t\|t-1}$	predictive law of x_t given $y_{1:t-1}$ before assimilating y_t
$\pi_{t\|t}$	filtering law of x_t given $y_{1:t}$ after assimilating y_t
$\pi_{t,\lambda}$	normalized homotopy density at pseudo-time $\lambda \in [0,1]$
$Z_t(\lambda)$	normalizing constant of the homotopy path
$v_{t,\lambda}$	velocity field whose flow is intended to transport $\pi_{t,\lambda}$
$\Phi_{t,\lambda}$	flow map generated by the particle ODE from pseudo-time 0 to λ
$A_t(\lambda), b_t(\lambda)$	affine EDH velocity coefficients under a Gaussian closure
$P_{t,\lambda}, m_{t,\lambda}$	covariance and mean of the Gaussian homotopy closure
$G_t^{(i)}$	particle-local observation Jacobian in LEDH
$\Lambda_t, \Lambda_t^{(i)}$	global and particle-local likelihood precision contributions
$\eta_t^{(i)}$	particle-local Gaussian information vector in LEDH

The source role is also deliberately limited. Daum–Huang particle flow [Daum and Huang, 2008] motivates the EDH construction. Invertible particle-flow particle filters [Li and Coates, 2017] explain why later chapters must treat a flow as a proposal map with a density correction rather than as an automatically correct posterior sampler. Particle-flow applications such as Hu and van Leeuwen [2021] motivate numerical and high-dimensional caution. These sources locate the method family; they do not remove the need for the local derivations below, and they do not establish a final HMC target for BayesFilter.

21.2 From discrete reweighting to continuous transport

Let the predictive law at time t be

$$\pi_{t|t-1}(dx) = p_\theta(x_t \in dx \mid y_{1:t-1}),$$

and let the filtering law be

$$\pi_{t|t}(dx) = p_\theta(x_t \in dx \mid y_{1:t}).$$

The bootstrap particle filter approximates the transition from $\pi_{t|t-1}$ to $\pi_{t|t}$ by assigning observation-driven weights and then resampling the cloud. Particle-flow methods replace this discrete update by a continuous pseudo-time transport from a prior-like distribution to a posterior-like distribution. The intent is to move particles toward regions of high posterior density before resampling is needed, thereby reducing weight collapse.

The conceptual shift is therefore from

(i) a *weighted empirical update* followed by discrete resampling,

to

(ii) a *continuous transport path* whose endpoint approximates the filtering law.

The advantage of the second view is geometric: the cloud is moved by a velocity field rather than by repeated duplication and deletion of particles. The cost is mathematical and numerical: the transport field must be derived, integrated, and interpreted correctly.

21.3 Homotopy path between predictive and filtering laws

Particle-flow methods introduce a pseudo-time parameter $\lambda \in [0,1]$ and construct an interpolating family of densities $\pi_{t,\lambda}$. Let

$$\bar{\pi}_{t,\lambda}(x) = p_\theta(x_t = x \mid y_{1:t-1}) p_\theta(y_t \mid x)^\lambda$$

be the unnormalized path. The normalized path is

$$\pi_{t,\lambda}(x) = \frac{\bar{\pi}_{t,\lambda}(x)}{Z_t(\lambda)}.$$

Equivalently, in the classical Daum–Huang construction one defines the log-homotopy

$$\log \pi_{t,\lambda}(x) = \log p_\theta(x_t = x \mid y_{1:t-1}) + \lambda \log p_\theta(y_t \mid x) - \log Z_t(\lambda), \tag{21.1}$$

where

$$Z_t(\lambda) = \int p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \, dx_t \tag{21.2}$$

is the normalizing constant.

This definition deserves more emphasis than a short formula often receives. First, the endpoint interpretation is exact:

$$\pi_{t,0}(x) = p_\theta(x_t = x \mid y_{1:t-1}), \qquad \pi_{t,1}(x) = p_\theta(x_t = x \mid y_{1:t}).$$

Thus the family joins the predictive law to the filtering law without changing what the endpoints mean. Second, the normalization is not merely a technical constant. It is the term that keeps the intermediate object a probability density rather than an unnormalized score. Third, the homotopy does not yet choose a transport field; it only chooses the density path the field is supposed to realize.

Differentiating (21.1) with respect to λ gives

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t \mid x) - \partial_\lambda \log Z_t(\lambda). \tag{21.3}$$

The term involving $Z_t(\lambda)$ is crucial: it keeps the intermediate density normalized. Omitting it may produce a convenient algebraic simplification, but it changes the law being transported. This is one of the first places where a future implementation can quietly go wrong: if the code is written only in terms of an unnormalized path, the resulting transported object may no longer match the intended predictive-to-filtering homotopy.

One can make the normalizer term explicit. Differentiating (21.2) gives

$$\frac{dZ_t(\lambda)}{d\lambda} = \int p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \log p_\theta(y_t \mid x_t) \, dx_t.$$

This differentiation under the integral sign is not automatic. A sufficient local condition is that, on the pseudo-time interval being used, the likelihood is positive where the predictive density has support and there is an integrable envelope dominating

$$p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \left|\log p_\theta(y_t \mid x_t)\right|.$$

Equivalently, the predictive law must give finite expectation to the log-likelihood factor along the homotopy and permit dominated differentiation of $Z_t(\lambda)$. If this condition fails, the centered identity below should be treated as a formal diagnostic rather than as a justified derivative calculation. Therefore

$$\partial_\lambda \log Z_t(\lambda) = \int \log p_\theta(y_t \mid x)\, \pi_{t,\lambda}(x)\, dx.$$

The derivative in (21.3) is thus the centered log-likelihood under the intermediate law:

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t \mid x) - \mathbb{E}_{\pi_{t,\lambda}}[\log p_\theta(y_t \mid X_t)].$$

This centered form is a useful audit identity. It shows that the density path preserves total probability, since integrating $\partial_\lambda \pi_{t,\lambda}$ over x gives zero.

For debugging purposes, the main lesson of this section is simple: if a claimed particle-flow implementation does not recover the correct endpoint target even in a simple controlled case, one of the first questions to ask is whether the normalized homotopy was implemented or whether an unnormalized shortcut was silently substituted.

21.4 Continuity equation and transport PDE

Suppose the particle cloud is transported by a velocity field $v_{t,\lambda}(x)$ through the pseudo-time ODE

$$\frac{dx(\lambda)}{d\lambda} = v_{t,\lambda}(x(\lambda)). \tag{21.4}$$

The following derivation is a classical strong-form calculation. It assumes that the density is differentiable in λ, continuously differentiable in x on the region of interest, that $\pi_{t,\lambda} v_{t,\lambda}$ is integrable, and that the boundary term in the integration-by-parts step vanishes. This is satisfied, for example, for smooth rapidly decaying densities on $\mathbb{R}^{n_x}$ with controlled velocity, for compactly supported smooth test functions in the weak form, or for bounded domains with an explicitly imposed no-flux or otherwise stated boundary condition. Without one of these admissible settings, the PDE should be read as a formal transport equation rather than a fully justified global statement. For reviewer-grade use, the weak-form identity with compactly supported test functions is the safer default; the displayed strong-form PDE is used only when the stated differentiability, integrability, and boundary-flux hypotheses hold. Mass conservation over any regular test region then implies that the evolving density must satisfy the continuity equation

$$\partial_\lambda \pi_{t,\lambda}(x) + \nabla \cdot \big(\pi_{t,\lambda}(x) v_{t,\lambda}(x)\big) = 0. \tag{21.5}$$

Equivalently, for any smooth compactly supported test function φ,

$$\frac{d}{d\lambda} \int \varphi(x) \pi_{t,\lambda}(x)\, dx = \int \nabla \varphi(x)^\top v_{t,\lambda}(x) \pi_{t,\lambda}(x)\, dx,$$

and integration by parts gives (21.5). Dividing by $\pi_{t,\lambda}(x)$ where the density is positive yields

$$\partial_\lambda \log \pi_{t,\lambda}(x) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \tag{21.6}$$

Combining (21.3) and (21.6) gives the basic particle-flow PDE:

$$\log p_\theta(y_t \mid x) - \partial_\lambda \log Z_t(\lambda) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \tag{21.7}$$

This equation is exact as a conservation statement. What is not yet fixed is a unique velocity field. The PDE constrains the field but does not determine it without additional structure. In particular, if a vector field has zero weighted divergence with respect to $\pi_{t,\lambda}$, adding it to one solution can leave the same density evolution. EDH and LEDH should therefore be read as particular closure choices, not as the only possible solution of the transport PDE.

21.5 EDH under Gaussian closure

The classical exact Daum–Huang (EDH) construction becomes explicit after a Gaussian closure. The word "exact" in the name is historical and must be read with the regime stated here: the derivation is exact for the Gaussian family it postulates, and it is an exact filtering transport only when that Gaussian family is the true homotopy family.

Assume that the predictive law is approximated by

$$p_\theta(x_t = x \mid y_{1:t-1}) \approx \mathcal{N}(x; m_{t|t-1}, P_{t|t-1})$$

and that the observation density is Gaussian with linear observation map H_t and covariance R_t:

$$p_\theta(y_t \mid x) \propto \exp\left\{-\frac{1}{2}(y_t - H_t x)^\top R_t^{-1}(y_t - H_t x)\right\}.$$

Here $P_{t|t-1}$ and R_t are assumed symmetric positive definite, and the matrix products have the conformable dimensions $H_t \in \mathbb{R}^{n_y \times n_x}$, $P_{t|t-1} \in \mathbb{R}^{n_x \times n_x}$, and $R_t \in \mathbb{R}^{n_y \times n_y}$. These assumptions are what make the inverse and solve notation below meaningful; an implementation should use linear solves and record conditioning diagnostics rather than treating the displayed inverses as a numerical instruction. At pseudo-time λ the interpolating density is then approximated by

$$\pi_{t,\lambda}(x) \approx \mathcal{N}(x; m_{t,\lambda}, P_{t,\lambda}).$$

The likelihood precision is

$$\Lambda_t = H_t^\top R_t^{-1} H_t. \tag{21.8}$$

Collecting the quadratic terms in the log-homotopy gives

$$P_{t,\lambda}^{-1} = P_{t|t-1}^{-1} + \lambda \Lambda_t,$$

so the homotopy covariance satisfies

$$P_{t,\lambda} = \left(P_{t|t-1}^{-1} + \lambda \Lambda_t\right)^{-1}, \tag{21.9}$$

and the matrix inverse derivative gives the covariance evolution. To see the step explicitly, set $M_{t,\lambda} = P_{t|t-1}^{-1} + \lambda \Lambda_t$, so that $M_{t,\lambda} P_{t,\lambda} = I$. Differentiating this identity yields $\Lambda_t P_{t,\lambda} + M_{t,\lambda}(dP_{t,\lambda}/d\lambda) = 0$ and hence

$$\frac{dP_{t,\lambda}}{d\lambda} = -P_{t,\lambda} \Lambda_t P_{t,\lambda}. \tag{21.10}$$

The corresponding information vector is

$$h_{t,\lambda} = P_{t|t-1}^{-1} m_{t|t-1} + \lambda H_t^\top R_t^{-1} y_t, \qquad m_{t,\lambda} = P_{t,\lambda} h_{t,\lambda}.$$

Differentiating $m_{t,\lambda}$ gives

$$\frac{dm_{t,\lambda}}{d\lambda} = -P_{t,\lambda}\Lambda_t m_{t,\lambda} + P_{t,\lambda}H_t^\top R_t^{-1} y_t.$$

The EDH ansatz now chooses an affine velocity field

$$v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda), \tag{21.11}$$

whose mean and covariance evolution match the Gaussian homotopy. If $X_\lambda \sim \mathcal{N}(m_{t,\lambda}, P_{t,\lambda})$ evolves under this affine field, then

$$\frac{dm_{t,\lambda}}{d\lambda} = A_t(\lambda)m_{t,\lambda} + b_t(\lambda), \qquad \frac{dP_{t,\lambda}}{d\lambda} = A_t(\lambda)P_{t,\lambda} + P_{t,\lambda}A_t(\lambda)^\top.$$

Choosing $A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t$ matches (21.10). The offset $b_t(\lambda)$ is then determined by the mean equation:

$$b_t(\lambda) = \frac{dm_{t,\lambda}}{d\lambda} - A_t(\lambda)m_{t,\lambda}.$$

Using

$$m_{t,\lambda} = m_{t|t-1} + \lambda P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1})$$

rewrites this offset in the standard BayesFilter EDH convention. The coefficients are

$$A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t, \tag{21.12}$$

$$b_t(\lambda) = -A_t(\lambda)m_{t|t-1} + \big(I + \lambda A_t(\lambda)\big)P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1}). \tag{21.13}$$

Thus the EDH pseudo-time ODE is

$$\frac{dx}{d\lambda} = A_t(\lambda)x + b_t(\lambda). \tag{21.14}$$

The Gaussian closure should be read with care. The continuity equation and the homotopy construction are exact at their own level. The approximation enters when one assumes that the intermediate family remains Gaussian and then chooses an affine velocity field that is consistent with that Gaussian family. In other words, EDH is exact as a transport formula only when the closure assumption is exactly true; otherwise it is a mathematically structured approximation.

A future implementation should therefore separate four tasks explicitly:

(i) evaluate the predictive Gaussian inputs $(m_{t|t-1}, P_{t|t-1})$;

(ii) build the precision contribution Λ_t;

(iii) build the information contribution $H_t^\top R_t^{-1} y_t$ or its locally shifted analogue;

(iv) integrate the affine pseudo-time dynamics (21.14) using a numerically stable scheme.

If the transported cloud behaves unexpectedly, the first debugging question is not merely whether the ODE solver was stable, but also whether the Gaussian closure itself was plausible for the state-space regime under study.

21.6 Linear-Gaussian recovery as the exact special case

The most important exactness statement for EDH is the linear-Gaussian recovery result. Suppose the predictive law is Gaussian and the observation model is truly linear-Gaussian. Then the homotopy family remains Gaussian for all λ. The covariance endpoint at $\lambda = 1$ is

$$P_{t,1} = \left(P_{t|t-1}^{-1} + H_t^\top R_t^{-1} H_t\right)^{-1}, \tag{21.15}$$

which is exactly the Kalman posterior covariance in information form. The mean endpoint follows from the endpoint information vector:

$$m_{t,1} = P_{t,1}\left(P_{t|t-1}^{-1} m_{t|t-1} + H_t^\top R_t^{-1} y_t\right). \tag{21.16}$$

Using the Kalman gain

$$K_t = P_{t|t-1} H_t^\top (H_t P_{t|t-1} H_t^\top + R_t)^{-1},$$

the same mean can be written as

$$m_{t,1} = m_{t|t-1} + K_t(y_t - H_t m_{t|t-1}).$$

Thus, in the linear-Gaussian case, EDH is not merely an approximation: it transports the predictive Gaussian law to the exact Kalman update, provided the affine ODE is integrated exactly or to controlled numerical tolerance.

This statement is crucial for BayesFilter because it supplies the exact special case against which any EDH implementation should be tested. It is also the boundary line for the rest of the chapter: outside this special case, the particle-flow path is no longer guaranteed exact merely by writing down the homotopy.

21.7 LEDH and local linearization

The local EDH (LEDH) construction replaces the single global observation linearization by a particle-specific local linearization. This is not merely an implementation refinement. It changes the closure from one Gaussian homotopy shared by the cloud to a family of particle-local Gaussian approximations.

For particle $x^{(i)}$, let

$$G_t^{(i)} = \left.\frac{\partial g_\theta(x)}{\partial x}\right|_{x=x^{(i)}} \tag{21.17}$$

and write the local linear approximation of the observation map as

$$g_\theta(x) \approx g_\theta(x^{(i)}) + G_t^{(i)}(x - x^{(i)}). \tag{21.18}$$

Equivalently, the local observation equation is approximated by

$$y_t \approx G_t^{(i)} x + \left(g_\theta(x^{(i)}) - G_t^{(i)} x^{(i)}\right) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_t).$$

Collecting quadratic and linear terms in this local Gaussian likelihood gives the local precision

$$\Lambda_t^{(i)} = (G_t^{(i)})^\top R_t^{-1} G_t^{(i)} \tag{21.19}$$

and the local Gaussian information vector

$$\eta_t^{(i)} = (G_t^{(i)})^\top R_t^{-1}\big(y_t - g_\theta(x^{(i)}) + G_t^{(i)} x^{(i)}\big), \tag{21.20}$$

which is the particle-wise analogue of the global Gaussian information term $H_t^\top R_t^{-1} y_t$ in EDH. The corresponding local homotopy covariance is

$$P_{t,\lambda}^{(i)} = \big(P_{t|t-1}^{-1} + \lambda \Lambda_t^{(i)}\big)^{-1}. \tag{21.21}$$

The associated local mean is

$$m_{t,\lambda}^{(i)} = P_{t,\lambda}^{(i)} \left(P_{t|t-1}^{-1} m_{t|t-1} + \lambda \eta_t^{(i)}\right).$$

The local linearization of the observation map near particle $x^{(i)}$ therefore produces particle-specific flow coefficients

$$A_t^{(i)}(\lambda) = -\frac{1}{2} P_{t,\lambda}^{(i)} \Lambda_t^{(i)}, \tag{21.22}$$

$$b_t^{(i)}(\lambda) = -A_t^{(i)}(\lambda) m_{t|t-1} + \big(I + \lambda A_t^{(i)}(\lambda)\big) P_{t,\lambda}^{(i)} \eta_t^{(i)}, \tag{21.23}$$

with pseudo-time evolution

$$\frac{dx^{(i)}}{d\lambda} = A_t^{(i)}(\lambda) x^{(i)} + b_t^{(i)}(\lambda). \tag{21.24}$$

This construction explains both the attraction and the danger of LEDH. The attraction is that each particle uses a local approximation intended to track nonlinear observation geometry more closely than a single global linearization. The danger is that the transported law now depends on a family of particle-specific linearizations, each of which may behave poorly if the Jacobian is unstable, ill-conditioned, or evaluated in a region where the local approximation is misleading. Hence LEDH is potentially more adaptive than EDH, but it is also more fragile numerically and interpretively. The chapter should not phrase LEDH as guaranteed improvement without model-specific evidence.

From an implementation and debugging standpoint, LEDH adds at least three new failure routes relative to EDH:

(i) inaccurate or unstable local Jacobians;

(ii) poor local-information-vector construction;

(iii) path-dependent variation in stiffness across particles.

These are not secondary engineering details. They are direct consequences of how the local approximation enters the mathematics.

21.8 Stiffness and discretization

Even when the flow equations are well defined, their numerical integration may be difficult. The main difficulty is stiffness. When the observation noise is small or the effective likelihood precision is large, the eigenvalues of the drift matrix in the EDH or LEDH ODE can span several orders of magnitude. In that regime, explicit solvers require very small pseudo-time steps to avoid large truncation error or instability.

Formally, the stiffness is tied to the conditioning of the homotopy covariance or precision family. In the local case, for example, large eigenvalue spreads in $\Lambda_t^{(i)}$ may force extremely fine pseudo-time resolution near the posterior end of the path. This matters for implementation because the flow construction is only useful if it can be integrated stably and repeatedly inside a filtering recursion.

The practical burden can be stated more concretely. If the pseudo-time ODE is integrated with an explicit method such as RK4, then the local truncation error is governed by powers of the step size, but the stability envelope is controlled by the operator norm and spectral spread of the drift matrix. In sharp- likelihood regimes, decreasing the observation noise or increasing the local precision can force the stable step size down dramatically. For LEDH this may happen non-uniformly across the cloud, because different particles see different local Jacobians and therefore different local precision matrices.

This is exactly why stiffness belongs in the monograph as more than a short warning. It creates a direct literature-to-debugging bridge:

- if a flow implementation behaves erratically only when observation noise becomes small, stiffness is a prime mathematical suspect;

- if certain particles show unstable trajectories while others remain well behaved, the local LEDH precision field is a prime suspect;

- if a nominally correct flow map fails under a coarse pseudo-time grid, the first question should be whether the flow is underresolved rather than whether the underlying theory is wrong.

For BayesFilter, the main consequence is therefore both conceptual and operational. Stiffness is not merely a performance issue. It is one of the main ways in which a formally attractive flow construction can become numerically unreliable enough to alter the practical behavior of the filter. That is why later implementation-facing chapters must record discretization policy, stiffness diagnostics, and stress tests explicitly, and why the literature pointers for stiffness should be treated as debugging tools rather than as optional background reading.

21.9 Exactness and approximation ledger

The particle-flow chapter has several layers that must not be collapsed.

Layer	Status	Main implementation diagnostic
Normalized homotopy endpoints	exact model identity when the predictive density and likelihood are the intended model objects	recover predictive law at $\lambda = 0$ and filtering law at $\lambda = 1$ in analytic test cases
Continuity equation	exact conservation statement under regularity and boundary assumptions	verify transported test-function moments against ODE integration in controlled problems
Velocity field choice	not unique; EDH/LEDH are closure choices	record the chosen field and compare alternatives on benchmark models
EDH Gaussian closure	exact for the Gaussian family; exact filtering only in linear-Gaussian recovery	test covariance and mean endpoints against the Kalman update
LEDH local linearization	approximate closure using particle-local Jacobians and information vectors	monitor Jacobian conditioning, local precision spectra, and particle-wise endpoint dispersion
Pseudo-time integration	numerical approximation to the chosen ODE	run step size, tolerance, stiffness, and log-det sensitivity checks

This ledger is deliberately conservative. It permits a strong statement about linear-Gaussian recovery, but it blocks any inference that a raw EDH or LEDH cloud is already a corrected nonlinear filter, a pseudo-marginal likelihood estimator, or an HMC-ready target.

21.10 What this chapter does and does not claim

This chapter develops the particle-flow transport mathematics itself. It does *not* yet claim:

- that the transported cloud alone defines the final corrected filtering law in the nonlinear case;
- that raw EDH or LEDH transport is already the final HMC target;
- that the Jacobian/proposal correction issue has been resolved;
- that differentiable resampling or OT relaxations have been introduced.

Those belong to the later PF-PF and resampling chapters. The role of the present chapter is narrower and mathematically prior: it explains how the transport path is constructed, where the Gaussian-closure or local-linearization approximation enters, and why the linear-Gaussian case supplies the exact special benchmark.

Chapter 22

Particle-Flow Particle Filters and Proposal Correction

The previous chapter developed particle flow as a transport construction. The main unresolved question left there was whether the transported cloud itself may already be treated as the corrected filtering object. In general, the answer is no. Outside exact special cases, the flow should be interpreted first as a proposal mechanism. The role of the present chapter is to make that proposal interpretation explicit and to derive the corresponding importance correction.

This chapter is therefore the mathematical bridge between raw flow transport and a more principled particle-based likelihood interpretation. It does not yet introduce differentiable resampling or optimal transport relaxations. Instead, it asks a prior question: once a particle has been moved by a deterministic flow map, what density does that transported particle actually have, and what importance weight is required to target the intended posterior law?

22.1 Objects, assumptions, and source roles

The PF-PF argument depends on naming the target and proposal objects before writing any weight. This chapter uses:

Object	Meaning
x_{t-1}	ancestor state after the previous filtering step
$x_{t,0}$	pre-flow particle sampled from a one-step proposal
$x_{t,1}$	post-flow particle after applying the flow map
$q_{t,0}$	pre-flow proposal density for $x_{t,0}$ conditional on (x_{t-1}, y_t)
Φ_t	differentiable flow map from pre-flow to post-flow coordinates
Φ_t^{-1}	inverse map used to express the pre-image of a post-flow particle
$J_t = D\Phi_t$	forward Jacobian matrix of the flow map
$q_{t,1}$	induced post-flow proposal density for $x_{t,1}$
γ_t	unnormalized one-step filtering target density
$\tilde{w}_t$	unnormalized importance weight correcting $q_{t,1}$ to γ_t

The source role is narrow. General SMC references [Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020] supply the proposal and importance-weight framework. Invertible particle-flow particle filtering [Li and Coates, 2017] motivates treating particle flow as a

proposal transformation with a Jacobian correction. These sources do not remove the need to state the target/proposal ratio locally, and they do not imply that finite-particle, numerical, or HMC target issues have disappeared.

22.2 Why proposal correction is needed

In the linear-Gaussian special case studied in the previous chapter, EDH can recover the exact Kalman update. Outside that case, however, the transport map is typically built under Gaussian closure or local linearization. Even if the transport moves particles toward high posterior density, the moved cloud is not automatically distributed according to the exact filtering law.

The right interpretation is therefore proposal-theoretic. Let $x_{t,0}$ denote a pre-flow particle sampled from a proposal law, and let the flow map transport it to

$$x_{t,1} = \Phi_t(x_{t,0}). \tag{22.1}$$

The question is then not merely where the particle moved, but which density on $x_{t,1}$ is induced by the transformation and how that induced proposal density must be corrected to target the filtering posterior.

This is the basic PF-PF point emphasized in the invertible particle-flow literature: the flow is best viewed as a proposal-improving device until its proposal-density effect has been accounted for. The exact change-of-variables identity is not itself the approximation; the approximation enters through the choice of flow family, the numerical integration of the flow, and the finite particle system.

22.3 Change of variables under the flow map

Assume that for each fixed time step and observation, the flow map $\Phi_t : \mathbb{R}^{n_x} \to \mathbb{R}^{n_x}$ is a differentiable bijection on the region of interest and that its forward Jacobian $D\Phi_t(x_{t,0})$ is an $n_x \times n_x$ nonsingular matrix along the proposal paths being weighted. Equivalently, the calculation below is a local diffeomorphism calculation on the support actually used by the proposal. Let the pre-flow proposal density be

$$q_{t,0}(x_{t,0} \mid x_{t-1}, y_t). \tag{22.2}$$

Then the transported particle density is given by the standard change-of-variables formula,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(x_{t,0} \mid x_{t-1}, y_t) \left|\det D\Phi_t(x_{t,0})\right|^{-1}, \tag{22.3}$$

where $x_{t,0} = \Phi_t^{-1}(x_{t,1})$. The determinant in (22.3) is the determinant of the *forward* map from $x_{t,0}$ to $x_{t,1}$. Equivalently,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(\Phi_t^{-1}(x_{t,1}) \mid x_{t-1}, y_t) \left|\det D\Phi_t^{-1}(x_{t,1})\right|.$$

These two forms are identical because $|\det D\Phi_t^{-1}(x_{t,1})| = |\det D\Phi_t(x_{t,0})|^{-1}$. The chapter uses the forward-map convention because the flow ODE naturally evolves $x_{t,0}$ to $x_{t,1}$ and the log-determinant ODE below accumulates $\log|\det D\Phi_t(x_{t,0})|$.

Thus the Jacobian determinant is not an optional numerical decoration. It is part of the proposal density itself. Any flow-based importance scheme that forgets this factor changes the proposal law and therefore changes the targeting relationship of the resulting importance weights.

22.4 Proposal-corrected PF-PF weights

Let the intended one-step filtering target be proportional to

$$\gamma_t(x_t) \propto p_\theta(y_t \mid x_t) p_\theta(x_t \mid x_{t-1}). \tag{22.4}$$

This target is conditional on the ancestor state. In a full particle filter, the ancestor itself is random and selected from the previous weighted cloud. The present derivation is therefore a one-step conditional density-ratio calculation; the full likelihood estimator also depends on ancestor selection, resampling policy, particle count, and time recursion.

For a generic pre-flow proposal $q_{t,0}$, the corrected importance weight is

$$\tilde{w}_t \propto \frac{\gamma_t(x_{t,1})}{q_{t,1}(x_{t,1} \mid x_{t-1}, y_t)} = \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|}{q_{t,0}(x_{t,0} \mid x_{t-1}, y_t)}. \tag{22.5}$$

The bootstrap/prior-proposal formula below is the special case $q_{t,0} = p_\theta(\cdot \mid x_{t-1})$.

If the pre-flow particle is sampled from the transition prior,

$$q_{t,0}(x_{t,0} \mid x_{t-1}, y_t) = p_\theta(x_{t,0} \mid x_{t-1}), \tag{22.6}$$

and then transported to $x_{t,1} = \Phi_t(x_{t,0})$, the post-flow proposal density is

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = p_\theta(x_{t,0} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|^{-1}. \tag{22.7}$$

The corresponding unnormalized importance weight is therefore

$$\tilde{w}_t \propto \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1})}{q_{t,1}(x_{t,1} \mid x_{t-1}, y_t)} = \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|}{p_\theta(x_{t,0} \mid x_{t-1})}. \tag{22.8}$$

Equation (22.8) is the central correction formula for the PF-PF viewpoint. It shows exactly what proposal correction restores: once the transformation of proposal density has been accounted for, the flow-transported particle may be reweighted against the desired posterior factor in the usual importance-sampling way.

The formula also shows exactly what is *not* optional in code. A PF-PF implementation must evaluate, on the same path, the observation density at the post-flow particle, the transition density at the post-flow particle, the pre-flow proposal density at the pre-image, and the forward log determinant. Changing any one of these objects changes the proposal-to-target ratio being computed.

22.5 EDH/PF and LEDH/PF

The correction formula is structurally the same whether the flow map comes from EDH or LEDH. The difference lies in how the map itself is built.

22.5.1 EDH/PF

For EDH/PF, the transport map is generated by the global affine ODE from the previous chapter. The resulting proposal is easier to analyze because the flow field is common across the cloud. The main approximation point is the Gaussian closure used to derive the affine coefficients. If the affine ODE is integrated with a common map Φ_t, the same forward log determinant applies to every particle up to its path through the time-varying affine field.

22.5.2 LEDH/PF

For LEDH/PF, the map is particle-specific: each particle evolves under a local linearization and therefore under its own drift coefficients. This can improve local adaptation to nonlinear observation structure, but it also makes the proposal analysis more delicate. The local map remains proposal-correctable in principle through the same change-of-variables logic, but the numerical burden of evaluating particle-specific Jacobians or log-determinants becomes more significant. A code path that reuses an EDH log determinant for LEDH has changed the proposal density unless the local maps are actually identical.

The important mathematical point is that EDH/PF and LEDH/PF are not merely flow algorithms. They are proposal-corrected particle methods. That is why they occupy a different target-status category from raw flow transport.

22.6 Jacobian determinant and log-determinant evolution

Directly forming a full Jacobian matrix for every particle and time step may be numerically and computationally burdensome. A more useful identity is obtained by differentiating the Jacobian matrix along the flow.

Let

$$J_t(\lambda) = \frac{\partial x_t(\lambda)}{\partial x_t(0)}. \tag{22.9}$$

The terminal forward determinant in (22.3) is $\det J_t(1)$. The inverse-density form would use $-\log|\det J_t(1)|$ instead. This sign distinction is one of the most common PF-PF implementation errors.

If the flow satisfies

$$\frac{dx_t(\lambda)}{d\lambda} = v_{t,\lambda}(x_t(\lambda)), \tag{22.10}$$

then differentiation with respect to the initial point gives

$$\frac{dJ_t(\lambda)}{d\lambda} = \nabla v_{t,\lambda}(x_t(\lambda))J_t(\lambda), \qquad J_t(0) = I. \tag{22.11}$$

The derivation is direct: differentiate the flow equation with respect to the initial coordinate $x_t(0)$ and apply the chain rule. Applying Jacobi's formula $d\log|\det J|/d\lambda = \operatorname{tr}(J^{-1}dJ/d\lambda)$ then yields

$$\frac{d}{d\lambda}\log|\det J_t(\lambda)| = \operatorname{tr}\big(\nabla v_{t,\lambda}(x_t(\lambda))\big). \tag{22.12}$$

because $J_t^{-1}(\nabla v_{t,\lambda})J_t$ has the same trace as $\nabla v_{t,\lambda}$. Thus

$$\log|\det D\Phi_t(x_{t,0})| = \int_0^1 \operatorname{tr}\big(\nabla v_{t,\lambda}(x_t(\lambda))\big)\, d\lambda.$$

For an affine flow $v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda)$, this simplifies to

$$\frac{d}{d\lambda}\log|\det J_t(\lambda)| = \operatorname{tr}(A_t(\lambda)). \tag{22.13}$$

The practical consequence is clear: one may integrate a scalar log-determinant alongside the particle-flow ODE rather than carrying a full dense Jacobian for every particle in the affine case. In more general settings, the same identity still shows that the determinant is part of the dynamical system, not an external afterthought. This also gives a direct debugging criterion: if the corrected weights behave inconsistently with the transported cloud, the first mathematical suspects are the Jacobian/log-determinant path, the numerical flow integration, and the finite-particle approximation rather than the formal change-of-variables identity itself.

22.7 What the correction restores, and what it does not

The proposal correction in (22.8) restores a clearer importance-sampling interpretation. More precisely, it restores the correct proposal-to-target density ratio for the transported proposal generated by the flow map. This is mathematically important: it distinguishes PF-PF from a pure transport approximation that simply declares the endpoint cloud to be posterior- like without density correction.

What the correction does *not* automatically restore is an exact analytic filter in the nonlinear case. Several approximation layers may still remain:

- the flow may itself be derived under Gaussian closure or local linearization;
- the ODE is integrated numerically rather than in closed form;
- the particle system still uses a finite number of particles;
- Jacobian or log-determinant quantities may be computed with numerical approximation.

Therefore PF-PF is mathematically stronger than raw flow transport, but it is not automatically equivalent to an exact nonlinear filtering likelihood. The exact identity in this chapter is the change-of-variables correction for the proposal density; what remains approximate are the flow construction itself when closure or local linearization is used, the numerical integration of that flow, and the finite-particle system used to estimate the target. Its status is better described as a proposal-corrected particle approximation whose value-side interpretation is clearer than that of uncorrected flow transport.

At trajectory level, this one-step correction is multiplied through the filtering recursion in the same spirit as ordinary sequential importance sampling. For a particle path and ancestor history, the product of incremental corrections defines the path proposal weight. That statement is still conditional on using the stated proposal, the stated flow map, and the stated determinant convention at each time step. It is not a separate theorem that the finite-particle likelihood estimate is exact in the nonlinear case.

22.8 Implementation audit obligations

The formulas above imply concrete checks before PF-PF can be used as a value-side candidate in later HMC experiments.

(i) **Affine density check**: choose a low-dimensional affine map with known determinant and compare (22.3) against a direct transformed-density calculation.

(ii) **Forward/inverse sign check**: verify that the proposal density uses $-\log|\det D\Phi_t|$ while the numerator form in (22.8) contributes $+\log|\det D\Phi_t|$.

(iii) **Jacobian ODE check**: compare the integrated trace in (22.12) with an automatic-differentiation or finite-difference determinant in small dimension.

(iv) **Corrected versus uncorrected weights**: run the same transported cloud with and without the determinant/proposal-density term to confirm that the diagnostic detects the expected discrepancy.

(v) **Path consistency**: on fixed random seeds, verify that the scalar value and any autodiff gradient use the same proposal density, flow map, and log-determinant path.

These are not tuning diagnostics. They are algebraic and numerical checks of the target/proposal object itself.

22.9 Why this chapter matters for the later HMC discussion

This chapter is the first point in the rebuilt DPF block where a sharper target question becomes possible. Raw flow transport gives a geometrically appealing approximation, but PF-PF introduces an explicit proposal density and an explicit correction factor. That is why later HMC-suitability analysis will treat proposal-corrected EDH/PF or LEDH/PF as the first rungs in this ladder with an explicit proposal-corrected value-side interpretation. That is a narrower statement than saying they are HMC-ready.

The key lesson is not that PF-PF solves all target issues. The key lesson is that it restores the proposal-correction logic that a valid particle method requires. This is exactly the right place in the monograph to separate flow-as-transport from flow-as-proposal.

22.10 What this chapter does and does not claim

This chapter derives the proposal-correction structure for flow-based particle methods. It does *not* yet claim:

- that differentiable resampling has been resolved;
- that the resulting method is already the final HMC target for nonlinear DSGE or MacroFinance models;
- that OT or learned transport relaxations have been introduced;
- that finite-particle, discretization, and Jacobian numerical issues are negligible.

Those belong to the later resampling and HMC-assessment chapters. The role of this chapter is more specific: it explains mathematically how a particle flow is turned into a proposal-corrected particle method and what that correction does and does not restore.

Chapter 23

Differentiable Resampling and Optimal Transport

The previous chapter showed how a particle flow may be turned into a proposal- corrected particle method. The present chapter addresses a different mathematical difficulty: the standard resampling step is discrete and therefore not pathwise differentiable. If one wants a differentiable particle method, one must either abandon pathwise differentiation at the resampling step or replace the resampling mechanism by a smooth relaxation.

This chapter studies the main relaxations relevant to the BayesFilter DPF program. The key point is that differentiability is not obtained for free. It is obtained by changing the resampling map, and therefore by changing the object whose gradient is computed. The chapter is organized to make that trade-off explicit. Its source spine is the differentiable-resampling literature [Zhu et al., 2020, Corenflos et al., 2021] together with the optimal-transport and Sinkhorn background in Villani [2003], Cuturi [2013], and Peyré and Cuturi [2019].

23.1 Objects, conventions, and source roles

The chapter uses the following objects. Naming them up front is important because several objects below are sometimes all called "resampling" even though they are different mathematical maps.

Object	Meaning
$\hat{\pi}_t^N$	weighted empirical measure before resampling
$\hat{\pi}_{t,\mathrm{eq}}^N$	equal-weight empirical measure after a resampling or transport step
a_j	discrete ancestor index selected by a categorical resampling rule
α	soft-resampling interpolation parameter
Π	transport coupling between source particles and equal target masses
$\mathcal{U}(w,u)$	feasible set of nonnegative couplings with source marginal w and target marginal u
C	transport cost matrix
$H(\Pi)$	entropy convention, defined below as $-\sum_{ij}\Pi_{ij}(\log\Pi_{ij}-1)$
Π_ε^*	exact optimizer of the entropic OT problem for fixed $\varepsilon>0$
a,b	Sinkhorn scaling vectors in the factorization $\Pi = \mathrm{diag}(a)K_\varepsilon\,\mathrm{diag}(b)$
$\tilde{x}_t^{(j)}$	barycentric output particle after the coupling is converted to a deterministic equal-weight cloud
$\Pi_{\varepsilon,K}$	finite-iteration Sinkhorn approximation computed in code

The source roles are deliberately limited. The differentiable-resampling sources motivate the need for a smooth replacement of hard ancestor selection; the OT sources justify the coupling formulation; the Sinkhorn sources justify the entropic scaling form and numerical caveats. None of these citations by itself proves that a relaxed resampling operator preserves the original categorical law, preserves a pseudo-marginal likelihood estimator, or supplies an HMC-valid target. Those statuses must be established by the local equations and by later target analysis.

23.2 The resampling bottleneck

At time t, suppose the particle system after propagation and weighting is represented by the weighted empirical measure

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^N w_t^{(i)}\,\delta_{x_t^{(i)}}(dx), \qquad w_t^{(i)} \geq 0, \quad \sum_{i=1}^N w_t^{(i)} = 1. \tag{23.1}$$

Classical resampling attempts to replace this weighted cloud by an equally weighted empirical measure

$$\hat{\pi}_{t,\mathrm{eq}}^N(dx) = \frac{1}{N}\sum_{j=1}^N \delta_{\tilde{x}_t^{(j)}}(dx), \tag{23.2}$$

while preserving the filtering information carried by the weighted cloud as well as possible.

For standard multinomial resampling, the new support points are selected by random ancestor indices $a_j \sim \mathrm{Categorical}(w_t^{(1:N)})$ and then setting $\tilde{x}_t^{(j)} = x_t^{(a_j)}$. This map is perfectly natural as a sampling operation, but it is not pathwise differentiable with respect to the weights. That is the resampling bottleneck.

23.3 Standard resampling as a discontinuous map

Let $U_j \sim \text{Uniform}(0,1)$ and write the cumulative weights as

$$C_k = \sum_{i=1}^{k} w_t^{(i)}.$$

Multinomial resampling chooses the ancestor index a_j such that

$$C_{a_j-1} < U_j \leq C_{a_j}. \tag{23.3}$$

Thus the map from weights to ancestors is piecewise constant. Away from the measure-zero boundaries where a uniform draw lands exactly on a cumulative weight break, the selected index does not change under an infinitesimal weight perturbation. Consequently, the pathwise derivative of the resampled particle with respect to the weights is zero almost everywhere.

The one-dimensional two-particle case makes the discontinuity concrete. Let $N = 2$, $w = (p, 1-p)$, and draw a fixed common random number $U \in (0,1)$. The first ancestor is selected when $U \leq p$ and the second ancestor is selected when $U > p$:

$$a(U,p) = \begin{cases} 1, & U \leq p, \\ 2, & U > p. \end{cases} \tag{23.4}$$

For any fixed U, the realized map $p \mapsto a(U,p)$ is a step function. Its classical derivative is zero away from $p = U$ and undefined at $p = U$. Reusing the same U makes the discontinuity visible; redrawing U changes the random map rather than supplying a pathwise derivative of the realized map.

This is the precise sense in which standard resampling blocks a naive pathwise autodiff chain. The issue is not that gradients are numerically inconvenient. It is that the ancestor-selection map itself is discontinuous. Score-function or common-random-number estimators can still be designed around the discrete operation, but they are not the same as differentiating the realized ancestor selection path. BayesFilter's DPF question is specifically about the latter: whether the filtering likelihood path supplied to an HMC or training routine is a smooth value-gradient computation.

23.4 Soft resampling

A simple differentiable replacement is soft resampling. Instead of selecting a new support point only from the categorical ancestor, one interpolates between the ancestor particle and the weighted mean of the cloud [Zhu et al., 2020]. A representative form is

$$\tilde{x}_t^{(j)} = \alpha x_t^{(a_j)} + (1-\alpha)\bar{x}_t, \qquad \bar{x}_t = \sum_{i=1}^{N} w_t^{(i)} x_t^{(i)}, \tag{23.5}$$

where $\alpha \in [0,1]$ is an interpolation parameter.

For the bias calculation below, take $x_i, \bar{x} \in \mathbb{R}^{n_x}$, weights $w_i \geq 0$ with $\sum_i w_i = 1$, and a scalar test function $\varphi : \mathbb{R}^{n_x} \to \mathbb{R}$ that is twice continuously differentiable on a neighborhood containing the line segments between x_i and $\bar{x}$. The gradient $\nabla\varphi(x_i)$ is treated as a column vector, so $\nabla\varphi(x_i)'(x_i - \bar{x})$ is a scalar directional derivative.

The key structural point is immediate:

- if $\alpha = 1$, one recovers standard multinomial resampling and loses pathwise differentiability;
- if $\alpha < 1$, the interpolation term depends smoothly on the weighted mean, but the sampled ancestor a_j is still a discrete random index unless the method also replaces or conditions on that draw.

The benefit is therefore a smoother surrogate path, not an exact pathwise derivative of categorical resampling. If the ancestor draw is held fixed, the map is differentiable only on that conditional branch; if the ancestor draw is also relaxed, the target object has changed further and must be named. The cost is bias. The weighted mean is preserved because, conditional on the current weighted cloud,

$$\mathbb{E}[\tilde{x}_t^{(j)} \mid X, w] = \alpha \sum_{i=1}^{N} w_i x_i + (1-\alpha)\bar{x} = \bar{x}. \tag{23.6}$$

Consequently every affine test function $\varphi(x) = c + d'x$ is preserved in conditional expectation. Nonlinear functionals of the cloud need not be preserved, because the argument of the test function is changed before the expectation is taken. For a smooth scalar test function φ,

$$\mathbb{E}[\varphi(\tilde{x}_t^{(j)}) \mid X, w] = \sum_i w_i \varphi(\alpha x_i + (1-\alpha)\bar{x})\,. \tag{23.7}$$

A first-order Taylor expansion around x_i gives the displayed bias. To see the sign and order explicitly, write $d_i = (1-\alpha)(\bar{x} - x_i)$. Then

$$\alpha x_i + (1-\alpha)\bar{x} = x_i + d_i$$

and, for each i,

$$\varphi(x_i + d_i) = \varphi(x_i) + \nabla\varphi(x_i)'d_i + \frac{1}{2} d_i' \nabla^2 \varphi(\xi_i) d_i$$

for some point ξ_i on the segment between x_i and $\alpha x_i + (1-\alpha)\bar{x}$. If the Hessian is bounded on the segment neighborhood, summing the remainders gives $O((1-\alpha)^2)$. Therefore

$$\mathbb{E}[\varphi(\tilde{x}_t^{(j)}) \mid X, w] - \sum_i w_i \varphi(x_i) = -(1-\alpha) \sum_i w_i \nabla\varphi(x_i)'(x_i - \bar{x}) + O((1-\alpha)^2). \tag{23.8}$$

Thus the linear test-function case is preserved, but nonlinear summaries are generally shifted. Hence soft resampling is naturally interpreted as a differentiable surrogate rather than as an exact replacement for multinomial resampling. The shrinkage can also reduce cloud diversity: if $\alpha < 1$, each selected ancestor is pulled toward the same weighted mean. This may stabilize gradients, but it changes the empirical distribution whose downstream likelihood or loss is being differentiated.

23.5 Resampling as a transport problem

A more geometric way to formulate equal-weight resampling is to view it as a transport problem. The starting measure is the weighted empirical measure $\hat{\pi}_t^N$ in (23.1); the target class is the set of equal-weight empirical measures of size N [Villani, 2003, Reich, 2013].

This makes it natural to ask for a transport plan that moves the weighted cloud to an equally weighted cloud while minimizing a transport cost. In particular, one may interpret equal-weighting as a projection problem in the Wasserstein geometry of probability measures. Standard resampling does not solve this projection problem exactly; it samples ancestor locations from the original support. Transport-based resampling instead tries to produce an equally weighted cloud through an explicit coupling between the weighted and uniform measures.

With source atoms x_i, target weights $u_j = 1/N$, and nonnegative cost C_{ij}, the coupling feasible set is

$$\mathcal{U}(w,u) = \left\{ \Pi \in \mathbb{R}_+^{N\times N} : \sum_{j=1}^{N} \Pi_{ij} = w_i,\ \sum_{i=1}^{N} \Pi_{ij} = u_j \right\}. \tag{23.9}$$

The unregularized coupling problem is

$$\Pi_0^* \in \arg\min_{\Pi\in\mathcal{U}(w,u)} \sum_{i=1}^{N}\sum_{j=1}^{N} \Pi_{ij} C_{ij}. \tag{23.10}$$

This is an exact linear-programming object for the chosen cost and feasible set. It is not a statement that categorical resampling has become differentiable, and it is not a proof that the categorical resampling law has been preserved. Categorical resampling samples support points from the source cloud according to ancestor indices. The OT formulation instead chooses a coupling with prescribed marginals and then usually converts that coupling into a deterministic output cloud. The equality of total mass and target weights is exact within (23.9); equivalence to categorical resampling is not being claimed.

This transport view is mathematically useful because it makes the relaxation explicit: a differentiable transport map is not the same object as the discrete categorical selection map. It also separates two levels that are often blurred in implementation discussions: the mathematical transport problem one wishes to solve, and the numerical or regularized surrogate one actually computes.

23.6 Entropic optimal transport resampling

Let $w = (w_1, \ldots, w_N)$ be the source weights and let $u = (1/N, \ldots, 1/N)$ be the target equal weights. Let the transport cost matrix be

$$C_{ij} = \|x_t^{(i)} - z_j\|^2, \tag{23.11}$$

where z_j are target support points or, in the barycentric form, support locations constructed from the transport plan itself.

Assume for the derivation that $w_i > 0$, $u_j > 0$, $\sum_i w_i = \sum_j u_j = 1$, and that all entries of $C \in \mathbb{R}^{N\times N}$ are finite. A coupling $\Pi \in \mathbb{R}_+^{N\times N}$ has row sums in $\mathbb{R}^N$ and column sums in $\mathbb{R}^N$. The scaling vectors a, b used below also lie in $\mathbb{R}_+^N$; consequently $K_\varepsilon b$, $K_\varepsilon' a$, and the elementwise products in (23.17) are conformable N-vectors.

Following the entropy-regularized OT construction used by Corenflos et al. [2021], the entropic optimal transport problem is

$$\Pi_\varepsilon^* = \arg\min_{\Pi\in\mathcal{U}(w,u)} \langle \Pi, C\rangle - \varepsilon H(\Pi), \tag{23.12}$$

where this chapter uses the convention

$$H(\Pi) = -\sum_{i=1}^{N}\sum_{j=1}^{N} \Pi_{ij}(\log \Pi_{ij} - 1), \qquad \Pi_{ij} > 0. \tag{23.13}$$

With this convention, minimizing $\langle \Pi, C\rangle - \varepsilon H(\Pi)$ is equivalent, up to constants, to minimizing $\sum_{ij} \Pi_{ij} C_{ij} + \varepsilon \sum_{ij} \Pi_{ij}(\log \Pi_{ij} - 1)$ over the coupling polytope. For fixed positive marginals and $\varepsilon > 0$, the entropy term makes the objective strictly convex on the positive part of the feasible set, giving a unique optimizer for the regularized problem. It also changes the mathematical object: Π^*_ε solves a regularized problem, whereas Π^*_0 solves (23.10).

The scaling form follows from the Lagrange first-order conditions. Introduce row and column multipliers λ_i and μ_j and form

$$\mathcal{L}(\Pi, \lambda, \mu) = \sum_{i,j} \Pi_{ij} C_{ij} + \varepsilon \sum_{i,j} \Pi_{ij}(\log \Pi_{ij} - 1) + \sum_i \lambda_i \left(\sum_j \Pi_{ij} - w_i \right) + \sum_j \mu_j \left(\sum_i \Pi_{ij} - u_j \right).$$

For an interior optimizer, differentiating with respect to Π_{ij} gives the stationarity condition

$$C_{ij} + \varepsilon \log \Pi_{ij} + \lambda_i + \mu_j = 0. \tag{23.14}$$

Therefore

$$\Pi_{ij} = \exp(-\lambda_i/\varepsilon) \exp(-C_{ij}/\varepsilon) \exp(-\mu_j/\varepsilon). \tag{23.15}$$

Absorbing the multiplier exponentials into positive scaling vectors gives the Sinkhorn representation

$$\Pi^*_\varepsilon = \operatorname{diag}(a) K_\varepsilon \operatorname{diag}(b), \qquad (K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon), \tag{23.16}$$

with $a_i > 0$ and $b_j > 0$ chosen to satisfy the marginal constraints. Substituting (23.16) into (23.9) gives the scaling equations

$$a \odot (K_\varepsilon b) = w, \tag{23.17}$$
$$b \odot (K'_\varepsilon a) = u. \tag{23.18}$$

The classical Sinkhorn updates are therefore

$$a^{(k+1)} = w \oslash (K_\varepsilon b^{(k)}), \tag{23.19}$$
$$b^{(k+1)} = u \oslash (K'_\varepsilon a^{(k+1)}), \tag{23.20}$$

where $\odot$ and $\oslash$ denote elementwise multiplication and division. These updates solve the exact EOT problem only in the limit of convergence and only for the stated kernel, marginals, cost matrix, and regularization [Cuturi, 2013].

The resulting equally weighted cloud is obtained by barycentric projection. In general, a column with mass $u_j > 0$ has conditional source weights Π^*_{ij}/u_j, so its barycenter is

$$\tilde{x}^{(j)}_t = \frac{1}{u_j} \sum_{i=1}^{N} \Pi^*_{ij} x^{(i)}_t.$$

For the equal-weight target $u_j = 1/N$, this becomes

$$\tilde{x}_t^{(j)} = N \sum_{i=1}^{N} \Pi^*_{ij} x_t^{(i)}. \tag{23.21}$$

The factor N appears because the target marginal is $u_j = 1/N$, so $N\Pi^*_{ij}$ are the conditional weights of source atoms contributing to the jth output particle. If $x_i \in \mathbb{R}^{n_x}$, then $\tilde{x}_t^{(j)} \in \mathbb{R}^{n_x}$ and the full output cloud again has N support points with equal weights. This barycentric step converts a coupling into a deterministic support cloud. It is a map derived from the coupling, not the coupling itself, and not a categorical ancestor draw.

This map is differentiable with respect to the particles and weights only after one specifies what is being differentiated: the exact EOT optimizer Π^*_ε via an implicit differentiation argument, or a finite unrolled Sinkhorn computation. These are related but not identical objects. At this point the chapter must keep four layers distinct: the exact categorical resampling law, the unregularized OT problem, the entropically regularized OT problem, and the finite-iteration Sinkhorn solve used to approximate the regularized OT optimum in practice.

Differentiated object	Value object	Derivative route	Non-claim
Exact regularized EOT optimizer	$\Pi^\star_\varepsilon$ and its barycentric projection for fixed C, w, u, ε	Implicit differentiation of the optimality or scaling equations, under regularity and positive-marginal conditions	Not a derivative of categorical resampling and not an unregularized OT claim.
Finite unrolled Sinkhorn solver	$\Pi_{\varepsilon,K}$ produced by exactly K implemented iterations	Autodiff through the implemented computational graph	Not a derivative of the converged EOT optimizer unless convergence and truncation error are separately controlled.
Implicit fixed-point solver	A fixed point satisfying the stated Sinkhorn equations to a declared tolerance	Linearized fixed-point or KKT system	Not valid without the fixed-point regularity, residual, and linear-solve conditions.
HMC or training scalar in code	Whatever scalar value actually uses the transport layer	Derivative of that same scalar only	Not a gradient of the original filtering likelihood unless a correction or equivalence proof is supplied.

23.7 Numerical-analysis obligations for Sinkhorn

The limit $\varepsilon \downarrow 0$ connects EOT heuristically to the unregularized OT objective, but it is not a harmless numerical limit. The kernel entries

$$(K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon)$$

can underflow when costs are large relative to ε. Underflow changes the effective support of the kernel and can make the scaling equations (23.17) numerically singular. Stabilized implementations therefore often work in log-scaled dual variables rather than forming all entries of K_ε in floating point [Schmitzer, 2019].

In practice, code computes a finite-iteration plan $\Pi_{\varepsilon,K}$ rather than the exact optimizer Π^*_ε. The relevant residuals are

$$r_{\text{row}} = \left\| \Pi_{\varepsilon,K}\mathbf{1} - w \right\|_1, \tag{23.22}$$

$$r_{\text{col}} = \left\| \Pi'_{\varepsilon,K}\mathbf{1} - u \right\|_1. \tag{23.23}$$

Here $\mathbf{1} \in \mathbb{R}^N$, so both residual arguments are N-vectors. The row-residual checks the source marginal and the column-residual checks the equal-weight target marginal. Small residuals are necessary for a credible transport computation, but they do not remove entropic bias relative to (23.10) or the target change relative to categorical resampling.

The gradient path must also be named. Unrolled differentiation treats the K Sinkhorn iterations as the computational graph. It is transparent but requires memory proportional to the number of stored iterations, unless checkpointing or recomputation is used. Implicit differentiation instead differentiates the fixed-point or optimality equations; it can reduce memory but requires solving a linearized system and is valid only under the regularity conditions of that fixed-point representation. Either way, the gradient is the gradient of the selected numerical or regularized object, not automatically the gradient of a categorical-resampling likelihood estimator.

The leading memory footprint of a dense Sinkhorn layer is $O(N^2)$ for the cost, kernel, or plan matrices, and the naive runtime is $O(N^2K)$ for K scaling iterations. Those costs matter in banking-scale state-space applications because large particle counts, long time series, and repeated HMC evaluations compound the expense. A reviewer should therefore see the numerical route, the residual tolerance, the stabilization method, and the gradient path before accepting any claim about differentiable-resampling suitability.

23.8 Bias versus differentiability

The central mathematical trade-off of this chapter is now explicit.

23.8.1 Standard multinomial resampling

Standard multinomial resampling preserves the intended categorical resampling law, but it is not pathwise differentiable because the selected ancestor index is locally constant in the weights except at discontinuity boundaries.

23.8.2 Soft resampling

Soft resampling is pathwise differentiable through the mean-interpolation term, but it changes the resampling map by shrinking particles toward the weighted mean. Equation (23.8) is the local expression of the resulting nonlinear-functional bias.

23.8.3 Entropic OT resampling

Entropic OT resampling is differentiable through the Sinkhorn/transport solve, but it replaces categorical resampling by a relaxed transport projection. It therefore introduces approximation through the entropic regularization and, in practice, through finite Sinkhorn iteration budgets.

Thus differentiability is obtained by replacing the exact categorical resampling map with a smoother object. The smoother object may be a useful approximation, but it is not the same map. More precisely, one should distinguish:

(i) the exact categorical resampling law;

(ii) the exact solution of the unregularized OT problem when such a transport interpretation is used;

(iii) the entropically regularized OT problem actually chosen for smoothness;

(iv) the finite-iteration Sinkhorn approximation used in code;

(v) the solver-differentiated objective defined by unrolling or implicitly differentiating the chosen Sinkhorn computation.

This distinction is essential for later HMC interpretation: if one differentiates through a relaxed resampling operator, one must be explicit about whether the resulting chain is intended to target the original posterior, a relaxed posterior, or a surrogate objective.

Layer	Mathematical object	Exact or approximate?	Source of approximation
Categorical resampling	ancestor draw from Categorical(w)	exact for the classical resampling law	no pathwise derivative through realized ancestors
Unregularized OT	coupling Π_0^* solving (23.10)	exact for the chosen transport projection	it is already a transport formulation, not categorical resampling
Entropic OT	coupling Π_ε^* solving (23.12)	exact for the regularized problem	entropy changes the unregularized OT objective
Finite Sinkhorn	numerical plan $\Pi_{\varepsilon,K}$	approximate	finite iterations, tolerance, log-domain stabilization, floating-point error
Solver-differentiated objective	scalar produced by differentiating a finite or implicit Sinkhorn path	exact only for the selected computational graph or fixed-point equations	gradient belongs to the relaxed numerical object, not automatically to categorical resampling

Table 23.1: OT object versus solver approximation.

23.9 Implementation debug map

The main implementation failures are diagnostic of different mathematical layers. A gradient that vanishes at resampling points points to accidental hard ancestor selection. Marginal mismatch in the transport plan points to an incomplete Sinkhorn solve. Exploding or NaN gradients at small ε usually point to unstabilized kernel scaling rather than to the OT formulation itself.

Method	Exactness status	Differentiability source	Bias source	Runtime issue
Categorical	exact resampling law	none pathwise	none relative to the classical resampling law	cheap but discontinuous
Soft	surrogate law	smooth mean interpolation	shrinkage toward $\bar{x}$ for nonlinear functionals	cheap but diversity-reducing
EOT	exact for regularized OT	smooth Sinkhorn scaling	entropy relative to unregularized OT and categorical law	$O(N^2K)$ Sinkhorn work
Finite Sinkhorn	numerical approximation to EOT	unrolled or implicit differentiated iterations	solver residual and tolerance choice	stability, memory, iteration count
Solver gradient	selected computational objective	autodiff or implicit fixed-point derivative	mismatch if reported as original likelihood gradient	memory or linear-solve burden

Table 23.2: Resampling family comparison.

Problem	Likely mathematical cause	Relevant literature
Zero pathwise gradient at resampling	hard categorical ancestor selection	differentiable resampling in Zhu et al. [2020]
Particle cloud collapses toward one point	soft-resampling shrinkage too strong	soft-resampling bias analysis; equation (23.8)
Sinkhorn rows or columns do not match desired marginals	finite-iteration residual or numerical underflow	Cuturi [2013], Peyré and Cuturi [2019]
NaN or unstable gradients for small ε	Gibbs kernel underflow or unstable scaling	stabilized/log-domain Sinkhorn [Schmitzer, 2019]
Posterior shifts after adopting EOT	relaxed resampling target differs from categorical law	EOT resampling [Corenflos et al., 2021] and HMC target analysis in Chapter 25
Gradient changes when iteration budget changes	differentiated scalar is tied to $\Pi_{\varepsilon,K}$ or to an implicit solver tolerance	numerical-analysis obligations in Section 23.7

Table 23.3: Implementation debug map for differentiable resampling and OT.

Item	Source claim	Local derivation task	Debugging relevance
Soft resampling	differentiable relaxation of hard resampling	derive preserved mean and nonlinear bias in (23.8)	explains shrinkage and diversity loss
OT projection	equal-weighting can be posed as a transport coupling	separate (23.10) from categorical resampling	prevents exactness overclaims
EOT/Sinkhorn	entropy gives smooth scaling form	identify Π_ε^* and barycentric map	locates regularization and solver errors
Finite solver	implementation computes $\Pi_{\varepsilon,K}$	distinguish numerical residual from OT definition	gives tolerance and stability diagnostics
Solver differentiation	code differentiates an unrolled or implicit solver path	identify the differentiated scalar and residual contract	prevents gradient-object overclaims

Table 23.4: Source claim, local derivation, and debugging use for E3.

23.10 Why learned or amortized OT comes later

A learned or amortized OT operator is a further approximation layer on top of the OT baseline developed above. It does not replace the need to understand the OT resampling map itself. For this reason, the present chapter stops at the OT and Sinkhorn level. The next chapter will treat learned or amortized transport operators explicitly as approximations to an already relaxed transport map.

23.11 What this chapter does and does not claim

This chapter develops the mathematical structure of differentiable resampling and transport-based equal-weighting. It does *not* yet claim:

- that soft resampling preserves the exact categorical resampling law;
- that OT resampling is identical to standard multinomial resampling;
- that learned/amortized OT is already covered by the same mathematics as the exact OT relaxation;
- that finite Sinkhorn is a black-box differentiable layer whose gradient has the same status as the original categorical resampling likelihood;
- that any of these relaxed resampling schemes are automatically final HMC targets for nonlinear structural models.

Its role is narrower and prior to those questions: it makes explicit how resampling is modified to obtain differentiability and where the resulting bias or relaxation enters.

Chapter 24

Learned Transport Operators and Implementation Mathematics

The previous chapter developed differentiable resampling up to the level of entropic optimal transport, finite Sinkhorn iteration, and barycentric projection. This chapter studies a further approximation layer: learned or amortized transport operators trained to imitate a previously defined transport map. The central point is deliberately conservative. A learned operator is not a speed trick that automatically preserves target correctness. It is a student map trained against a teacher object, and the teacher object may already be a relaxed or finite-solver approximation.

The source spine is correspondingly narrow. The entropy-regularized particle-filter teacher comes from the differentiable PF construction of Corenflos et al. [2021]; Sinkhorn and computational OT background comes from Cuturi [2013] and Peyré and Cuturi [2019]; and the architecture discussion uses permutation-aware set operators such as DeepSets and Set Transformer [Zaheer et al., 2017, Lee et al., 2019]. These sources identify plausible teacher and architecture families. They do not prove posterior preservation, HMC target correctness, out-of-distribution reliability, or banking suitability for a learned map.

24.1 Objects and conventions

Let $X = (x_1, \dots, x_N)^\top \in \mathbb{R}^{N \times d_x}$ be a particle cloud and let $w \in \Delta_N$ be normalized weights. Let $u = (1/N, \dots, 1/N)$ be the equal-weight target marginal. The cost matrix is $C(X, Z) \in \mathbb{R}^{N \times M}$ for source cloud X and target support $Z = (z_1, \dots, z_M)^\top$. In the square resampling case used below, $M = N$ and the barycentric output is an equal-weight cloud $\tilde{X} \in \mathbb{R}^{N \times d_x}$.

Table 24.1: Learned-OT object inventory.

Object	Definition	Audit question
Categorical baseline	Realized ancestor resampling from weights w.	Is the learned map being compared with categorical resampling, or only with an OT teacher?

Object	Definition	Audit question
Unregularized OT teacher	Optimizer $\Pi_0^\star$ of a transport objective with marginals (w, u).	Is the teacher the unregularized coupling, or is this only a limiting reference?
EOT teacher	Optimizer $\Pi_\varepsilon^\star$ of an entropy-regularized OT objective.	Is ε fixed and recorded, and is the entropy convention stated?
Finite Sinkhorn teacher	Finite-iteration coupling $\Pi_{\varepsilon,K}$ computed by a declared solver.	Is the student trained on a numerical teacher rather than the exact EOT optimizer?
Barycentric teacher output	Equal-weight cloud $B(\Pi) \in \mathbb{R}^{N\times d_x}$ derived from a coupling.	Is the student imitating the coupling, the barycentric cloud, or a scalar summary?
Learned student map	Parametric map T_ψ from weighted particle data and settings to an output cloud or coupling.	Are inputs, outputs, architecture class, particle count, dimension, and ε dependence declared?
Training distribution	Distribution $\mathcal{D}$ over clouds, weights, model regimes, and solver settings used to generate supervision.	Is every learned-map claim restricted to the support or stress envelope of $\mathcal{D}$?
Loss function	Empirical objective used to fit the student to the teacher.	Does the loss measure the object later claimed, or only a proxy such as map MSE?
Permutation action	Permutation matrix P acting on particle rows and weights.	Does the architecture respect row relabeling of an unordered particle cloud?
Residuals	Teacher-object, solver, learning, distribution-shift, filtering, posterior, and HMC value-gradient errors.	Is each residual tied to a mathematical object and a stated "does not prove" limit?

24.2 Teacher variants

There is no single object called "the OT teacher" unless the variant is named. For a cost matrix C and feasible set

$$\mathcal{U}(w,u) = \left\{\Pi \in \mathbb{R}_+^{N\times N} : \Pi\mathbf{1} = w,\ \Pi^\top\mathbf{1} = u\right\},$$

the unregularized teacher is

$$\Pi_0^\star \in \arg\min_{\Pi\in\mathcal{U}(w,u)} \langle C(X), \Pi\rangle. \tag{24.1}$$

This teacher is a transport projection, not categorical ancestor sampling. The EOT teacher for the entropy convention used in the preceding chapter is

$$\Pi_\varepsilon^\star \in \arg\min_{\Pi\in\mathcal{U}(w,u)} \left\{\langle C(X), \Pi\rangle + \varepsilon \sum_{i,j} \Pi_{ij}(\log \Pi_{ij} - 1)\right\}, \qquad \varepsilon > 0. \tag{24.2}$$

The finite numerical teacher is the output of a declared Sinkhorn procedure:

$$\Pi_{\varepsilon,K} = \mathrm{Sinkhorn}_K(C(X), w, u, \varepsilon), \tag{24.3}$$

where K records the iteration budget and the implementation also fixes stabilization, tolerances, and stopping policy. In many amortized designs the student is trained on $\Pi_{\varepsilon,K}$ or on its barycentric projection, not on $\Pi^\star_\varepsilon$ and not on $\Pi^\star_0$.

For a coupling Π, define the equal-weight barycentric output, for $j \in \{1, \dots, N\}$, by

$$B(\Pi)_j = \frac{\sum_{i=1}^N \Pi_{ij} x_i}{\sum_{i=1}^N \Pi_{ij}}, \tag{24.4}$$

whenever the denominator is positive. For the target marginal $u_j = 1/N$, this is $B(\Pi)_j = N \sum_i \Pi_{ij} x_i$. A training set must state whether its teacher output is $B(\Pi^\star_0)$, $B(\Pi^\star_\varepsilon)$, $B(\Pi_{\varepsilon,K})$, the coupling itself, or a lower-dimensional summary.

Table 24.2: Teacher variants and what they do not prove.

Teacher variant	Object actually used	Direct status	Does not prove
Categorical baseline	Ancestor draw from Categorical(w).	Classical resampling law, discontinuous pathwise.	Pathwise differentiability or learned-map correctness.
Unregularized OT	$\Pi^\star_0$ or $B(\Pi^\star_0)$.	Exact for the chosen OT projection.	Equivalence to categorical resampling or EOT.
EOT	$\Pi^\star_\varepsilon$ or $B(\Pi^\star_\varepsilon)$.	Exact for the regularized objective.	Small ε stability, finite-solver accuracy, or posterior invariance.
Finite Sinkhorn	$\Pi_{\varepsilon,K}$ or $B(\Pi_{\varepsilon,K})$.	Numerical approximation to the EOT teacher.	Convergence to EOT, absence of stabilization artifacts, or HMC validity.
Learned student supervision	Samples from whichever teacher was computed.	Empirical approximation problem on $\mathcal{D}$.	Out-of-distribution accuracy, posterior preservation, or bank-facing evidence.

24.3 Student map and training objective

A learned or amortized transport operator is a parametric map. In the barycentric-output version, write

$$\tilde{X}_\psi = T_\psi(X, w; \varepsilon, N, d_x, a), \tag{24.5}$$

where ψ are trainable parameters and a records architecture choices such as shared-row networks, pooling, attention blocks, depth, width, and positional or feature encodings. The dependence on N, d_x, and ε is not a software detail. It says which mathematical family of maps is being fit.

If the teacher output is

$$Y_\tau(X, w, \varepsilon) = B(\Pi_\tau(X, w, \varepsilon)),$$

where $\tau \in \{0, \varepsilon, (\varepsilon, K)\}$ names the teacher variant, a supervised objective has the form

$$\psi^\star \in \arg\min_\psi \mathbb{E}_{(X,w,\varepsilon,m)\sim\mathcal{D}} \left[\mathcal{L}\{T_\psi(X, w; \varepsilon, N, d_x, a), Y_\tau(X, w, \varepsilon)\}\right], \tag{24.6}$$

where m denotes the state-space model or regime that generated the cloud. In practice the expectation is replaced by an empirical average:

$$\widehat{J}(\psi) = \frac{1}{S}\sum_{s=1}^{S} \mathcal{L}\{T_\psi(X^{(s)}, w^{(s)}; \varepsilon^{(s)}, N^{(s)}, d_x^{(s)}, a), Y_\tau(X^{(s)}, w^{(s)}, \varepsilon^{(s)})\}. \tag{24.7}$$

Equations (24.6) and (24.7) prove only what they define: an empirical teacher-student fitting problem. They do not prove small filtering error, posterior error, HMC target correctness, or validity outside the training distribution. Those require additional diagnostics tied to the filtering recursion and posterior target.

24.4 Permutation symmetry

Particle labels are arbitrary. Let $P \in \mathbb{R}^{N\times N}$ be a permutation matrix. For output clouds whose rows are tied to the same particle ordering convention, the natural requirement is permutation equivariance:

$$T_\psi(PX, Pw; \varepsilon, N, d_x, a) = P\, T_\psi(X, w; \varepsilon, N, d_x, a). \tag{24.8}$$

For scalar summaries $s_\psi(X, w)$, the corresponding requirement is permutation invariance:

$$s_\psi(PX, Pw) = s_\psi(X, w). \tag{24.9}$$

DeepSets-style shared row maps and global aggregation [Zaheer et al., 2017], and attention-based set operators [Lee et al., 2019], are relevant because they can encode such symmetries.

Symmetry is necessary but not target-sufficient. A perfectly equivariant student can still approximate the wrong teacher, extrapolate outside $\mathcal{D}$, collapse the output cloud, or produce a value-gradient path that does not match the scalar target used by HMC. Conversely, symmetry violation is already a mathematical failure: the output measure depends on arbitrary input ordering rather than on the empirical measure represented by the cloud.

24.5 Approximation residuals and target shift

Residuals must be named by object. A small residual at one layer does not automatically bound a later residual. The map-level residual against the teacher τ is

$$R_{\psi,\tau}(X, w, \varepsilon) = T_\psi(X, w; \varepsilon, N, d_x, a) - Y_\tau(X, w, \varepsilon). \tag{24.10}$$

If the teacher is finite Sinkhorn, there is also a teacher-object residual:

$$E_{\text{teach}}(X, w, \varepsilon, K) = B(\Pi_{\varepsilon,K}) - B(\Pi^{\star}_{\varepsilon}). \tag{24.11}$$

A filtering-summary residual can be written for a test function φ as

$$E_{\varphi} = \frac{1}{N}\sum_{j=1}^{N} \varphi\{T_{\psi}(X, w)_j\} - \frac{1}{N}\sum_{j=1}^{N} \varphi\{Y_{\tau}(X, w)_j\}. \tag{24.12}$$

An HMC value-gradient residual for a learned scalar ℓ_{ψ} and a teacher scalar ℓ_{τ} is different again:

$$E_{\nabla} = \nabla_{\theta}\ell_{\psi}(\theta) - \nabla_{\theta}\ell_{\tau}(\theta). \tag{24.13}$$

This residual does not decide whether HMC is correct. It measures discrepancy between two targets. HMC target correctness still requires the gradient used by the integrator to be the derivative of the same scalar value used in the Metropolis correction.

Table 24.3: Residual hierarchy for learned OT.

Residual layer	Direct object	Does not prove	Required next diagnostic
Teacher-object error	$B(\Pi_{\varepsilon,K}) - B(\Pi^{\star}_{\varepsilon})$ or EOT versus unregularized OT.	That the teacher matches categorical resampling or the exact posterior.	Solver residuals, ε sensitivity, and teacher-target comparison.
Finite-solver error	Marginal residuals, cost gap, and stopping residuals of Sinkhorn.	That the learned student is accurate or stable.	Fixed-iteration sensitivity and stabilized versus unstabilized comparisons.
Supervised learning error	$R_{\psi,\tau}$ on training and held-out clouds.	Small filtering-summary error or posterior error.	Stratified errors by weight degeneracy, dimension, particle count, and ε.
Distribution shift	Difference between deployment clouds and $\mathcal{D}$.	That held-out in-distribution residuals transfer.	Stress sets for sharp weights, high dimension, particle-count shift, and model regions.
Filtering-summary error	Test-function errors such as E_{φ}.	Global target equivalence.	Multiple nonlinear test functions and full filtering likelihood comparisons.
Posterior error	Parameter posterior difference under learned and teacher filters.	HMC value-gradient consistency or production suitability.	Posterior comparisons against teacher OT and trusted references.

Residual layer	Direct object	Does not prove	Required next diagnostic
HMC value-gradient error	Difference between gradients or scalar paths used by HMC.	Correctness for either target unless the value-gradient contract is satisfied.	Same-scalar value/gradient checks, finite differences, and accept/reject audit.

24.6 Training-distribution dependence

The training distribution must be part of the mathematical claim. Write

$$\mathcal{D} = \mathcal{D}(N, d_x, \varepsilon, m, \eta),$$

where m is the model class or simulation regime and η records weight geometry, particle degeneracy, missing-data pattern, and other cloud-generating features. A claim about T_ψ is local to the support and stress envelope of this distribution unless a separate generalization argument is given.

Important distribution coordinates include:

- state dimension d_x and latent-factor structure;
- particle count N and any batching or padding convention;
- weight-simplex geometry, especially sharp or nearly collapsed weights;
- regularization range for ε and Sinkhorn iteration budget K;
- structural-model region, including invalid or near-boundary parameter values;
- missing-data, mixed-frequency, and long-panel regimes;
- hardware or compiled-execution regime if the learned map is used inside repeated likelihood evaluations.

The phrase "the learned map works" is therefore not acceptable by itself. A defensible statement has the form: relative to teacher τ, training distribution $\mathcal{D}$, architecture class a, particle count and dimension envelope, and recorded stress tests, the student residuals remain below stated thresholds. Even that statement remains a teacher-student statement until filtering and posterior diagnostics are added.

24.7 Failure regimes

Table 24.4: Failure regimes for learned OT.

Regime	Why it is hard	Likely failure mode	Required diagnostic
Sharp weights	Mass concentrates on few particles.	Student smooths or collapses the cloud when resampling matters most.	Residuals stratified by ESS and maximum weight.
High dimension	Transport geometry and network capacity change with d_x.	Low-dimensional residuals do not transfer.	Dimension ladder and coordinate-scaling tests.
Particle-count shift	Deployment N differs from training N.	Architecture, normalization, or attention cost changes the learned map.	Train/test ladders in N with fixed model and fixed ε.
ε shift	Regularization controls both teacher smoothness and target drift.	Student trained at one smoothness level is used at another.	Sensitivity over ε and teacher residual comparisons.
Structural-model regions	Invalid, near-determinacy, or stiff parameter regions change cloud geometry.	Student extrapolates where the filter is already fragile.	DSGE/MacroFinance stress clouds, invalid-region policy, and failure logging.
Learned-map collapse	Network maps diverse inputs to low-diversity outputs.	Particle diversity and likelihood curvature are distorted.	Output covariance, duplicate-count, and nonlinear test-function diagnostics.
Compiled repetition	The map is used repeatedly inside a likelihood or HMC path.	Tiny deterministic biases accumulate across long panels.	Long-panel repeated-evaluation tests and seed/hardware reproducibility.

24.8 Implementation-facing mathematics

Runtime gains are real engineering evidence, but they are not target evidence. If T_ψ replaces a Sinkhorn solve by a single forward pass, the cost can fall substantially. The reason is that the computation has changed from a solver for a declared teacher to an evaluation of a fitted student. A runtime table must therefore sit next to teacher residuals, filtering-summary diagnostics, and posterior comparisons. It cannot replace them.

Deterministic evaluation is similarly double-edged. A deterministic learned map can simplify autodiff and repeated HMC likelihood evaluation compared with a stochastic resampling path. But determinism only makes the surrogate easier to differentiate; it does not identify the posterior. The HMC question remains: what scalar value is being evaluated, and is the supplied gradient the derivative of that same scalar?

Architecture choices also have mathematical status. Shared-row maps, pooling, attention blocks, equivariance constraints, fixed particle counts, padding, and normalization layers define the class of transport maps available to the student. They are not software-only details. When a learned OT result is reported, the architecture is part of the approximation theorem that the work does not yet have.

24.9 Internal evidence ladder for bank-facing use

Table 24.5: Internal evidence ladder for learned-OT use in bank-facing filtering.

Gate	Evidence required	What the evidence supports	What it does not support
Teacher residual tests	Marginal residuals, cost gaps, ε sensitivity, and finite-Sinkhorn stability.	The teacher computation is numerically interpretable.	Learned-map accuracy or posterior preservation.
Student residual tests	Train/held-out residuals by N, d_x, ESS, ε, and model regime.	The student imitates the chosen teacher on the tested envelope.	Global generalization or HMC target correctness.
Filtering stress tests	Test-function errors, likelihood comparisons, cloud-diversity diagnostics, and failure logs.	The learned map is not visibly breaking the filtering summaries tested.	Posterior equivalence or bank-facing deployment.
Posterior comparison	Teacher versus student posterior summaries on named datasets and model classes.	Credible research evidence for the tested posterior quantities.	Production readiness or untested structural regimes.
Gradient and HMC checks	Same-scalar value/gradient checks, finite differences, compiled repetition, and accept/reject path audit.	HMC coherence for the learned scalar if the target contract passes.	Correctness for the original posterior unless corrected.
Governance limitations	Versioned model, data, architecture, weights, diagnostics, monitoring, and rollback plan.	A controlled internal research or governance package after review.	Automatic approval from speed, smoothness, or a small residual table.

This internal ladder intentionally prevents a common overclaim. It is not a substitute for a bank's formal model-risk or production-governance framework. A learned map can be fast, smooth, equivariant, and close to a finite Sinkhorn teacher on a held-out set, while still lacking evidence for posterior preservation under nonlinear DSGE or MacroFinance

targets. Bank-facing credibility requires the later gates, not only the early ones, and formal use would require a separate institution-specific governance process.

24.10 Why this chapter stops short of an HMC verdict

This chapter identifies learned or amortized OT as a layered surrogate above an OT or finite-Sinkhorn teacher. That is enough to prepare the HMC target chapter, but it does not settle the HMC question. The unresolved HMC question requires a joint analysis of:

- the baseline particle-filter or structural filtering target;
- the OT relaxation layer;
- the finite-solver layer, if the teacher is numerical;
- the learned-student approximation layer;
- the scalar value and gradient path supplied to HMC;
- the correction mechanism, if any.

If HMC uses the learned scalar and its matching gradient, the result may be coherent for the learned target. It is not thereby coherent for the original categorical or structural posterior unless a correction or posterior-error argument is supplied.

24.11 What this chapter does and does not claim

This chapter claims that learned OT must be documented as a teacher-student surrogate with explicit residual and extrapolation risks. It does not claim:

- that learned OT is equivalent to the OT teacher;
- that an OT teacher is equivalent to categorical resampling;
- that a finite Sinkhorn teacher is the exact EOT optimizer;
- that a small map-level residual implies small filtering or posterior error;
- that permutation equivariance implies target correctness;
- that runtime improvement is evidence of posterior preservation;
- that learned OT is justified as a final HMC target for nonlinear DSGE or Macro-Finance models;
- that learned OT is ready for bank-facing or production-governance use.

The chapter's role is narrower and mathematically prior: it identifies the teacher actually being approximated, the student map being trained, the distribution on which the claim is local, and the residual ladder that must be closed before stronger target or banking language would be defensible.

Chapter 25

DPF HMC Target Correctness and Structural-Model Suitability

The preceding DPF chapters built a ladder of increasingly differentiable filtering constructions: raw particle-flow proposals, proposal-corrected PF-PF, differentiable resampling, entropic optimal-transport relaxation, and learned or amortized transport. This chapter asks a narrower and more severe question. If one of those constructions is used inside Hamiltonian Monte Carlo for a nonlinear DSGE or MacroFinance model, what target is the Markov chain intended to leave invariant?

The answer cannot be inferred from the label "differentiable particle filter." HMC is a target-specific algorithm. Its mathematical interpretation depends on the scalar log target used for accept/reject decisions, the gradient used by the numerical integrator, and the correction mechanism, if any [Neal, 2011, Betancourt, 2017]. Particle MCMC adds a separate target logic: an unbiased, nonnegative likelihood estimator can define a corrected extended-space chain under pseudo-marginal assumptions, but that is not the same object as a smooth surrogate likelihood differentiated by HMC [Andrieu and Roberts, 2009, Andrieu et al., 2010]. The DPF ladder must therefore separate three questions:

(i) whether the scalar value supplied to HMC is the intended posterior value, an unbiased estimator inside a corrected construction, or a deliberately approximate value;

(ii) whether the gradient supplied to the integrator is the derivative of that same scalar value;

(iii) whether the evidence is strong enough for research, bank-facing, or production-governance language.

25.1 Objects in the HMC target contract

Let $\theta \in \Theta$ be the structural parameter and let $u \in \mathbb{R}^{d_\theta}$ be the unconstrained parameter used by the sampler, with transform $\theta = T(u)$. Let

$$\ell(u) = \log p(T(u)) + \log L(y_{1:T} \mid T(u)) + \log |\det DT(u)|$$

denote the scalar log target in unconstrained coordinates when the filtering likelihood is exact. This expression assumes that the transform is differentiable on the sampler-valid region and that $DT(u)$ is the square Jacobian relevant to the change of variables,

or the appropriate constrained- to-unconstrained volume term if the parameterization is more general. In a DPF construction the likelihood term may be replaced by a particle, relaxed, or learned object. The HMC contract is therefore stated for the actual scalar value $\ell_\star(u)$ used by the sampler, not for the method family name.

Table 25.1: Objects that must be named before a DPF construction can be called an HMC target.

Object	Mathematical role	Reviewer audit question
Parameter u and transform T	Coordinates integrated by HMC and the map back to structural parameters.	Are support constraints, Jacobian terms, and invalid parameter regions included in the scalar target?
Scalar log target $\ell_\star(u)$	The value whose density $\pi_\star(u) \propto \exp\{\ell_\star(u)\}$ is claimed as the invariant target.	Is $\ell_\star$ exact, pseudo-marginal extended-space, relaxed, or learned?
Potential $U_\star(u)$	$U_\star(u) = -\ell_\star(u)$, up to an irrelevant additive constant.	Is the sign convention consistent between value, gradient, and accept/reject?
Momentum p and mass M	Auxiliary Gaussian momentum $p \sim \mathcal{N}(0, M)$ used to define Hamiltonian dynamics.	Is M fixed or adapted under a declared warmup policy, and is it positive definite?
Hamiltonian $H_\star(u, p)$	$H_\star(u,p) = U_\star(u) + \frac{1}{2}p^\top M^{-1}p$ in ordinary separable HMC.	Does the accept/reject step evaluate this same $H_\star$?
Integrator $\Phi_{\epsilon,L}$	Numerical map used to propose (u', p') after L steps of size ϵ.	Is the proposal reversible and volume preserving for the stated correction, or is a different Metropolis ratio supplied?
Gradient $g(u)$	Vector supplied to the integrator in place of $\nabla_u \ell_\star(u)$.	Is $g(u) = \nabla_u \ell_\star(u)$ for the same scalar value, or is the mismatch explicitly corrected and diagnosed?
Accept/reject value	Scalar value used in the Metropolis correction.	Does the correction use the intended target, the relaxed target, or a learned surrogate?
Target-status category	Label attached to the resulting chain.	Does the label distinguish exact, pseudo-marginal, relaxed, learned, and diagnostic-only use?

Assume that $\ell_\star : \mathbb{R}^{d_\theta} \to \mathbb{R}$ is differentiable on the interior of the sampler's valid region, and that $g : \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_\theta}$ is the vector field supplied to the integrator in the same coordinates. The mass matrix M in the ordinary separable Hamiltonian is a symmetric positive-definite $d_\theta \times d_\theta$ matrix, so $p^\top M^{-1} p$ is well defined. The minimum value-gradient consistency condition is

$$g(u) = \nabla_u \ell_\star(u). \tag{25.1}$$

Equation (25.1) is not a claim that $\ell_\star$ is the exact nonlinear filtering likelihood. It

is a claim that the gradient used by the integrator and the value used by the target interpretation are the same mathematical object.

If the value is $\ell_\star$ but the integrator uses a vector field $g \neq \nabla \ell_\star$, the usual leapfrog derivation no longer describes the Hamiltonian flow of $H_\star$. A Metropolis correction may still be valid for an arbitrary proposal only after reversibility, volume preservation, and the correct acceptance ratio have been established. Without that separate proposal-level proof, a smooth chain with mismatched value and gradient is not evidence of correct HMC. If both value and gradient are changed to a relaxed or learned scalar, then the chain may be internally coherent, but the target has changed to $\pi_\star$.

25.2 Target-status taxonomy

The DPF chapters use the following target-status categories. The categories are deliberately conservative because a reviewer should be able to tell exactly what posterior, extended posterior, or surrogate posterior is being discussed.

Exact-likelihood HMC
: HMC evaluates a deterministic scalar log posterior and its exact or audited derivative. Numerical integration error is corrected by the standard Metropolis step, provided the implemented proposal satisfies the usual HMC reversibility and volume-preservation assumptions.

Pseudo-marginal MCMC
: A random nonnegative likelihood estimator is included as part of an extended-space target. The claim is not that the estimator is a smooth likelihood surrogate. The claim is that marginal correctness follows from the pseudo-marginal construction under its assumptions [Andrieu and Roberts, 2009, Andrieu et al., 2010].

Noisy-gradient method
: A stochastic or approximate gradient is used to move the chain. This is not exact HMC unless a valid correction theory is stated. It may be useful as an optimization, adaptation, or diagnostic device, but that usefulness does not by itself identify the invariant posterior.

Delayed-acceptance or surrogate-corrected MCMC
: A surrogate or particle object proposes, screens, or accelerates moves, but a later correction evaluates the intended target. The target claim belongs to the corrected chain, not to the surrogate alone.

Relaxed-target HMC
: The scalar log target deliberately replaces a discontinuous or discrete filtering operation by a smooth relaxed operation, such as soft resampling or entropic OT. HMC can be coherent for this scalar if Equation (25.1) holds, but the posterior is the relaxed posterior.

Learned-surrogate HMC
: A neural or amortized map defines part of the scalar value. The chain targets the learned scalar only if value and gradient are consistent. It

targets the original or teacher posterior only after a separate correction or error-bound argument strong enough for that claim.

25.3 Rung-by-rung target analysis

25.3.1 EDH and LEDH flow proposals

At the raw EDH or LEDH rung, the value object is a flow-based approximate filtering quantity. The flow is derived from Gaussian closure or local linearization assumptions and is integrated numerically. If HMC differentiates the same scalar flow object that it evaluates, then the construction can satisfy value-gradient consistency for that scalar. Its target status is nevertheless limited:

- exact-target HMC only in recovery cases where the flow construction equals the exact filtering likelihood, such as the linear-Gaussian benchmark;
- approximate-target or value-side benchmark HMC outside those recovery cases;
- diagnostic-only use if the implementation differentiates a path that differs from the scalar value used for accept/reject.

Smoothness of the flow ODE, by itself, does not upgrade a closure-based filtering approximation into an exact nonlinear posterior target.

25.3.2 PF-PF proposal correction

At the PF-PF rung, the flow is used as a proposal and the scalar value includes the proposal-to-target correction through importance weights and change-of-variables terms. This is the first rung in the BayesFilter DPF ladder with an explicit proposal-correction interpretation. That statement is not a production or validation claim. It means only that the value-side target story is more defensible than raw flow transport because the proposal density, Jacobian or log determinant, and particle weight enter the same scalar object. The improvement is proposal-accounting clarity for the stated one-step conditional construction in Chapter 22; it is not evidence, by itself, of multi-step posterior fidelity, finite-particle stability, or HMC correctness.

For HMC, the required scalar is the corrected particle objective actually used by the sampler. The required gradient is the derivative of that same corrected objective, including every differentiable term in the proposal density, transition density, measurement density, flow map, and log determinant. The remaining evidence burden is substantial:

- finite-particle variability must be measured rather than hidden by a deterministic seed;
- log-determinant and Jacobian paths must be audited against the same value path;
- flow discretization error must be separated from Monte Carlo error;
- compiled repeated evaluation must remain finite and reproducible on the structural model class being claimed;

- if randomness enters the HMC target, the construction must state whether it is pseudo-marginal extended-space MCMC, a fixed-randomness deterministic approximation, or a noisy-gradient diagnostic.

Thus PF-PF is a plausible first research rung for DPF-HMC experiments, not an HMC-validity theorem and not a bank-governance result.

25.3.3 Soft differentiable resampling

Soft resampling replaces categorical ancestor selection by a differentiable mixture or shrinkage construction. The scalar value is therefore a soft-resampling filtering value. If HMC differentiates exactly that scalar, the resulting chain can be internally coherent for the soft target. It should not be described as sampling the categorical-resampling posterior unless a separate correction is supplied.

The core mathematical issue is nonlinear test-function bias. Even if a soft-resampling map preserves a weighted mean under special conditions, it does not generally preserve expectations of nonlinear functions, path genealogies, or the likelihood contribution of the original categorical particle filter. The HMC label must therefore be relaxed-target HMC, surrogate-target HMC, or diagnostic use, depending on the scalar value actually corrected by the sampler.

25.3.4 OT and entropic OT resampling

At the OT/EOT rung, resampling is written as a transport problem. Entropic regularization and a finite Sinkhorn solve make the construction differentiable and numerically structured, but they also define a different map from exact categorical resampling. The scalar value must specify the cost, entropy convention, regularization parameter, stopping tolerance, and whether the unrolled solver path or an implicit derivative is used.

For HMC, the target status is relaxed-target HMC when the accept/reject value and gradient both use the same EOT/Sinkhorn scalar. Finite solver residuals, underflow handling, stabilization, and ε sensitivity are target diagnostics, not mere implementation details. A finite Sinkhorn layer does not become an exact categorical resampling layer because its gradient exists.

25.3.5 Learned or amortized OT

Learned OT introduces a second approximation layer: a neural or amortized map is trained to imitate, accelerate, or residual-correct a transport teacher. The scalar value is then the learned scalar unless a correction evaluates the teacher or original target. The gradient path is the derivative through the learned map and any explicitly retained teacher terms.

The target-status label must therefore be learned-surrogate HMC unless the implementation supplies a valid correction or an error analysis strong enough to promote the claim. Runtime reduction, smoothness, and low teacher residual on training clouds are not target-correctness evidence. The missing validation includes out-of-distribution particle-cloud tests, seed sensitivity, posterior comparison against the EOT teacher, and stress tests on structural parameter regions that are invalid, nearly invalid, or geometrically stiff.

25.4 Structured target maps

Table 25.2: Rung-by-rung HMC target map.

Rung	Scalar value	Gradient path	Target status	Missing validation before promotion
EDH/LEDH	Gaussian-closure or local-flow filtering scalar.	Derivative of the same flow scalar, if implemented consistently.	Exact only in recovery cases; otherwise approximate benchmark.	Linear-Gaussian recovery, nonlinear likelihood-error studies, flow discretization checks, invalid-region behavior.
PF-PF	Proposal-corrected particle scalar with weights and flow Jacobian/log-det terms.	Derivative of the same corrected scalar, including correction terms.	First rung with explicit proposal-correction interpretation; research candidate only.	Finite-particle variance, Jacobian/log-det audit, value-gradient parity, compiled repeated evaluation, randomness policy.
Soft resampling	Soft or shrinkage resampling scalar.	Derivative through the same soft map.	Relaxed or surrogate target.	Posterior sensitivity against categorical or trusted references, nonlinear test-function bias, seed and temperature sensitivity.
OT/EOT	Transport-relaxed scalar with cost, entropy, ε, and solver budget.	Derivative through the same unrolled or implicit solver path.	Relaxed target.	ε and Sinkhorn-budget sensitivity, residual tolerances, stabilization audit, teacher posterior comparison.
Learned OT	Learned approximation to an OT/EOT or residual-corrected scalar.	Derivative through the learned map and retained teacher terms.	Learned surrogate unless corrected.	Teacher residual to posterior-error analysis, out-of-distribution cloud tests, structural stress tests, correction evidence.

Table 25.2 is a target map, not a performance table. It does not say that the lower rungs are useless or that the higher rungs are invalid. It says that every rung must name the scalar target before an HMC claim can be interpreted.

25.5 Pseudo-marginal and surrogate distinctions

The pseudo-marginal result and the DPF surrogate story are often confused because both involve particle approximations. They are different claims. Suppose $\widehat{L}(y \mid \theta, \omega) \geq 0$ is an unbiased estimator of $L(y \mid \theta)$ under auxiliary randomness ω:

$$\mathbb{E}_{\omega|\theta}\left[\widehat{L}(y \mid \theta, \omega)\right] = L(y \mid \theta).$$

Pseudo-marginal MCMC targets an extended density involving ω and can recover the exact marginal posterior under its assumptions. The validity argument is an extended-target argument. It is not an argument that $\log \widehat{L}$ is a smooth deterministic likelihood whose pathwise gradient is the gradient of $\log L$.

By contrast, a relaxed DPF target usually chooses a deterministic or fixed-randomness scalar $\ell_\star(\theta)$ and differentiates it. HMC may be valid for $\ell_\star$ if the value-gradient contract and proposal correction hold. That does not imply validity for the original posterior $\ell(\theta)$. A delayed-acceptance or surrogate-corrected scheme can bridge the gap only if the final correction evaluates the intended target with the right acceptance probability.

25.6 Why nonlinear DSGE models sharpen the issue

Nonlinear DSGE models make target drift especially costly because the filtering law is tied to structural economics rather than only to a generic latent-state transition. The DPF chapter does not need to solve the economics, but it must not hide target changes inside a numerical method. The main DSGE stress points are:

(i) **Determinacy and invalid regions**: the sampler will propose parameters outside or near determinacy regions. The target must return a declared finite penalty or $-\infty$ policy, not an uncaught solver failure.

(ii) **Pruning and approximation convention**: first-order, higher-order, pruned, and simulation-based approximations can define different latent laws. A DPF likelihood is only meaningful after the structural law is fixed.

(iii) **Structural timing**: observables, controls, states, and shocks must enter the transition and measurement equations with the intended timing. A smooth filter for the wrong timing is still the wrong target.

(iv) **Latent-state semantics**: deterministic-completion coordinates and stochastic state coordinates are not interchangeable. Particle clouds must represent the declared latent law.

(v) **Local Jacobian fragility**: EDH, LEDH, and proposal-corrected flow methods inherit local-linearization and derivative fragility near boundaries or stiff regions.

For DSGE work, the correct conclusion is conservative. A DPF rung may be a useful research target after it states its scalar value and diagnostics. It is not bank-facing evidence until the structural law, invalid-region policy, value-gradient parity, and posterior sensitivity have been tested on the named model class.

25.7 Why MacroFinance models sharpen the issue

MacroFinance latent-factor systems create a different target-risk profile. Their long panels can amplify small likelihood changes, and their latent factors often interact with volatility, measurement noise, and missing-data patterns. A relaxed or learned resampling map that looks harmless on a short toy filter can move posterior mass materially in a long panel.

The minimum MacroFinance limitations to record are:

- latent dimension and particle degeneracy pressure;
- long observation spans that compound small per-period target changes;
- missing, ragged-edge, or mixed-frequency observation patterns;
- volatility and measurement-noise directions with sharp curvature;
- compiled repeatability across seeds, batches, and hardware settings;
- model-risk governance, including reproducible artifacts and claim boundaries.

These limitations do not rule out DPF research. They rule out treating a smooth differentiable filtering path as banking evidence before the target and validation ladder have been passed.

25.8 Relation to surrogate-HMC acceleration

Surrogate-HMC and Hamiltonian-neural-network ideas address a different point in the inference stack [Greydanus et al., 2019]. They attempt to improve movement through an already chosen target, or to use a surrogate proposal that is later corrected against that target. DPF methods may instead alter the filtering likelihood itself through flow approximation, relaxed resampling, or learned transport. The distinction is operational:

- a corrected surrogate-HMC proposal can preserve the intended target if the correction evaluates that target and the proposal assumptions hold;
- a relaxed DPF likelihood defines a new target unless a correction returns to the original likelihood;
- a learned DPF transport map defines a learned target unless the sampler corrects against the teacher or original target.

Table 25.3: Method-family distinctions relevant to DPF-HMC claims.

Method family	What is approximated?	Target implication	Required wording
Classical exact HMC	Hamiltonian trajectory by numerical integration.	Target preserved after Metropolis correction when value, gradient, and proposal conditions hold.	Exact for the named scalar target, not exact because the trajectory is exact.
Pseudo-marginal MCMC	Likelihood value by an unbiased nonnegative estimator on extended space.	Marginal posterior can be exact under pseudo-marginal assumptions.	Corrected extended-space chain, not smooth surrogate-gradient HMC.
Noisy-gradient method	Gradient or force field used for movement.	No exact posterior claim without correction theory.	Diagnostic or approximate unless corrected.
Delayed-acceptance surrogate	Early-stage value or proposal geometry.	Can preserve intended target if final correction evaluates intended target.	Surrogate-assisted, not target-defining, when corrected.
Relaxed DPF	Filtering likelihood construction itself.	Posterior changes to relaxed posterior.	Relaxed-target HMC.
Learned DPF	Transport, residual, or filtering map by a learned function.	Posterior changes to learned posterior unless corrected.	Learned-surrogate HMC pending correction or validation.

25.9 Evidence gates for banking language

The chapter's recommendation is evidence-gated. A method can be mathematically interesting, credible as a research prototype, and still not ready for a bank-facing model-risk claim. The following gates constrain all wording in this chapter and in later experiments.

Table 25.4: Evidence ladder for DPF-HMC banking and governance language.

Claim level	Allowed claim	Required evidence	Disallowed shortcut
Exploratory use	The rung is worth studying on controlled examples.	Named scalar value, value-gradient check, finite smoke tests, clear target status.	Calling the method suitable because it is differentiable.

Claim level	Allowed claim	Required evidence	Disallowed shortcut
Credible research use	The rung has a defensible target interpretation for a named model class.	Source-grounded derivation, repeated fixed-seed and varied-seed tests, posterior comparisons, failure-region diagnostics.	Treating a toy result as evidence for DSGE or MacroFinance deployment.
Bank-facing research claim	The rung can support a restricted internal research conclusion under model-risk controls.	Reproducible artifacts, model-specific stress tests, challenger comparisons, audit trail, documented limitations.	Using runtime improvement as target-correctness evidence.
Production or governance claim	The rung is approved for controlled production use in a defined workflow.	Independent review, change control, monitoring, rollback, data and model governance, stability across releases.	Inferring production readiness from chapter-level mathematics.

No rung in this chapter is promoted to production or bank-governance status. The strongest current recommendation is a bounded experimental hypothesis: prioritize proposal-corrected PF-PF first because it is the first rung with an explicit proposal-correction interpretation, then compare relaxed and learned rungs as deliberately changed targets. The hypothesis remains conditional on the finite-particle, Jacobian, randomness, compilation, and posterior-comparison gates stated above.

25.10 BayesFilter recommendation

The disciplined recommendation is:

(i) use EDH and LEDH as recovery and approximation benchmarks, not as generic exact HMC targets;

(ii) treat proposal-corrected PF-PF as the first experimental hypothesis with an explicit proposal-correction interpretation and a defensible value-side target story, subject to finite-particle, Jacobian, randomness, compilation, and posterior-comparison gates;

(iii) label soft resampling and OT/EOT as relaxed-target HMC unless a correction restores the original categorical or trusted target;

(iv) label learned OT as learned-surrogate HMC unless a correction or posterior-error argument justifies stronger wording;

(v) require DSGE and MacroFinance promotion evidence at the model-class level before any bank-facing claim.

25.11 What this chapter does and does not claim

This chapter claims that DPF-HMC analysis must be organized around named scalar targets, value-gradient consistency, and evidence gates. It does not claim:

- that PF-PF has already passed finite-particle, Jacobian, compiled-path, or posterior-comparison gates in BayesFilter;
- that relaxed or learned resampling targets are unusable;
- that pseudo-marginal validity transfers to deterministic relaxed gradients;
- that nonlinear DSGE or MacroFinance DPF-HMC has been validated in this repository;
- that any DPF rung is approved for production use or bank governance.

The chapter's role is to make future claims auditable. A skeptical reviewer should be able to identify the target sampled by each rung, the correction mechanism if one is claimed, and the exact evidence still missing before stronger banking language would be defensible.

Chapter 26

DPF Literature-to-Debugging Verification Contract

The preceding DPF chapters separate the mathematical layers: classical particle filtering, particle-flow transport, proposal correction, differentiable resampling, OT/Sinkhorn relaxation, learned transport, and HMC target interpretation. This chapter turns that separation into a verification contract. Its purpose is not to validate any implementation by prose. Its purpose is to tell a coding agent or reviewer which equation, controlled example, implementation quantity, tolerance, and failure interpretation belongs to each layer.

The contract is deliberately ordered. Target and value-gradient checks come before tuning. Teacher-map diagnostics come before learned-map diagnostics. Equation-local failures should be interpreted narrowly: a failed Sinkhorn marginal test is evidence about a finite solver, not evidence that particle flows are wrong; a learned-map residual is evidence about the student and its training distribution, not evidence that the OT teacher is invalid.

26.1 Diagnostic order

Use this order before changing code or tuning numerical parameters:

(1) name the scalar object being estimated, relaxed, learned, or sampled;

(2) verify that the gradient path corresponds to that same scalar object whenever HMC or autodiff claims are made;

(3) test the classical particle-filter recursion and likelihood estimator on a controlled model;

(4) test the particle-flow endpoint and homotopy derivative before testing PF-PF proposal correction;

(5) test PF-PF weights and log-determinant accounting before interpreting resampling or OT behavior;

(6) test categorical, soft, EOT, Sinkhorn, and barycentric objects before testing a learned student map;

(7) only after those checks pass, run posterior comparisons and sampler diagnostics.

This order prevents a common error: making a numerically smooth path smoother without first proving that it is the intended path.

26.2 Equation-to-test matrix

Table 26.1: Equation-indexed verification matrix for the DPF block.

Layer	Equation or claim	Controlled example	Implementation quantity	Pass signal	Narrow failure interpretation
Filtering recursion	(20.3), (20.4)	linear-Gaussian state-space model with Kalman reference	predict/update law, normalized weights, filtered mean/covariance	moments match reference within declared Monte Carlo error	recursion, normalization, or model-interface bug; not an OT or learned-map claim
Bootstrap likelihood estimator	(20.16), (20.17)	same linear-Gaussian model; fixed seeds and increasing N	per-period average weights and log likelihood estimator	Monte Carlo mean approaches Kalman likelihood; variance decreases with N	particle proposal or weight bug; not evidence about differentiability
Homotopy derivative	(21.1), (21.3)	scalar nonlinear observation with analytic log-likelihood derivative	finite difference in λ versus analytic derivative	central difference and analytic derivative agree under step refinement	homotopy algebra or likelihood derivative issue; not yet a PF-PF correction issue
EDH endpoint	(21.11), (21.15)	linear-Gaussian recovery with known posterior mean/covariance	flow endpoint particles, mean, covariance, and endpoint residual	endpoint moments match Kalman update within flow and Monte Carlo tolerances	flow ODE or Gaussian-closure implementation issue; not HMC target validity
LEDH local endpoint	(21.17), (21.24)	synthetic observation with known local Jacobian variation	particle-local Jacobians, local precision, and endpoint continuity	local quantities are finite and stable under small perturbations	local linearization fragility; not proof that global EDH is wrong
PF-PF weight	(22.2), (22.3), (22.8)	synthetic affine flow with known determinant	proposal density, target density, corrected log weight	manual density ratio and implemented weight agree	proposal-density or correction algebra bug; not a resampling diagnosis
Log-determinant ODE	(22.11), (22.12), (22.13)	affine flow $x_\lambda = A_\lambda x + b_\lambda$	integrated log determinant and finite-difference determinant	sign and magnitude match under step-size refinement	Jacobian sign, time direction, or integrator issue; not a likelihood theorem
Categorical discontinuity	(23.3), (23.4)	two-particle resampling with weight crossing	ancestor index path and realized resampled cloud	jump occurs as expected; no false pathwise gradient is reported	discrete resampling is being differentiated incorrectly; not proof soft resampling is valid
Soft-resampling bias	(23.5), (23.8)	two-particle nonlinear test function	mean preservation and nonlinear test-function error	affine tests behave as derived; nonlinear bias is visible when expected	surrogate-law effect; not evidence of categorical-target preservation
EOT/Sinkhorn marginals	(23.12), (23.17), (23.22)	small cost matrix with prescribed marginals	row residual, column residual, cost, entropy, ε, iteration count	residuals fall below tolerance and are stable under log-domain implementation	finite-solver or stabilization issue; not proof of posterior invariance
Barycentric projection	(23.21)	small coupling with known column masses	projected equal-weight support cloud and dimensions	output has declared shape and column-mass normalization	projection or shape bug; not a coupling-optimality result

Table 26.1: Equation-indexed verification matrix for the DPF block, continued.

Layer	Equation or claim	Controlled example	Implementation quantity	Pass signal	Narrow failure interpretation
Learned-map residual	(24.5), (24.10), (24.8)	permutation-equivariance test and held-out teacher clouds	teacher/student residuals, permutation residual, training-distribution tags	residuals stay below threshold on the declared envelope	student or distribution-shift issue; not a teacher-map refutation
HMC value-gradient contract	(25.1)	finite-difference check on a fixed scalar target and fixed random seed policy	scalar value, autodiff gradient, finite-difference gradient, accept/reject value	all paths refer to the same scalar and gradients agree within tolerance	target-interface bug; do not tune HMC before resolving it

26.3 Minimal controlled examples

Table 26.2: Minimal controlled examples for DPF verification.

Example	Required inputs	Expected output	Does not prove
Linear-Gaussian recovery	Known transition, observation, covariance matrices, Kalman likelihood, and Kalman filtering moments.	Particle recursion, EDH endpoint, and likelihood estimates approach the Kalman reference under increasing particles or decreasing flow step size.	Nonlinear DSGE validity or learned-map generalization.
Synthetic affine flow	Invertible affine map, analytic determinant, and known proposal density.	PF-PF density ratio, weight, and log determinant match hand calculation.	Correctness for nonlinear flow fields.
Scalar nonlinear observation	One-dimensional observation function with analytic derivatives and controlled curvature.	Homotopy derivative and local-linearization diagnostics show expected error as curvature changes.	High-dimensional stability.
Two-particle soft-resampling bias	Two particles, weights crossing 1/2, and nonlinear test function.	Categorical path is discontinuous; soft path has the predicted surrogate bias.	Acceptability of soft resampling as an HMC target.
Small Sinkhorn problem	Positive cost matrix, weights, equal target marginal, fixed ε, and known residual tolerance.	Row/column residuals and stabilized objective behave consistently with the solver budget.	Posterior equivalence to categorical resampling.

Example	Required inputs	Expected output	Does not prove
Permutation-equivariance learned-map test	Teacher clouds, learned map, and permutation matrices.	Permuting input rows permutes output rows, and scalar summaries remain invariant when required.	Small teacher/student error or target preservation.
Finite-difference HMC gradient test	Fixed scalar target, fixed random seed policy, finite-difference step ladder, and autodiff gradient.	Autodiff and finite-difference gradients agree for the scalar used by accept/reject.	Convergence of the Markov chain or correctness for another scalar target.

26.4 Diagnostic specification

Every diagnostic result should be recorded with the following fields:

- **Equation or claim**: label from the chapter, not only a method name.
- **Inputs**: model, particles, weights, random seed policy, ε, iteration budget, architecture, and scalar target as applicable.
- **Expected output**: scalar, vector, matrix, empirical measure, residual, or posterior summary.
- **Tolerance**: absolute and relative thresholds, Monte Carlo standard-error logic, or convergence slope.
- **Failure interpretation**: the narrow mathematical layer that the failure implicates.
- **Non-implication**: what the failure does not prove.

Table 26.3: Diagnostic contract by layer.

Diagnostic	Inputs	Tolerance rule	Failure interpretation	Non-implication
Filtering recursion parity	Kalman reference, fixed particles, fixed model.	Monte Carlo error decreases with N; deterministic quantities match reference when applicable.	State-space interface, normalization, or weight update bug.	Does not reject OT, learned OT, or HMC.
Flow endpoint parity	Linear-Gaussian recovery, flow step ladder.	Endpoint error decreases as flow integration is refined.	Flow ODE, closure, or integration issue.	Does not prove PF-PF correction is wrong.

Diagnostic	Inputs	Tolerance rule	Failure interpretation	Non-implication
PF-PF algebra parity	Affine flow, analytic determinant, explicit proposal density.	Log-weight difference below deterministic tolerance.	Density-ratio, Jacobian, or sign error.	Does not diagnose resampling.
Soft-resampling bias check	Two-particle nonlinear test.	Bias sign and magnitude match derived expression within arithmetic tolerance.	Surrogate target is being confused with categorical target.	Does not prove soft resampling is unusable.
Sinkhorn residual check	Small cost matrix, fixed ε, solver budget.	Row/column residuals below declared threshold and improve with budget.	Finite-solver or stabilization issue.	Does not prove EOT is a valid posterior target.
Learned residual check	Teacher outputs, held-out clouds, distribution tags.	Residuals reported by regime; thresholds tied to training envelope.	Student approximation or extrapolation issue.	Does not invalidate the teacher.
HMC value-gradient check	Scalar value, autodiff path, finite-difference ladder.	Gradient errors shrink under step refinement until numerical limits.	Value-gradient mismatch or nondifferentiable scalar path.	Does not prove chain convergence.

26.5 Source and implementation map

Table 26.4: Source, chapter, equation, and implementation links.

Layer	Chapter location	Equation labels	Source family	Implementation quantity
Classical PF	Chapter 20, Sections 20.3–20.7	(20.4), (20.17)	SMC literature spine	weights, ESS, likelihood estimator, empirical moments
Particle flow	Chapter 21, Sections 21.3–21.7	(21.3), (21.14), (21.24)	Daum–Huang, PF-PF flow literature	flow field, endpoint residual, local Jacobians
PF-PF correction	Chapter 22, Sections 22.3–22.6	(22.8), (22.12)	proposal correction and change-of-variables spine	proposal density, target density, log determinant, corrected weight
Resampling and OT	Chapter 23, Sections 23.3–23.7	(23.4), (23.8), (23.17)	differentiable resampling, OT, Sinkhorn sources	ancestor path, soft surrogate, coupling residuals, barycentric cloud
Learned OT	Chapter 24, Sections 24.1–24.9	(24.5), (24.10), (24.8)	EOT teacher and set-architecture sources	teacher output, student output, residuals, distribution tags
HMC target	Chapter 25, Sections 25.1–25.9	(25.1)	HMC, pseudo-marginal, surrogate-HMC spine	scalar value, gradient, accept/reject value, target-status label

26.6 Promotion thresholds

Table 26.5: Promotion thresholds for DPF implementation claims.

Claim level	Required passing evidence	Disallowed interpretation	Gate
Exploratory implementation	Layer-local smoke tests pass on minimal controlled examples; failures are recorded with equation labels.	Treating smoothness or runtime as target correctness.	May run more experiments.
Credible research implementation	Classical PF, flow, PF-PF, resampling/OT, learned-map, and HMC value-gradient checks pass on named model classes with sensitivity reports.	Generalizing beyond tested N, d_x, ε, model class, or target status.	May make bounded research claims.
Bank-facing research claim	Research checks plus posterior comparisons, stress regimes, reproducible artifacts, and documented failure policies.	Claiming production suitability from a chapter or benchmark alone.	Requires model-risk review.
Production or governance claim	Independent review, change control, monitoring, rollback, data/model governance, release stability, and ongoing diagnostics.	Assuming academic derivation or local tests are sufficient approval.	Requires separate governance process.

26.7 Boundary of this contract

This chapter does not validate any DPF implementation. It defines the minimum diagnostic evidence needed to interpret failures and to avoid promoting a method beyond its evidence. The word "validate" is therefore reserved for a specific equation-local check or for a separately governed model-risk process; it is not used here as a blanket claim about DPF, learned OT, HMC, or banking deployment.

Chapter 27

Filter Choice

Filter choice is a target-definition decision, not only a speed decision. For HMC, the selected backend determines the log likelihood, the derivative path, the finite-failure policy, and the compilation surface. BayesFilter should choose the simplest backend that gives the required target fidelity and a validated gradient.

27.1 Decision Register

The backend decision should be recorded in a short register:

Exact Kalman, solve, or square-root LGSSM Best for linear Gaussian likelihoods and regression oracles. The main risks are SPD failures, mask handling, and large-scale factor failures. The HMC gate is analytic/custom score parity plus compiled value-gradient parity.

EKF Best as a cheap nonlinear baseline and affine recovery check. The main risks are linearization bias and Jacobian fragility. The HMC gate is affine Kalman recovery plus finite-gradient stress tests.

Sigma-point Best for moderate nonlinear Gaussian approximations. The main risks are an approximate target and covariance-update instability. The HMC gate is reference recovery, static sigma-point shapes, and derivative parity.

Square-root sigma-point Best for value-side robustness under near-singular covariance. The main risk is false confidence when the implementation reconstructs covariance matrices internally. The HMC gate is factor reconstruction, finite gradients, and compiled parity.

SVD sigma-point Best for robust approximate value paths and spectral diagnostics. The main risks are close or repeated spectral values and raw autodiff instability. The HMC gate is eigen-gap telemetry, stress gates, and a custom-gradient fallback.

Particle or differentiable particle filter Best for non-Gaussian diagnostics and frontier approximations. The main risks are Monte Carlo variance, resampling non-differentiability, and relaxed target bias. The HMC gate is an estimator-variance report and a source-backed correction policy.

27.2 Recommended Order

For a new model, BayesFilter should proceed in this order:

(i) reduce to an LGSSM or affine test case and validate the exact Kalman likelihood and derivatives;

(ii) add masks, missing-data patterns, and mixed-frequency timing while retaining exact references;

(iii) test EKF or sigma-point approximations on low-dimensional nonlinear examples with dense or trusted references;

(iv) only then attempt DSGE-scale SVD sigma-point HMC;

(v) treat particle methods and transport methods as separate evidence streams unless a correction policy makes them target-preserving.

27.3 Industrial Default

For NAWM-sized or industrial applications, the default should be an exact or controlled approximate likelihood with an analytic or custom derivative path. Raw tape gradients through spectral decompositions are acceptable as diagnostic tools and small-case oracles, but they should not be the final production plan unless spectral-gap, finite-gradient, and compiled parity evidence survives the intended stress regime.

This policy is intentionally conservative. The SVD sigma-point debugging work showed that value robustness and HMC-gradient robustness are different problems. The next implementation phases should close that gap deliberately instead of discovering it during long chains.

Chapter 28

High-Dimensional Nonlinear Filtering Foundations

28.1 Chapter Claim And Source Discipline

This chapter fixes the notation, exact identities, and evidence boundaries used by Chapters 29–32. It is deliberately strict about what follows from mathematics, what follows from a BayesFilter execution artifact, and what remains a research hypothesis. The chapter does not certify any BayesFilter backend as NAWM-ready, HMC-ready, GPU-ready, or production-ready.

Literature statements in this block use four support classes:

(i) *primary technical source*: a cited paper/book is used for a specific method or diagnostic claim;

(ii) *local ResearchAssistant summary*: the local MCP summary is used only as review material and never as theorem-level support;

(iii) *BayesFilter artifact*: a code path, benchmark row, result note, or source-map entry supports only the exact observed engineering fact;

(iv) *source-gap blocker*: the idea is named because it matters to the research program, but the present chapter refuses to use it as support for a technical claim.

This convention is part of the content, not clerical metadata: it prevents a survey sentence from becoming an accidental validation claim.

28.2 State-Space Contract

Let $\theta \in \Theta$ be static parameters, $x_t \in \mathbb{R}^n$ a latent state, and $y_t \in \mathbb{R}^m$ an observation. A discrete-time nonlinear state-space model is specified by an initial law $\mu_\theta(dx_0)$, transition kernel $f_\theta(dx_t \mid x_{t-1})$, and observation likelihood $g_\theta(y_t \mid x_t)$:

$$x_0 \sim \mu_\theta, \qquad x_t \mid x_{t-1}, \theta \sim f_\theta(\cdot \mid x_{t-1}), \qquad y_t \mid x_t, \theta \sim g_\theta(\cdot \mid x_t). \tag{28.1}$$

When simulation maps exist, we write

$$x_t = F_\theta(x_{t-1}, \varepsilon_t), \qquad y_t = H_\theta(x_t, \eta_t), \tag{28.2}$$

with innovations ε_t and observation shocks η_t. In structural models the full state may contain deterministic completion coordinates. The dimension that controls particles or quadrature is then the declared stochastic integration dimension, not merely the length of the storage vector.

Assumption 28.1 (Existence and dominated differentiation)**.** *For exact filtering identities, the transition and observation kernels are defined on a common measurable state space, the predictive densities appearing below are finite and positive on the observed path, and all integrals are with respect to the model-induced measure. For score identities, differentiation under the integral sign is justified by a local dominating function.*

28.3 Prediction, Update, And Likelihood

Definition 28.1 (Exact filtering recursion)**.** *Under Assumption 28.1, the prediction and update density recursions are*

$$p_\theta(x_t \mid y_{1:t-1}) = \int f_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}, \tag{28.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}, \tag{28.4}$$

where

$$p_\theta(y_t \mid y_{1:t-1}) = \int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{28.5}$$

Proposition 28.1 (Likelihood factorization)**.** *If the conditional predictive densities in (28.5) exist, are finite, and are positive, then*

$$\ell_T(\theta) = \log p_\theta(y_{1:T}) = \sum_{t=1}^{T} \log p_\theta(y_t \mid y_{1:t-1}). \tag{28.6}$$

Proof. The probability chain rule gives $p_\theta(y_{1:T}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{1:t-1})$. Positivity and finiteness permit taking logarithms term by term. □

Proposition 28.2 (Predictive-score identity)**.** *Let $Z_t(\theta) = p_\theta(y_t \mid y_{1:t-1})$ be finite and positive, and assume dominated differentiation. Then*

$$\nabla_\theta \log Z_t(\theta) = \frac{\nabla_\theta Z_t(\theta)}{Z_t(\theta)}. \tag{28.7}$$

If $Z_t(\theta) = \int a_t(x_t, \theta)\, dx_t$ for $a_t(x_t, \theta) = g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})$, then

$$\nabla_\theta \log Z_t(\theta) = \int \nabla_\theta \log a_t(x_t, \theta)\, p_\theta(x_t \mid y_{1:t})\, dx_t, \tag{28.8}$$

whenever $a_t > 0$ on the posterior support.

Proof. Equation (28.7) is the derivative of $\log Z_t$. For (28.8), dominated differentiation gives $\nabla_\theta Z_t = \int \nabla_\theta a_t\, dx_t = \int a_t \nabla_\theta \log a_t\, dx_t$. Divide by Z_t and use $a_t / Z_t = p_\theta(x_t \mid y_{1:t})$ from (28.4). □

The score identity is not an implementation instruction by itself. In a large nonlinear SSM the derivative of the predictive density also depends on the filtering distribution inherited from the previous step. A backend must state which approximation it differentiates and must verify that the scalar value and gradient refer to the same approximate target.

28.4 Exact Posterior Versus Approximate Posterior

For a parameter prior $\pi(\theta)$, the exact parameter posterior is

$$\log \pi(\theta \mid y_{1:T}) = \ell_T(\theta) + \log \pi(\theta) - \log p(y_{1:T}). \tag{28.9}$$

A nonlinear filter used inside HMC or optimization usually supplies $\widehat{\ell}_T(\theta)$ instead. Unless an unbiased pseudo-marginal or otherwise target-correcting mechanism is supplied, the resulting target is

$$\widehat{\pi}(\theta \mid y_{1:T}) \propto \exp\{\widehat{\ell}_T(\theta)\}\pi(\theta), \tag{28.10}$$

not the exact posterior in (28.9). This is the main boundary behind the later HMC chapter: an exact integrator for an approximate likelihood samples the approximate posterior.

28.5 High-Dimensional Failure Mechanisms

High dimension is not a single obstacle. It is a collection of failure modes that show up in different ledgers:

- *Gaussian projection*: the propagated covariance can lose positive semidefiniteness, the local linearization can be wrong in the posterior mass, or a low-order moment closure can hide multimodality.
- *Quadrature*: global tensor rules grow exponentially, and high-degree cubature may spend points on interactions that the model does not actually have.
- *Particles*: normalized weights $\bar{w}_i = w_i / \sum_j w_j$ can concentrate, giving ESS $= 1/\sum_i \bar{w}_i^2$ close to one even with many particles; this is the obstacle emphasized in high-dimensional particle-filter analyses such as Bengtsson et al. [2008] and Snyder et al. [2008].
- *HMC*: finite log density and gradients do not guarantee valid exploration. Divergences, poor energy exploration, high $\widehat{R}$, low effective sample size, and strong step-size sensitivity can veto a chain before speed is meaningful; see the HMC diagnostics framing of Neal [2011] and Betancourt [2017].
- *Tensor compression*: rank truncation can reduce storage while introducing signed-density artifacts, normalization drift, covariance invalidity, or posterior distortion.

28.6 NAWM-Like Industrial Stress

The NAWM-style nonlinear DSGE target is used here as a stress standard, not as validation evidence. A method that looks plausible on a ten-dimensional fixture can fail at NAWM-like scale because the state vector mixes stocks, forward-looking controls, shocks, measurement channels, occasionally binding features, deterministic completion equations, and parameter transforms. The three rescue structures that matter most are:

(1) *locality*: economic blocks or shock blocks interact sparsely;

(2) *low rank*: observation or transition sensitivity is concentrated in a smaller active subspace;

(3) *transport geometry*: an invertible or approximately invertible map removes the worst curvature before filtering or sampling.

Each is a hypothesis. Promotion requires evidence on the downstream filtering or posterior computation, not only a local residual, a training loss, or a small smoke row.

28.7 BayesFilter Evidence Boundary

The current BayesFilter evidence relevant to this chapter is narrow:

- existing nonlinear sigma-point tests support finite value and score diagnostics on selected Model A–C cells;
- the May 27 high-dimensional smoke artifact, abbreviated here as the P8 smoke JSON, contains 18 CPU-only rows, of which 16 completed and 2 were skipped by the declared point cap;
- those rows recorded comparator, shape, dtype, seed policy, tolerance, finite/shape status, runtime, command, environment, CPU/GPU policy, labels, and non-implication text;
- no row establishes posterior accuracy, HMC convergence, GPU speedup, broad XLA readiness, tensor validation, production defaults, or NAWM readiness.

28.8 Ledgers And Promotion Rule

The remaining chapters keep five ledgers:

(i) engineering correctness: contracts, shapes, dtypes, finite status, and reproducible commands;

(ii) numerical validity: parity against a declared comparator, positive covariance factors, and stable residuals;

(iii) sampler validity: divergences, acceptance, $\widehat{R}$, ESS, E-BFMI, energy error, and posterior/reference checks under a stated sampler plan;

(iv) scientific interpretation: whether the approximation answers the statistical or economic question;

(v) performance evidence: runtime, memory, compilation, and device behavior after validity vetoes have passed.

No result moves from one ledger to another without an explicit promotion criterion. A fast invalid row is invalid first and fast second.

28.9 Source And Evidence Ledger

Claim	Support class	Boundary
Filtering recursion, likelihood factorization, and predictive-score identity	Derivations in this chapter; MathDevMCP audit attempt recorded in the execution note	Exact only under Assumption 28.1.
Particle-filter collapse is a high-dimensional obstacle	Primary sources Bengtsson et al. [2008], Snyder et al. [2008]	Qualitative warning here; no new theorem is proved.
HMC diagnostics can veto speed comparisons	Primary HMC sources Neal [2011], Betancourt [2017]	The chapter does not validate BayesFilter nonlinear HMC.
May 27 P8 rows are bounded execution diagnostics	BayesFilter benchmark JSON and markdown artifact	No posterior, GPU, XLA, NAWM, or production claim.

28.10 Unresolved Claim Register

- No theorem proves that any candidate filter scales to NAWM.
- No exact nonlinear reference likelihood is established for BayesFilter Models B–C in this chapter block.
- No HMC convergence, tensor-method validation, or GPU speedup claim is made.
- Source gaps in later tensor and transport sections remain visible until primary technical papers are audited.

28.11 What Is Not Concluded

This chapter does not conclude production readiness, high-dimensional tractability, posterior accuracy, GPU speedup, tensor-rank adequacy, HMC convergence, or NAWM validation. It provides the notation and claim discipline needed for skeptical review of the methods that follow.

Chapter 29

Gaussian Projection and High-Order Quadrature Filters

29.1 Chapter Claim

Gaussian projection filters are the most inspectable first rung for high-dimensional nonlinear SSMs because their assumptions, derivatives, moments, covariance failures, and computational costs can be audited locally. They are not automatically accurate. Unstructured high-order quadrature is especially dangerous: it can be mathematically elegant at small dimension and computationally impossible at the dimension of a serious DSGE target. The defensible industrial direction is block-local, sparse, or low-rank moment projection with explicit diagnostics.

29.2 Gaussian Projection Contract

Let the prediction law be approximated by $X_t \mid y_{1:t-1} \approx \mathcal{N}(m_t^-, P_t^-)$ and let $Y_t = h_\theta(X_t) + v_t$ with $v_t \sim \mathcal{N}(0, R_t)$ independent of X_t. Define the exact moments under the approximate predictive Gaussian:

$$\bar{y}_t = \mathbb{E}[h_\theta(X_t)], \tag{29.1}$$

$$S_t = \mathrm{Var}(h_\theta(X_t)) + R_t, \tag{29.2}$$

$$C_t = \mathrm{Cov}(X_t, h_\theta(X_t)). \tag{29.3}$$

The Gaussian projection update is

$$K_t = C_t S_t^{-1}, \tag{29.4}$$

$$m_t = m_t^- + K_t(y_t - \bar{y}_t), \tag{29.5}$$

$$P_t = P_t^- - K_t S_t K_t^\top. \tag{29.6}$$

Proposition 29.1 (Moment projection update)**.** *Among affine estimators $a + B\,Y_t$ of X_t, the mean-square optimal affine estimator under the joint moments (29.1)–(29.3) uses $B = C_t S_t^{-1}$ and gives (29.5)–(29.6).*

Proof. For centered variables $\tilde{X} = X_t - m_t^-$ and $\tilde{Y} = Y_t - \bar{y}_t$, minimize $\mathbb{E}\|\tilde{X} - B\tilde{Y}\|^2$. Differentiating with respect to B gives the normal equation $B\mathbb{E}[\tilde{Y}\tilde{Y}^\top] = \mathbb{E}[\tilde{X}\tilde{Y}^\top]$, hence $B = C_t S_t^{-1}$ when S_t is nonsingular. The residual covariance is $P_t^- - C_t S_t^{-1} C_t^\top$, equal to (29.6). □

The approximation is not in the affine update itself. It is in replacing the true prediction law by a Gaussian and in approximating the moments above.

29.3 Derivative-Based Filters

The extended Kalman filter linearizes f_θ and h_θ:

$$f_\theta(x) \approx f_\theta(m) + F_t(x - m), \qquad h_\theta(x) \approx h_\theta(m^-) + H_t(x - m^-). \tag{29.7}$$

This yields the familiar predicted covariance and update

$$P_t^- \approx F_t P_{t-1} F_t^\top + Q_t, \qquad S_t \approx H_t P_t^- H_t^\top + R_t, \qquad C_t \approx P_t^- H_t^\top. \tag{29.8}$$

An iterated EKF repeats the observation linearization around a current state estimate; it is closer to a local Gauss–Newton correction than a global posterior approximation. A second-order EKF adds Hessian corrections. For $X = m + \delta$, $\mathbb{E}[\delta] = 0$, and twice differentiable scalar component h_r,

$$\mathbb{E}[h_r(X)] \approx h_r(m) + \frac{1}{2}\operatorname{tr}(H_r(m)P)\,, \tag{29.9}$$

where $H_r(m)$ is the Hessian of h_r at m.

Derivation sketch for (29.9). Taylor expand $h_r(m + \delta)$ through second order: $h_r(m) + J_r\delta + \frac{1}{2}\delta^\top H_r\delta$. The linear term vanishes because $\mathbb{E}[\delta] = 0$, and $\mathbb{E}[\delta^\top H_r\delta] = \operatorname{tr}(H_r\mathbb{E}[\delta\delta^\top]) = \operatorname{tr}(H_rP)$. □

Derivative-based filters scale well only when derivatives are available and structured. Dense Hessian corrections can cost $O(mn^2)$ storage and computation per observation block before any factorization is considered. For NAWM-like models, selective second-order corrections must therefore be tied to economic blocks or low-rank sensitivities.

29.4 Sigma-Point And Cubature Rules

A sigma-point rule approximates Gaussian expectations by

$$\mathbb{E}[\varphi(X)] \approx \mathcal{Q}[\varphi] = \sum_{j=1}^{N_q} w_j \varphi(m + L\xi_j), \qquad LL^\top = P. \tag{29.10}$$

The unscented transform and sigma-point Kalman filters are associated with Julier and Uhlmann [1996, 1997], van der Merwe [2004]. Cubature and CUT-style rules choose points to integrate polynomials exactly up to a stated degree under a Gaussian weight; BayesFilter's CUT4-G support follows the conjugate unscented-transform line of Adurthi et al. [2012a,b].

Implementation-grade review of Algorithm 1 focuses on the covariance factor, weight signs, solve stability, finite nonlinear evaluations, PSD residual, and whether the same rule is used for the value and score path.

Algorithm 1 Gaussian sigma-point filtering step

Require: Mean m^-, covariance factor L, observation y_t, rule $\{(w_j, \xi_j)\}_{j=1}^{N_q}$, nonlinear observation map h_θ, observation covariance R_t.

1: Form sigma states $x_j = m^- + L\xi_j$.
2: Evaluate $z_j = h_\theta(x_j)$.
3: Compute $\bar{y} = \sum_j w_j z_j$.
4: Compute $S = R_t + \sum_j w_j (z_j - \bar{y})(z_j - \bar{y})^\top$.
5: Compute $C = \sum_j w_j (x_j - m^-)(z_j - \bar{y})^\top$.
6: Solve $K = CS^{-1}$ using a stable factorization.
7: Return $m = m^- + K(y_t - \bar{y})$ and $P = P^- - KSK^\top$ with a PSD check.

29.5 Point Growth And Sparse Alternatives

Tensor-product Gauss–Hermite rules with r one-dimensional nodes use r^d points in dimension d. The implemented CUT4-G point count in the current BayesFilter sigma-point chapter is

$$N_{\mathrm{CUT4}}(d) = 2d + 2^d. \tag{29.11}$$

At $d = 15$, this is 32,798 points; at $d = 100$, the corner set is not a credible production proposal.

Sparse-grid and Smolyak rules are often proposed as an antidote to tensor-product growth because they combine lower-level tensor rules. In this lane they remain a source-gated candidate, not an accepted filtering result: the local ResearchAssistant index did not supply an audited sparse-grid filtering source, and this execution did not perform a primary-source technical review. The only claim retained here is the blocker logic. A sparse rule would still require smoothness, low effective interaction order, bounded point count, and a measured residual for interactions omitted by the sparse construction. If the neglected interaction is where the likelihood is informative, the approximation can fail while its point count looks attractive.

29.6 Block-Local High-Order Filtering

Let the stochastic integration coordinates be partitioned into blocks $B_1, \ldots, B_K$ with sizes d_k. A block quadrature approximation applies a rule inside each block and uses a reconstruction policy for cross-block moments. If a rule uses $N(d_k)$ points in block k, the per-step nonlinear evaluation count is approximately

$$N_{\mathrm{block}} = \sum_{k=1}^{K} N(d_k) \tag{29.12}$$

for additive/block-separable propagation, rather than $N(\sum_k d_k)$. The price is an approximation to cross-block interactions.

The cross-block policy is the mathematical weak point. A reviewer should ask whether the policy is exact by model structure, an approximation with measured residuals, or merely a convenience.

Algorithm 2 Block-local Gaussian projection with selective high order

Require: Block partition $B_1, \dots, B_K$, predictive Gaussian (m^-, P^-), per-block rules, cross-block reconstruction policy, observation y_t.

1: **for** $k = 1, \dots, K$ **do**
2: Extract block marginal $(m^-_{B_k}, P^-_{B_k B_k})$.
3: Choose EKF, CKF/UKF, CUT4, or a separately audited sparse-grid rule according to the block diagnostic budget.
4: Propagate block moments and local cross-covariances.
5: **end for**
6: Reconstruct global $\bar{y}$, S, and C using the declared cross-block policy.
7: Apply the Gaussian projection update.
8: Record point count, PSD residual, neglected-cross-block diagnostic, finite status, runtime, and comparator status.

29.7 Complexity And Failure Diagnostics

Method	Scaling and memory	Main failure diagnostics	NAWM-like status
EKF/IEKF	Jacobian evaluation plus dense solves, often $O(n^3)$ if dense; $O(n^2)$ covariance memory	derivative mismatch, innovation residual, PSD failure, iteration nonconvergence	first rung if derivatives and blocks are available
Second-order EKF	Hessian contractions, dense worst case $O(mn^2)$ plus solves and $O(n^2)$ plus Hessian storage or products	Hessian noise, excessive local bias, non-PSD covariance	selective block correction only
UKF/CKF	$O(N_q)$ model evaluations, usually $N_q = O(n)$ for common rules; $O(n^2)$ covariance memory and batch point storage	weight/pathology, PSD residual, score mismatch	useful small/block reference
CUT4-G	$O(2d + 2^d)$ model evaluations; covariance plus point batch storage	point-cap skip, runtime explosion, PSD failure	not global; block reference
Tensor Gauss–Hermite	$O(r^d)$ model evaluations and point batch storage	impossible point count, overflow, memory exhaustion	validation-only at tiny dimension
Block cubature and source-gated sparse grids	rank/block dependent; exponential or source-dependent in block size, plus covariance and block point batches	neglected cross-block residual, point-cap skip, rule bias	CUT4/CKF can be a block reference; sparse-grid use is blocked until primary-source audit

29.8 BayesFilter Evidence Box

BayesFilter currently supports nonlinear sigma-point value and score diagnostics in narrow V1 cells, and the May 27 high-dimensional smoke artifact records CPU-only execution rows for existing Model B and deterministic block extensions. The artifact intentionally sets `CUDA_VISIBLE_DEVICES=-1` before TensorFlow import and records CPU-only policy. The two skipped CUT4 block-4 rows are successful stop-rule evidence, not failures of LaTeX or code: the point cap prevented an unsupported large-point diagnostic.

The evidence supports engineering statements such as "this command completed with finite value and shape status for this row." It does not support posterior accuracy, default policy, broad XLA readiness, GPU speedup, or NAWM readiness.

29.9 Industrial Practitioner Stress Test

At NAWM-like scale, global Gaussian projection can break because the posterior mass may live away from the linearization point, shocks can enter different economic blocks with different curvature, and observation channels can create nearly singular innovation covariance. The rescue structure is not "use a higher order rule everywhere." The plausible rescue is:

(1) block IEKF/IEKS to get a stable local trajectory;

(2) selective second-order corrections only for blocks with demonstrated curvature impact;

(3) CUT4 or source-gated sparse-grid block references to audit the local approximation only where point counts and source support permit;

(4) a covariance-validity gate before any HMC or posterior use.

29.10 Source And Evidence Ledger

Claim	Support class	Boundary
Gaussian projection update	Proposition 29.1 and standard Kalman/Gaussian filtering literature, e.g. Durbin and Koopman [2012]	Exact only for the stated joint moments; moment calculation may be approximate.
UKF/sigma-point lineage	Primary sources Julier and Uhlmann [1996, 1997], van der Merwe [2004]	Does not imply high-dimensional accuracy.
CUT4-G point count	BayesFilter chapter source and Adurthi et al. [2012a,b]	Applies to the discussed implemented rule, not every high-order cubature rule.

Claim	Support class	Boundary
Sparse-grid filtering	Explicit source-gap blocker; requires further primary sparse-grid filtering audit	Roadmap-only candidate; no theorem, accuracy, or production claim.
P8 high-dimensional rows	BayesFilter JSON and markdown artifacts	Execution diagnostics only.

29.11 Unresolved Claim Register

- No sparse-grid backend is implemented or validated in this phase.
- No block-local quadrature theorem is proved for a NAWM nonlinear DSGE.
- No default policy change is justified.
- Sparse-grid source support is inadequate for theorem-level prose; it is retained only as an explicit roadmap blocker until a primary technical sparse-grid filtering source is audited.

29.12 What Is Not Concluded

This chapter does not conclude that EKF, IEKF, second-order EKF, UKF, CKF, CUT4, sparse grids, or block quadrature is accurate for any particular high-dimensional DSGE model. It explains how to make these methods auditable and why their failures should be expected unless structure is demonstrated.

Chapter 30

Particle, Transport, Tensor-Train, and Tensor-Network Filters

30.1 Chapter Claim

Particle, transport, and tensor methods are high-payoff research directions for high-dimensional nonlinear filtering only when their approximation errors are measured on the downstream filtering or posterior task. Particles give non-Gaussian support but suffer weight degeneracy. Transport maps can move samples toward posterior mass but must carry density corrections. Tensor trains and tensor networks can compress structured functions or covariance objects but can fail through rank growth, truncation, positivity, and normalization errors. None of these mechanisms removes the curse of dimensionality by declaration.

30.2 Particle Filter Baseline

The bootstrap particle filter of Gordon et al. [1993] and related SMC methods [Kitagawa, 1996, Arulampalam et al., 2002, Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020] represent the filtering law by a weighted empirical measure

$$\widehat{\pi}_t^N(dx) = \sum_{i=1}^N \bar{w}_t^{(i)} \delta_{x_t^{(i)}}(dx), \qquad \bar{w}_t^{(i)} = \frac{w_t^{(i)}}{\sum_{j=1}^N w_t^{(j)}}. \tag{30.1}$$

With a proposal $q_t(x_t \mid x_{t-1}, y_t)$, the incremental weight is

$$\alpha_t^{(i)} = \frac{g_\theta(y_t \mid x_t^{(i)}) f_\theta(x_t^{(i)} \mid x_{t-1}^{a(i)})}{q_t(x_t^{(i)} \mid x_{t-1}^{a(i)}, y_t)}. \tag{30.2}$$

The bootstrap filter is the special case $q_t = f_\theta$, so the incremental weight is the observation likelihood. The standard likelihood estimator is

$$\widehat{p}_\theta(y_{1:T}) = \prod_{t=1}^T \left(\frac{1}{N} \sum_{i=1}^N w_t^{(i)} \right), \tag{30.3}$$

with the usual SMC caveat that unbiasedness statements concern the likelihood estimator, not the log likelihood.

Algorithm 3 Sequential importance resampling filter

Require: Particle count N, initial sampler, transition f_θ, observation likelihood g_θ, resampling threshold τ.

Sample $x_0^{(i)} \sim \mu_\theta$ and set $\bar{w}_0^{(i)} = 1/N$.
for $t = 1, \dots, T$ **do**
  if $\mathrm{ESS}_{t-1} < \tau N$ **then**
    Resample ancestor indices $a(i)$ from $\bar{w}_{t-1}$.
    Reset ancestor weights to $1/N$.
  else
    Set $a(i) = i$.
  end if
  Propagate $x_t^{(i)} \sim f_\theta(\cdot \mid x_{t-1}^{a(i)})$.
  Weight $w_t^{(i)} = g_\theta(y_t \mid x_t^{(i)})$.
  Normalize weights and record ESS, finite status, and ancestor count.
end for

The effective sample size diagnostic

$$\mathrm{ESS}_t = \frac{1}{\sum_{i=1}^{N} (\bar{w}_t^{(i)})^2} \tag{30.4}$$

is a degeneracy signal, not a posterior-accuracy certificate. In a high-dimensional observation, small likelihood mismatches compound across coordinates and the maximum weight can dominate. The high-dimensional collapse warnings of Bengtsson et al. [2008] and Snyder et al. [2008] are therefore central, not incidental.

30.3 Guided And Auxiliary Proposals

Auxiliary and guided particle filters spend computation before weighting to choose better ancestors or proposals. The implementation contract is simple: if the proposal changes, the density ratio in (30.2) must change with it. A guide can be an EKF linearization, block IEKF smoother, flow proposal, local Laplace approximation, or learned policy, but it remains a proposal until the correction is explicit.

Algorithm 4 Guided proposal particle step

Require: Ancestor particles, guide state r_t, proposal density $q_t(\cdot \mid x_{t-1}, y_t, r_t)$, transition density f_θ, likelihood g_θ.

for $i = 1, \dots, N$ **do**
  Select ancestor $a(i)$ using the declared auxiliary policy.
  Draw $x_t^{(i)} \sim q_t(\cdot \mid x_{t-1}^{a(i)}, y_t, r_t)$.
  Set $\alpha_t^{(i)}$ using (30.2).
end for
Normalize, resample if required, and record finite density ratios, ESS, and guide residuals.

The practitioner question is not whether the guide looks plausible. It is whether the corrected weights avoid collapse and whether the resulting filtering estimate agrees with a reference on controlled cells.

30.4 Transport And Flow Proposals

Let $z \sim r(z)$ and let T_ϕ be an invertible differentiable map with $x = T_\phi(z)$. The proposal density induced by the map is

$$q_\phi(x) = r(T_\phi^{-1}(x)) \left|\det DT_\phi^{-1}(x)\right|. \tag{30.5}$$

If this proposal is used for filtering, the importance weight must divide by q_ϕ. Normalizing-flow references such as Papamakarios et al. [2021] and transport-map MCMC work such as Parno and Marzouk [2018] support the change-of-variables and geometry-preconditioning framing, but they do not by themselves validate a BayesFilter nonlinear SSM proposal.

Proposition 30.1 (Transport proposal correction)**.** *For any invertible differentiable proposal map with density* (30.5)*, importance sampling estimates expectations under a target density $\gamma(x)/Z$ using weights proportional to $\gamma(x^{(i)})/q_\phi(x^{(i)})$, provided $q_\phi > 0$ wherever $\gamma > 0$.*

Proof. For integrable φ, $\int \varphi(x)\gamma(x)\,dx = \int \varphi(x)\gamma(x)q_\phi(x)^{-1}q_\phi(x)\,dx$ on the target support. Monte Carlo under q_ϕ gives the usual self-normalized estimator after dividing by the estimated normalizing constant. □

The diagnostics are finite inverse map, finite log determinant, finite density ratio, ESS, support mismatch, and posterior or filtering checks. A map-training loss is explanatory only unless the phase plan explicitly makes it a promotion criterion and connects it to the downstream computation.

30.5 Ensemble Transport

Ensemble transform methods build a coupling between forecast particles and analysis particles. Reich's ensemble transform method [Reich, 2013] and entropy-regularized optimal transport [Villani, 2003, Cuturi, 2013, Peyré and Cuturi, 2019, Schmitzer, 2019] are useful because they expose a deterministic map or coupling matrix. For filtering, the approximation is the finite ensemble, the localization or map class, the regularization, and the numerical transport solve.

Algorithm 5 Localized ensemble transport diagnostic step

Require: Forecast ensemble $\{x_f^{(i)}\}_{i=1}^N$, weights $\bar{w}_i$, localization blocks, transport regularization ϵ.

1: **for** each localization block **do**
2: Build a block cost matrix between forecast and analysis support.
3: Solve a constrained coupling or entropy-regularized transport problem.
4: Transform the block ensemble and record marginal mismatch.
5: **end for**
6: Recombine blocks using the declared cross-block policy.
7: Record ESS before transport, coupling residual, localization residual, finite status, and posterior/reference diagnostics where available.

For NAWM-like use, localization is the likely rescue structure. Without it, the transport problem itself becomes high-dimensional and finite-ensemble collapse returns under a different name.

30.6 Tensor-Train Density And Operator Compression Blocker

A tensor train represents a d-way array $A(i_1, \dots, i_d)$ as

$$A(i_1, \dots, i_d) = G_1(i_1)G_2(i_2)\cdots G_d(i_d), \tag{30.6}$$

where each $G_k(i_k)$ is an $r_{k-1} \times r_k$ matrix with $r_0 = r_d = 1$. For equal mode size M and rank bound r, storage is approximately $O(dMr^2)$ rather than $O(M^d)$. This is compression only if the ranks remain small.

The filtering research idea is to tensorize a density, likelihood, transition operator, observation map, or quadrature rule and propagate it through a sequential update. This execution does not have enough source support to present that idea as an implementation recommendation. The specific tensor-train filtering papers identified in the May 27 plan remain metadata/source-gap support until their technical sections are audited. Therefore the only review-ready content here is the quarantine contract: any future TT pilot must expose rank growth, normalization error, positivity defects, and downstream likelihood/score distortion before it can be used as more than a diagnostic.

Algorithm 6 Blocked tensor-train filtering pilot checklist

Require: A separately accepted source-review note, low-dimensional or block-structured fixture, tensorization map, initial TT approximation, rank cap $r_{\max}$, low-dimensional reference.

1: Stop if the source-review note is missing; do not implement from this chapter alone.
2: **for** $t = 1, \dots, T$ **do**
3: Apply the transition/update operator in TT form or project it into TT form.
4: Round/truncate ranks with declared tolerance.
5: Renormalize if the representation is a density.
6: Record TT ranks, normalization residual, positivity residual, moment error, and value/score parity against the reference.
7: **end for**
8: Keep the TT object quarantined as a diagnostic/proposal unless downstream posterior checks pass.

30.7 Tensor-Network And Square-Root Blocker

Tensor-network Kalman filters are reported in the planning lane as a way to compress lifted linear dynamics or covariance objects, but this run has only metadata-level access to those sources. Consequently this chapter does not make a technical claim about tensor-network Kalman filtering. It records a BayesFilter blocker instead: if a future phase compresses covariance or lifted dynamics, it must preserve an inspectable factor or square-root validity gate, because direct rounding of covariance-like objects can lose positive semidefiniteness. That statement is a general numerical-safeguard requirement, not evidence that the named tensor-network methods work for nonlinear SSMs.

30.8 Low-Rank Tensor Observation Compression

The NeuroImage 2023 low-rank tensor UKF paper discussed in the planning lane is domain-specific and currently only metadata-supported here. This chapter therefore does not cite it as support for a BayesFilter method. It records only the question a future source-review phase would have to answer: can nonlinear observation geometry be compressed while preserving the filtering quantity of interest? For a DSGE model, possible hypotheses include sparse factor loadings, block observation maps, or active subspaces in measurement equations, but each requires its own source review and downstream posterior checks.

30.9 Complexity And Failure Diagnostics

Family	Scaling and memory	Degeneracy/failure diagnostics	NAWM-like rescue
Bootstrap PF	$O(N)$ transition/likelihood evaluations per step; $O(Nn)$ particle memory	ESS, unique ancestors, max weight, support gaps	not viable without guide
Guided/APF	$O(N)$ proposal plus guide cost; $O(Nn)$ plus guide state	finite density ratios, guide residual, ESS	block IEKF/transport guide
Flow PF	$O(N)$ map evaluations plus Jacobians; particles plus map parameters	log-det instability, support mismatch, weight collapse	triangular/block maps
Ensemble transport	transport solve plus ensemble cost; ensemble plus cost or coupling matrices	coupling residual, localization bias, rank/ensemble collapse	localization and sparse couplings
TT density/operator blocker	rank dependent; rough TT storage formulas are illustrative until source audit; $O(dMr^2)$ stored cores under a rank cap	rank explosion, negative density, normalization drift	blocked until source audit
TN square-root covariance blocker	rank/network dependent; factor-network storage if implemented	PSD/factor residual, rounding drift	blocked; require factor validity gate

30.10 Industrial Practitioner Stress Test

At NAWM-like scale, particle filters break first through support and weight collapse. Transport methods break when the map class cannot express the posterior geometry or when Jacobian/density evaluation becomes unstable. Tensor methods break when ranks grow with shocks, regimes, or observation channels. The only credible rescue is structure:

- block-local proposals tied to economic model blocks;

- localization for ensemble transport;
- triangular or conditional transports whose density correction is cheap;
- tensor pilots restricted to source-reviewed maps or likelihood factors with measured low rank;
- downstream posterior checks before any method informs HMC or policy.

30.11 BayesFilter Evidence Boundary

BayesFilter has particle-filter and differentiable-resampling monograph material elsewhere, but this high-dimensional chapter does not import DPF implementation evidence or student baselines as authority. The May 27 P8 harness is cited only for CPU-only execution metadata in deterministic block extensions. There is no BayesFilter tensor backend, no promoted transport particle filter, no accepted sparse-grid filtering source review, and no high-dimensional posterior recovery benchmark in this lane.

30.12 Literature Matrix

Classical SMC.
Gordon et al. [1993], Kitagawa [1996], Arulampalam et al. [2002], Doucet et al. [2001], Chopin and Papaspiliopoulos [2020] are primary technical sources for baseline algorithms and diagnostics.

High-dimensional particle-filter collapse.
Bengtsson et al. [2008], Snyder et al. [2008] are primary technical sources for the degeneracy warning and evidence burden.

Particle flow.
Daum and Huang [2008], Li and Coates [2017], Hu and van Leeuwen [2021] are primary sources for the particle-flow family. In this lane they support proposal/research framing only.

Differentiable and OT resampling.
Zhu et al. [2020], Corenflos et al. [2021], Jonschkowski et al. [2018], Karkus et al. [2018] are primary sources, but the DPF implementation lane remains quarantined. They are background only here.

Optimal transport and ensemble transform.
Villani [2003], Cuturi [2013], Peyré and Cuturi [2019], Schmitzer [2019], Reich [2013] are primary technical sources for coupling and ensemble-transform diagnostics.

Normalizing and transport maps.
Papamakarios et al. [2021], Parno and Marzouk [2018] support proposal-density and geometry framing. They do not validate a BayesFilter filtering backend.

Tensor-train and tensor-network entries.
Li–Wang–Yau–Zhang arXiv:1908.04010, Zhao–Cui JMLR 2024, Batselier–Chen–Wong arXiv:1610.05434, Menzen–Kok–Batselier arXiv:2409.03276, and the NeuroImage 2023 low-rank tensor UKF paper remain source-gap blockers. They may

motivate future source review, but they do not support theorem-level BayesFilter claims in this chapter.

30.13 Unresolved Claim Register

- Tensor-train and tensor-network filtering sources still require technical full-paper review before theorem-level claims.
- No tensor, flow-particle, or ensemble-transport backend is implemented or validated by this phase.
- No particle/transport method is promoted to HMC readiness.
- DPF student and controlled-baseline files remain outside this lane.

30.14 What Is Not Concluded

This chapter does not conclude that particle, flow, transport, tensor-train, or tensor-network methods solve high-dimensional filtering. It states the corrections, diagnostics, and source gaps that must be closed before any method can be treated as more than a candidate.

Chapter 31

HMC as a Research Program for Nonlinear State-Space Models

31.1 Chapter Claim

Hamiltonian Monte Carlo for nonlinear state-space models is a per-model research problem. A finite scalar target, a dense mass matrix, or a short chain is not enough. Nonlinear filters introduce approximate likelihoods, invalid regions, branch-dependent gradients, and curvature that can defeat plain Euclidean HMC. BayesFilter's defensible production direction is an inspectable fixed-trajectory HMC layer with explicit value/score contracts; TFP NUTS is diagnostic/reference only in this project.

31.2 Target, Coordinates, And Correction

Let $q \in \mathbb{R}^d$ be the unconstrained coordinate used by the integrator and let $\theta = \tau(q)$. If the filter supplies log likelihood $\ell_T(\theta)$, the exact potential in q coordinates is

$$U(q) = -\ell_T(\tau(q)) - \log \pi(\tau(q)) - \log \left|\det D\tau(q)\right|. \tag{31.1}$$

If the filter supplies $\widehat{\ell}_T$, the HMC target is the corresponding approximate posterior unless an exact correction is supplied. The gradient used by the integrator must be the gradient of the same scalar potential used in the Metropolis correction:

$$\nabla_q U(q) = -D\tau(q)^\top \nabla_\theta \{\ell_T(\theta) + \log \pi(\theta)\}\big|_{\theta=\tau(q)} - \nabla_q \log |\det D\tau(q)|. \tag{31.2}$$

Proposition 31.1 (Value/score same-scalar condition)**.** *For a Metropolis-corrected HMC transition to target the density proportional to* $\exp\{-U(q)\}$*, the numerical trajectory may use an approximate integrator, but the acceptance probability must evaluate the same Hamiltonian* $H(q,p) = U(q) + \frac{1}{2}p^\top M^{-1}p$ *whose gradient drives the integrator.*

Proof. The standard HMC correction applies Metropolis–Hastings to a deterministic, volume-preserving, reversible proposal map generated by the numerical integrator; see Neal [2011] and Betancourt [2017]. If the gradient corresponds to a different scalar than the accepted Hamiltonian, the proposal is no longer the intended discretization of that Hamiltonian and the usual detailed-balance argument no longer applies. □

31.3 Why Plain Mass-Matrix HMC Can Diverge

A diagonal or dense mass matrix handles global scale but not arbitrary target geometry. It cannot by itself remove funnels, hard constraints, invalid regions, sharp ridges, local curvature rotations, discontinuous branches, or filter-induced roughness. Divergences therefore matter: they indicate that the numerical Hamiltonian trajectory is not faithfully following the target at the chosen step size and trajectory length. Speed comparisons are meaningless until divergence and validity gates pass.

The BayesFilter Model B/C HMC ladder remains a candidate diagnostic only: finite values and samples were observed in short rows, but acceptance was near 1.0 and maximum $\widehat{R}$ was around 2.0. That is evidence that the target can be evaluated in the tested cells and that research should continue. It is not convergence evidence.

31.4 Fixed-Trajectory HMC Baseline

Algorithm 7 BayesFilter-owned fixed-trajectory HMC diagnostic step

Require: Potential $U(q)$, gradient $\nabla U(q)$, mass matrix M, step size ϵ, leapfrog steps L, current q.

1: Check value and gradient finite at q.
2: Draw $p \sim \mathcal{N}(0, M)$.
3: Run L leapfrog steps with the declared invalid-region policy.
4: Evaluate initial and proposed Hamiltonians with the same scalar U.
5: Accept or reject by the Metropolis probability.
6: Record divergence flag, energy error, acceptance, finite status, and invalid-region hits.

This baseline is intentionally simple. It makes failures visible and keeps implementation ownership inside BayesFilter. A production candidate would need adaptation, diagnostics, and careful warmup, but the research ladder starts with the smallest transition whose target contract can be audited.

31.5 Diagnostic-Only NUTS Policy

NUTS [Hoffman and Gelman, 2014] is an important reference algorithm, and Stan's implementation is the benchmark many practitioners trust. In this project, TFP NUTS is not the strategic production path for industrial nonlinear SSMs: dynamic control flow, adaptation internals, and nested kernel structure make full-pipeline XLA behavior, failure triage, and model-specific diagnostics less inspectable than a BayesFilter-owned fixed transition. This is an engineering policy, not a mathematical claim that NUTS is inferior.

31.6 NeuTra And Transport-Preconditioned HMC

NeuTra-style HMC learns an invertible neural transport so that HMC runs in a warped space; local ResearchAssistant identifies Hoffman et al. [2021] as the relevant source but marks the local summary as needing manual review. The safe claim is therefore bounded: neural transport is a geometry-preconditioning idea, not a correctness substitute.

If $q = T_\phi(z)$ and the target density in q coordinates is $\pi_q(q)$, the warped target in z coordinates is

$$\log \pi_z(z) = \log \pi_q(T_\phi(z)) + \log |\det DT_\phi(z)|. \tag{31.3}$$

HMC in z must use the gradient of (31.3) and a Metropolis correction for that same scalar. A trained map can reduce curvature or can make tails worse; only sampler diagnostics decide.

31.7 RMHMC, HNN, And Learned Dynamics

Riemannian HMC replaces the fixed mass matrix by a position-dependent metric [Girolami and Calderhead, 2011]. The conceptual fit is strong when curvature rotates over the posterior, but the implementation burden is severe: metric positive definiteness, derivatives of the metric, implicit solves, and filtering Hessian stability become part of the target contract.

Hamiltonian neural networks [Greydanus et al., 2019] and learned HMC variants can learn proposal dynamics. The local ResearchAssistant learned-HMC summary is not domain-specific to nonlinear SSMs, so transfer claims are forbidden. Learned dynamics can enter BayesFilter only as a proposal with target-preserving correction and diagnostics stronger than training loss.

Algorithm 8 Acceleration promotion ladder

Require: Baseline fixed HMC result, candidate accelerator, controlled target.

1: Verify same-scalar value/score parity in eager and compiled modes.
2: Run plain fixed HMC until validity diagnostics are interpretable.
3: Add transport, RMHMC metric, or learned proposal with exact correction.
4: Reject the acceleration if divergences, nonfinite values, unavailable $\widehat{R}$/ESS, or posterior-reference checks fail.
5: Compare runtime per effective sample only after validity gates pass.

31.8 Diagnostics

The minimum nonlinear SSM HMC diagnostic ledger is:

- finite target and finite gradient at representative points;
- value/score parity for the scalar used by the sampler;
- divergence count and energy error;
- acceptance distribution, step-size sensitivity, and trajectory-length sensitivity;
- $\widehat{R}$, bulk/tail ESS, and E-BFMI where chains are long enough for these fields to be meaningful;
- posterior recovery on a controlled reference before scientific claims;
- runtime per effective sample only after the validity vetoes pass.

Any divergence, nonfinite value, failed same-scalar check, unavailable $\widehat{R}$/ESS on a claimed convergence row, or failed posterior-reference check reroutes the result to repair. It must not be ranked by speed.

31.9 XLA And GPU Boundary

XLA and GPU execution are performance mechanisms, not scientific validation. For HMC, XLA readiness requires static-enough control flow, stable shapes, same-scalar value/gradient parity after compilation, deterministic invalid region policy, and reproducible diagnostics. GPU speedup requires trusted GPU execution, meaningful problem size, CPU comparator rows, and validity gates before speed ranking. CPU-only artifacts must hide GPU devices before framework import and say so.

31.10 Industrial Practitioner Stress Test

At NAWM-like scale, plain HMC can fail because posterior geometry is shaped by structural solution maps, occasionally stiff measurement equations, parameter constraints, and filter approximation error. The plausible rescue sequence is:

(1) stabilize the filtering target and value/score path first;

(2) establish a plain fixed-HMC baseline and record its divergences;

(3) use block or transport preconditioning to reduce geometry pathologies;

(4) consider HNN or learned proposals only after exact correction and sampler diagnostics are routine;

(5) treat every model as a new research target until posterior checks pass.

31.11 Source And Evidence Ledger

Claim	Support class	Boundary
HMC same-scalar and diagnostic discipline	Primary sources Neal [2011], Betancourt [2017] plus Proposition 31.1	Does not validate any BayesFilter nonlinear chain.
NUTS is diagnostic/reference-only here	Project policy informed by Hoffman and Gelman [2014] and local engineering constraints	Not a claim against Stan or NUTS mathematically.
NeuTra is geometry preconditioning	Hoffman et al. [2021]; local ResearchAssistant summary is needs-review	No BayesFilter NeuTra validation or Gate-1 promotion in this lane.
RMHMC is geometry-aware	Girolami and Calderhead [2011]; local summary needs-review	Hessian/metric implementation unresolved.

Claim	Support class	Boundary
HNN/learned dynamics are proposal acceleration ideas	Greydanus et al. [2019] and local learned-HMC summary	No nonlinear SSM transfer claim.
Model B/C HMC ladder is not convergence evidence	BayesFilter V1 result notes	Finite short rows nominate research only.

31.12 Unresolved Claim Register

- No BayesFilter nonlinear HMC convergence result exists.
- No NeuTra, RMHMC, or HNN BayesFilter backend is implemented or promoted here.
- No nonlinear Hessian readiness or NAWM HMC diagnostic is claimed.
- ResearchAssistant summaries for NeuTra/RMHMC/learned HMC are useful review material but remain needs-review, so they cannot support theorem-level prose.

31.13 What Is Not Concluded

This chapter does not conclude that HMC, NeuTra, HNN, RMHMC, TFP NUTS, XLA, or GPU execution is production-ready for nonlinear SSMs. It defines the per-model research ladder required before such a claim could be considered.

Chapter 32

Candidate Synthesis for Industrial-Scale Nonlinear Filtering

32.1 Chapter Claim

The most defensible industrial strategy is hybrid and staged. Start with block-local Gaussian and smoothing structure because it is inspectable. Add high-order block checks where they answer a specific local question. Use transport, flow, particle, tensor, NeuTra, or HNN components only behind correction and diagnostic gates. The ranking in this chapter is a reviewer-defensible evidence-building order, not a leaderboard for posterior accuracy, speed, or production readiness.

32.2 Assumptions And Current Evidence State

The ranking assumes the current BayesFilter record:

- V1 nonlinear sigma-point diagnostics are narrow finite/parity cells.
- The May 27 P8 harness is CPU-only and records execution metadata for selected existing and deterministic-block rows.
- No tensor, ensemble-transport, NeuTra, HNN, RMHMC, or flow-particle backend is validated in this lane.
- No high-dimensional posterior recovery benchmark and no nonlinear NAWM validation exists.

Therefore the ranking is split into implementation-near candidates and source-gated research-watchlist candidates. It is a research prior for where to spend evidence, not a scientific conclusion.

32.3 Ranking Criteria

Each candidate is judged by:

(i) expected computational and memory scaling;

(ii) required model structure;

(iii) degeneracy and failure behavior;

(iv) differentiability and same-scalar compatibility;

(v) XLA/GPU readiness as an engineering property;

(vi) BayesFilter implementation burden;

(vii) evidence needed before promotion;

(viii) suitability for a NAWM-like nonlinear DSGE stress target.

32.4 Ranked Candidate Tables

32.4.1 Implementation-Near Evidence-Building Order

These entries have enough BayesFilter-local or primary-source support to define near-term evidence ladders. They still are not validated for NAWM-like deployment.

1. **Block IEKF/IEKS plus selective second-order correction.** Scaling is near-linear across blocks plus block-cubic solves if the partition is real. It needs sparse derivatives and stable economic blocks. Vetoes are local linearization bias, failed iterates, bad Hessian blocks, and PSD loss. Promotion requires derivative parity, innovation checks, PSD residuals, and block-reference recovery.

2. **Block CKF/UKF/CUT4 references.** Scaling is exponential only inside chosen blocks, so bounded block size and controlled cross-block reconstruction are mandatory. Vetoes are point-cap skips, covariance reconstruction error, and local rule bias. Promotion requires small-block reference comparisons, point-count and memory gates, and cross-block residuals.

3. **Guided or flow particle filtering.** Scaling is N times proposal or map cost and requires strong guides with support coverage. Vetoes are weight collapse, support mismatch, and unstable Jacobians. Promotion requires corrected-weight ESS, finite density ratios, and low-dimensional reference likelihood or posterior checks.

4. **NeuTra or transport-preconditioned fixed HMC.** Scaling is training cost plus HMC cost. It needs an invertible map and the exact transformed target. Vetoes are bad tails, map bias, and failed transformed diagnostics. Promotion requires a plain-HMC baseline, transformed value/score parity, zero-divergence controlled rows, and posterior checks.

5. **HNN or learned Hamiltonian acceleration.** Scaling is training plus cheap learned steps, but the proposal must still be corrected against the intended target. Vetoes are invalid proposal mechanics, domain shift, and poor Metropolis efficiency. ESS per gradient is considered only after validity diagnostics pass.

6. **Hybrid filter-assisted HMC proposals.** Scaling combines filter and sampler cost and requires explicit target/proposal separation. Vetoes are approximate-target drift, double use of data, and a mismatch between filter diagnostics and posterior diagnostics. Promotion requires same-scalar audit, proposal correction, posterior recovery, and no failed filter diagnostics.

32.4.2 Source-Gated Research Watchlist

The next entries are not lower because they are uninteresting; they are separated because this execution lacks enough technical source review or BayesFilter evidence to rank them as implementable choices.

Ensemble transport filtering/smoothing.
Primary OT and ensemble-transform sources are cited, but this lane has no BayesFilter transport backend or localized posterior check. Promotion would require coupling residuals, localization sensitivity, ESS, and low-dimensional posterior/reference checks. NAWM-like use is promising only with localization.

Sparse-grid filtering beyond block references.
The local run has no audited primary sparse-grid filtering source. Promotion would require primary-source review, point-count and interaction residual gates, and comparison to CUT4/CKF blocks. It remains a blocked candidate.

TT/TN density or likelihood compression.
Named tensor sources are metadata/source-gap support only. Promotion would require technical source review, rank stability, positivity, normalization, value/score parity, and downstream checks. It remains a blocked pilot.

32.5 Synthesis Architecture

A credible BayesFilter research architecture for a NAWM-like nonlinear DSGE should proceed in layers:

(1) *Structural partition*: declare stochastic, deterministic, and economic block coordinates before choosing a filter.

(2) *Gaussian scaffold*: run IEKF/IEKS and selective second-order diagnostics to locate curvature and invalid covariance behavior.

(3) *Block references*: use CKF/UKF/CUT4 blocks where point counts and cross-block residuals are controlled; add sparse-grid blocks only after a primary-source review unblocks them.

(4) *Non-Gaussian proposals*: add guided particles or flow proposals with density correction and ESS gates.

(5) *Compression pilots*: keep TT/TN or low-rank observation compression as source-gated isolated pilots before using them in filtering.

(6) *Sampler layer*: use fixed HMC on the declared approximate or exact target; add NeuTra/HNN acceleration only after validity diagnostics pass.

32.6 Decision Table

Move from EKF to block high order.
Promote only if residuals show local curvature matters and point count is bounded. Veto on PSD failure, derivative mismatch, or disagreement with a block reference. Passing this gate does not prove global posterior accuracy.

Move from bootstrap PF to guided or flow PF.
Promote only if ESS and support diagnostics improve with corrected weights. Veto on nonfinite density ratios or support mismatch. Passing does not validate a learned proposal scientifically.

Use ensemble transport.
Promote only after source review, coupling residual checks, and localized posterior checks. Veto if localization materially changes posterior summaries. Passing does not prove an exact Bayesian update.

Use TT/TN compression.
Promote only after source review and stable rank, positivity, normalization, and value/score parity. Veto if source review is missing, rank grows unbounded, or signed-density defects appear. Passing does not solve high-dimensional filtering generally.

Use NeuTra or HNN acceleration.
Promote only after the plain target gates pass and the accelerated chain passes sampler diagnostics. Veto on any persistent divergence or same-scalar failure. Passing does not establish production HMC.

32.7 Industrial Practitioner Questions

A skeptical former professor working in industry will ask five questions before accepting any candidate:

(1) What exact scalar target is being approximated or sampled?

(2) Which structure reduces dimension: blocks, sparsity, low rank, localization, transport, or tensor rank?

(3) What diagnostic vetoes the method before speed is considered?

(4) What BayesFilter artifact proves the implementation fact being cited?

(5) What remains unproved for NAWM-like deployment?

Every chapter in this block is organized to answer those questions. When an answer is unavailable, the correct label is blocker or research hypothesis, not implicit acceptance.

32.8 BayesFilter Evidence Boundary

The current evidence supports a planning conclusion: BayesFilter has enough nonlinear value/score and execution scaffolding to design a disciplined high-dimensional research ladder. It does not yet have evidence for an industrial default. The P8 rows should be used to test harness shape and metadata discipline; they should not rank candidate families by speed or accuracy.

32.9 Source And Evidence Ledger

Block Gaussian methods are first-rung candidates.
Support comes from Chapter 29 and current BayesFilter nonlinear sigma-point evidence. The boundary is ranking judgment, not accuracy proof.

Particle and transport methods need correction and degeneracy gates.
Support comes from Chapter 30 and primary SMC/OT sources. The boundary is no BayesFilter promotion.

Sparse-grid and tensor methods are source-gated watchlist candidates.
Support comes from the source-gap ledgers in Chapters 29 and 30. The boundary is no sparse-grid or tensor validation claim.

HMC acceleration comes after target validity.
Support comes from Chapter 31 and primary HMC sources. The boundary is no convergence or production claim.

Hybrid strategy is the recommended evidence-building order.
Support comes from synthesis across chapters and current BayesFilter artifacts. The boundary is research roadmap only.

32.10 Unresolved Claim Register

- No candidate is validated on NAWM.
- No candidate has a full high-dimensional posterior recovery benchmark.
- No GPU/XLA policy change is supported.
- No HMC acceleration method is promoted.
- Tensor and sparse-grid claims need deeper primary-source technical audit before theorem-level monograph prose.

32.11 What Is Not Concluded

The ranking does not conclude that the top method is scientifically correct or that lower-ranked methods are poor. It states a practical order for building evidence under current BayesFilter constraints while preserving all no-overclaim boundaries.

Part V

HMC, Geometry, and Diagnostics

Chapter 33

HMC for State-Space Likelihoods

This chapter will develop Hamiltonian Monte Carlo and NUTS for filtering-based posterior targets. The mathematical foundations are supported by the Phase 2B literature gate, but implementation claims remain gated by BayesFilter's target, gradient, and compilation contracts.

33.1 Target Contract

For BayesFilter, an HMC transition is valid only relative to the target in Definition 3.1. The sampler needs the scalar log target and its gradient in the coordinates used by the integrator. If the filter supplies a custom score, that score must correspond to the same log likelihood value used in the Metropolis correction.

The HMC chapter therefore depends on the earlier derivative chapters. A filtering backend that can evaluate $\ell(\theta)$ but has no declared gradient policy is value-only evidence. It can be used for optimization, diagnostics, or likelihood comparison, but it should not be promoted to HMC production use.

33.2 Implementation Position

TensorFlow Probability provides useful reference implementations of HMC and NUTS, but BayesFilter should not treat TFP NUTS as the strategic production backend. This is a project implementation decision, not a mathematical criticism of NUTS. The decision is recorded in the source map and reset memo and is supported by the minimal Gaussian benchmark in `docs/benchmarks/benchmark_tfp_nuts_gaussian.py`.

The reasons to keep TFP NUTS as a reference rather than the default production path are:

- full-trajectory NUTS control flow is hard to keep reliably XLA-compiled across model-specific likelihoods, adaptation logic, and diagnostics;
- nested TensorFlow Probability kernels can obscure whether the complete value-and-gradient pipeline is compiled or only partially traced;
- debugging numerical failures inside TFP kernel internals is slower than debugging a small BayesFilter-owned HMC/NUTS transition;

- large DSGE targets require explicit policies for finite log density, finite gradients, static shapes, filter failure handling, and custom derivative backends.

Consequently, this chapter will consolidate NUTS foundations from the approved literature sources while treating TFP NUTS as a reference and diagnostic baseline. BayesFilter's production direction is a small, inspectable HMC layer whose contracts are designed around filtering likelihoods, analytic/custom gradients, and XLA/JIT discipline. Any future NUTS implementation should satisfy the same filter-aware contracts before it is promoted beyond diagnostic use.

33.3 Diagnostic Use of NUTS

NUTS remains valuable as an algorithmic reference and as a diagnostic sampler on small controlled targets. The bounded BayesFilter decision is narrower: the project should not treat TFP NUTS as the default answer to every convergence, geometry, or compilation problem. The Gaussian benchmark shows that TFP NUTS can XLA-compile on a toy target, but it is still materially heavier than fixed-step HMC there. Larger filtering targets add the hard parts omitted by that benchmark: spectral derivative risk, missing-data branches, model-specific invalid regions, and backend fallbacks.

Chapter 34

Mass Matrices and Local Geometry

Mass matrices translate derivative information into sampler geometry. In a linear Gaussian validation problem, a local posterior Hessian or observed information matrix can be a useful preconditioner. In a nonlinear or weakly identified model, the same matrix can be misleading unless it is regularized and checked against diagnostics.

34.1 Sources of Curvature

BayesFilter recognizes several curvature sources:

- analytic likelihood Hessian plus prior curvature;
- observed-information or expected-information approximations;
- finite-difference Hessian diagnostics at a MAP point;
- empirical covariance from warmup draws;
- diagonal fallbacks when dense curvature is unstable.

The source must be recorded because each source has a different failure mode. MacroFinance mass-matrix tests check eigenvalue regularization, dense versus diagonal choices, whitening finiteness, and covariance construction from Hessian helpers. These are geometry utilities; they are not identification proofs.

34.2 Regularization Policy

A mass matrix used by HMC must be symmetric positive definite. If a curvature estimate is indefinite or ill-conditioned, BayesFilter should regularize it, record the regularization, and expose the eigenvalue diagnostics. Silently using an indefinite curvature matrix as if it were a covariance is a target geometry error.

Chapter 35

Boundary Handling and Safe Gradients

Boundary handling is part of the posterior target. Parameters can leave the stationary, positive-definite, or economically admissible region during HMC warmup. A robust implementation must handle those proposals without crashing the chain or returning non-finite gradients.

35.1 Finite Invalid Returns

The preferred policy is:

(i) detect invalid regions before fragile factorizations where possible;

(ii) return a deterministic low log density;

(iii) return a finite fallback gradient compatible with rejection;

(iv) record the invalid reason in diagnostics;

(v) keep the behavior stable under JIT compilation.

This policy is not a mathematical trick to make an invalid model valid. It is an engineering rule that keeps sampler control flow alive long enough for HMC to reject bad proposals and continue exploring the admissible region.

35.2 Boundary Checklist

Every HMC target should state its policy for prior support, parameter transforms, stationarity or determinacy restrictions, covariance pivots, solve residuals, non-finite likelihoods, and non-finite gradients.

Chapter 36

XLA and JIT Compilation

BayesFilter uses JIT compilation as a production tool, not as a correctness argument. A sampler is not fully compiled merely because an outer function is wrapped in `tf.function`. The complete value-and-gradient path must be inside the compiled region, with static shapes and compatible device placement.

36.1 Full-Pipeline Requirement

For HMC, the compiled path includes:

- parameter transform;
- model-provider maps;
- filtering likelihood;
- custom or autodiff gradient path;
- invalid-region branch;
- leapfrog transition or equivalent integrator.

If any component falls back to Python, retraces dynamically, or executes an opaque noncompiled kernel, the implementation should be described as partially compiled.

36.2 Device and Shape Constraints

The DSGE XLA source notes emphasize two lessons that carry over to BayesFilter. First, XLA compiles a function for a target platform, so mixing CPU-only custom calls with GPU-targeted operations inside one JIT block can fail. Second, XLA requires static shapes. Dynamic metadata should be encoded as values or opaque compile-time metadata, not as changing tensor ranks.

The practical policy is phase separation: solve or model-construction phases may be compiled separately from the filtering/HMC phase if they have different device requirements. A monolithic JIT block is valuable only when it actually contains compatible operations.

Chapter 37

Diagnostics and Failure Taxonomy

Diagnostics are evidence about failures and geometry. They are not proofs of convergence. BayesFilter should report diagnostics at three levels: filter evaluation, derivative evaluation, and sampler behavior.

37.1 Filter and Derivative Diagnostics

Filter diagnostics include factor pivots, reconstruction errors, solve residuals, condition numbers, invalid-region counts, and finite-value checks. Derivative diagnostics include score/Hessian finite checks, symmetry errors, backend parity deltas, finite-difference residuals, and custom-gradient parity. For SVD or eigendecomposition paths, singular-value or eigenvalue gap telemetry is mandatory before HMC claims are made.

37.2 Sampler Diagnostics

Sampler diagnostics include acceptance rate, divergences, energy behavior, effective sample size, split R-hat, mass-matrix conditioning, and tree depth if NUTS is used diagnostically. Short clean runs are smoke tests. They can justify the next experiment, but they do not establish convergence for an industrial target.

37.3 Failure Taxonomy

Common failure classes are:

- target-domain failures;
- factorization or solve failures;
- non-finite value or gradient failures;
- spectral derivative instability;
- JIT retracing or partial compilation;
- sampler geometry failures;
- unsupported missing-data or mask patterns.

Each failure should point to a next action: fix the model contract, change the backend, add jitter or regularization, strengthen derivative validation, or downgrade the backend to diagnostic use.

Chapter 38

Transport Maps and Surrogate Geometry

Transport maps and surrogates are downstream geometry tools. They can make an HMC target easier to explore, but they cannot repair an invalid likelihood, an inconsistent custom gradient, or a non-finite filter boundary. This chapter therefore depends on the target, derivative, and diagnostics contracts already defined in the earlier chapters.

38.1 Transported Targets

Let $q \in \mathbb{R}^d$ denote the original unconstrained parameter and let

$$q = T_\psi(z)$$

be an invertible differentiable map from transported coordinates z to the original coordinates. If the original target density is

$$\pi_Q(q) \propto \exp[-U_Q(q)],$$

then the transported target is

$$U_Z(z) = U_Q(T_\psi(z)) - \log \left| \det J_{T_\psi}(z) \right|. \tag{38.1}$$

Running HMC in z-space is target-preserving only when the sampler evaluates (38.1) and its gradient. The Jacobian term is not optional. A learned map that is imperfect can still be useful as a preconditioner, but omitting the change-of-variables correction changes the target.

38.2 NeuTra Position

The Phase 2B literature gate accepted NeuTra only as transport and geometry support. That is exactly the right status for BayesFilter. A NeuTra-like map can improve posterior geometry by making the transformed target closer to a simple reference distribution. It is not a correctness guarantee, and it is not a substitute for a valid filtering likelihood or a derivative provider that matches the Metropolis value.

The strategic use is:

(i) certify the filtering target and gradient first;

(ii) train or choose a transport map to improve geometry;

(iii) run HMC on the Jacobian-corrected transported target;

(iv) compare diagnostics and effective sample size per unit time to the untransported or affine baseline.

If the map has poor tail behavior, bad inverse stability, or large Jacobian curvature, it can make sampling worse even when it improves a training loss.

38.3 Surrogate Targets

A surrogate likelihood or surrogate Hamiltonian is a different object from a transport map. If the surrogate changes the scalar log target used in the Metropolis correction, the Markov chain targets the surrogate posterior unless a correction mechanism is added. BayesFilter should record one of three statuses for every surrogate:

diagnostic surrogate used only for exploration, initialization, or comparison; no posterior correctness claim is made;

corrected surrogate used inside a delayed-acceptance or Metropolis-corrected scheme that evaluates the intended target before a sample is accepted;

target-defining surrogate deliberately defines the approximate posterior being sampled, with approximation error documented separately.

This distinction prevents a common failure mode: using a fast approximate filter or neural surrogate for gradients and then interpreting the resulting chain as if it sampled the exact filtering posterior.

38.4 Diagnostics

Transport diagnostics should be tied to sampler performance and target integrity:

- transformed-target finite value and finite gradient checks;
- log-Jacobian finite checks and inverse/forward reconstruction residuals;
- tail probes for large $\|z\|$;
- acceptance rate, divergences, E-BFMI, and effective sample size per second relative to an affine baseline;
- shell-level Hessian or force diagnostics where available.

These diagnostics explain why a map helps or fails. They do not override a failed target contract.

38.5 BayesFilter Policy

Transport and surrogate methods should enter BayesFilter after the exact or controlled approximate likelihood path has been validated. They are valuable for large models because geometry can dominate HMC cost, but they sit on top of the filter contract rather than underneath it. In particular, they should not be proposed as a fix for SVD spectral-gradient instability, missing derivative guards, or partial XLA compilation. Those problems must be solved at the filter and sampler-contract layer first.

Part VI

Industrial Applications and Case Studies

Chapter 39

LGSSM Validation Ladder

The linear Gaussian state-space model is the first validation target for BayesFilter because it supplies an exact likelihood, controlled derivatives, and tractable regression tests. Every nonlinear or DSGE backend should be reduced to an LGSSM special case before it is trusted in HMC.

39.1 Evidence Already Available

The MacroFinance project provides the strongest local evidence stream:

- exact prediction-error likelihood formulas and analytic derivative notes;
- covariance, solve, Cholesky square-root, QR square-root, TensorFlow, and SVD-flavored backends;
- finite-difference and autodiff parity tests on small models;
- large-scale backend-ladder tests that report reconstruction errors, solve residuals, QR pivots, and finite derivative deltas;
- missing-data policy tests that fail unsupported sparse panels before the likelihood silently changes meaning.

This evidence justifies using the LGSSM as BayesFilter's regression oracle. It does not yet replace the BayesFilter-specific packaging and public API tests that will be needed when the code is moved into this project.

39.2 Validation Rungs

The LGSSM ladder is:

(i) fixed-parameter likelihood agreement against an independent Kalman implementation;

(ii) score agreement against finite differences and small-model autodiff;

(iii) Hessian agreement where the backend claims observed-information support;

(iv) backend parity among solve, Cholesky, QR, and TensorFlow paths;

(v) mask and mixed-frequency parity against dense special cases;

(vi) compiled/eager parity for the full value-and-gradient target;

(vii) short HMC plumbing checks with finite diagnostics;

(viii) medium and strict HMC runs only after the preceding rungs pass.

39.3 Interpretation

Passing the LGSSM ladder means the sampler interface, derivative convention, and linear algebra backends are coherent on an exact target. It does not prove that a nonlinear approximation is accurate, and it does not prove that a DSGE posterior has good HMC geometry. It is the floor, not the ceiling.

Chapter 40

Nonlinear SSM Validation Ladder

The nonlinear SSM ladder tests whether approximate nonlinear filtering can be used inside HMC before DSGE perturbation solvers and economic boundaries enter the picture. It should use small synthetic models with known parameters and independent references, not large application models.

40.1 Controlled Models

The first nonlinear validation suite is deliberately small. It is designed to test the filtering law, structural deterministic completion, and sigma-point moment approximation before DSGE perturbation solvers, client adapters, or HMC geometry enter the picture. The suite contains three first-rung models.

40.1.1 Model A: Affine Gaussian Structural Oracle

The affine oracle is a BayesFilter-local synthetic model:

$$x_t = (m_t, \ell_t)^\top, \qquad m_t = 0.35 m_{t-1} - 0.10 \ell_{t-1} + 0.25 \varepsilon_t, \qquad \ell_t = m_{t-1},$$

$$y_t = m_t + \eta_t, \qquad \varepsilon_t \sim N(0,1), \qquad \eta_t \sim N(0, 0.15^2).$$

The initial law is Gaussian and the deterministic coordinate ℓ_t stores the previous m. The oracle is the exact linear Gaussian Kalman likelihood after converting the structural law to its full-state LGSSM. The test must verify that SVD cubature, SVD-UKF, and SVD-CUT4 collapse to the same prediction-error likelihood and that deterministic-completion residuals are zero up to numerical tolerance.

40.1.2 Model B: Nonlinear Accumulation

The nonlinear accumulation model is a smooth BayesFilter-local structural fixture:

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = \alpha k_{t-1} + \beta \tanh(m_t),$$

$$y_t = m_t + k_t + \eta_t, \qquad \varepsilon_t \sim N(0,1), \qquad \eta_t \sim N(0, \sigma_\eta^2).$$

The first V1 defaults are

$$\rho = 0.70, \qquad \sigma = 0.25, \qquad \alpha = 0.55, \qquad \beta = 0.80, \qquad \sigma_\eta = 0.30.$$

This model tests nonlinear deterministic completion without a time-indexed transition hook. The reference oracle is a dense one-step Gaussian moment-projection quadrature: it integrates the augmented predictive Gaussian, computes the transformed prediction and observation moments, and then applies the same Gaussian update used by the sigma-point filters. It is not an exact nonlinear likelihood.

40.1.3 Model C: Nonlinear Growth Benchmark

The scalar nonlinear growth benchmark is the standard model associated with Kitagawa's Monte Carlo filtering examples [Kitagawa, 1996] and widely reused in sequential Monte Carlo discussions. Its time-inhomogeneous law is

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1+x_{t-1}^2} + 8\cos(1.2(t-1)) + \sigma_x\varepsilon_t,$$

$$y_t = \frac{x_t^2}{20} + \sigma_y\eta_t, \qquad \varepsilon_t, \eta_t \sim N(0,1).$$

Because the current BayesFilter structural TensorFlow transition does not pass an explicit time index, the V1 testing fixture uses an autonomous deterministic phase coordinate:

$$\tau_t = \tau_{t-1} + 1, \qquad \tau_1 = 1,$$

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1+x_{t-1}^2} + 8\cos(1.2\tau_{t-1}) + \sigma_x\varepsilon_t.$$

The observation equation remains $y_t = x_t^2/20 + \sigma_y\eta_t$. The first defaults are $x_1 \sim N(0, 0.2)$, $\sigma_x = 1$, and $\sigma_y = 1$. As in Model B, the first oracle is dense one-step Gaussian moment projection, not the exact nonlinear likelihood.

40.1.4 Deferred Models

Bearings-only tracking. Gordon et al. [1993] and Arulampalam et al. [2002] are useful sources, but the test should wait until angle residual and wrapping policy are fixed.

Radar range-bearing tracking. This should follow the bearings-only residual policy and should add mixed-scale observation diagnostics.

Stochastic volatility. This should wait until the sigma-point interface supports non-additive observation noise or an explicit approximation contract, since current filters assume additive observation covariance after $h(x_t)$.

Deferred means that the model is useful later, not that it is rejected. The first V1 suite stops at Models A–C so that value, score, branch, and benchmark tests share a stable set of fixtures.

40.2 Current V1 Filter Evidence

The first CPU nonlinear benchmark compares three SVD/eigen sigma-point value filters:

$$\mathcal{B} = \{\text{SVD cubature}, \text{SVD-UKF}, \text{SVD-CUT4}\}.$$

For Model A the reference is the exact Kalman prediction-error likelihood after structural-to-LGSSM conversion. The benchmark rows must satisfy

$$|\ell_b(y_{1:T}) - \ell_{\mathrm{KF}}(y_{1:T})| \approx 0, \qquad b \in \mathcal{B},$$

up to floating-point tolerance, and the filtered means and covariances must match the exact Kalman recursion. This verifies that the nonlinear sigma-point code collapses to the correct affine Gaussian law.

For Models B and C the benchmark uses only a dense one-step Gaussian projection reference. If Q_{dense} denotes a tensor-product Gaussian rule and Π_b denotes one of the sigma-point rules, the artifact reports

$$\left|\ell^{(1)}_{\Pi_b} - \ell^{(1)}_{Q_{\mathrm{dense}}}\right|, \qquad \left\|\hat{x}^{(1)}_{\Pi_b} - \hat{x}^{(1)}_{Q_{\mathrm{dense}}}\right\|_2, \qquad \left\|\hat{P}^{(1)}_{\Pi_b} - \hat{P}^{(1)}_{Q_{\mathrm{dense}}}\right\|_F.$$

These are approximation-quality diagnostics for the first filtering step, not exact full-likelihood errors. The current artifact therefore records the full Models B–C log likelihoods as filter outputs and blocks any claim that those values have been certified against an exact nonlinear likelihood.

The same artifact records the shared branch diagnostics

$$r_{\mathrm{det}}, \quad r_{\mathrm{supp}}, \quad g_{\mathrm{place}}, \quad g_{\mathrm{innov}}, \quad c_{\mathrm{floor}}, \quad N_{\mathrm{points}},$$

where r_{det} is the deterministic-completion residual, r_{supp} is the residual in the inactive SVD support, g_{place} and g_{innov} are placement and innovation spectral-gap diagnostics, c_{floor} counts active covariance floors, and N_{points} is the rule size. These diagnostics keep model-law failures separate from numerical regularization. They are also the gate before analytic score or HMC claims.

The analytic first-order score now has a second rung beyond the affine fixture. For Model B we use

$$\theta_B = (\rho, \sigma, \beta)$$

with α, the initial law, the innovation variance, and the observation variance held fixed. The derivative provider supplies

$$\frac{\partial F}{\partial x_{t-1}} = \begin{bmatrix} \rho & 0 \\ \beta(1-\tanh^2 m_t)\rho & \alpha \end{bmatrix}, \qquad \frac{\partial F}{\partial \varepsilon_t} = \begin{bmatrix} \sigma \\ \beta(1-\tanh^2 m_t)\sigma \end{bmatrix},$$

and parameter partials

$$F_\rho = \begin{bmatrix} m_{t-1} \\ \beta(1-\tanh^2 m_t)m_{t-1} \end{bmatrix}, \quad F_\sigma = \begin{bmatrix} \varepsilon_t \\ \beta(1-\tanh^2 m_t)\varepsilon_t \end{bmatrix}, \quad F_\beta = \begin{bmatrix} 0 \\ \tanh(m_t) \end{bmatrix}.$$

The observation derivative is $H_x = [1, 1]$. Centered finite differences of the implemented SVD cubature, SVD-UKF, and SVD-CUT4 likelihoods match the analytic scores on the selected smooth branch.

For Model C we use

$$\theta_C = (\sigma_u, \sigma_y, P_{0,x}).$$

The derivative provider supplies

$$F_x = \begin{bmatrix} \frac{1}{2} + 25\frac{1-x^2}{(1+x^2)^2} & -9.6\sin(1.2\tau) \\ 0 & 1 \end{bmatrix}, \qquad F_\varepsilon = \begin{bmatrix} \sigma_u \\ 0 \end{bmatrix},$$

$$F_{\sigma_u} = (\varepsilon_t, 0)', \qquad \partial_{\sigma_y} R = 2\sigma_y, \qquad \partial_{P_{0,x}} P_0 = \mathrm{diag}(1, 0),$$

and $H_x = [x/10, 0]$. Score parity is certified on a nondegenerate phase-state testing variant with positive phase variance. The default Model C value benchmark still has zero phase variance. Under the smooth simple-spectrum/no-active-floor SVD score branch, that default law remains blocked by an active placement floor. This is an intentional diagnostic. The structural fixed-support branch in Chapter 18 instead keeps the phase direction as a declared structural null direction: the factor column and its derivative are zero, the pre-transition sigma-point variable is (x_{t-1}, ε_t), and the propagated phase coordinate is completed pointwise by the transition map. On that branch, default Model C is now a score-ready testing target for SVD cubature, SVD-UKF, and SVD-CUT4, subject to the fixed-null covariance, fixed-null derivative, active-gap, and finite-difference diagnostics.

Consequently, GPU/XLA and HMC evidence for nonlinear Models B–C remains deferred until the score-ready parameter boxes and branch diagnostics have been selected and tested.

40.3 Required Outputs

A nonlinear SSM HMC validation run should write:

- JSON diagnostics with chain counts, warmup/draw counts, step sizes, acceptance rate, divergence counts, R-hat, ESS, MCSE/SD, and runtime;
- NPZ chain artifacts for independent audit;
- true-parameter coverage when synthetic truth exists;
- filter failure counts, conditioning-guard counts, and finite-gradient failures.

Medium runs are sampler-usability evidence. Strict runs require at least four chains, finite log targets, zero or justified divergences, R-hat below 1.01, bulk and tail ESS above the planned thresholds, and saved artifacts.

40.4 Stop Rules

If the LGSSM rung fails, the nonlinear rung should not start. If the nonlinear SSM rung fails while LGSSM passes, the failure should be attributed first to the approximation, nonlinear target geometry, transition hook, or derivative path. DSGE/NK debugging should wait until this simpler explanation is ruled out.

Chapter 41

NK SVD Case Study

The NK SVD case study is a warning label with useful engineering evidence. It shows how far a numerically hardened SVD sigma-point path can get, and it also shows why BayesFilter should not equate bounded HMC output with convergence or industrial readiness.

41.1 What Passed

The source reset memos record several meaningful passes:

- focused SVD, target-boundary, adjoint, and generic nonlinear SSM contracts passed during the post-refactor work;
- generic SSM SVD likelihoods and gradients compile under production XLA;
- NK SVD fixed-solution and full-solve likelihood/gradient paths compile under XLA with explicit opt-in;
- tiny NK SVD production-XLA HMC smoke tests run to completion with finite output;
- generic nonlinear SSM medium HMC writes diagnostics with finite chains and zero divergences under tuned short-run defaults.

These results are valuable sampler-usability and compilation evidence.

41.2 What Remains Blocked

The same reset memos record why NK convergence remains blocked. Earlier bounded stress runs produced nonfinite adjoint and lam1 inputs, SVD conditioning guard activations, nonfinite intermediate arrays, and extreme but finite transformed-theta states. The later production-XLA smoke gates do not erase that artifact history, because they are tiny runs and not clean stress experiments.

Therefore the next NK claim must be a clean stress gate, not a convergence claim. A valid clean gate requires no checked-nonfinite adjoint captures, no nonfinite lam1 inputs, no nonfinite SVD-filter captures, no unexplained transformed-theta scale explosion, and zero or separately justified divergences.

41.3 Lesson for BayesFilter

The case study separates four statuses:

filter-correct likelihood and gradients agree with controlled references;

sampler-usable bounded HMC runs produce finite chains and useful diagnostics;

converged strict multi-chain diagnostics pass on the intended target;

production-XLA-gated the full value-and-gradient path compiles and runs under the declared production policy.

NK SVD has meaningful production-XLA and smoke evidence, but it is not yet a converged case study. That distinction should stay visible in every future BayesFilter report.

Chapter 42

CIP/AFNS Case Study

The CIP/AFNS material is useful to BayesFilter because it is a large structured linear Gaussian application with economically meaningful observation equations, missing-data pressure, and analytic derivative requirements. The economics belongs in the source CIP monograph. BayesFilter should import only the filtering lessons.

42.1 Filtering Lessons

The reusable lessons are:

- affine or linearized macro-finance measurement systems can become large LGSSMs with structured observation matrices;
- yield-curve blocks create dense measurement panels with heterogeneous maturities and noise scales;
- missing observations must be represented by explicit masks, not by silent imputation or dynamic shape changes;
- analytic derivatives and backend parity matter because HMC mass matrices and MAP curvature depend on the same likelihood being sampled.

42.2 BayesFilter Boundary

BayesFilter should not re-derive AFNS economics, currency-basis theory, or asset-pricing restrictions in this monograph. It should define the state-space objects that an application supplies:

$$F_\theta, \quad Q_\theta, \quad H_\theta, \quad R_\theta, \quad m_{0|0,\theta}, \quad P_{0|0,\theta},$$

and then validate that the filter, derivative provider, masks, and HMC diagnostics behave as promised. This boundary keeps BayesFilter reusable and prevents application monographs from becoming maintenance forks of the filtering code.

42.3 Readiness Use

CIP/AFNS should be used as a large LGSSM readiness case once the BayesFilter package exists. The required evidence is backend parity, missing-data policy, finite analytic gradi-

ents and Hessians, MAP/mass-matrix diagnostics, and short HMC plumbing checks. It is not the right first target for SVD sigma-point HMC claims, because its main reusable strength is exact or controlled linear Gaussian filtering.

Chapter 43

NAWM-Scale Design Target

NAWM-scale models define the industrial target for BayesFilter. They should not be presented as completed evidence in this monograph pass. Instead, they set the design requirements for robust filtering and HMC on high-dimensional state-space systems.

43.1 Why NAWM Changes the Standard

As state dimension, parameter dimension, and observation dimension grow, several risks become routine rather than exceptional:

- covariance spectra become clustered or nearly singular;
- HMC proposals visit invalid or barely valid structural regions;
- raw autodiff through eigendecompositions or SVDs becomes fragile;
- compile boundaries and dynamic branches become performance bottlenecks;
- short smoke runs become less informative about long-chain behavior.

This is why BayesFilter should prefer analytic or custom derivative paths for production filtering wherever feasible.

43.2 Design Requirements

A NAWM-grade BayesFilter backend needs:

(i) a fixed, documented target contract and parameter transform;

(ii) explicit state-space shape contracts;

(iii) finite invalid-region returns and finite gradients where HMC moves;

(iv) analytic/custom gradients for exact or controlled approximate likelihoods;

(v) spectral-gap and factor diagnostics for any spectral backend;

(vi) full-pipeline JIT evidence, not only outer-wrapper tracing;

(vii) reproducible diagnostics and reset memos for every long experiment.

43.3 Hypothesis

The working hypothesis is that NAWM-scale HMC will be more robust with source-backed analytic or custom filtering gradients than with raw tape gradients through spectral decompositions. This is not yet a theorem or a completed benchmark. It is the prevention-first design target motivated by the SVD debugging history and by the derivative risks documented in the Phase 2B literature gate.

Chapter 44

Production Checklist

This checklist is the operating definition of BayesFilter production readiness. It is intentionally stricter than a successful smoke test.

44.1 Target and Source

Before HMC:

(i) the scalar log target is defined in writing;

(ii) every formula has provenance in source notes, local code/tests, or reviewed literature;

(iii) parameter transforms and Jacobian terms are included;

(iv) invalid structural regions return documented finite values;

(v) approximate filters are labeled as target-defining approximations or corrected surrogates.

44.2 Derivative and Filter

The filter backend must provide:

(i) shape, dtype, mask, and static-compilation contracts;

(ii) finite value and finite gradient checks;

(iii) derivative parity against finite differences or autodiff on controlled cases;

(iv) backend parity where multiple linear algebra forms exist;

(v) factor, solve, condition-number, and spectral-gap diagnostics;

(vi) an explicit custom-gradient policy when raw autodiff is not the final production path.

44.3 Sampler and JIT

The sampler must report:

(i) whether the complete value-and-gradient path is compiled;

(ii) eager/compiled parity on small targets;

(iii) mass-matrix source and conditioning;

(iv) acceptance rate, divergences, E-BFMI or equivalent energy diagnostics, R-hat, ESS, MCSE/SD, and runtime;

(v) saved JSON and chain artifacts for medium and strict runs;

(vi) sidecar capture artifacts only for bounded debugging, not for every production transition.

44.4 Claim Discipline

Use the narrowest honest label:

value-valid the likelihood value is finite and reference-checked;

gradient-valid the derivative matches the target value and passes parity checks;

sampler-usable finite HMC diagnostics justify the next experiment;

converged strict multi-chain diagnostics pass and artifacts are saved;

production-ready all preceding gates pass under the intended model, data, hardware, and compilation policy.

Anything less should stay in the reset memo as a gap, not disappear into prose.

Appendices

Appendix A

Notation and Shape Conventions

This appendix fixes the notation used throughout BayesFilter. Source projects use overlapping but not identical symbols; this monograph uses the following conventions unless a chapter states otherwise.

Symbol	Meaning
$t = 1, \dots, T$	observation time index
$x_t \in \mathbb{R}^{n_x}$	latent state
$y_t \in \mathbb{R}^{n_y}$	observation vector before masking
$\theta \in \mathbb{R}^{d_\theta}$	model parameter vector
$u \in \mathbb{R}^{d_u}$	unconstrained sampler coordinate
c, F, Q	linear Gaussian transition offset, matrix, covariance
a, H, R	linear Gaussian observation offset, matrix, covariance
$m_{t\|s}, P_{t\|s}$	conditional state mean and covariance
v_t, S_t	innovation and innovation covariance
K_t	Kalman gain
$\ell(\theta)$	filtering log likelihood
$g(\theta)$	likelihood score $\nabla_\theta \ell(\theta)$
$G(\theta)$	likelihood Hessian $\nabla^2_{\theta\theta} \ell(\theta)$

Matrix transposes are written with $^\top$. Positive definiteness and positive semidefiniteness are written $A \succ 0$ and $A \succeq 0$. The operator $\operatorname{tr}(\cdot)$ denotes trace, and $\operatorname{diag}(\cdot)$ forms or extracts a diagonal depending on context.

A.1 Derivative Shapes

For a parameter-dependent vector $b(\theta) \in \mathbb{R}^n$, first derivatives have shape $d_\theta \times n$ and second derivatives have shape $d_\theta \times d_\theta \times n$. For a matrix $A(\theta) \in \mathbb{R}^{m \times n}$, first derivatives have shape $d_\theta \times m \times n$ and second derivatives have shape $d_\theta \times d_\theta \times m \times n$.

This convention matches the MacroFinance derivative-provider layout used as the current implementation reference. Chapters that discuss TensorFlow or JAX execution may store tensors differently for performance, but must state the mathematical shape convention before changing memory layout.

A.2 Label Namespace

New labels use a BayesFilter namespace:

`ch:bf-*`, `sec:bf-*`, `def:bf-*`, `asm:bf-*`, `eq:bf-*`, `tab:bf-*`.

Source-project labels should not be copied verbatim. When a source label is important for provenance, it should be recorded in the reset memo or source map.

Appendix B

Matrix Calculus Identities

The derivative chapters repeatedly use a small set of matrix identities. This appendix records them as proof obligations to be checked against the source derivations.

For an invertible matrix $S(\theta)$,

$$\partial_i S^{-1} = -S^{-1}(\partial_i S)S^{-1}.$$

For a symmetric positive definite matrix $S(\theta)$,

$$\partial_i \log\det S = \operatorname{tr}(S^{-1}\partial_i S).$$

For $q = v^\top S^{-1} v$ and $Sw = v$,

$$\partial_i q = 2(\partial_i v)^\top w - w^\top(\partial_i S)w.$$

These identities are the local algebra behind the solve-form score in Chapter 9. Second-order formulas should be expanded only with explicit source labels or proof notes because the noncommutative matrix products are easy to reorder incorrectly.

Appendix C

Factor Derivative Proofs

This appendix is a placeholder for full Cholesky and QR factor derivative proofs. The current writing pass records the required evidence policy rather than attempting to re-prove every factor identity.

Before this appendix is promoted beyond outline form, it should include:

- the source MacroFinance labels for Cholesky derivative formulas;
- the QR perturbation equations used by the square-root Kalman backend;
- reconstruction identities connecting factor derivatives to covariance derivatives;
- code references to QR and Cholesky differentiated Kalman tests;
- MathDevMCP or manual proof-obligation results for each nontrivial step.

Appendix D

MathDevMCP Workflows

MathDevMCP is used as an audit assistant, not as an oracle. Its results should be recorded with their status and limitations.

Useful commands for this monograph include:

- `search-latex` to find source labels and neighborhoods;
- `extract-latex-context` to record exact source provenance;
- `audit-derivation-label` for bounded symbolic checks;
- `audit-kalman-recursion` for AST-level implementation structure.

The current solve-form Kalman audit found the expected solve-oriented operation graph but reported missing strict log-determinant and trace classification, as well as missing explicit shape and covariance guards. The correct response is to document those gaps and use the MacroFinance parity tests as additional evidence, not to relabel the audit as a proof.

Appendix E

ResearchAssistant Workflows

ResearchAssistant is the literature-ingestion sidecar for this monograph. It should be used when a chapter needs primary-source support rather than local implementation provenance. It is not a replacement for reading the source and checking whether the cited paper supports the exact BayesFilter claim.

E.1 Workspace Policy

Use a persistent, ignored workspace under the BayesFilter tree:

`/home/chakwong/BayesFilter/.research/ra-bayesfilter-monograph`.

Downloaded papers, source packages, extracted text, and inbox files stay out of Git. Durable decisions belong in 'docs/plans' result notes, the reset memo, and the source map.

E.2 Ingestion Workflow

The standard workflow is:

(i) search or fetch the primary source;

(ii) verify metadata against title, authors, venue, DOI or arXiv id;

(iii) reject mismatches explicitly;

(iv) extract the citation neighborhood or source labels relevant to the BayesFilter claim;

(v) write a short result note stating supported claims and limits;

(vi) only then add or use a bibliography entry.

The Phase 2B result note is the model: it accepted exact arXiv source packages for HMC, NUTS, NeuTra, Matrix Backprop, Kitagawa score/Hessian material, and the differentiable particle-filter frontier, while rejecting several metadata mismatches.

E.3 Claim Gate

A literature-heavy paragraph is draftable only when one of the following is true:

- an accepted ResearchAssistant note supports the claim and records its limits;
- a local source document plus code/test audit supports the claim;
- the paragraph is explicitly labeled as a project decision or an open hypothesis.

Raw '.bib' entries, downloaded PDFs, and search results are not claim support.

E.4 Current Blockers

The current blockers are:

- primary Julier–Uhlmann or equivalent UKF support for detailed sigma-point literature claims;
- primary PMCMC and pseudo-marginal support before particle-filter correctness claims;
- square-root filtering primary sources if final square-root bibliographic prose is expanded;
- a dedicated derivation/code audit before Kalman score and Hessian formulas are marked peer-review ready;
- industrial-scale SVD-HMC safety evidence before NAWM-scale claims.

Until these blockers are closed, the monograph should keep the conservative wording used in the current draft.

Appendix F

Source Map

The machine-readable source map is

`docs/source_map.yml`.

It records which source project, chapter, reset memo, code file, or test file supports each BayesFilter chapter. The source map is part of the monograph's quality-control system, not merely an index.

Each imported item should move through explicit states:

- `candidate`: relevant but not yet drafted;
- `drafted`: consolidated into BayesFilter prose;
- `audited`: checked against source math, code, tests, or reviewed literature;
- `superseded`: replaced by another source;
- `excluded`: intentionally outside BayesFilter scope.

The source map also records duplicate source groups. For example, Kalman likelihood material appears in the DSGE monograph, CIP monograph, and the MacroFinance analytic derivative note. BayesFilter treats the MacroFinance note and associated tests as the derivative spine while using the monographs as expository sources.

Generated artifacts, paper caches, and local PDFs are not provenance by themselves. A claim becomes monograph-ready only when the relevant source or test has been summarized in a committed note, mapped in the source map, or explicitly cited in the chapter.

Appendix G

Experiment Templates

This appendix gives reusable experiment templates for future BayesFilter implementation work. The templates are intentionally short. Each experiment should write a reset-memo entry with commands, artifacts, interpretation, and the next justified phase.

G.1 Filter Reference Template

Use this template before HMC:

(i) define the model, parameter transform, data seed, and fixed structural parameters;

(ii) compute a reference likelihood from an independent exact, dense, or trusted implementation;

(iii) compare the BayesFilter backend value to the reference;

(iv) compare gradients by finite differences or small-model autodiff;

(v) record finite-value, factor, solve, mask, and failure-code diagnostics;

(vi) save the command and tolerance rationale.

Passing this template earns value-valid or gradient-valid status, depending on which checks are included. It does not earn sampler-usability status.

G.2 HMC Smoke Template

Use this template to test sampler plumbing:

(i) use a small model and static shapes;

(ii) run one or two short chains with deliberately small draw counts;

(iii) require finite chains, finite log targets, and no unhandled exceptions;

(iv) record acceptance rate, divergence count, step size, runtime, and whether the complete value-gradient path was compiled;

(v) state explicitly that the run is not convergence evidence.

Smoke runs justify the next experiment only when the failure taxonomy is clean.

G.3 Medium Recovery Template

Use this template for controlled posterior recovery:

(i) run at least four chains when feasible;

(ii) save JSON diagnostics and NPZ chain artifacts;

(iii) report R-hat, bulk ESS, tail ESS, MCSE/SD, divergences, acceptance, and runtime;

(iv) compare posterior intervals to synthetic truth or a trusted reference;

(v) classify the result as sampler-usable or not sampler-usable.

Medium recovery is allowed to have looser thresholds than strict convergence, but the looseness must be stated in the result note.

G.4 Strict Convergence Template

Use this template only after the preceding gates pass:

(i) run at least four chains with planned warmup and draw counts;

(ii) require finite log targets and finite gradients throughout accepted trajectories;

(iii) require zero divergences unless a bounded exception is justified;

(iv) require the planned R-hat, ESS, and MCSE/SD thresholds;

(v) audit saved artifacts before writing a convergence claim.

If any required diagnostic fails, the reset memo should name the blocker and the next diagnostic hypothesis instead of weakening the claim label.

Bibliography

Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. The conjugate unscented transform: An approach to evaluate multi-dimensional expectation integrals. In *2012 American Control Conference*, pages 5556–5561, 2012a. Article 6314970.

Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. Conjugate unscented transform and its application to filtering and stochastic integral calculation. In *AIAA Guidance, Navigation, and Control Conference 2012*, 2012b. AIAA Guidance, Navigation, and Control Conference, Minneapolis, Minnesota.

Sungbae An and Frank Schorfheide. Bayesian analysis of dsge models. *Econometric Reviews*, 26(2–4):113–172, 2007.

Martin M. Andreasen, Jesús Fernández-Villaverde, and Juan F. Rubio-Ramírez. The pruned state-space system for non-linear dsge models: Theory and empirical applications. *Review of Economic Studies*, 85(1):1–49, 2018.

Christophe Andrieu and Gareth O. Roberts. The pseudo-marginal approach for efficient monte carlo computations. *The Annals of Statistics*, 37(2):697–725, 2009.

Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. *Journal of the Royal Statistical Society: Series B*, 72(3):269–342, 2010.

M. Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002. doi: 10.1109/78.978374.

Thomas Bengtsson, Peter Bickel, and Bo Li. Curse-of-dimensionality revisited: Collapse of the particle filter in very large scale systems. *Probability and Statistics: Essays in Honor of David A. Freedman*, pages 316–334, 2008.

Michael Betancourt. A conceptual introduction to hamiltonian monte carlo. *arXiv preprint arXiv:1701.02434*, 2017.

Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Springer, 2020. doi: 10.1007/978-3-030-47845-2.

Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport. In *Proceedings of the 38th International Conference on Machine Learning*, pages 2100–2111, 2021.

Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013.

Fred Daum and Jim Huang. Nonlinear filters with particle flow. In *Signal Processing, Sensor Fusion, and Target Recognition XVII*, 2008.

Arnaud Doucet, Nando de Freitas, and Neil Gordon, editors. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.

J. Durbin and S. J. Koopman. *Time Series Analysis by State Space Methods*. Oxford University Press, 2012.

Mark Girolami and Ben Calderhead. Riemann manifold langevin and Hamiltonian monte carlo methods. *Journal of the Royal Statistical Society: Series B*, 73(2):123–214, 2011. doi: 10.1111/j.1467-9868.2010.00765.x.

Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F: Radar and Signal Processing*, 140(2):107–113, 1993.

Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 15379–15389, 2019.

Edward P. Herbst and Frank Schorfheide. *Bayesian Estimation of DSGE Models*. Princeton University Press, 2015.

Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15: 1593–1623, 2014.

Matthew D. Hoffman, Pavel Sountsov, Joshua V. Dillon, Ian Langmore, Dustin Tran, and Srinivas Vasudevan. Neutra-lizing bad geometry in Hamiltonian Monte Carlo using neural transport. *Journal of Machine Learning Research*, 22(52):1–64, 2021. arXiv:1903.03704.

Chao Hu and Peter Jan van Leeuwen. A particle flow filter for high-dimensional system applications. *Quarterly Journal of the Royal Meteorological Society*, 147(739):2358–2377, 2021.

Catalin Ionescu, Orestis Vantzos, and Cristian Sminchisescu. Training deep networks with structured layers by matrix backpropagation. *arXiv preprint arXiv:1509.07838*, 2015. doi: 10.48550/arXiv.1509.07838.

Rico Jonschkowski, Deepak Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors. In *Robotics: Science and Systems*, 2018.

Simon J. Julier and Jeffrey K. Uhlmann. A general method for approximating nonlinear transformations of probability distributions. Technical report, Robotics Research Group, Department of Engineering Science, University of Oxford, 1996. 1 November 1996.

Simon J. Julier and Jeffrey K. Uhlmann. A new extension of the kalman filter to nonlinear systems. *Proceedings of AeroSense*, 1997.

Peter Karkus, David Hsu, and Wee Sun Lee. Particle filter networks with application to visual localization. In *Conference on Robot Learning*, 2018.

Jinill Kim, Sunghyun Kim, Ernst Schaumburg, and Christopher A. Sims. Calculating and using second-order accurate solutions of discrete time dynamic equilibrium models. *Journal of Economic Dynamics and Control*, 32(11):3397–3414, 2008.

Genshiro Kitagawa. Monte carlo filter and smoother for non-gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25, 1996. doi: 10.1080/10618600.1996.10474692.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant processing. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

Yanghai Li and Mark Coates. Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116, 2017.

Radford M. Neal. Mcmc using hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

Matthew D. Parno and Youssef M. Marzouk. Transport map accelerated Markov chain Monte Carlo. *SIAM/ASA Journal on Uncertainty Quantification*, 6(2):645–682, 2018. doi: 10.1137/17M1134640.

Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.

Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.

Bernhard Schmitzer. Stabilized sparse scaling algorithms for entropy regularized transport problems. *SIAM Journal on Scientific Computing*, 41(3):A1443–A1481, 2019.

Chris Snyder, Thomas Bengtsson, Peter Bickel, and Jeffrey Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008.

Rudolph van der Merwe. *Sigma-Point Kalman Filters for Probabilistic Inference in Dynamic State-Space Models*. PhD thesis, OGI School of Science and Engineering at Oregon Health and Science University, 2004.

Cédric Villani. *Topics in Optimal Transportation*, volume 58 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2003.

Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *Advances in Neural Information Processing Systems*, 2017.

Meixin Zhu, Kevin Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.